STUDY GUIDE

Archive Algorithms Handbook

Every algorithm and problem from the course slides, with dry runs

This is Part 5 of the master guide on its own, ready to print for exam revision.

Generated 10 October 2026 from the same lesson data that powers the website.
Every Java program in this guide was compiled with --release 17 and run against its expected output.

Archive algorithms

5.1 Euclid's Algorithm (GCD)

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 40 min · ★ on the minimum-viable path

Find the greatest common divisor of two numbers in a handful of steps by repeatedly taking remainders.

YOU WILL BE ABLE TO

- Explain what a GCD is and compute small ones by listing divisors.
- Explain WHY gcd(a, b) = gcd(b, a mod b) — not just that it works.
- Write Euclid's algorithm in Java both as a loop and recursively, and trace it by hand.
- State its running time, O(log min(a, b)), and why it is so much faster than trying every divisor.

Before this lesson: Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading · Divisibility, GCD/LCM, Primes and Modular Arithmetic · Recursion: Base Case, Recursive Case and the Call Stack

The idea

The **greatest common divisor** (GCD) of two numbers is the largest number that divides both of them with no remainder. gcd(12, 18) = 6 because 6 divides 12 and 18, and nothing bigger does.

Picture a floor that is 50 tiles long and 12 tiles wide. You want the **largest square tile** that covers the whole floor with no cutting. Euclid's trick: cut off as many 12×12 squares as you can. Four fit (4 × 12 = 48), leaving a thin strip 12 long and **2** wide. Any tile that works for the big floor must also work for the leftover strip, so now solve the strip: cut 2×2 squares from 12×2 — six fit and nothing is left over. The tile is 2×2, so gcd(50, 12) = 2.

Each line replaces the pair (a, b) with the smaller pair (b, a mod b).

```
50 = 4 × 12 + 2 → gcd(50, 12) = gcd(12, 2)
12 = 6 × 2 + 0 → gcd(12, 2) = gcd( 2, 0) = 2
```

WHY THIS WORKS

Let d be any number that divides both a and b. Write a = q·b + r. Then r = a − q·b, which is also divisible by d. So d divides b and r too. The same argument works backwards: anything dividing b and r also divides a = q·b + r. Hence **a,b and b,r have exactly the same common divisors**, so they have the same greatest one. That single idea is the entire algorithm; the program is just that idea repeated until the remainder is 0.

Syntax

You need three Java ideas, all from earlier lessons:

- `%` is the remainder operator: `50 % 12` is `2`.
- A `while (b != 0)` loop repeats until the condition becomes false.
- A method can call itself (recursion) as long as it has a **base case** that stops it.

A tiny example

The smallest useful version just does what the pen-and-paper table does:

```
int a = 50, b = 12;
while (b != 0) {
 int r = a % b; // remainder of this step
 a = b; // the old divisor becomes the new dividend
 b = r; // the remainder becomes the new divisor
}
System.out.println(a); // 2
```

Guided example

The lecture slides write the work as a **Q | A | B | R** table (quotient, dividend, divisor, remainder). Each new row takes A from the previous B and B from the previous R. For gcd(33, 12):

The answer is the A on the last row, where B has become 0: gcd(33, 12) = 3.

Q	A	B	R
2	33	12	9
1	12	9	3
3	9	3	0
—	3	0	—

The program `EuclidGcd` prints exactly this table so you can compare it with your hand calculation. `NaiveGcd` shows the slow alternative — try every candidate from `min(a, b)` downwards — so you can feel the difference.

NaiveGcd.java — Naive GCD (try every divisor) (simple solution)

```
1  import java.util.Scanner;
2
3  public class NaiveGcd {
4   // Try every number from the smaller input down to 1.
5   // The first one that divides BOTH numbers is the greatest common divisor.
6   static int gcd(int a, int b) {
7   int smaller = Math.min(a, b);                                          (1)
8   for (int d = smaller; d >= 1; d--) {                                   (2)
9   if (a % d == 0 && b % d == 0) {                                        (3)
10  return d;
11  }
12  }
13  return 1; // only reached if a or b is 0 or negative; see the lesson's edge-case notes   (4)
14  }
15
16  public static void main(String[] args) {
17  Scanner sc = new Scanner(System.in);
18  int a = sc.nextInt();
19  int b = sc.nextInt();
20  System.out.println("gcd(" + a + ", " + b + ") = " + gcd(a, b));
21  }
22 }
```

Counts down from min(a, b) and returns the first number that divides both. Easy to believe, but slow for big inputs.

Sample input

```
50 12
```

Output when you run it

```
gcd(50, 12) = 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	A common divisor can never be larger than the smaller number, so we start there.
2	line 8	Counting DOWN means the first hit is automatically the greatest.
3	line 9	d is a common divisor only if it divides both with remainder 0.
4	line 13	Reached only for inputs the lesson excludes (0 or negatives). Every positive pair shares divisor 1, so the loop always returns earlier.

EuclidGcd.java — Euclid's algorithm (iterative) (optimised solution)

```
1  import java.util.Scanner;
2
3  public class EuclidGcd {
4   // Euclid's algorithm: gcd(a, b) = gcd(b, a mod b), and gcd(a, 0) = a.
5   static long gcd(long a, long b) {
6   a = Math.abs(a);                                                    1
7   b = Math.abs(b);
8   while (b != 0) {                                                    2
9   long remainder = a % b;                                             3
10  a = b;                                                              4
11  b = remainder;                                                      5
12  }
13  return a;
14  }
15
16  public static void main(String[] args) {
17  Scanner sc = new Scanner(System.in);
18  long a = sc.nextLong();
19  long b = sc.nextLong();
20
21  // Print the same Q | A | B | R table used in the lecture slides.
22  System.out.println(" Q A B R");
23  long x = Math.abs(a);
24  long y = Math.abs(b);
25  while (y != 0) {
26  long q = x / y;                                                     6
27  long r = x % y;
28  System.out.printf("%4d %10d %10d %10d%n", q, x, y, r);
29  x = y;
30  y = r;
31  }
32  System.out.printf("%4s %10d %10d %10s%n", "-", x, y, "-");
33  System.out.println("gcd(" + a + ", " + b + ") = " + gcd(a, b));
34  }
35 }
```

Replaces (a, b) by (b, a mod b) until b is 0. Uses long and Math.abs so large and negative inputs behave.

Sample input

```
33 12
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	gcd ignores signs: gcd(-12, 18) = gcd(12, 18) = 6.
2	line 8	Stop when the divisor reaches 0; at that moment a holds the answer.
3	line 9	Compute the remainder BEFORE overwriting a — the order of the next three lines matters.
4	line 10	Slide the window: old divisor becomes the new dividend.
5	line 11	The remainder becomes the new divisor (it is strictly smaller than b, so progress is guaranteed).
6	line 26	The table printing loop repeats the algorithm just to display Q, A, B, R. The real work is the gcd method above.

EuclidRecursive.java — Euclid's algorithm (recursive) (demonstration)

```
1  public class EuclidRecursive {
2   // The recursive form is a direct copy of the rule gcd(a, b) = gcd(b, a mod b).
3   static int gcd(int a, int b) {
4   System.out.println("gcd(" + a + ", " + b + ")");
5   if (b == 0) {                                                    (1)
6   return a; // base case: nothing left to divide by
7   }
8   return gcd(b, a % b); // smaller problem: same answer, smaller numbers   (2)
9   }
10
11  public static void main(String[] args) {
12  System.out.println("Answer: " + gcd(83, 19));
13  }
14 }
```

The same rule written as a self-calling method, with a print so you can watch the call stack unwind.

Line-by-line walkthrough

#	Where	What happens
1	line 5	Base case — without it the method would recurse forever (and then divide by zero).
2	line 8	The recursive step is the rule gcd(a, b) = gcd(b, a mod b), copied literally.

Dry run: watch it execute

Slide example 2: gcd(83, 19) — the numbers shrink until the remainder reaches 0.

Algorithm: Euclid's GCD. **Input:** a = `83`, b = `19`.

Pseudocode

```
while (b != 0) {
 r = a % b;
 a = b;
 b = r;
}
return a;
```

Step 1 of 12. Start with a = 83, b = 19.

a = 83 · b = 19 · r = —

Step 2 of 12. 83 mod 19 = 7 (quotient 4)

a = 83 · b = 19 · r = 7

Step 3 of 12. Slide: a becomes 19, b becomes 7.

a = 19 · b = 7 · r = 7

Step 4 of 12. 19 mod 7 = 5 (quotient 2)

a = 19 · b = 7 · r = 5

Step 5 of 12. Slide: a becomes 7, b becomes 5.

a = 7 · b = 5 · r = 5

Step 6 of 12. 7 mod 5 = 2 (quotient 1)

a = 7 · b = 5 · r = 2

Step 7 of 12. Slide: a becomes 5, b becomes 2.

a = 5 · b = 2 · r = 2

Step 8 of 12. 5 mod 2 = 1 (quotient 2)

a = 5 · b = 2 · r = 1

Step 9 of 12. Slide: a becomes 2, b becomes 1.

a = 2 · b = 1 · r = 1

Step 10 of 12. 2 mod 1 = 0 (quotient 2)

a = 2 · b = 1 · r = 0

Step 11 of 12. Slide: a becomes 1, b becomes 0.

a = 1 · b = 0 · r = 0

Step 12 of 12. b is 0, so the answer is a = 1.

a = 1 · b = 0 · r = 0

> **RESULT**
> gcd = 1

Complexity

Time

Naive: the loop may run `min(a, b)` times → O(min(a, b)). For gcd(1 000 000 007, 1 000 000 000) that is up to a billion iterations.

Euclid: look at two consecutive steps. Claim: a mod b < a/2 always.

- If b ≤ a/2, then a mod b < b ≤ a/2.
- If b > a/2, then a mod b = a − b < a/2.

So every step the value in the "A" position is more than halved after two steps. A number can only be halved about $\log_2(n)$ times before it hits 0, so the loop runs at most about $2 \cdot \log_2(\min(a, b))$ times.

```
time = O(log min(a, b)) space = O(1) iterative, O(log min(a, b))
                        recursive (call stack)
```

The worst case for Euclid is two consecutive Fibonacci numbers (Lamé's theorem), and even that is only logarithmic.

Input	Naive iterations (worst)	Euclid steps
gcd(50, 12)	12	2
gcd(83, 19)	19	5
gcd(1 000 000 007, 1 000 000 000)	≈ 1 000 000 000	≈ 6
gcd(Fibonacci F_{40}, F_{39})	≈ 63 000 000	38 (the worst case)

Naive vs optimized

Both are correct. Euclid wins by a factor of up to **millions** on large inputs because every step throws away a big chunk of the problem instead of one candidate.

Common mistakes

Overwriting a before using it

Problem

```
while (b != 0) {
 a = b;
 b = a % b; // a was already replaced!
}
```

Why: After `a = b`, the expression `a % b` is `b % b` which is always 0, so the loop stops with the wrong answer.

Fix: while (b != 0) { int r = a % b; a = b; b = r; }

Forgetting the base case

Problem

```
static int gcd(int a, int b) {
 return gcd(b, a % b);
}
```

Why: When b becomes 0, `a % 0` throws ArithmeticException ("/ by zero"). Recursion needs a stopping rule.

Fix: static int gcd(int a, int b) { if (b == 0) return a; return gcd(b, a % b); }

*LCM computed as a b / gcd**

Problem

```
int lcm = a * b / gcd(a, b); // a * b can overflow int
```

Why: For a = b = 100 000 the product 10 000 000 000 does not fit in an int, so the result is garbage.

Fix: long lcm = (long) a / gcd(a, b) * b; // divide first, then multiply

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Reduce a fraction — *Beginner*

Read a numerator and a denominator (both positive). Print the fraction in lowest terms as `numerator/denominator`.

- 1 ≤ numerator, denominator ≤ 10^9

Example input	Expected output
12 18	2/3

Hint 1: What number can you divide BOTH parts by without leaving a remainder? **Hint 2:** Use Euclid's algorithm for that number, then divide.

Exercise 2 · LCM of a list — *Intermediate*

Read n and then n positive integers. Print their least common multiple. The answer fits in a `long`.

- 1 ≤ n ≤ 20
- 1 ≤ each number ≤ 10^6
- answer ≤ $9 \cdot 10^{18}$

Example input	Expected output
3 ⏎ 4 6 8	24

Hint 1: lcm(a, b) = a / gcd(a, b) * b. **Hint 2:** Fold the list: lcm = lcm(lcm, next). **Hint 3:** Use long and divide before multiplying.

Exercise 3 · Modular inverse — *Challenge*

Given a and m, print the number x in [0, m−1] such that (a · x) mod m = 1, or -1 if no such x exists.

- 1 ≤ a, m ≤ 10^9
- m ≥ 2

Example input	Expected output
3 11	4
6 9	−1

Hint 1: An inverse exists exactly when gcd(a, m) = 1. **Hint 2:** Extended Euclid finds x and y with a·x + m·y = gcd(a, m). Then x is the inverse. **Hint 3:** The x it returns can be negative — shift it into [0, m−1] with ((x % m) + m) % m.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is gcd(48, 18)?

- **A.** 3
- **B.** 6
- **C.** 9
- **D.** 12

Q2. What does this program print?

```
int a = 84, b = 36;
while (b != 0) {
 int t = a % b;
 a = b;
 b = t;
}
System.out.println(a);
```

Your answer: _____

Q3. Which line makes this gcd method return the wrong answer?

```
static int gcd(int a, int b) {
 while (b != 0) {
 a = b;
 b = a % b;
 }
 return a;
}
```

Which line contains the bug? _____

Q4. Complete the recursive base case and recursive call.

```
static int gcd(int a, int b) {
 if (b == ______) return ______;
 return gcd(______, a % b);
}
```

Q5. What is the running time of Euclid's algorithm for gcd(a, b)?

- **A.** O(min(a, b))
- **B.** O(log min(a, b))
- **C.** O(a + b)
- **D.** O(1)

Q6. Trace Euclid's algorithm for (a, b) = (50, 12).

```
while (b != 0) {
 int r = a % b;
 a = b;
 b = r;
}
```

- a after iteration 1: _____
- b after iteration 1: _____
- b after iteration 2: _____
- final a: _____

Interview / exam questions

- Prove that gcd(a, b) = gcd(b, a mod b).
- What is the worst-case input for Euclid's algorithm and why?
- How do you compute the GCD of an array of n numbers? What is the total cost?
- How is the LCM related to the GCD, and how do you avoid overflow when computing it?
- What does the extended Euclidean algorithm compute and where is it used?

Watch & read

Video: GCD - Euclidean Algorithm (Method 1) — Neso Academy · 14:38. *Why gcd(a, b) = gcd(b, a mod b)*

Video: Euclidean Algorithm (GCD). Java — Aleksandr Azimut · 8:08. *Writing Euclid's algorithm in Java*

REVISION CARD (30–60 SECONDS)

- gcd(a, b) = gcd(b, a mod b); gcd(a, 0) = a.
- Loop: r = a % b; a = b; b = r; stop when b == 0, answer is a.
- Time O(log min(a, b)); naive is O(min(a, b)).
- lcm(a, b) = a / gcd(a, b) * b (divide first to avoid overflow).
- Worst case: consecutive Fibonacci numbers.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide pseudocode states the prerequisite "a > b". It is not needed: if a < b the first step swaps them (gcd(12, 50) → gcd(50, 12)).

5.2 Euler's Phi (Totient) Function

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 45 min · ★ on the minimum-viable path

Count the numbers up to n that share no factor with n — using prime factorisation instead of testing each one.

YOU WILL BE ABLE TO

- Define coprime numbers and Euler's φ(n) and compute φ for small n by hand.
- Derive φ(n) = n · ∏(1 − 1/p) from the idea "remove the multiples of each prime factor".
- Write the O(√n) Java method and explain every line, including the "leftover prime" step.
- Spot and explain the error in the lecture's worked example φ(60).

Before this lesson: Euclid's Algorithm (GCD) · Divisibility, GCD/LCM, Primes and Modular Arithmetic · Loops: for, while, do-while, break and continue

The idea

Two numbers are **coprime** when their only common divisor is 1 (gcd = 1). 8 and 15 are coprime; 8 and 12 are not (both divisible by 4). **Euler's totient** φ(n) counts how many numbers from 1 to n are coprime to n.

A number fails to be coprime to 12 exactly when it shares the factor 2 or the factor 3.

```
n = 12 → 1 2 3 4 5 6 7 8 9 10 11 12
coprime → ✓ · · · ✓ · ✓ · · · ✓ · φ(12) = 4 (1, 5, 7, 11)
```

So φ(n) is "n minus everything that shares a prime factor with n". Take n = 12 = 2² · 3:

- Half of the numbers are multiples of 2 → keep **1/2** of them (× (1 − 1/2)).
- A third are multiples of 3 → keep **2/3** of those (× (1 − 1/3)).

$$\varphi(12) = 12 \times (1 - 1/2) \times (1 - 1/3) = 12 \times 1/2 \times 2/3 = 4$$

WHY THIS WORKS

The formula φ(n) = n · ∏ (1 − 1/p) takes the product over the **distinct** primes p that divide n. Multiples of different primes are independent of each other (that is what the Chinese Remainder Theorem says), so the "survival fractions" multiply. A prime appearing 5 times in n counts only once — its multiples are the same numbers however many times it divides n.

Syntax

Java ideas used: `for` loops with a condition that involves multiplication (`p * p <= n`), the remainder operator `%`, and `n /= p` (shorthand for `n = n / p`). Use `long` so that `p * p` cannot overflow for large n.

A tiny example

The smallest correct program literally counts coprime numbers (using the Euclid lesson):

```
int count = 0;
for (int k = 1; k <= n; k++) {
 if (gcd(k, n) == 1) count++;
}
```

Guided example

The efficient program never tests individual numbers. It **finds the prime factors of n** by trial division and applies "× (1 − 1/p)" once for each. Trace n = 60 = $2^2 \cdot 3 \cdot 5$:

φ(60) = 16.

step	p	n before	p divides n?	n after	result
start	—	60	—	60	60
1	2	60	yes (strip 2,2)	15	60 − 60/2 = 30
2	3	15	yes (strip 3)	5	30 − 30/3 = 20
3	4	5	loop stops: 4·4 > 5	5	20
leftover	—	5 > 1	prime factor 5 remains	5	20 − 20/5 = 16

BruteForcePhi.java — Brute force φ(n) (simple solution)

```
import java.util.Scanner;

public class BruteForcePhi {
 static int gcd(int a, int b) {                                    1
 while (b != 0) {
 int r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 // phi(n) = how many numbers k in 1..n have gcd(k, n) = 1 ("coprime to n")
 static int phi(int n) {
 int count = 0;
 for (int k = 1; k <= n; k++) {                                    2
 if (gcd(k, n) == 1) {                                             3
 count++;
 }
 }
 return count;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 System.out.println("phi(" + n + ") = " + phi(n));
 }
}
```

Counts every k in 1..n with gcd(k, n) = 1. O(n log n).

Sample input

```
24
```

Line-by-line walkthrough

#	Where	What happens
1	line 4	Euclid's algorithm from the previous lesson, reused as a helper.
2	line 16	Test every number from 1 to n.
3	line 17	k is coprime to n exactly when the gcd is 1.

EulerPhi.java — φ(n) from prime factors (optimised solution)

```
1  import java.util.Scanner;
2
3  public class EulerPhi {
4   // phi(n) = n * (1 - 1/p1) * (1 - 1/p2) * ... over the DISTINCT primes p dividing n.
5   // We apply each factor as result -= result / p which stays in whole numbers.
6   static long phi(long n) {
7   long result = n;                                                    (1)
8   for (long p = 2; p * p <= n; p++) {                                 (2)
9   if (n % p == 0) { // p is a prime factor of n                       (3)
10  while (n % p == 0) { // strip every copy of p so we never test it again
11  n /= p;                                                             (4)
12  }
13  result -= result / p; // multiply result by (1 - 1/p)               (5)
14  }
15  }
16  if (n > 1) { // what is left is one prime factor larger than sqrt(original n)  (6)
17  result -= result / n;
18  }
19  return result;
20  }
21
22  public static void main(String[] args) {
23  Scanner sc = new Scanner(System.in);
24  long n = sc.nextLong();
25  System.out.println("phi(" + n + ") = " + phi(n));
26  }
27 }
```

Finds each distinct prime factor p ≤ √n and applies result × (1 − 1/p) in integer form. O(√n).

Sample input

```
60
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Start with "all n numbers survive" and remove the unwanted ones prime by prime.
2	line 8	A composite number has a prime factor ≤ its square root, so after this loop at most ONE prime factor is left over. Using `long` prevents p*p overflowing.
3	line 9	The first p that divides n is always prime (any smaller prime factor was already removed).
4	line 11	Strip every copy of p so that n shrinks and p is never counted twice.

#	Where	What happens
5	line 13	Multiply result by (1 – 1/p). `result / p` is exact because result is still divisible by p, so there is no rounding.
6	line 16	Whatever is left is one prime factor bigger than √(original n). It needs its own (1 – 1/p) factor.

EulerPhiTable.java — φ for every number up to N (demonstration)

```
1  public class EulerPhiTable {
2   // Computes phi(1..limit) for all numbers at once with a sieve-like pass.
3   static int[] phiUpTo(int limit) {
4   int[] phi = new int[limit + 1];
5   for (int i = 0; i <= limit; i++) {
6   phi[i] = i;                                                    (1)
7   }
8   for (int p = 2; p <= limit; p++) {
9   if (phi[p] == p) { // untouched so far => p is prime           (2)
10  for (int multiple = p; multiple <= limit; multiple += p) {
11  phi[multiple] -= phi[multiple] / p;                            (3)
12  }
13  }
14  }
15  return phi;
16  }
17
18  public static void main(String[] args) {
19  int[] phi = phiUpTo(20);
20  for (int n = 1; n <= 20; n++) {
21  System.out.println("phi(" + n + ") = " + phi[n]);
22  }
23  }
24 }
```

Same formula applied sieve-style: for each prime p, update all its multiples. O(N log log N).

Line-by-line walkthrough

#	Where	What happens
1	line 6	Every φ starts at i itself.
2	line 9	Nobody has reduced phi[p] yet, so p has no smaller prime factor: p is prime.
3	line 11	Apply (1 – 1/p) to every multiple of the prime p.

Dry run: watch it execute

φ(60): strip each prime factor of n and multiply result by (1 – 1/p).

Algorithm: Euler's φ (totient). **Input:** n = `60`.

Pseudocode

```
result = n;
for (p = 2; p * p <= n; p++) {
 if (n % p == 0) {
 while (n % p == 0) n /= p;
 result -= result / p;
 }
}
if (n > 1) result -= result / n;
return result;
```

Step 1 of 6. result starts as n = 60.

p = — · n = 60 · result = 60

Step 2 of 6. 2 divides n: strip every 2 (n becomes 15); result = 60 − 60/2 = 30.

p = 2 · n = 15 · result = 30

Step 3 of 6. 3 divides n: strip every 3 (n becomes 5); result = 30 − 30/3 = 20.

p = 3 · n = 5 · result = 20

Step 4 of 6. Loop ends: p·p = 16 > n = 5.

p = 4 · n = 5 · result = 20

Step 5 of 6. Leftover prime 5: result = 20 − 20/5 = 16.

p = 4 · n = 5 · result = 16

Step 6 of 6. φ(60) = 16.

p = 4 · n = 5 · result = 16

> **RESULT**
> φ(60) = 16

Complexity

Brute force: n gcd calls, each O(log n) → O(n log n).

Factorisation method: the loop variable p goes up to √n (n only gets smaller) and the divisions inside the loop add at most O(log n) in total → **O(√n) time, O(1) space**. For n ≈ 10^{12} that is about 10^6 steps instead of 10^{12}.

Table for 1..N: each prime p touches N/p multiples; the sum of N/p over primes is N · log log N.

n	Brute force steps	Factor method steps
10^6	10^6 gcds	≤ 1 000
10^{12}	10^{12} gcds (impossible)	≤ 10^6

Common mistakes

Using the formula with (p – 1) · p^(k–1) but also dividing again

Problem

```
// φ(60) the "slide" way: (2-1)*2^1 * (2-1)*2^2 * (3-1)*3^0 * (5-1)*5^0 * (5-1)*5^1 = 32
```

Why: The lecture example repeats the factors for 2 and 5. Each distinct prime contributes ONE factor (p − 1)·p^(k−1). The correct product is (2−1)·2^1 × (3−1)·3^0 × (5−1)·5^0 = 2 × 2 × 4 = 16.

Fix: // one factor per distinct prime phi = (2-1)*2* (3-1)*1* (5-1)*1; // = 16

Forgetting the leftover prime

Problem

```
for (int p = 2; p * p <= n; p++) { ... }
return result; // missing: if (n > 1) result -= result / n;
```

Why: For n = 14 the loop strips 2 and leaves n = 7. 7·7 > 7 ends the loop before 7 is handled, so φ(14) would come out as 7 instead of 6.

Fix: if (n > 1) result -= result / n;

*int overflow in p p**

Problem

```
for (int p = 2; p * p <= n; p++) // n up to 2·10^9
```

Why: When p reaches ~46 341, p * p exceeds Integer.MAX_VALUE and becomes negative, so the loop never ends or ends wrongly.

Fix: for (long p = 2; p * p <= n; p++)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · List the coprime numbers — *Beginner*

Read n. Print, in increasing order and separated by spaces, every k with 1 ≤ k ≤ n and gcd(k, n) = 1.

- 1 ≤ n ≤ 10^5

Example input	Expected output
12	1 5 7 11

Hint 1: You need a gcd helper — reuse Euclid. **Hint 2:** Build the output in a StringBuilder and print once.

Exercise 2 · Sum of totients — *Intermediate*

Read n and print φ(1) + φ(2) + ... + φ(n).

- 1 ≤ n ≤ 10^6

Example input	Expected output
10	32

Hint 1: Computing φ separately for each i with trial division is too slow in the worst case — use the table approach. **Hint 2:** Remember the answer can exceed int range for large n: use long for the sum.

Exercise 3 · Coprime pairs — *Challenge*

Count ordered pairs (a, b) with 1 ≤ a, b ≤ n and gcd(a, b) = 1.

- 1 ≤ n ≤ 10^6

Example input	Expected output
3	7

Hint 1: For a fixed larger value b ≥ 2, how many a < b are coprime to b? **Hint 2:** Each such unordered pair counts twice (a,b) and (b,a). Don't forget (1,1).

Quick check

Answers are in the answer key at the end of this part.

Q1. What is φ(17)?

- **A.** 8
- **B.** 15
- **C.** 16
- **D.** 17

Q2. What is φ(60)?

- **A.** 16
- **B.** 24
- **C.** 32
- **D.** 48

Q3. What does phi(36) return for the optimized method?

```
long result = 36, n = 36;
for (long p = 2; p * p <= n; p++) {
 if (n % p == 0) {
 while (n % p == 0) n /= p;
 result -= result / p;
 }
}
if (n > 1) result -= result / n;
System.out.println(result);
```

Your answer: _____

Q4. Complete the line that applies the factor (1 – 1/p).

```
if (n % p == 0) {
 while (n % p == 0) n /= p;
 result ______ result / p;
}
```

Q5. Why is `if (n > 1)` needed after the loop?

- **A.** To avoid dividing by zero
- **B.** A prime factor larger than √n can remain
- **C.** Because n is always 1 after the loop
- **D.** To handle n = 0

Q6. Match each n with φ(n).

Item	Your match
φ(1)	_____
φ(9)	_____
φ(10)	_____
φ(13)	_____

Choose from: `1`, `12`, `4`, `6`

Interview / exam questions

- What does Euler's totient count? Give φ(p) and φ(p^k).
- Prove or explain why φ(ab) = φ(a)φ(b) when gcd(a, b) = 1.
- How would you compute φ for all numbers up to 10^7?
- Where does φ appear? (Euler's theorem, RSA, modular inverses).

Watch & read

Video: Euler's Totient Function (Phi Function) — Neso Academy · 8:40. *Definition and meaning of φ(n)*

Video: Euler's Totient Function (Solved Examples) — Neso Academy · 12:53. *Computing φ(n) from prime factors*

REVISION CARD (30–60 SECONDS)

- φ(n) = count of k in 1..n with gcd(k, n) = 1.
- φ(n) = n · Π(1 − 1/p) over DISTINCT primes p | n.
- Code: for p ≤ √n strip p, result −= result/p; finally if n > 1: result −= result/n.
- O(√n) time, O(1) space. Prime: φ(p) = p − 1.
- φ(60) = 16 (not 32).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide computes φ(60) = 32. The correct value is 16: 60 × (1/2) × (2/3) × (4/5) = 16. The slide repeated factors for the primes 2 and 5. The Java code on the slide is correct and returns 16. • The slide's product formula φ(n) = $(p_1-1)(p_2-1)\ldots \cdot p_1\text{^}(k_1-1)\ldots$ is right, but its intermediate text for φ(60) lists the factors of 5 twice.

5.3 Chinese Remainder Theorem

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 45 min · complete path

Recover a number from its remainders modulo several pairwise-coprime numbers.

YOU WILL BE ABLE TO

- Read a system of congruences x ≡ r (mod m) and explain what it asks.
- Solve small systems by searching, and larger ones with the CRT formula.
- Compute a modular inverse and use it inside the formula.
- Explain why the answer is unique only modulo M = $m_1 \cdot m_2 \cdot \ldots$

Before this lesson: Euclid's Algorithm (GCD) · Divisibility, GCD/LCM, Primes and Modular Arithmetic

The idea

A general lines up soldiers in rows of 3 and 2 are left over; in rows of 5, 3 are left over; in rows of 7, 2 are left over. How many soldiers are there? The question "find x such that x % 3 = 2, x % 5 = 3, x % 7 = 2" is called a **system of congruences**, and it is written x ≡ 2 (mod 3), x ≡ 3 (mod 5), x ≡ 2 (mod 7).

Think of three clocks with 3, 5 and 7 positions. A counter x makes all three tick. The three clock positions together are different for every x between 0 and 104, and the pattern repeats every 3 × 5 × 7 = 105 steps. So **the triple of remainders identifies x uniquely, up to multiples of 105**. That is the theorem (it needs the moduli to be pairwise coprime — share no factors).

WHY THIS WORKS

To build the answer we make three "switches". For modulus 3 we need a number that is 1 mod 3 but a multiple of 5 and 7, i.e. a multiple of M_1 = 105/3 = 35 that is ≡ 1 (mod 3): 35 · inverse(35 mod 3) = 35 · 2 = 70. Multiply by the wanted remainder 2 and it is "on" for modulus 3 and invisible (≡ 0) for 5 and 7. Add the three switched-on parts and take mod 105.

```
x = 2·(35·2) + 3·(21·1) + 2·(15·1) = 140 + 63 + 30 = 233
233 mod 105 = 23 ← the smallest non-negative solution
check: 23 mod 3 = 2 ✓ 23 mod 5 = 3 ✓ 23 mod 7 = 2 ✓
```

Syntax

Java ideas: arrays for the moduli and remainders, nested loops with `break`, `long` arithmetic for products of moduli, and the remainder operator on possibly large numbers.

A tiny example

Brute force: try x = 0, 1, 2, ... until all three remainders match.

```java
int x = 0;
while (!(x % 3 == 2 && x % 5 == 3 && x % 7 == 2)) {
 x++;
}
System.out.println(x); // 23
```

Guided example

The slide solution is exactly this search generalised to arrays. It is perfectly fine while M is small, but needs up to M iterations. The formula needs only a few multiplications and one modular inverse per modulus.

CrtBruteForce.java — CRT by search (simple solution)

```
import java.util.Scanner;

public class CrtBruteForce {
 // Slide approach: try x = 1, 2, 3, ... until x % m[i] == r[i] for every i.
 static long solve(int[] mods, int[] rems) {
 long limit = 1;
 for (int m : mods) {
 limit *= m; // the answer repeats every M = product of moduli                    (1)
 }
 for (long x = 0; x < limit; x++) {
 boolean ok = true;
 for (int i = 0; i < mods.length; i++) {
 if (x % mods[i] != rems[i]) {                                                    (2)
 ok = false;
 break;                                                                           (3)
 }
 }
 if (ok) {
 return x;
 }
 }
 return -1; // no solution (moduli not pairwise coprime / inconsistent)
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] mods = new int[n];
 int[] rems = new int[n];
 for (int i = 0; i < n; i++) mods[i] = sc.nextInt();
 for (int i = 0; i < n; i++) rems[i] = sc.nextInt();
 System.out.println(solve(mods, rems));
 }
}
```

Tries x = 0 … M−1 until x % m[i] == r[i] for every i. At most M iterations.

Sample input

```
3
3 5 7
2 3 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	The answers repeat every M = $m_1 \cdot m_2 \cdot \ldots$, so searching below M is enough; if nothing is found there is no solution.
2	line 13	One failed congruence rejects x immediately.
3	line 15	Leave the inner loop early — no need to test the remaining moduli.

CrtFormula.java — CRT by formula (optimised solution)

```
1  import java.util.Scanner;
2
3  public class CrtFormula {
4   // modular inverse of a modulo m by trying 1..m-1 (fine for small moduli; see Euclid
        lesson for the fast way)
5   static long inverse(long a, long m) {
6   a %= m;
7   for (long x = 1; x < m; x++) {
8   if (a * x % m == 1) {
9   return x;
10  }
11  }
12  throw new IllegalArgumentException("no inverse: moduli are not coprime");
13  }
14
15  // x = sum( r_i * M_i * inv(M_i mod m_i) ) mod M where M_i = M / m_i
16  static long solve(int[] mods, int[] rems) {
17  long M = 1;
18  for (int m : mods) {
19  M *= m;
20  }
21  long x = 0;
22  for (int i = 0; i < mods.length; i++) {
23  long Mi = M / mods[i];                                                    (1)
24  long b = inverse(Mi, mods[i]);                                           (2)
25  x += rems[i] * Mi % M * b % M;                                           (3)
26  x %= M;
27  }
28  return x;
29  }
30
31  public static void main(String[] args) {
32  Scanner sc = new Scanner(System.in);
33  int n = sc.nextInt();
34  int[] mods = new int[n];
35  int[] rems = new int[n];
36  for (int i = 0; i < n; i++) mods[i] = sc.nextInt();
37  for (int i = 0; i < n; i++) rems[i] = sc.nextInt();
38  System.out.println(solve(mods, rems));
39  }
40 }
```

x = Σ r_i · M_i · inverse(M_i mod m_i), reduced mod M.

Sample input

```
3
3 5 7
2 3 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 23	M_i is the product of all the OTHER moduli. It is ≡ 0 modulo each of them and invertible modulo m_i.
2	line 24	b·M_i ≡ 1 (mod m_i). Here the inverse is found by trying 1..m−1; the Euclid lesson gives a faster way.
3	line 25	Scale the "switch" by the required remainder. `% M` after each multiplication keeps numbers small.

Dry run: watch it execute

Sun Tzu's classic: x ≡ 2 (mod 3), 3 (mod 5), 2 (mod 7) → 23.

Algorithm: Chinese remainder theorem by sieving. **Input:** pairs = `2:3,3:5,2:7`.

Pseudocode

```
x = r1; step = m1
for each further (r, m):
 while x % m != r: x += step
 step = lcm(step, m)
x is the smallest solution
```

Step 1 of 5. x ≡ 2 (mod 3): the candidates are 2, 5, 8, ... (step 3).

x = 2 · step = 3

Step 2 of 5. Next condition x ≡ 3 (mod 5). Candidates 2, 5, 8: 8 % 5 = 3 ✓.

x = 8 · step = 3 · mod = 5

candidates for mod 5

2	5	8
0	1	2

Step 3 of 5. All later solutions differ by the lcm(3, 5) = 15, so the step grows to 15.

x = 8 · step = 15

Step 4 of 5. Next condition x ≡ 2 (mod 7). Candidates 8, 23: 23 % 7 = 2 ✓.

x = 23 · step = 15 · mod = 7

candidates for mod 7

8	23
0	1

Step 5 of 5. All later solutions differ by the lcm(15, 7) = 105, so the step grows to 105.

x = 23 · step = 105

> **RESULT**
> x = 23 (repeats every 105)

Complexity

Search: O(M · k) where M = ∏ m_i and k is the number of congruences — exponential in the number of moduli.

Formula: for each of the k congruences one inverse (O(log m) with extended Euclid, O(m) with the loop used here) → O(k · log M) with Euclid. Space O(k).

Common mistakes

Moduli that are not coprime

Problem

```
// x % 6 == 2 and x % 4 == 1 → no x exists, but the formula silently returns garbage
```

Why: The CRT formula needs gcd(m_i, m_j) = 1. 6 and 4 share the factor 2, so x must be even (from mod 6) and odd (from mod 4) — impossible. Without coprimality the inverse does not exist.

Fix: // check gcd(mi, mj) == 1 first, or use the general merge from the challenge exercise

Reading the answer as THE solution

Problem

```
// "x = 233" for the slide example
```

Why: Any x + 105k is a solution. The conventional answer is the smallest non-negative one: 233 mod 105 = 23.

Fix: x = ((x % M) + M) % M;

Overflow in the products

Problem

```
x += rems[i] * Mi * b; // can exceed long for big moduli
```

Why: Mi alone may be near 10^{12}, and times b and remainder may overflow 64 bits.

Fix: x = (x + rems[i] % M * Mi % M * b) % M;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Two congruences — *Beginner*

Given coprime moduli a and b and remainders r1, r2 (read in the order `a r1 b r2`), print the smallest non-negative x with x % a = r1 and x % b = r2.

- 2 ≤ a, b ≤ 1000, gcd(a, b) = 1
- 0 ≤ r1 < a, 0 ≤ r2 < b

Example input	Expected output
3 2 5 3	8

Hint 1: The answer is below a·b. **Hint 2:** Loop x from 0 and test both conditions.

Exercise 2 · CRT with the formula — *Intermediate*

Read n, then n lines `m r`. All moduli are pairwise coprime. Print the smallest non-negative x with x ≡ r (mod m) for all lines.

- 1 ≤ n ≤ 5
- 2 ≤ m ≤ 1000
- product of moduli ≤ 10^{12}

Example input	Expected output
3 ⏎ 3 2 ⏎ 5 3 ⏎ 7 2	23

Hint 1: M_i = M / m_i. **Hint 2:** Inverse of M_i modulo m_i: extended Euclid, or try 1..m_i−1. **Hint 3:** Reduce mod M after every multiplication.

Exercise 3 · Non-coprime moduli — *Challenge*

Read `r1 m1 r2 m2` (the moduli may share factors). Print the smallest non-negative x with x ≡ r1 (mod m1) and x ≡ r2 (mod m2), or -1 if none exists.

- 1 ≤ m1, m2 ≤ 10^6
- 0 ≤ r1 < m1, 0 ≤ r2 < m2

Example input	Expected output
2 6 4 10	14
2 6 1 4	−1

Hint 1: Only numbers r1, r1+m1, r1+2·m1, … satisfy the first congruence. **Hint 2:** Their remainders modulo m2 repeat after at most m2 steps — try that many.

Quick check

Answers are in the answer key at the end of this part.

Q1. The smallest non-negative x with x ≡ 2 (mod 3), x ≡ 3 (mod 5), x ≡ 2 (mod 7) is…

- **A.** 23
- **B.** 233
- **C.** 105
- **D.** 35

Q2. Which condition do the moduli need for the CRT formula?

- **A.** All equal
- **B.** All prime
- **C.** Pairwise coprime
- **D.** Sum to 100

Q3. What is printed?

```
int x = 0;
while (!(x % 4 == 1 && x % 5 == 2)) x++;
System.out.println(x);
```

Your answer: _____

Q4. In the formula, what is M_i?

- **A.** m_i squared
- **B.** M / m_i
- **C.** M − m_i
- **D.** r_i · m_i

Q5. The system x ≡ 1 (mod 6), x ≡ 2 (mod 4) has…

- **A.** x = 13
- **B.** x = 5
- **C.** no solution
- **D.** infinitely many but none below 24

Interview / exam questions

- State the CRT. When is the solution unique?
- Compute the modular inverse of 35 modulo 3 by hand.
- How would you solve a system whose moduli are not coprime?
- Where does CRT appear in cryptography (RSA speed-up)?

Watch & read

Video: The Chinese Remainder Theorem (Solved Example 1) — Neso Academy · 14:22. *Solving simultaneous congruences*

Video: The Chinese Remainder Theorem (Solved Example 2) — Neso Academy · 12:06. *A worked CRT example*

REVISION CARD (30–60 SECONDS)

- x ≡ r_i (mod m_i), m_i pairwise coprime → unique x modulo M = Πm_i.
- x = Σ r_i·M_i·inv(M_i mod m_i), M_i = M / m_i, then x mod M.
- Slide example: x = 233 ≡ 23 (mod 105).
- Not coprime? Check consistency (gcd) first.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide gives the answer as "x ≡ 233 (mod 105)". That is a correct congruence but the canonical answer is the reduced value 23. • The slide's Java program has a compile error (`for(int i=0; i<n; i++)` uses n, which is never declared — it should be `size`) and its brute-force loop never terminates when no solution exists. The versions in this lesson fix both.

ASSUMPTIONS MADE

• The slide's program takes the divisors first and then the remainders; the lesson programs use the same input order (`n`, then moduli, then remainders).

5.4 Combinations (nCr)

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 40 min · complete path

Count the ways to choose r items from n, and list them — without overflowing on big factorials.

YOU WILL BE ABLE TO

- Explain the difference between a permutation (order matters) and a combination (order does not).
- Compute nCr with the factorial formula and see why it overflows for n ≥ 13.
- Compute nCr safely with a multiply-then-divide loop and with Pascal's triangle.
- Generate all combinations of an array with recursion and backtracking.

Before this lesson: Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading · Recursion: Base Case, Recursive Case and the Call Stack

The idea

Choosing 2 friends from {Ann, Ben, Cy} for a pair project: {Ann, Ben}, {Ann, Cy}, {Ben, Cy} — **3** ways. {Ann, Ben} and {Ben, Ann} are the same team, so **order does not matter**. That count is written **C(3, 2)** or ${}^{3}C_{2}$ ("3 choose 2").

If order did matter (president and vice-president), there would be 3 × 2 = 6 arrangements (a **permutation**). Every team of 2 was counted 2! = 2 times, so the number of teams is 6 / 2 = 3. In general:

```
C(n, r) = n! / (r! · (n − r)!)  (what the lecture slide uses)
```

WHY THIS WORKS

Another way to see the formula is Pascal's rule: C(n, r) = C(n−1, r−1) + C(n−1, r). Take one special item. Either it is in the team (choose r−1 more from the other n−1) or it is not (choose all r from the other n−1). That recurrence builds Pascal's triangle and never needs a factorial.

Syntax

```
long result = 1;
for (int i = 1; i <= r; i++) result = result * (n - r + i) / i; // multiply THEN divide: stays
    whole

// import java.math.BigInteger; // when long overflows (n > ~60)
BigInteger f = BigInteger.ONE;
f = f.multiply(BigInteger.valueOf(k)).divide(BigInteger.valueOf(i));
long safe = Math.multiplyExact(a, b); // throws ArithmeticException instead of wrapping
```

A tiny example

```
// 5C2 = 5! / (2! * 3!) = 120 / (2 * 6) = 10
```

Guided example

The lecture program puts numbers in an `ArrayList` and uses only its **size** n = 5 — the list contents (9, 12, 19, 61, 19) play no role in counting. `fact` is written with a `while` loop. It prints 10.

> **WATCH OUT**
> `int` can only hold up to 2 147 483 647. 13! = 6 227 020 800 already overflows, so the factorial formula silently gives wrong answers for n ≥ 13 — even when the final answer is small.

CombinationNaive.java — nCr with factorials (simple solution)

```
1  import java.util.Scanner;
2
3  public class CombinationNaive {
4   // The slide's formula nCr = n! / (r! * (n-r)!) -- correct, but n! overflows int for n
        > 12.
5   static int fact(int k) {                                                    (1)
6   int f = 1;
7   for (int j = 2; j <= k; j++) {
8   f *= j;
9   }
10  return f;
11  }
12
13  public static void main(String[] args) {
14  Scanner sc = new Scanner(System.in);
15  int n = sc.nextInt();
16  int r = sc.nextInt();
17  System.out.println(n + "C" + r + " = " + fact(n) / (fact(r) * fact(n - r)));   (2)
18  }
19 }
```

Direct translation of n! / (r! (n−r)!). Correct for n ≤ 12, breaks for larger n because of overflow.

Sample input

```
5 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 5	Multiplies 2 · 3 · … · k. The starting value 1 also handles 0! = 1.
2	line 17	Integer division is exact because the true quotient is a whole number — if nothing overflowed.

Combination.java — nCr multiplicatively (optimised solution)

```java
import java.util.Scanner;

public class Combination {
 // nCr without ever building n!: multiply and divide one step at a time.
 // After step i the running value is exactly C(n - r + i, i), always a whole number.
 static long nCr(int n, int r) {
 if (r < 0 || r > n) {
 return 0;
 }
 r = Math.min(r, n - r); // C(n, r) == C(n, n - r); fewer steps (1)
 long result = 1;
 for (int i = 1; i <= r; i++) {
 result = result * (n - r + i) / i; (2)
 }
 return result;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 System.out.println(n + "C" + r + " = " + nCr(n, r));
 }
}
```

Builds the answer in r steps: after step i the value is C(n−r+i, i). Uses long and the symmetry C(n, r) = C(n, n−r).

Sample input

```
30 15
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	C(100, 98) = C(100, 2): choose the smaller side so the loop is short.
2	line 13	Multiply FIRST then divide. The product of i consecutive numbers is always divisible by i!, so `/ i` is exact at every step.

PascalTriangle.java — Pascal's triangle (demonstration)

```
1  public class PascalTriangle {
2   // C(n, r) = C(n-1, r-1) + C(n-1, r) -- pick the last item or skip it.
3   public static void main(String[] args) {
4   int rows = 6;
5   long[][] c = new long[rows][rows];
6   for (int n = 0; n < rows; n++) {
7   c[n][0] = 1;                                                    (1)
8   for (int r = 1; r <= n; r++) {
9   c[n][r] = c[n - 1][r - 1] + (r <= n - 1 ? c[n - 1][r] : 0);     (2)
10  }
11  }
12  for (int n = 0; n < rows; n++) {
13  StringBuilder line = new StringBuilder();
14  for (int r = 0; r <= n; r++) {
15  line.append(c[n][r]).append(' ');
16  }
17  System.out.println(line.toString().trim());
18  }
19  }
20 }
```

Each entry is the sum of the two entries above it. Only addition, so no overflow until the numbers themselves are huge.

Line-by-line walkthrough

#	Where	What happens
1	line 7	There is one way to choose nothing.
2	line 9	Pascal's rule: include the last item, or skip it.

CombinationsList.java — List all combinations (backtracking) (demonstration)

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class CombinationsList {
5   // Prints every way to choose r items from the array (the "harder" part mentioned in
        the lecture).
6   static void combos(int[] items, int r, int start, List<Integer> current) {
7   if (current.size() == r) {                                      (1)
8   System.out.println(current);
9   return;
10  }
11  for (int i = start; i < items.length; i++) {
12  current.add(items[i]); // choose items[i]                       (2)
13  combos(items, r, i + 1, current); // go on with the items to its right   (3)
14  current.remove(current.size() - 1); // un-choose (backtrack)    (4)
15  }
16  }
17
18  public static void main(String[] args) {
19  combos(new int[] {9, 12, 19, 61}, 2, 0, new ArrayList<>());
20  }
21 }
```

Choose an item, recurse on the items to its right, then un-choose it and try the next item.

Line-by-line walkthrough

#	Where	What happens
1	line 7	Base case: we have exactly r items — print this combination.
2	line 12	Choose.
3	line 13	Explore: only items to the RIGHT can be added, which prevents duplicates like [12, 9].
4	line 14	Un-choose (backtrack) so the next loop iteration starts from a clean selection.

Dry run: watch it execute

Pascal's triangle and the multiplicative formula agree.

Algorithm: nCr from Pascal's triangle and from the formula. **Input:** n = `6`, r = `2`.

Pseudocode

```
Pascal: C(i, j) = C(i−1, j−1) + C(i−1, j)
multiplicative:
r = min(r, n − r); result = 1
for i = 1..r: result = result ∗ (n − r + i) / i
```

Step 1 of 11. Row 0: the edges are always 1.

row = 0

1						

Step 2 of 11. Row 1: the edges are always 1.

row = 1

1						
1	1					

Step 3 of 11. Row 2: each inner entry is the sum of the two entries above it.

row = 2

1						
1	1					
1	2	1				

Step 4 of 11. Row 3: each inner entry is the sum of the two entries above it.

row = 3

1						
1	1					
1	2	1				
1	3	3	1			

Step 5 of 11. Row 4: each inner entry is the sum of the two entries above it.

row = 4

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		

Step 6 of 11. Row 5: each inner entry is the sum of the two entries above it.

row = 5

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		
1	5	10	10	5	1	

Step 7 of 11. Row 6: each inner entry is the sum of the two entries above it.

row = 6

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		
1	5	10	10	5	1	
1	6	15	20	15	6	1

Step 8 of 11. C(6, 2) is the entry in row 6, column 2 = 15.

C(6,2) = 15

1						
1	1					
1	2	1				
1	3	3	1			
1	4	6	4	1		
1	5	10	10	5	1	
1	6	15	20	15	6	1

Step 9 of 11. Formula version: use symmetry C(6, 2) = C(6, 4) and take r = 2 (fewer steps). result starts at 1.

r = 2 · result = 1

Step 10 of 11. i = 1: result = result × (n − r + i) / i = × 5 / 1 → 5. It is always a whole number, so integer division is exact.

i = 1 · result = 5

Step 11 of 11. i = 2: result = result × (n − r + i) / i = × 6 / 2 → 15. It is always a whole number, so integer division is exact.

i = 2 · result = 15

> **RESULT**
> 6C2 = 15

Complexity

Factorial formula: O(n) multiplications, O(1) space — but incorrect for n ≥ 13 in `int` (n ≥ 21 in `long`).

Multiplicative loop: O(min(r, n−r)) time, O(1) space. Valid whenever the *result* fits in the type.

Pascal: O(n · r) time, O(n · r) space (can be O(r) with a single row).

Listing all combinations: there are C(n, r) of them, each costing O(r) to print → O(r · C(n, r)). The count itself explodes — C(40, 20) ≈ 1.4 × 10^{11}.

Common mistakes

Overflow in the factorial

Problem

```
int c = fact(20) / (fact(2) * fact(18));
```

Why: 20! overflows int (and even long is borderline). The result printed is nonsense, with no error message.

Fix: long result = 1; for (int i = 1; i <= r; i++) result = result * (n - r + i) / i;

Divide before multiply

Problem

```
result = result / i * (n - r + i);
```

Why: Integer division truncates: 4/3 becomes 1 and the answer is wrong.

Fix: result = result * (n - r + i) / i; // multiply first

Treating order as important

Problem

```
// printing [9,12] and [12,9] as two different combinations
```

Why: A combination is a set. Starting the recursive loop at i + 1 (not 0) prevents revisiting earlier items.

Fix: for (int i = start; i < items.length; i++) ...

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · nCr as a number — *Beginner*

Read n and r (0 ≤ r ≤ n ≤ 30). Print C(n, r).

Example input	Expected output
5 2	10

Hint 1: Multiply first, then divide, one step at a time. **Hint 2:** Use long.

Exercise 2 · nCr modulo 1 000 000 007 — *Intermediate*

Read n and r (0 ≤ r ≤ n ≤ 1000). Print C(n, r) mod 1 000 000 007.

Example input	Expected output
10 3	120

Hint 1: The true value is far too large for long — use Pascal's rule and take % MOD after each addition.

Exercise 3 · List all combinations — *Challenge*

Read n and r (1 ≤ r ≤ n ≤ 10). Print every r-element subset of {1..n}, one per line, elements in increasing order, subsets in lexicographic order.

Example input	Expected output
4 2	1 2 ⏎ 1 3 ⏎ 1 4 ⏎ 2 3 ⏎ 2 4 ⏎ 3 4

Hint 1: Recursion: choose(start, count). **Hint 2:** Only pick values larger than the previous one.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is C(6, 2)?

- **A.** 12
- **B.** 15
- **C.** 30
- **D.** 36

Q2. How many ways to arrange (order matters) 2 of 6 items?

- **A.** 12
- **B.** 15
- **C.** 30
- **D.** 36

Q3. What is printed?

```
long result = 1;
int n = 6, r = 2;
for (int i = 1; i <= r; i++)
 result = result * (n - r + i) / i;
System.out.println(result);
```

Your answer: _____

Q4. Why does `fact(15) / (fact(2) * fact(13))` print a wrong answer in int arithmetic?

- **A.** Division is wrong
- **B.** fact(15) overflows int
- **C.** r is too small
- **D.** nCr is undefined for n=15

Q5. C(n, r) equals...

- **A.** C(n, n−r)
- **B.** C(n, r+1)
- **C.** C(n−1, r)
- **D.** n · r

Interview / exam questions

- Difference between nCr and nPr?
- Compute C(1000, 3) without overflow.
- Why is Pascal's rule true?
- How many subsets does an n-element set have? ($2^n = \Sigma\ C(n, r)$)

Watch & read

Video: The Combination Formula and Why it Works | Combinatorics, Combinations and Permutations — Wrath of Math · 8:14. *The nCr formula and its computation*

Video: Pascal Triangle | Finding nCr in minimal time — take U forward · 26:45. *Pascal's triangle and nCr by DP*

REVISION CARD (30–60 SECONDS)

- C(n,r) = n! / (r!(n−r)!)
- C(n,r) = C(n,n−r); C(n,0) = C(n,n) = 1
- Loop: result = result*(n−r+i)/i for i=1..r
- Pascal: C(n,r) = C(n−1,r−1) + C(n−1,r)
- Overflow: 13! > int, 21! > long

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide places an ArrayList of 5 numbers into the program, but only list.size() is ever used, so the "9, 12, 19, 61, 19" values have no effect on the result (10). • The slide's factorial uses int and would overflow for n ≥ 13 — fine for the sample, wrong in general.

5.5 Karatsuba's Multiplication

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · complete path

Multiply two big numbers with three recursive multiplications instead of four — a first taste of divide and conquer.

YOU WILL BE ABLE TO

- Explain school-book multiplication and its O(n²) digit multiplications.
- Derive the 3-multiplication trick (a+b)(c+d) − ac − bd = ad + bc.
- Implement Karatsuba for long and for BigInteger and test it against *.
- Solve the recurrence T(n) = 3T(n/2) + O(n) = O(n^1.585).

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Algorithm Design Paradigms · Euclid's Algorithm (GCD)

The idea

Multiply 1234 × 5678 by splitting each number into two halves: 1234 = **12**·100 + **34** and 5678 = **56**·100 + **78**. Call the halves a = 12, b = 34, c = 56, d = 78.

$$(a \cdot 100 + b)(c \cdot 100 + d) = ac \cdot 100^2 + (ad + bc) \cdot 100 + bd$$

That needs **four** smaller multiplications: ac, ad, bc, bd. Karatsuba noticed that the middle term can be obtained from ONE extra multiplication that we already half-have:

$$(a + b)(c + d) = ac + ad + bc + bd \Rightarrow ad + bc = (a+b)(c+d) - ac - bd$$

WHY THIS WORKS

We already compute ac and bd for the outer terms. One more product, (a+b)(c+d), gives the middle term after two subtractions. Subtractions are cheap (linear time), multiplications are expensive, so trading one multiplication for a few additions makes the recursion branch 3 ways instead of 4. Over many levels that is the difference between n² and n^1.585.

```
1234 × 5678 a=12 b=34 c=56 d=78
ac = 12 × 56 = 672
bd = 34 × 78 = 2652
(a+b)(c+d)= 46 × 134 = 6164
ad + bc = 6164 − 672 − 2652 = 2840
result = 672·10000 + 2840·100 + 2652 = 7 006 652 ✓
```

Syntax

Java ideas: recursion with a base case, `Long.toString(x).length()` to count digits, integer division `/` and remainder `%` to split a number, and `BigInteger` with `shiftLeft/shiftRight` for the bit-splitting version.

A tiny example

```java
// split 1234 into halves with p = 100
long a = 1234 / 100; // 12
long b = 1234 % 100; // 34
```

Guided example

Karatsuba.java — Karatsuba (decimal split) (simple solution)

```java
import java.util.Scanner;

public class Karatsuba {
 static int baseCases = 0; // how many single-digit multiplications we performed

 static long pow10(int k) {
 long p = 1;
 for (int i = 0; i < k; i++) {
 p *= 10;
 }
 return p;
 }

 // Multiplies two non-negative numbers using 3 recursive multiplications instead of 4.
 static long multiply(long x, long y) {
 if (x < 10 || y < 10) { // base case: a one-digit factor => plain multiplication (1)
 baseCases++;
 return x * y;
 }
 int n = Math.max(Long.toString(x).length(), Long.toString(y).length());
 int half = n / 2; // number of digits in the LOW part (2)
 long p = pow10(half);

 long a = x / p, b = x % p; // x = a * 10^half + b (3)
 long c = y / p, d = y % p; // y = c * 10^half + d

 long ac = multiply(a, c);
 long bd = multiply(b, d);
 long abcd = multiply(a + b, c + d); // (a+b)(c+d) = ac + ad + bc + bd (4)
 long adPlusBc = abcd - ac - bd; // the middle term without a 4th multiplication (5)

 return ac * p * p + adPlusBc * p + bd; (6)
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long x = sc.nextLong();
 long y = sc.nextLong();
 long result = multiply(x, y);
 System.out.println(x + " x " + y + " = " + result);
 System.out.println("check with *: " + (x * y) + " -> " + (result == x * y ? "match" :
 "MISMATCH"));
 System.out.println("single-digit multiplications used: " + baseCases);
 }
}
```

Splits both numbers at half their digit count and recurses on three products.

Sample input

```
1234 5678
```

Line-by-line walkthrough

#	Where	What happens
1	line 16	Base case: if either factor has one digit, ordinary multiplication is already cheap.
2	line 21	The LOW part gets `half` digits; the high part gets the rest. This differs from the slide (which used (n+1)/2 with Math.pow) but both are valid.
3	line 24	x = a·10^half + b, so division gives the high part and remainder gives the low part.
4	line 29	The single extra multiplication that replaces both ad and bc.
5	line 30	Remove the terms we already know. What remains is exactly ad + bc.
6	line 32	Shift the three parts into place and add them (shifting = multiplying by powers of 10).

KaratsubaBig.java — Karatsuba (bit split, BigInteger) (optimised solution)

```
1  import java.math.BigInteger;
2  import java.util.Random;
3
4  public class KaratsubaBig {
5   // Karatsuba on BigInteger, splitting by BITS (shifts are cheaper than powers of 10).
6   static BigInteger multiply(BigInteger x, BigInteger y) {
7   int n = Math.max(x.bitLength(), y.bitLength());
8   if (n <= 32) { // small enough: use the machine multiply                    (1)
9   return x.multiply(y);
10  }
11  int half = n / 2;
12  BigInteger a = x.shiftRight(half); // high part                            (2)
13  BigInteger b = x.subtract(a.shiftLeft(half)); // low part
14  BigInteger c = y.shiftRight(half);
15  BigInteger d = y.subtract(c.shiftLeft(half));
16
17  BigInteger ac = multiply(a, c);
18  BigInteger bd = multiply(b, d);
19  BigInteger mid = multiply(a.add(b), c.add(d)).subtract(ac).subtract(bd);   (3)
20
21  return ac.shiftLeft(2 * half).add(mid.shiftLeft(half)).add(bd);           (4)
22  }
23
24  public static void main(String[] args) {
25  Random rnd = new Random(42);
26  int ok = 0;
27  for (int t = 0; t < 200; t++) {
28  BigInteger x = new BigInteger(200 + rnd.nextInt(300), rnd);
29  BigInteger y = new BigInteger(200 + rnd.nextInt(300), rnd);
30  if (multiply(x, y).equals(x.multiply(y))) {
31  ok++;
32  }
33  }
34  System.out.println(ok + " of 200 random 200–500 bit products match BigInteger.multiply")
        ;
35  BigInteger big = new BigInteger("123456789012345678901234567890");
36  System.out.println(multiply(big, big));
37  }
38 }
```

Same idea using shifts on BigInteger; handles numbers of hundreds of digits and is self-checked against BigInteger.multiply.

Line-by-line walkthrough

#	Where	What happens
1	line 8	Small enough for the hardware: use the built-in multiply instead of recursing further.
2	line 12	Shifting right by `half` bits is x / 2^half — the high part.
3	line 19	The same three-product identity, with BigInteger operations.
4	line 21	Multiplying by 2^(2·half) is a left shift — much cheaper than a decimal power.

For 1234 × 5678 the program reports **11** single-digit multiplications. The school method needs 4 × 4 = 16 digit multiplications — and the gap widens fast with the number of digits.

Dry run: watch it execute

Three recursive multiplications per level instead of four.

Algorithm: Karatsuba multiplication. **Input:** x = `1234`, y = `5678`.

Pseudocode

```
multiply(x, y):
 if x < 10 or y < 10: return x * y
 p = 10^(n/2); x = a·p + b; y = c·p + d
 ac = multiply(a, c); bd = multiply(b, d)
 abcd = multiply(a + b, c + d)
 return ac·p² + (abcd − ac − bd)·p + bd
```

The full trace has 16 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 16. base case: 1 × 5 = 5 (one factor has a single digit).

depth = 2 · x = 1 · y = 5 · result = 5 · base multiplications = 1

Step 2 of 16. base case: 2 × 6 = 12 (one factor has a single digit).

depth = 2 · x = 2 · y = 6 · result = 12 · base multiplications = 2

Step 3 of 16. base case: 3 × 11 = 33 (one factor has a single digit).

depth = 2 · x = 3 · y = 11 · result = 33 · base multiplications = 3

Step 4 of 16. 12 × 56: split at 10^1 → a=1, b=2, c=5, d=6. ac=5, bd=12, (a+b)(c+d)=33, middle = 33 − 5 − 12 = 16. Result 5·100 + 16·10 + 12 = 672.

depth = 1 · ac = 5 · bd = 12 · a·d + b·c = 16 · result = 672

Step 6 of 16. base case: 4 × 8 = 32 (one factor has a single digit).

depth = 2 · x = 4 · y = 8 · result = 32 · base multiplications = 5

Step 7 of 16. base case: 7 × 15 = 105 (one factor has a single digit).

depth = 2 · x = 7 · y = 15 · result = 105 · base multiplications = 6

Step 8 of 16. 34 × 78: split at 10^1 → a=3, b=4, c=7, d=8. ac=21, bd=32, (a+b)(c+d)=105, middle = 105 − 21 − 32 = 52. Result 21·100 + 52·10 + 32 = 2652.

depth = 1 · ac = 21 · bd = 32 · a·d + b·c = 52 · result = 2652

Step 9 of 16. base case: 4 × 13 = 52 (one factor has a single digit).

depth = 2 · x = 4 · y = 13 · result = 52 · base multiplications = 7

Step 10 of 16. base case: 6 × 4 = 24 (one factor has a single digit).

depth = 2 · x = 6 · y = 4 · result = 24 · base multiplications = 8

Step 11 of 16. base case: 1 × 1 = 1 (one factor has a single digit).

depth = 3 · x = 1 · y = 1 · result = 1 · base multiplications = 9

Step 13 of 16. base case: 1 × 8 = 8 (one factor has a single digit).

depth = 3 · x = 1 · y = 8 · result = 8 · base multiplications = 11

Step 14 of 16. 10 × 17: split at 10^1 → a=1, b=0, c=1, d=7. ac=1, bd=0, (a+b)(c+d)=8, middle = 8 − 1 − 0 = 7. Result 1·100 + 7·10 + 0 = 170.

depth = 2 · ac = 1 · bd = 0 · a·d + b·c = 7 · result = 170

Step 15 of 16. 46 × 134: split at 10^1 → a=4, b=6, c=13, d=4. ac=52, bd=24, (a+b)(c+d)=170, middle = 170 − 52 − 24 = 94. Result 52·100 + 94·10 + 24 = 6164.

depth = 1 · ac = 52 · bd = 24 · a·d + b·c = 94 · result = 6164

Step 16 of 16. 1234 × 5678: split at 10^2 → a=12, b=34, c=56, d=78. ac=672, bd=2652, (a+b)(c+d)=6164, middle = 6164 − 672 − 2652 = 2840. Result 672·10000 + 2840·100 + 2652 = 7006652.

depth = 0 · ac = 672 · bd = 2652 · a·d + b·c = 2840 · result = 7006652

RESULT
1234 × 5678 = 7006652 (matches *), 11 single-digit multiplications

Complexity

Let T(n) be the cost for n-digit numbers. We make **3** recursive calls on numbers about half as long, plus linear work for splitting, adding and subtracting:

$$T(n) = 3 \cdot T(n/2) + O(n) \Rightarrow T(n) = O(n^{(\log_2 3)}) \approx O(n^{1.585})$$

School-book multiplication satisfies $T(n) = 4 \cdot T(n/2) + O(n) = O(n^2)$. For 1000-digit numbers that is about 10^6 vs $1000^{1.585} \approx 5.6 \times 10^4$ digit operations.

Space: the recursion depth is $\log_2 n$, but every level allocates numbers of size O(n), so O(n) extra space overall.

digits n	school n^2	Karatsuba n^1.585	speed-up
10	100	38	≈ 2.6×
100	10 000	1 479	≈ 6.8×
1 000	1 000 000	56 000	≈ 18×
10 000	100 000 000	2 200 000	≈ 45×

NOTE
For small inputs ordinary multiplication is faster because Karatsuba has overhead. Real libraries switch to schoolbook below a threshold (BigInteger uses 80 ints) — exactly what the base case does.

Common mistakes

Using Math.pow for powers of 10

Problem

```
long p = (long) Math.pow(10, half);
```

Why: Math.pow returns a double. Beyond 2^{53} it cannot represent every integer, so results can silently be off by one.

Fix: long p = 1; for (int i = 0; i < half; i++) p *= 10;

Declaring a variable named ad+bc

Problem

```
long ad+bc = multiply(a + b, c + d) − ac − bd; // the slide
```

Why: Identifiers cannot contain "+". This is a compile-time error.

Fix: long adPlusBc = multiply(a + b, c + d) - ac - bd;

Forgetting that a+b may have more digits

Problem

```
// assuming (a+b) has n/2 digits
```

Why: The sum of two half-length numbers can carry into one more digit. The algorithm still works because every recursion strictly decreases the numbers, but the digit count of a+b is not n/2.

Fix: Recompute the length inside every call (the code above does this)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · School-book multiplication — *Beginner*

Read two non-negative integers as digit strings (up to 100 digits each). Print their product without using BigInteger or `long` arithmetic on the whole numbers.

Example input	Expected output
12345 6789	83810205

Hint 1: Digit i of x times digit j of y contributes to position i + j + 1 of the result. **Hint 2:** Keep a carry as you add.

Exercise 2 · Count the base multiplications — *Intermediate*

Run the lesson's Karatsuba algorithm (split at n/2 low digits; base case when either factor < 10) on x and y (both < 10^9). Print how many times the base case is reached.

Example input	Expected output
1234 5678	11

Hint 1: Add a static counter incremented in the base case. **Hint 2:** Do not change the splitting rule — the answer depends on it.

Exercise 3 · Big Karatsuba — *Challenge*

Read two non-negative integers (up to 1000 digits). Multiply them with your own Karatsuba on BigInteger (do not call BigInteger.multiply except below a 30-bit threshold) and print the product.

Example input	Expected output
123456789 987654321	121932631112635269

Hint 1: Use bitLength() to decide the split point. **Hint 2:** Use shiftLeft/shiftRight and subtract to obtain the low part.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many recursive multiplications does Karatsuba make per level?

- **A.** 2
- **B.** 3
- **C.** 4
- **D.** 5

Q2. Which expression equals ad + bc?

- **A.** (a+b)(c+d) − ac − bd
- **B.** (a−b)(c−d)
- **C.** ac + bd
- **D.** (a+c)(b+d)

Q3. The running time of Karatsuba is...

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^1.585)
- **D.** O(n^2)

Q4. With a=12, b=34, c=56, d=78, what is (a+b)(c+d) − ac − bd?

```
// ac = 672, bd = 2652
long abcd = 46 * 134;
System.out.println(abcd - 672 - 2652);
```

Your answer: _____

Q5. Why do libraries use school-book multiplication for small numbers?

- **A.** It is more accurate
- **B.** Karatsuba has overhead that dominates for small n
- **C.** It is required by the JVM
- **D.** Karatsuba cannot handle small numbers

Interview / exam questions

- Derive Karatsuba's 3-multiplication identity.
- Solve T(n) = 3T(n/2) + n with the master theorem.
- Why does BigInteger switch algorithms with size?
- Name an algorithm faster than Karatsuba for huge numbers (Toom–Cook, FFT/Schönhage–Strassen).

Watch & read

Video: 1 3 Karatsuba Multiplication 13 min — Stanford Algorithms · 12:40. *Three multiplications instead of four*

Video: Understanding Karatsuba Algorithm based on Divide and Conquer Implementation — MathPhy · 20:36. *Why Karatsuba is O(n^1.585)*

REVISION CARD (30–60 SECONDS)

- x=a·10^m+b, y=c·10^m+d
- ac, bd, (a+b)(c+d); middle = third − ac − bd
- x·y = ac·10^(2m) + mid·10^m + bd
- T(n) = 3T(n/2) + O(n) → O(n^1.585)
- Base case: small factor → plain multiply

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide code contains `long ad+bc = …` and later uses `adbc`; neither compiles. The slide also uses Math.pow (floating point) to build powers of ten. • The slide's explanation labels z0 = ac in one paragraph and z0 = bd in the next. In this lesson ac is the high product and bd the low product, consistently. • The slide says the three-product count "reduces overall complexity" but never states it: it is O(n^1.585) versus O(n²).

5.6 Booth's Algorithm (signed binary multiplication)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 60 min · complete path

Multiply two signed two's-complement numbers with additions, subtractions and shifts — the way hardware does.

YOU WILL BE ABLE TO

- Convert numbers to and from n-bit two's complement and negate them.
- Explain an arithmetic right shift and why it preserves the sign.
- Apply the three Booth rules (Q_0Q_{-1} = 00/11, 10, 01) by hand for n = 4.
- Implement the algorithm in Java and verify it against * for every input.

Before this lesson: Number Systems, Binary and Two's Complement · Bitwise Operators and Shifts · Loops: for, while, do-while, break and continue

The idea

Ordinary binary multiplication adds a shifted copy of the multiplicand for every 1 in the multiplier. 0111 (7) has three 1s so it costs three additions. But 7 = 8 − 1: **one shift and one subtraction**. A run of 1s from bit position j to k equals 2^(k+1) − 2^j.

Booth's algorithm scans the multiplier bit pairs from right to left and uses that observation: **subtract** the multiplicand where a run of 1s begins, **add** it where the run ends, and do nothing inside a run or inside a run of 0s.

Booth's conditions — the slide's "CONDITIONS" box.

Q_0	Q_{-1}	meaning	action
0	0	inside a run of 0s	only shift
1	1	inside a run of 1s	only shift
1	0	a run of 1s begins	AC = AC − M, then shift
0	1	a run of 1s ends	AC = AC + M, then shift

WHY THIS WORKS

Treat the multiplier as the sum $\Sigma(q_{i-1} - q_i)\cdot 2^i$. Each pair contributes +1, 0 or −1 times the multiplicand shifted into place. Doing the multiply with signed digits works for NEGATIVE multipliers too, because in two's complement the leading bit has a negative weight and the same pair rule gives it the right sign. The arithmetic shift (copying the sign bit) keeps the partial product's sign.

Syntax

Two's complement represents −x in n bits as (2^n − x): flip every bit and add 1. For n = 4: 7 = 0111; −7 = 1000 + 1 = 1001.

Arithmetic shift right moves every bit one place right and keeps the leftmost (sign) bit: 1100 → 1110. Java's `>>` does this for `int`; `>>>` shifts in zeros instead.

Java bit tools: `x & mask` (keep bits), `x | y`, `x << k`, `x >> k`, `(1 << n) - 1` (n ones).

A tiny example

```
int M = 7; // 0111
int negM = (~M + 1); // flip bits and add 1 → −7
System.out.println(negM); // −7
System.out.println(negM & 0xF); // 9 (= 1001, the 4-bit pattern of −7)
```

Guided example

The slide traces M = 0111 (7) and Q = 0011 (3) with registers **AC** (accumulator), **Q** (multiplier) and **Q_{-1}**. The table below is produced by the program `BoothMultiply` on the same input:

Result = AC:Q = 0001 0101 = 21 = 7 × 3

cycle	Q_0 Q_{-1}	operation	AC	Q	Q_{-1}
init			0000	0011	0
1	1 0	AC = AC − M, ASR	1100	1001	1
2	1 1	shift only	1110	0100	1
3	0 1	AC = AC + M, ASR	0010	1010	0
4	0 0	shift only	0001	0101	0

BoothMultiply.java — Booth's algorithm with trace (optimised solution)

```
import java.util.Scanner;

public class BoothMultiply {
 static String bits(int value, int width) {
 String s = Integer.toBinaryString(value & ((1 << width) - 1));
 return "0".repeat(width - s.length()) + s;
 }

 // Booth's algorithm for two signed numbers that each fit in `width` bits (two's complement).
 static int multiply(int multiplicand, int multiplier, int width, boolean trace) {
 if (multiplicand == -(1 << (width - 1))) {                                    (1)
 width++; // the most negative number cannot be negated in `width` bits: use one extra bit
 }
 int mask = (1 << width) - 1;
 int M = multiplicand & mask; // multiplicand register
 int negM = (-multiplicand) & mask; // two's complement of M (flip bits, add 1)   (2)
 int AC = 0; // accumulator
 int Q = multiplier & mask; // multiplier register
 int qMinus1 = 0; // the extra bit to the right of Q

 if (trace) {
 System.out.println("M = " + bits(M, width) + " -M = " + bits(negM, width));
 System.out.println("cycle Q0 Q-1 operation AC Q Q-1 (state AFTER the shift)");
 System.out.println("init - " + bits(AC, width) + " " + bits(Q, width) + " " + qMinus1);
 }
 for (int cycle = 1; cycle <= width; cycle++) {
 int q0 = Q & 1;
 int pairQ = qMinus1;
 String op = "shift only";
 if (q0 == 1 && qMinus1 == 0) { // pair "10": a run of 1s begins -> subtract M   (3)
```

(code continued)

```
    AC = (AC + negM) & mask;
    op = "AC = AC - M, ASR";
    } else if (q0 == 0 && qMinus1 == 1) { // pair "01": a run of 1s ends -> add M   4
    AC = (AC + M) & mask;
    op = "AC = AC + M, ASR";
    }
    // arithmetic shift right of the combined register AC | Q | Q-1
    qMinus1 = Q & 1;                                                               5
    Q = ((Q >> 1) | ((AC & 1) << (width - 1))) & mask;                             6
    int sign = AC & (1 << (width - 1)); // remember the sign bit...
    AC = (AC >> 1) | sign; // ...and put it back on the left                       7
    if (trace) {
    System.out.println(" " + cycle + " " + q0 + " " + pairQ + " " + String.format("%-15s",
        op) + " " + bits(AC, width) + " " + bits(Q, width) + " " + qMinus1);
    }
    }
    long raw = ((long) AC << width) | Q; // 2*width-bit result AC:Q                8
    long signBit = 1L << (2 * width - 1);
    return (int) ((raw & signBit) != 0 ? raw - (1L << (2 * width)) : raw);         9
    }

    public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int m = sc.nextInt();
    int q = sc.nextInt();
    int width = sc.nextInt();
    int product = multiply(m, q, width, true);
    System.out.println(m + " x " + q + " = " + product + " (check: " + (m * q) + ")");
    }
   }
```

Simulates AC, Q, Q_{-1} registers of `width` bits. Handles negative operands and the most-negative-number edge case.

Sample input

```
7 3 4
```

Line-by-line walkthrough

#	Where	What happens
1	line 11	Edge case: the most negative n-bit number (−8 in 4 bits) has no positive partner in n bits, so −M cannot be formed. One extra bit (sign extension) fixes it.
2	line 16	Two's complement of M as an n-bit pattern, precomputed so the loop can just ADD it to subtract M.
3	line 30	Pair "10": a run of 1s begins → AC = AC − M (adding −M).
4	line 33	Pair "01": a run of 1s ends → AC = AC + M.
5	line 38	Arithmetic shift of the 2n+1-bit chain AC|Q|Q_{-1}: the lowest bit of Q falls into Q_{-1}...
6	line 39	...Q shifts right and receives the lowest bit of AC at its top...
7	line 41	...and AC shifts right while the sign bit is kept: that is the arithmetic shift.
8	line 46	The 2n-bit product is AC concatenated with Q.
9	line 48	If the top bit is set the number is negative: subtract 2^(2n) to read it as signed.

BoothCheck.java — Exhaustive test (demonstration)

```
public class BoothCheck {
 // Compact copy of Booth's algorithm (no printing) so this file is self-contained.
 static int booth(int multiplicand, int multiplier, int width) {
 if (multiplicand == -(1 << (width - 1))) {
 width++;
 }
 int mask = (1 << width) - 1;
 int M = multiplicand & mask;
 int negM = (-multiplicand) & mask;
 int AC = 0;
 int Q = multiplier & mask;
 int qMinus1 = 0;
 for (int cycle = 0; cycle < width; cycle++) {
 int q0 = Q & 1;
 if (q0 == 1 && qMinus1 == 0) {
 AC = (AC + negM) & mask;
 } else if (q0 == 0 && qMinus1 == 1) {
 AC = (AC + M) & mask;
 }
 qMinus1 = Q & 1;
 Q = ((Q >> 1) | ((AC & 1) << (width - 1))) & mask;
 AC = (AC >> 1) | (AC & (1 << (width - 1)));
 }
 long raw = ((long) AC << width) | Q;
 return (int) ((raw & (1L << (2 * width - 1))) != 0 ? raw - (1L << (2 * width)) : raw);
 }

 // Exhaustively compares Booth's algorithm with the * operator for every pair of 6-bit
     signed numbers.
 public static void main(String[] args) {
 int width = 6;
 int lo = -(1 << (width - 1)); // -32
 int hi = (1 << (width - 1)) - 1; // 31
 int checked = 0;
 for (int m = lo; m <= hi; m++) {                                                    (1)
 for (int q = lo; q <= hi; q++) {
 int got = booth(m, q, width);
 if (got != m * q) {                                                                 (2)
 System.out.println("MISMATCH " + m + " x " + q + " gave " + got);
 return;
 }
 checked++;
 }
 }
 System.out.println("all " + checked + " products correct (including " + lo + " x " + lo
     + ")");
 }
}
```

*A compact copy of the algorithm compared with the * operator for all 64 × 64 six-bit pairs, including (−32) × (−32).*

Line-by-line walkthrough

#	Where	What happens
1	line 34	Every signed 6-bit multiplicand from −32 to 31.
2	line 37	Any disagreement with Java's own multiplication is reported immediately.

Dry run: watch it execute

The slide example M = 7, Q = 3 in 4 bits.

Algorithm: Booth's multiplication. **Input:** m = `7`, q = `3`, bits = `4`.

Pseudocode

```
AC = 0; Q−1 = 0
repeat n times:
 if (Q0,Q−1) == 10: AC = AC − M
 if (Q0,Q−1) == 01: AC = AC + M
 arithmetic shift right (AC, Q, Q−1)
product = AC:Q
```

Step 1 of 6. M = 0111 (7), −M = 1001, Q = 0011 (3). AC = 0, Q_{-1} = 0.

cycle = 0 · Q0 Q-1 = — — · AC = 0000 · Q = 0011 · Q-1 = 0

Step 2 of 6. Cycle 1: Q_0Q_{-1} = 10 → AC = AC − M (AC becomes 1001), then arithmetic shift right.

cycle = 1 · Q0 Q-1 = 10 · AC = 1100 · Q = 1001 · Q-1 = 1

Step 3 of 6. Cycle 2: Q_0Q_{-1} = 11 → only shift, then arithmetic shift right.

cycle = 2 · Q0 Q-1 = 11 · AC = 1110 · Q = 0100 · Q-1 = 1

Step 4 of 6. Cycle 3: Q_0Q_{-1} = 01 → AC = AC + M (AC becomes 0101), then arithmetic shift right.

cycle = 3 · Q0 Q-1 = 01 · AC = 0010 · Q = 1010 · Q-1 = 0

Step 5 of 6. Cycle 4: Q_0Q_{-1} = 00 → only shift, then arithmetic shift right.

cycle = 4 · Q0 Q-1 = 00 · AC = 0001 · Q = 0101 · Q-1 = 0

Step 6 of 6. Product = AC:Q = 0001 0101 = 21.

cycle = 4 · Q0 Q-1 = · AC = 0001 · Q = 0101 · Q-1 = 0

> **RESULT**
> 7 × 3 = 21

Complexity

The loop runs once per multiplier bit, and each cycle does at most one n-bit addition and one shift → **$O(n^2)$ bit operations** for n-bit operands (n cycles × n-bit add), O(n) cycles. Space O(n).

Why bother? In hardware shifting is nearly free, so the cost is the number of add/subtract operations. Booth turns a run of k ones into 2 operations instead of k. For multipliers with long runs of 1s (like −1 = 1111…1) that is a huge saving; worst case (alternating 0101…) is still n operations.

Common mistakes

Logical shift instead of arithmetic shift

Problem

```
AC = AC >> 1; // on an n-bit pattern held in a larger int
```

Why: The top bit of the n-bit pattern is not the int's sign bit, so >> shifts in 0. Negative partial products become positive.

Fix: AC = (AC >> 1) | (AC & signBit); // re-insert the sign bit

Forgetting to discard the carry

Problem

```
AC = AC + M; // AC is int, bit n silently appears
```

Why: The addition can produce an (n+1)th bit. Hardware drops it; in software you must mask.

Fix: AC = (AC + M) & mask;

Negating the most negative number

Problem

```
booth(-8, 3, 4) // -(-8) is 8, which does not fit in 4-bit two's complement
```

Why: −8 = 1000 and its two's complement is again 1000. The subtraction adds the wrong value.

Fix: Use width + 1 bits when the multiplicand is −2^(n−1) (as BoothMultiply does).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · n-bit two's complement — *Beginner*

Read an integer x and a width n (1 ≤ n ≤ 16, x fits in n bits). Print the n-bit two's complement bit pattern of x.

Example input	Expected output
−5 8	11111011

Hint 1: Walk from the top bit n−1 down to 0 and print (x >> bit) & 1. **Hint 2:** Java's >> on a negative int already yields the two's complement bits.

Exercise 2 · Booth's operations — *Intermediate*

Read an n-bit multiplier as a bit string. For each cycle (from the rightmost bit to the leftmost), print the operation Booth's algorithm performs: `−M`, `+M` or `0`. Separate with spaces.

Example input	Expected output
0110	0 −M 0 +M

Hint 1: Keep the previous bit (starting at 0). **Hint 2:** (1,0) → −M; (0,1) → +M; otherwise 0.

Exercise 3 · Booth multiplication — *Challenge*

Read two signed integers (each fits in 16-bit two's complement). Print their product, computed by simulating Booth's algorithm on registers (no `*`).

Example input	Expected output
−3 4	−12

Hint 1: Use width = 16 (17 if the multiplicand is −32768). **Hint 2:** Mask AC and M to the register width after every addition. **Hint 3:** Combine AC and Q at the end and sign-extend the 2n-bit value.

Quick check

Answers are in the answer key at the end of this part.

Q1. Booth's algorithm subtracts the multiplicand when (Q_0, Q_{-1}) is...

- **A.** 00
- **B.** 01
- **C.** 10
- **D.** 11

Q2. What is the 4-bit two's complement of 0111 (7)?

```
// flip: 1000, add 1
```

Your answer: _____

Q3. An arithmetic right shift of 1100 (4-bit) gives...

- **A.** 0110
- **B.** 1110
- **C.** 1100
- **D.** 0100

Q4. What is 0111 × 0011 after 4 Booth cycles in AC:Q?

- **A.** 00010101
- **B.** 00011001
- **C.** 00001111
- **D.** 00010011

Q5. Which multiplier makes Booth's algorithm most efficient?

- **A.** 01010101
- **B.** 11111111
- **C.** 10101010
- **D.** 00000001

Interview / exam questions

- Why does Booth's algorithm work for negative multipliers?
- What are the three registers and what does each do?
- Why is an extra bit needed for the most negative multiplicand?
- How does Booth compare to shift-and-add on an input with many consecutive 1s?

Watch & read

Video: Booth's Algorithm for Signed Multiplication — TutorialsPoint · 14:32. *Booth's signed multiplication*

Video: Booth's algorithm - Binary multiplication example | Computer Organization — Scholarly Things · 5:39. *A worked Booth example*

REVISION CARD (30–60 SECONDS)

- Pairs (Q0 Q-1): 10 → AC−M, 01 → AC+M, 00/11 → nothing; then arithmetic shift right.
- n cycles; product = AC:Q (2n bits).
- −M = ~M + 1.
- Run of 1s = 2^(k+1) − 2^j: one subtract, one add.
- Example: 7 × 3 = 0001 0101 = 21.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The Booth deck is entirely screenshots from a video lecture; it has no Java code. The program here is original and verified exhaustively.

ASSUMPTIONS MADE

- The slides use M = 0111, Q = 0011 (n = 4) and a second example with M = 01011, Q = 01110 (n = 5). Both are supported by the program.

5.7 Simple Sieve of Eratosthenes

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 40 min · ★ on the minimum-viable path

Find every prime up to n by crossing out multiples — far faster than testing each number.

YOU WILL BE ABLE TO

- Explain how crossing out multiples finds primes.
- Justify the two optimisations: start at p·p and stop at √n.
- Write the sieve with a boolean array and count / print primes.
- State O(n log log n) time and O(n) space and when a sieve is the right tool.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Divisibility, GCD/LCM, Primes and Modular Arithmetic

The idea

A **prime** is a number greater than 1 whose only divisors are 1 and itself. To list the primes up to 50 you could test each number — or you can do what Eratosthenes did 2 200 years ago: write the numbers down, and repeatedly cross out multiples.

1. Write 2, 3, 4, ..., n. Nothing is crossed out.
2. Take the first number that is not crossed out (2). It is prime. Cross out its multiples: 4, 6, 8, ...
3. Take the next surviving number (3). It is prime. Cross out its multiples 6, 9, 12, ...
4. Repeat. Every surviving number is prime.

```
n = 30 2 3 4 5 6 7 8 9 10 11 12 13 14 15 ...
p = 2 2 3 ✗ 5 ✗ 7 ✗ 9 ✗ 11 ✗ 13 ✗ 15 ...
p = 3 2 3 5 7 ✗ 11 13 ✗ ...
p = 5 2 3 5 7 11 13 ... (25 crossed; 5·5 = 25 ≤ 30)
```

WHY THIS WORKS

Why start at p·p? Any multiple p·k with k < p has a smaller prime factor than p (one of the factors of k), so it was already crossed out earlier. **Why stop at √n?** If a composite number m ≤ n had no prime factor ≤ √n, then m would be at least (√n)·(√n) > n — contradiction. So every composite is crossed out by a prime ≤ √n; primes larger than √n have nothing new to cross out.

Syntax

A `boolean[]` of size n+1 is indexed by the number itself: `isPrime[17]` answers "is 17 prime?". Java initialises boolean arrays to `false`, so the program either sets all to `true` first or flips the meaning to `composite[]`.

A tiny example

```
int n = 20;
boolean[] isPrime = new boolean[n + 1];
for (int i = 2; i <= n; i++) isPrime[i] = true;
for (int p = 2; p * p <= n; p++)
 if (isPrime[p])
 for (int m = p * p; m <= n; m += p)
 isPrime[m] = false;
```

Guided example

Compare with testing each number separately by trial division up to √k:

TrialDivision.java — Trial division (simple solution)

```
1  import java.util.Scanner;
2
3  public class TrialDivision {
4   // Test each number separately: n is prime if no d with 2 <= d <= sqrt(n) divides it.
5   static boolean isPrime(int n) {
6   if (n < 2) {                                                    1
7   return false;                                                   3
8   }
9   for (int d = 2; (long) d * d <= n; d++) {                       2
10  if (n % d == 0) {
11  return false;
12  }
13  }
14  return true;
15  }
16
17  public static void main(String[] args) {
18  Scanner sc = new Scanner(System.in);
19  int limit = sc.nextInt();
20  StringBuilder sb = new StringBuilder();
21  int count = 0;
22  for (int n = 2; n <= limit; n++) {
23  if (isPrime(n)) {
24  sb.append(n).append(' ');
25  count++;
26  }
27  }
28  System.out.println(sb.toString().trim());
29  System.out.println("count = " + count);
30  }
31 }
```

For every number k, test divisors up to √k. Total about n·√n operations.

Sample input

```
50
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	0 and 1 are not prime by definition.
2	line 9	Only divisors up to √n need testing; casting to long avoids overflow.
3	line 7	Found a divisor — n is composite. Return immediately.

SimpleSieve.java — Sieve of Eratosthenes (optimised solution)

```java
import java.util.Scanner;

public class SimpleSieve {
 // Sieve of Eratosthenes: cross out the multiples of every prime; whatever survives is prime.
 static boolean[] sieve(int limit) {
 boolean[] isPrime = new boolean[limit + 1];
 for (int i = 2; i <= limit; i++) {
 isPrime[i] = true; // assume prime until crossed out (1)
 }
 for (int p = 2; (long) p * p <= limit; p++) { // primes bigger than sqrt(limit) have nothing left to cross out (2)
 if (isPrime[p]) { (3)
 for (int m = p * p; m <= limit; m += p) { // smaller multiples were crossed out by smaller primes (4)
 isPrime[m] = false;
 }
 }
 }
 return isPrime;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int limit = sc.nextInt();
 boolean[] isPrime = sieve(limit);
 StringBuilder sb = new StringBuilder();
 int count = 0;
 for (int n = 2; n <= limit; n++) {
 if (isPrime[n]) {
 sb.append(n).append(' ');
 count++;
 }
 }
 System.out.println(sb.toString().trim());
 System.out.println("count = " + count);
 }
}
```

Crosses out multiples of each surviving prime starting at p·p. O(n log log n).

Sample input

```
50
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Optimistically mark every number prime; crossing out will correct this.
2	line 10	Only primes up to √limit need to cross out multiples.
3	line 11	p survived all smaller primes so p is prime. Composite p can be skipped because their multiples were already handled.
4	line 12	Smaller multiples were crossed out by smaller primes; stepping by p visits each multiple once.

Both print the primes up to 50: `2 3 5 7 11 13 17 19 23 29 31 37 41 43 47` (15 primes).

Dry run: watch it execute

Primes up to 30: watch each prime cross out its multiples.

Algorithm: Sieve of Eratosthenes. **Input:** n = `30`.

Pseudocode

```
isPrime[2..n] = true
for (p = 2; p * p <= n; p++)
 if (isPrime[p])
 for (m = p * p; m <= n; m += p)
 isPrime[m] = false
primes = all i with isPrime[i]
```

Step 1 of 6. Write down 2..n. Nothing is crossed out yet.

p = —

numbers (= crossed out)

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

20	21	22	23	24	25	26	27	28	29	30
20	21	22	23	24	25	26	27	28	29	30

Step 2 of 6. 2 is prime. Cross out 2·2 = 4, then every 2th number after it.

p = 2 · start p·p = 4

numbers (= crossed out)

2	3	✗	5	✗	7	✗	9	✗	11	✗	13	✗	15	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	21	✗	23	✗	25	✗	27	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

Step 3 of 6. 3 is prime. Cross out 3·3 = 9, then every 3th number after it.

p = 3 · start p·p = 9

numbers (= crossed out)

2	3	✗	5	✗	7	✗	✗	✗	11	✗	13	✗	✗	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	✗	✗	23	✗	25	✗	✗	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

Step 4 of 6. 4 is already crossed out, so it is not prime — skip it.

p = 4

numbers (= crossed out)

2	3	✗	5	✗	7	✗	✗	✗	11	✗	13	✗	✗	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	✗	✗	23	✗	25	✗	✗	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

Step 5 of 6. 5 is prime. Cross out 5·5 = 25, then every 5th number after it.

p = 5 · start p·p = 25

numbers (= crossed out)

2	3	✗	5	✗	7	✗	✗	✗	11	✗	13	✗	✗	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	✗	✗	23	✗	✗	✗	✗	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

Step 6 of 6. p·p exceeds 30, so we stop. Survivors are the primes.

count = 10

numbers (= crossed out)

2	3	✗	5	✗	7	✗	✗	✗	11	✗	13	✗	✗	✗	17	✗	19
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

✗	✗	✗	23	✗	✗	✗	✗	✗	29	✗
20	21	22	23	24	25	26	27	28	29	30

RESULT

10 primes: 2 3 5 7 11 13 17 19 23 29

Complexity

Prime p crosses out about n/p numbers. Summed over primes p ≤ √n that is n·(1/2 + 1/3 + 1/5 + ...) = n · log log n (a very slowly growing function: for n = 10^9 it is about 3).

```
time = O(n log log n) space = O(n) (one boolean per number)
```

n	Trial division ≈ n√n	Sieve ≈ n log log n
10^6	10^9	3×10^6
10^7	3×10^{10}	3×10^7

NOTE

Use a sieve when you need ALL primes up to n, or many primality queries. Use trial division (or Miller–Rabin) when you need to test a single large number. When n is too large for memory but the range is narrow, use the segmented sieve (next lesson).

Common mistakes

Index out of bounds

Problem

```
boolean[] isPrime = new boolean[limit];
isPrime[limit] = true;
```

Why: A boolean[limit] has indices 0..limit−1. To use the number `limit` itself the size must be limit + 1.

Fix: new boolean[limit + 1]

Starting at 2·p

Problem

```
for (int m = 2 * p; m <= limit; m += p)
```

Why: Correct, but does redundant work — those multiples were already crossed out. It does NOT break correctness, only speed (a common beginner version).

Fix: for (int m = p * p; ...)

Counting 0 and 1 as prime

Problem

```
for (int i = 0; i <= limit; i++) if (isPrime[i]) count++;
```

Why: With the "all true" initialisation, 0 and 1 stay true unless set to false.

Fix: Start both the initialisation and the counting loops at 2.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count primes — *Beginner*

Read n and print how many primes are ≤ n.

- $1 \le n \le 10^7$

Example input	Expected output
100	25

Hint 1: Use a boolean array of size n + 1. **Hint 2:** Count while you sieve.

Exercise 2 · The k-th prime — *Intermediate*

Read k ($1 \le k \le 100\,000$) and print the k-th prime (2 is the 1st).

Example input	Expected output
1000	7919

Hint 1: How large can the 100 000-th prime be? About 1.3 million. **Hint 2:** Sieve up to a fixed safe bound and count.

Exercise 3 · Prime factorisation queries — *Challenge*

Read q, then q numbers x ($2 \le x \le 10^6$). For each, print its prime factorisation as `x = p1^e1 * p2^e2` (exponent omitted when 1).

Example input	Expected output
3 ⏎ 12 ⏎ 97 ⏎ 360	12 = 2^2 * 3 ⏎ 97 = 97 ⏎ 360 = 2^3 * 3^2 * 5

Hint 1: Build an array spf[x] = smallest prime factor with a sieve. **Hint 2:** Repeatedly divide x by spf[x].

Quick check

Answers are in the answer key at the end of this part.

Q1. In the sieve, why can the outer loop stop at p·p ≤ n?

- **A.** Because larger primes are not prime
- **B.** Because every composite ≤ n has a prime factor ≤ √n
- **C.** Because arrays are limited to √n
- **D.** Because p·p is always prime

Q2. Which numbers does p = 3 cross out for n = 30 (starting at p·p)?

```
for (int m = 9; m <= 30; m += 3) System.out.print(m + " ");
```

Your answer: _____

Q3. Time complexity of the sieve of Eratosthenes?

- **A.** O(n)
- **B.** O(n log log n)
- **C.** O(n^2)
- **D.** O(n√n)

Q4. This sieve reports no primes at all. Which line is wrong?

```
1  boolean[] isPrime = new boolean[n + 1];
2  Arrays.fill(isPrime, true);
3  for (int p = 2; p * p <= n; p++)
4   for (int m = p; m <= n; m += p)
5   isPrime[m] = false;
```

Which line contains the bug? _____

Q5. How many primes are there below 50?

- **A.** 13
- **B.** 15
- **C.** 17
- **D.** 19

Interview / exam questions

- Why is the sieve O(n log log n)?
- How would you reduce the memory of a sieve?
- Difference between sieve and trial division — when to use which?
- How do you get the smallest prime factor of every number?

Watch & read

Video: Sieve of Eratosthenes | Journey into cryptography | Computer Science | Khan Academy — Khan Academy Labs · 4:12. *Crossing out multiples to find primes*

Video: L6. Sieve of Eratosthenes | Maths Playlist — take U forward · 18:27. *Sieve implementation in Java*

REVISION CARD (30–60 SECONDS)

- isPrime[0..n]=true (except 0,1)
- for p with p·p ≤ n and isPrime[p]: cross out p·p, p·p+p, ...
- Time O(n log log n), space O(n)
- Use for "all primes ≤ n" and many queries

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide walk-through for n = 20 is correct, but its last numbered step is cut off ("11. The rem..."); the final answer is 2 3 5 7 11 13 17 19.

5.8 Segmented & Incremental Sieve

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · complete path

Find primes inside a window [L, R] with R up to 10^{12}, or generate primes one at a time with no upper bound.

YOU WILL BE ABLE TO

- Explain why a simple sieve cannot handle R = 10^{12} and how segmenting fixes it.
- Implement the segmented sieve: base primes up to √R, then mark multiples inside [L, R].
- Handle the tricky parts: first multiple ≥ L, never marking p itself, and L < 2.
- Describe an incremental sieve that produces primes one at a time using a map of upcoming composites.

Before this lesson: Simple Sieve of Eratosthenes · Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

The idea

The ordinary sieve needs an array of size n. To find the primes between 10^{12} and $10^{12} + 10^6$ it would need a trillion cells. But the interval itself is only a million numbers wide. The key fact: **a composite number in [L, R] has a prime factor ≤ √R ≤ 10^6**. So we only need the small primes up to √R to cross out everything in the window.

1. Run the simple sieve to get all **base primes** up to √R.
2. Create a small array for the window; cell i stands for the number L + i.
3. For each base prime p find its first multiple inside the window and cross out every p-th cell.
4. The cells never crossed out are the primes in [L, R].

```
window [10, 30], base primes ≤ √30 = 5 → 2, 3, 5

index 0 1 2 3 4 5 ... number = 10 + index
n 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
p=2 ✗ ✗ ✗ ✗ ✗ ✗ ✗ ✗ ✗ ✗
p=3 · ✗ ✗ ✗ ✗ ✗ ✗ ✗
p=5 ✗ ✗ ✗ (10, 15, 20, 25, 30)
primes: 11 13 17 19 23 29
```

WHY THIS WORKS

The first multiple of p that is ≥ L is ⌈L/p⌉·p. But we must also never start below p·p: if L ≤ p ≤ R, then p itself lies in the window and must not be crossed out. Starting at max(p·p, ⌈L/p⌉·p) gives both guarantees.

Incremental sieve

The incremental (a.k.a. unbounded or "rolling") sieve produces **the next prime on demand**, with no limit decided in advance. It keeps a dictionary mapping an upcoming composite number to the list of primes that will cross it out. When the counter reaches a number: if it is in the dictionary it is composite — move each of its primes to the next multiple; if it is not, it is prime and we schedule its square.

Syntax

New Java here: `HashMap<Long, List<Long>>` (a map from a number to a list of numbers), `computeIfAbsent(key, k -> new ArrayList<>())` (get the list, creating it if missing), and `map.remove(key)` which returns the removed value or `null`.

A tiny example

(low + p − 1) / p rounds low/p UP using integer division.

```
long low = 10, p = 3;
long start = Math.max(p * p, (low + p - 1) / p * p); // 12: first multiple of 3 that is >= 10
```

Guided example

SegmentedSieve.java — Segmented sieve (optimised solution)

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class SegmentedSieve {
 // Step 1: ordinary sieve that returns every prime up to `limit`.
 static List<Integer> simpleSieve(int limit) {
 boolean[] composite = new boolean[limit + 1];
 List<Integer> primes = new ArrayList<>();
 for (int i = 2; i <= limit; i++) {
 if (!composite[i]) {
 primes.add(i);
 for (long m = (long) i * i; m <= limit; m += i) {
 composite[(int) m] = true;
 }
 }
 }
 return primes;
 }

 // Step 2: sieve only the window [low, high] using the small primes up to sqrt(high).
 static List<Long> segmentedSieve(long low, long high) {
 int root = (int) Math.sqrt((double) high);                                  (1)
 while ((long) root * root > high) root--; // guard against floating-point rounding
 while ((long) (root + 1) * (root + 1) <= high) root++;
 List<Integer> basePrimes = simpleSieve(root);                               (2)

 boolean[] composite = new boolean[(int) (high - low + 1)]; // index i <-> number low + i  (3)
 for (int p : basePrimes) {
 // first multiple of p that is >= low, but never below p*p (p itself must stay prime)
 long start = Math.max((long) p * p, (low + p - 1) / p * p);                 (4)
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;                                          (5)
 }
 }
 List<Long> primes = new ArrayList<>();
 for (long n = Math.max(low, 2); n <= high; n++) {                           (6)
 if (!composite[(int) (n - low)]) {
 primes.add(n);
 }
 }
 return primes;
```

(code continued)

```
    }

    public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    long low = sc.nextLong();
    long high = sc.nextLong();
    List<Long> primes = segmentedSieve(low, high);
    StringBuilder sb = new StringBuilder();
    for (long p : primes) {
    sb.append(p).append(' ');
    }
    System.out.println(sb.toString().trim());
    System.out.println("count = " + primes.size());
    }
   }
```

Simple sieve up to √high, then marks multiples inside the window. Memory O(√high + (high − low)).

Sample input

```
10 30
```

Line-by-line walkthrough

#	Where	What happens
1	line 23	We need base primes up to √high. The two while loops correct floating-point rounding.
2	line 26	The ordinary sieve from the previous lesson, but returning the primes.
3	line 28	Only the window is stored. Index i represents the number low + i.
4	line 31	First multiple ≥ low, but not below p·p so that p itself (if in the window) survives.
5	line 33	Translate the number m into its array index by subtracting low.
6	line 37	Numbers below 2 are not prime, so a window starting at 0 or 1 must skip them.

IncrementalSieve.java — Incremental sieve (assumed from the slide description) (demonstration)

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class IncrementalSieve {
 // An incremental (unbounded) sieve: no upper limit is needed in advance.
 // `composites` remembers, for each upcoming composite number, which primes will cross it out.
 private final Map<Long, List<Long>> composites = new HashMap<>();
 private long candidate = 1;

 long nextPrime() {
 while (true) {
 candidate++;
 List<Long> crossers = composites.remove(candidate);                                    (1)
 if (crossers == null) {
 // nobody has crossed candidate out, so it is prime; schedule its first multiple p*p
 composites.computeIfAbsent(candidate * candidate, k -> new ArrayList<>()).add(candidate)  (2)
 ;
 return candidate;
 }
 for (long p : crossers) { // candidate is composite: move each p on to its next multiple
 composites.computeIfAbsent(candidate + p, k -> new ArrayList<>()).add(p);              (3)
 }
 }
 }

 public static void main(String[] args) {
 IncrementalSieve s = new IncrementalSieve();
 StringBuilder first = new StringBuilder();
 for (int i = 0; i < 15; i++) {
 first.append(s.nextPrime()).append(' ');
 }
 System.out.println("first 15 primes: " + first.toString().trim());

 // "start from N": keep generating and skip everything below N
 long N = 100;
 IncrementalSieve t = new IncrementalSieve();
 long p = t.nextPrime();
 while (p < N) {
 p = t.nextPrime();
 }
 StringBuilder from = new StringBuilder();
 for (int i = 0; i < 5; i++) {
 from.append(p).append(' ');
 p = t.nextPrime();
 }
 System.out.println("first 5 primes >= " + N + ": " + from.toString().trim());
 }
}
```

An unbounded generator of primes using a map of upcoming composites. The slide only describes this algorithm in words, so the implementation is inferred.

Line-by-line walkthrough

#	Where	What happens
1	line 15	If someone scheduled this number, it is composite and the list holds its prime factors seen so far.
2	line 18	A new prime p can first cross out p·p (smaller multiples have smaller factors).
3	line 22	Each prime that hit a composite moves on to its next multiple.

Dry run: watch it execute

Window [10, 30]: base primes 2, 3, 5 cross out cells; survivors are prime.

Algorithm: Segmented sieve [L, R]. **Input:** low = `10`, high = `30`.

Pseudocode

```
base = primes up to sqrt(R)
window[i] stands for L + i
for each base prime p:
 start = max(p*p, ceil(L/p)*p)
 cross out start, start+p, ... <= R
survivors (>= 2) are prime
```

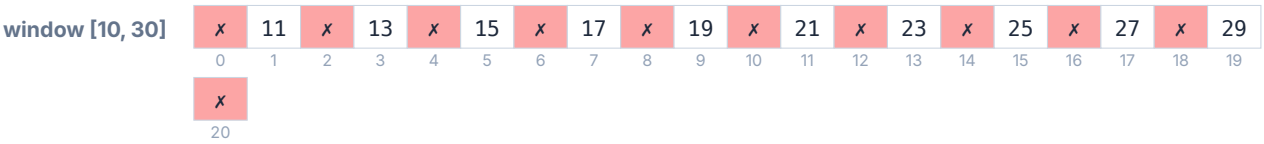
Step 1 of 5. √30 ≈ 5, so the base primes are 2, 3, 5.

base primes = 2 3 5



Step 2 of 5. p = 2: first multiple ≥ 10 is 10; starting at max(4, that) = 10.

p = 2 · start = 10



Step 3 of 5. p = 3: first multiple ≥ 10 is 12; starting at max(9, that) = 12.

p = 3 · start = 12



Step 4 of 5. p = 5: first multiple ≥ 10 is 10; starting at max(25, that) = 25.

p = 5 · start = 25



Step 5 of 5. Cells never crossed out (and ≥ 2) are prime.

count = 6



RESULT
6 primes: 11 13 17 19 23 29

Complexity

Segmented: base primes cost O(√R log log R). Marking the window costs about (R − L) · log log R. So time ≈ O((R − L) log log R + √R) and space O(√R + (R − L)). For R = 10^{12}, R − L = 10^6 this is a few million operations.

Incremental: each new prime p inserts one map entry; each composite c causes one map move per distinct prime factor. Memory is proportional to the number of primes found up to √(current) — the map keeps only "next composite" per prime.

Common mistakes

Marking the prime itself

Problem

```
for (long m = (low / p) * p; m <= high; m += p) composite[...] = true; // slide style
```

Why: If low ≤ p (for example low = 2), this crosses out p itself, so 2 is reported as composite.
Fix: long start = Math.max((long) p * p, (low + p - 1) / p * p);

Using int for the window bounds

Problem

```
int low = 1_000_000_000_000;
```

Why: 10^{12} is far above Integer.MAX_VALUE (2.1·10^9): it does not compile; computations on int bounds overflow silently.
Fix: long low, high; // but keep the ARRAY index (high − low) as an int

Forgetting to initialise (slide)

Problem

```
boolean prime[] = new boolean[h + 1]; // all false, then marks them false again
```

Why: The slide's array starts as all false and the commented-out Arrays.fill is never run, so the printed result is empty. It also writes `False` (capital F), which does not compile.
Fix: Use a composite[] array: false means "still possibly prime", so no fill is needed.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Primes in a small range — *Beginner*

Read L and R (1 ≤ L ≤ R ≤ 10^6). Print how many primes are in [L, R].

Example input	Expected output
10 30	6

Hint 1: A plain sieve up to R is fine here. **Hint 2:** Count only values ≥ max(L, 2).

Exercise 2 · Primes in a huge range — *Intermediate*

Read L and R with $R \le 10^{12}$ and $R - L \le 10^6$. Print how many primes lie in [L, R].

Example input	Expected output
1000000000000 1000000000100	4

Hint 1: The simple sieve cannot allocate 10^{12} cells. **Hint 2:** Base primes up to 10^6, then window marking.

Exercise 3 · Largest prime gap in a window — *Challenge*

Read L and R ($R - L \le 10^6$, $R \le 10^{12}$). Print the largest difference between two consecutive primes in [L, R], or -1 if there are fewer than two primes.

Example input	Expected output
1 100	8

Hint 1: Sieve the window, then scan keeping the previous prime.

Quick check

Answers are in the answer key at the end of this part.

Q1. To sieve [L, R] you need base primes up to...

- **A.** R
- **B.** R/2
- **C.** √R
- **D.** L

Q2. What is the first multiple of 7 that is ≥ 100 and ≥ 49?

```
long low = 100, p = 7;
System.out.println(Math.max(p * p, (low + p - 1) / p * p));
```

Your answer: _____

Q3. Why start marking at max(p·p, first multiple ≥ L)?

- **A.** To speed up only
- **B.** To avoid marking p itself and values below p·p
- **C.** Because arrays start at p
- **D.** It makes memory smaller

Q4. Which statement describes an incremental sieve?

- **A.** Needs the limit up front
- **B.** Produces primes one after another, remembering upcoming composites
- **C.** Only works on [L, R]
- **D.** Uses recursion on array halves

Interview / exam questions

- How do you find primes in $[10^{12}, 10^{12} + 10^6]$?
- Why is √R enough for base primes?
- What is the memory of a segmented sieve?
- How does an incremental sieve avoid fixing the upper bound?

Watch & read

Video: Segmented Sieve | Sample Video II for Essential Maths for CP | GeeksforGeeks — GeeksforGeeks · 11:42. *Sieving a range [L, R] with base primes*

Video: 15.1 Sieve of Eratosthenes | Challenge | C++ Placement Course — Apna College · 10:47. *Segmented sieve implementation (C++ — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- Base primes ≤ √R via simple sieve.
- window[i] ↔ L + i
- start = max(p·p, ⌈L/p⌉·p)
- skip numbers < 2
- Incremental: map composite → primes crossing it

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide's SegSieve does not compile (`False`, a commented Arrays.fill) and, if repaired naively, marks small primes in the window as composite and ignores the p·p optimisation. • The slide's array has size h + 1 (memory proportional to R), which defeats the purpose of segmenting.

ASSUMPTIONS MADE

• The slide describes the incremental sieve only in words ("check N + i using primes up to L"). The lesson implements the standard incremental (dictionary-based) sieve and labels it as such.

5.9 Binary Palindrome

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 35 min · ★ on the minimum-viable path

Decide whether the binary form of a number reads the same forwards and backwards, using strings and then pure bit operations.

YOU WILL BE ABLE TO

- Write a number in binary and decide by hand whether it is a palindrome.
- Explain what `x & 1`, `x >> 1`, `r << 1` and `r | bit` do and combine them to reverse bits.
- Implement both a String-based and a bit-based palindrome check and compare them.
- Understand why leading zeros are NOT part of the binary representation.

Before this lesson: Number Systems, Binary and Two's Complement · Bitwise Operators and Shifts · Loops: for, while, do-while, break and continue

The idea

A palindrome reads the same in both directions (`racecar`, `12321`). A **binary palindrome** is a number whose binary digits form a palindrome. 17 is `10001` — reversed it is still `10001`, so it is a binary palindrome. 16 is `10000`; reversed it is `00001`, which is not the same, so 16 is not.

n	binary	reversed	palindrome?
5	101	101	yes
6	110	011 → 11 ≠ 110	no
9	1001	1001	yes
10	1010	0101	no
51	110011	110011	yes

WHY THIS WORKS

Binary has **no leading zeros**: 6 is written `110`, never `0110`. So a number whose binary form ends in zeros can never be a palindrome (its reverse would start with zeros that vanish). This is why the bit-reversal trick below can compare plain integers: if the reversed bits give the same integer, they gave the same bit string.

To reverse the bits of x with arithmetic, peel bits off the right end of x (`x & 1` reads the lowest bit; `x >> 1` removes it) and push them onto the right end of a new number (`r << 1` makes room, `r | bit` adds the bit).

Syntax

expression	meaning	example with x = 13 (1101)
x & 1	lowest bit	1
x >> 1	drop lowest bit (÷ 2)	110 = 6
r << 1	make room (× 2)	1 → 10
r | b	set the lowest bit to b	10 | 1 = 11

A tiny example

```
int x = 5; // 101
System.out.println(Integer.toBinaryString(x)); // "101"
```

Guided example

BinaryPalindromeString.java — Binary palindrome with a String (simple solution)

```
1  import java.util.Scanner;
2
3  public class BinaryPalindromeString {
4   // Convert to a binary String and compare characters from both ends.
5   static boolean isBinaryPalindrome(int x) {
6   String bits = Integer.toBinaryString(x); // no leading zeros, e.g. 17 -> "10001"   (1)
7   int left = 0;                                                                      (2)
8   int right = bits.length() - 1;
9   while (left < right) {
10  if (bits.charAt(left) != bits.charAt(right)) {                                     (3)
11  return false;
12  }
13  left++;                                                                            (4)
14  right--;
15  }
16  return true;
17  }
18
19  public static void main(String[] args) {
20  Scanner sc = new Scanner(System.in);
21  int x = sc.nextInt();
22  System.out.println(x + " (" + Integer.toBinaryString(x) + ") " + (isBinaryPalindrome(x)
        ? "is" : "is not") + " a binary palindrome");
23  }
24 }
```

Uses Integer.toBinaryString and two pointers moving inwards.

Sample input

```
17
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Gives the bits without leading zeros, e.g. 17 → "10001".
2	line 7	Two pointers start at the two ends of the string.
3	line 10	One mismatch is enough to answer "no".
4	line 13	Move both pointers one step towards the middle.

BinaryPalindromeBits.java — Binary palindrome with bit operations (optimised solution)

```
1  import java.util.Scanner;
2
3  public class BinaryPalindromeBits {
4   // Build the bit-reversal of x with shifts and compare with x. O(log x) time, O(1)
        space, no String.
5   static boolean isBinaryPalindrome(int x) {
6   int reversed = 0;                                                              (1)
7   int rest = x;
8   while (rest > 0) {                                                             (2)
9   reversed <<= 1; // make room on the right                                      (3)
10  reversed |= (rest & 1); // copy the lowest bit of rest into the new lowest bit (4)
11  rest >>= 1; // drop the bit we just used                                       (5)
12  }
13  return reversed == x;                                                          (6)
14  }
15
16  public static void main(String[] args) {
17  Scanner sc = new Scanner(System.in);
18  int x = sc.nextInt();
19  System.out.println(x + (isBinaryPalindrome(x) ? " has a binary palindrome
        representation." : " does not have a binary palindrome representation."));
20  }
21 }
```

Builds the bit-reversed number and compares it with the original. No String, no array.

Sample input

```
51
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	The number being built, starting empty.
2	line 8	Each iteration consumes one bit of the copy `rest`; the loop ends when no bits remain.
3	line 9	Shift left: make room for the next bit on the right.
4	line 10	`rest & 1` is the lowest bit of rest; OR-ing puts it into the freed spot.
5	line 11	Remove the bit we just copied.
6	line 13	Both numbers have the same number of bits, so they are equal exactly when the bit strings are mirror images.

Dry run: watch it execute

Peel bits off x one at a time and push them onto reversed.

Algorithm: Binary palindrome (bit reversal). **Input:** n = `51`.

Pseudocode

```
reversed = 0; rest = x;
while (rest > 0) {
 reversed <<= 1;
 reversed |= (rest & 1);
 rest >>= 1;
}
return reversed == x;
```

Step 1 of 8. x = 51 = 110011 in binary.

x = 51 · rest = 51 · reversed = 0 · rest bin = 110011 · reversed bin = 0

Step 2 of 8. Lowest bit of rest was 1: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 25 · reversed = 1 · rest bin = 11001 · reversed bin = 1

Step 3 of 8. Lowest bit of rest was 1: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 12 · reversed = 3 · rest bin = 1100 · reversed bin = 11

Step 4 of 8. Lowest bit of rest was 0: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 6 · reversed = 6 · rest bin = 110 · reversed bin = 110

Step 5 of 8. Lowest bit of rest was 0: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 3 · reversed = 12 · rest bin = 11 · reversed bin = 1100

Step 6 of 8. Lowest bit of rest was 1: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 1 · reversed = 25 · rest bin = 1 · reversed bin = 11001

Step 7 of 8. Lowest bit of rest was 1: shift reversed left and OR it in; then shift rest right.

x = 51 · rest = 0 · reversed = 51 · rest bin = 0 · reversed bin = 110011

Step 8 of 8. reversed (51) equals x (51): a binary palindrome.

x = 51 · reversed = 51 · palindrome = true

> **RESULT**
> 51 (110011) is a binary palindrome

Complexity

A number x has $\lfloor \log_2 x \rfloor + 1$ bits, and each loop iteration removes one: **O(log x) time, O(1) space** for the bit method. The String method is also O(log x) time but allocates a String of that length (O(log x) space).

Why log? Doubling the number adds only one more bit, so 10^9 has just 30 bits — 30 iterations.

Common mistakes

Forgetting that zero bits at the end matter

Problem

```
// "10" is 2 in binary; someone treats 010 as a palindrome
```

Why: Leading zeros are not stored. 2 = `10` which reversed is `01` = 1 ≠ 2, so 2 is not a palindrome.

Fix: Compare the integer reversal with the original (the lesson code) — it handles this automatically.

Reversing the same variable you are reading

Problem

```
while (x > 0) {
 reversed = (reversed << 1) | (x & 1);
 x >>= 1;
}
return reversed == x; // x is 0 now!
```

Why: The loop destroys x. After the loop x == 0, so the comparison is almost always false.

Fix: int rest = x; // work on a copy; compare with the untouched x

Negative numbers

Problem

```
isBinaryPalindrome(-5)
```

Why: Java stores negatives in two's complement (32 bits starting with 1...). The loop `while (x > 0)` never runs and returns `0 == -5` → false, which is a reasonable answer but a different question.

Fix: Decide the problem's rule (usually: n ≥ 0) and document it.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Is it a binary palindrome? — *Beginner*

Read a non-negative integer n. Print YES if its binary representation is a palindrome, otherwise NO.

- $0 \le n \le 2^{31} - 1$

Example input	Expected output
5	YES
10	NO

Hint 1: Compare the number with its bit reversal. **Hint 2:** Remember 0 is written "0" — a palindrome.

Exercise 2 · Count binary palindromes up to n — *Intermediate*

Read n. Print how many integers from 1 to n have a palindromic binary representation.

- $1 \le n \le 10^6$

Example input	Expected output
10	5

Hint 1: Loop and reuse your checker. **Hint 2:** Check your answer by listing them for n = 10.

Exercise 3 · Next binary palindrome — *Challenge*

Read n. Print the smallest binary palindrome strictly greater than n.

- $0 \le n \le 10^9$

Example input	Expected output
7	9

Hint 1: Palindromes are fairly dense: the gap is never larger than about 2^(bits/2). **Hint 2:** Start at n + 1 and keep going.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which of these numbers is a binary palindrome?

- **A.** 6 (110)
- **B.** 9 (1001)
- **C.** 12 (1100)
- **D.** 14 (1110)

Q2. What does this print?

```
int x = 6, r = 0;
while (x > 0) {
 r = (r << 1) | (x & 1);
 x >>= 1;
}
System.out.println(r);
```

Your answer: _____

Q3. What does `r << 1` do to the number r?

- **A.** divides by 2
- **B.** multiplies by 2
- **C.** adds 1
- **D.** flips all bits

Q4. Fill in the bit that is copied each step.

```
reversed = (reversed << 1) | (rest ______ 1);
rest = rest >> 1;
```

Q5. Time complexity of the bit-reversal palindrome test?

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n log n)

Interview / exam questions

- How would you check a binary palindrome without converting to a String?
- Why can't a number that is a multiple of 2 be a binary palindrome (other than 0)?
- How would you reverse the 32 bits of an int?

Watch & read

Video: C++ Challenge: Check If a Number's Binary Form Is a Palindrome — Atshk.h programing · 6:09. *Testing a binary palindrome with bit operations (C++ — the idea transfers to Java)*

Video: Reverse Bits - Binary - Leetcode 190 - Python — NeetCode · 10:13. *Reversing the bits of an integer (Python — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- x & 1 = lowest bit; x >> 1 drops it; r << 1 | bit pushes it.
- Reverse the bits, compare with the original.
- O(log n) time, O(1) space.
- Even numbers (> 0) are never binary palindromes: they end in 0 but start with 1.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slides list 16 (10000) as a non-palindrome and 17 (10001) as a palindrome — both correct. The explanation text calls the reversed value "00001"; mathematically the reversed integer is 1.

5.10 Swap Two Nibbles in a Byte

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 30 min · complete path

Exchange the high and low 4-bit halves of a byte using masks and shifts.

YOU WILL BE ABLE TO

- Define a nibble and split a byte into its two nibbles with masks 0xF0 and 0x0F.
- Combine shifts and OR to rebuild the byte with nibbles swapped.
- Explain why Java's signed `byte` needs `& 0xFF` before shifting.
- Generalise: swap adjacent bits and reverse all 32 bits.

Before this lesson: Bitwise Operators and Shifts · Number Systems, Binary and Two's Complement · Type Conversion, Casting and Overflow

The idea

A **bit** is 0 or 1. A **nibble** is 4 bits (half a byte; it holds exactly one hexadecimal digit, 0–F). A **byte** is 8 bits = two nibbles. In `1010 1100` (0xAC) the high nibble is `1010` (A) and the low nibble is `1100` (C). Swapping the nibbles gives `1100 1010` (0xCA).

```
1010 1100 0xAC
& 0xF0 → 1010 0000 (keep high nibble) >>> 4 → 0000 1010
& 0x0F → 0000 1100 (keep low nibble) << 4 → 1100 0000
OR → 1100 1010 0xCA
```

WHY THIS WORKS

A mask is a pattern of 1s over the bits you want to keep: `x & mask` zeroes everything else. Shifting by 4 slides a nibble across the byte boundary: multiplying by 16 (`<< 4`) moves the low nibble up, dividing by 16 (`>>> 4`) moves the high nibble down. OR glues the two moved pieces back together because they occupy different bit positions.

Syntax

Hexadecimal literals start with `0x`: `0xF0` = 240 = `1111 0000`, `0x0F` = 15 = `0000 1111`.

BYTE IS SIGNED

Java's `byte` is **signed** (−128..127). When a byte is used in an expression it is converted to `int` with sign extension: `(byte) 0xAB` becomes `0xFFFFFFAB`. Always mask with `& 0xFF` to get the unsigned 0–255 value first.

A tiny example

```java
int b = 0xAC;
int swapped = ((b & 0xF0) >>> 4) | ((b & 0x0F) << 4);
System.out.println(Integer.toHexString(swapped)); // ca
```

Guided example

NibbleSwap.java — Swap nibbles (int based) (optimised solution)

```
1  import java.util.Scanner;
2
3  public class NibbleSwap {
4   // A byte has 8 bits = two 4-bit nibbles. Swap high and low nibble.
5   static int swapNibbles(int value) {
6   int b = value & 0xFF; // work on exactly one byte (0..255)                    (1)
7   int upper = (b & 0xF0) >>> 4; // keep the high 4 bits and move them to the right  (2)
8   int lower = (b & 0x0F) << 4; // keep the low 4 bits and move them to the left    (3)
9   return upper | lower; // glue them back together                              (4)
10  }
11
12  static String bits(int v) {
13  String s = Integer.toBinaryString(v & 0xFF);                                  (5)
14  return "0".repeat(8 - s.length()) + s;
15  }
16
17  public static void main(String[] args) {
18  Scanner sc = new Scanner(System.in);
19  int value = sc.nextInt();
20  int swapped = swapNibbles(value);
21  System.out.println("original: " + value + " = " + bits(value));
22  System.out.println("swapped : " + swapped + " = " + bits(swapped));
23  }
24 }
```

Works on an int holding 0..255, which avoids all the signed-byte surprises.

Sample input

```
172
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Keep exactly 8 bits so the value is 0–255 regardless of the input's sign.
2	line 7	Isolate the high nibble and slide it to the low position.
3	line 8	Isolate the low nibble and slide it to the high position.
4	line 9	OR combines the two non-overlapping pieces.
5	line 13	Pad with leading zeros so all 8 bits are shown.

NibbleSwapByte.java — Swap nibbles with the byte type (lecture version) (from the slides)

```
1  public class NibbleSwapByte {
2   // The lecture version, using the Java type byte (signed!). Casting back to byte needs
        care.
3   public static byte swapNibbles(byte b) {
4   byte upperNibble = (byte) ((b & 0xF0) >>> 4); // b is promoted to int; & 0xF0 removes
        sign-extension bits
5   byte lowerNibble = (byte) (b & 0x0F);
6   return (byte) ((lowerNibble << 4) | upperNibble);
7   }
8
9   public static void main(String[] args) {
10  byte byteValue = (byte) 0xAB; // 1010 1011
11  byte swappedByte = swapNibbles(byteValue);
12  System.out.println("Original byte: " + Integer.toBinaryString(byteValue & 0xFF));
13  System.out.println("Swapped byte: " + Integer.toBinaryString(swappedByte & 0xFF));
14  System.out.println("as hex: " + Integer.toHexString(byteValue & 0xFF) + " -> " +
        Integer.toHexString(swappedByte & 0xFF));
15   }
16  }
```

(1) (2)

Uses (byte) casts. Correct, but see the notes about sign extension.

Line-by-line walkthrough

#	Where	What happens
1	line 4	b is promoted to int; the mask clears the sign-extension bits.
2	line 12	Mask when printing, otherwise a negative byte prints as a 32-bit pattern with many leading 1s.

Dry run: watch it execute

172 = 1010 1100 → 1100 1010 = 202.

Algorithm: Swap nibbles. **Input:** n = `172`.

Pseudocode

```
b = value & 0xFF;
upper = (b & 0xF0) >>> 4;
lower = (b & 0x0F) << 4;
return upper | lower;
```

Step 1 of 4. b = 172 = 10101100

b = 172 · bits = 10101100

Step 2 of 4. b & 0xF0 keeps the high nibble 10100000; >>> 4 moves it to 00001010.

b = 172 · upper = 10 · upper bits = 00001010

Step 3 of 4. b & 0x0F keeps the low nibble 00001100; << 4 moves it to 11000000.

upper = 10 · lower = 192 · lower bits = 11000000

Step 4 of 4. upper | lower = 11001010 = 202.

result = 202 · bits = 11001010

> **RESULT**
> 10101100 → 11001010 (172 → 202)

Complexity

A fixed number of operations on a fixed-size value: **O(1) time, O(1) space**. (For an n-bit word the same pattern still takes O(1) word operations.)

Common mistakes

Using >> instead of >>> on a negative byte

Problem

```
byte b = (byte) 0xAB;
int high = (b & 0xF0) >> 4;
```

Why: Here the mask already removes the sign, so it happens to work — but without the mask `b >> 4` copies the sign bit and gives 0xFFFFFFFA. Always mask first.

Fix: int high = (b & 0xF0) >>> 4;

Printing a byte directly

Problem

```
System.out.println(Integer.toBinaryString(b)); // b = (byte) 0xAB
```

Why: b is sign-extended to 32 bits: prints 11111111111111111111111110101011.

Fix: System.out.println(Integer.toBinaryString(b & 0xFF));

Forgetting that << can overflow a byte

Problem

```
byte low = (byte) (b & 0x0F);
byte r = low << 4; // compile error: int to byte
```

Why: Arithmetic on bytes yields int; assigning back needs a cast.

Fix: byte r = (byte) (low << 4);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Swap the nibbles — *Beginner*

Read an integer 0 ≤ x ≤ 255 and print the value obtained by swapping its two 4-bit halves.

Example input	Expected output
171	186

Hint 1: High nibble: (x & 0xF0) >>> 4 **Hint 2:** Low nibble: (x & 0x0F) << 4

Exercise 2 · Swap adjacent bits — *Intermediate*

Read an unsigned 32-bit integer and swap bit 0 with bit 1, bit 2 with bit 3, and so on. Print the result as an unsigned number.

Example input	Expected output
23	43

Hint 1: Even-position bits: x & 0x55555555; odd-position bits: x & 0xAAAAAAAA. **Hint 2:** Shift evens left by 1 and odds right by 1 (use >>>). **Hint 3:** Print `result & 0xFFFFFFFFL` to see it unsigned.

Exercise 3 · Reverse 32 bits — *Challenge*

Read an unsigned 32-bit integer and print the integer whose binary form is the original's 32-bit pattern reversed.

Example input	Expected output
43261596	964176192

Hint 1: Loop 32 times: shift result left, OR in the lowest bit of x, shift x right with >>>. **Hint 2:** Print as unsigned (long).

Quick check

Answers are in the answer key at the end of this part.

Q1. How many bits are in a nibble?

- **A.** 2
- **B.** 4
- **C.** 8
- **D.** 16

Q2. What does this print (decimal)?

```
int b = 0x3C;
System.out.println(((b & 0xF0) >>> 4) | ((b & 0x0F) << 4));
```

Your answer: _____

Q3. Which mask keeps only the high nibble of a byte?

- **A.** 0x0F
- **B.** 0xF0
- **C.** 0xFF
- **D.** 0x80

Q4. Why write `b & 0xFF` for a Java byte b?

- **A.** To make it faster
- **B.** To remove sign extension and get 0–255
- **C.** To convert to hexadecimal
- **D.** To make it a char

Interview / exam questions

- How would you swap the two halves of a 32-bit int?
- What is the difference between >> and >>>?
- How many hex digits does a byte have?

Watch & read

Video: Swap two nibbles in a byte | GFG POTD | Bit Manipulation | CPP | AlgoBazaar — AlgoBazaar · 9:06. *Swapping the two nibbles of a byte (C++ — the idea transfers to Java)*

Video: Bitwise Operators in C (Part 2) — Neso Academy · 4:59. *Masks and shifts (C — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- byte = 8 bits = 2 nibbles; one nibble = one hex digit.
- swap = ((b & 0xF0) >>> 4) | ((b & 0x0F) << 4)
- Mask with & 0xFF before shifting a Java byte.
- O(1).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide's English steps are written as Python (`swap_nibbles(byte)`, `bin()`) but the program on the next slide is Java — the Java is what this lesson explains. In the example, the slide moves from `1010 1100` (0xAC) to `1100 1010` (0xCA), while its code demo uses 0xAB → 0xBA; both are consistent.

5.11 Longest Sequence of 1s (and after flipping a bit)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 45 min · complete path

Find the longest run of consecutive 1 bits, then the longest run you can create by flipping a single 0.

YOU WILL BE ABLE TO

- Count the longest run of 1s with a scan using `count` and `maxCount`.
- Handle the "flip one 0" variant by remembering the run before the last 0.
- Do it on 32-bit ints without strings and handle negative numbers and the all-ones case.
- Verify a bit trick against brute force.

Before this lesson: Binary Palindrome · Bitwise Operators and Shifts · Strings, StringBuilder and Characters

The idea

Look at a binary number as a string of 0s and 1s and find the longest block of consecutive 1s. `1101110111111001` has runs 11, 111, 111111, 1 — the longest is **6**.

```
1 1 0 1 1 1 0 1 1 1 1 1 1 0 0 1
└┘ └──┘ └─────────┘ └ 1
 2 3 6
```

Now you may flip exactly one 0 to a 1. Flipping the 0 between `111` and `111111` joins both runs: 3 + 1 + 6 = **10**. In general a flipped 0 can join the run on its left, the run on its right, and itself.

WHY THIS WORKS

Scan the bits once, always remembering two things: the length of the run of 1s you are in now (`current`) and the length of the run just before the last 0 (`previous`). When you stand on a 0, flipping it would give `previous + 1 + current` — but only if that 0 is a single 0 (if two zeros in a row precede, `previous` is not adjacent). So one pass with two counters is enough.

Syntax

call / idiom	result
`Integer.toBinaryString(x)`	the bits without leading zeros
`Integer.bitCount(x)`	number of 1 bits
`Integer.highestOneBit(x)`	only the top set bit
`Integer.numberOfTrailingZeros(x)`	index of the lowest set bit
`x & 1`	the lowest bit
`x >>= 1` / `x >>> 1`	drop the lowest bit (arithmetic / logical)
`x & (x - 1)`	clear the lowest set bit
`x | (x << 1)`	every run of 1s grows by one position

A tiny example

```
int current = 0, best = 0;
for (int i = 0; i < 32; i++) {
 if ((x & 1) == 1) { current++; best = Math.max(best, current); }
 else current = 0;
 x >>>= 1;
}
```

Guided example

LongestOnes.java — Longest run of 1s (simple solution)

```
1  import java.util.Scanner;
2
3  public class LongestOnes {
4   // Longest run of consecutive 1 bits, scanning the 32 bits with shifts (no String
        needed).
5   static int longestRun(int x) {
6   int current = 0;                                                          (4)
7   int best = 0;
8   for (int i = 0; i < 32; i++) {                                            (1)
9   if ((x & 1) == 1) {                                                       (2)
10  current++;
11  best = Math.max(best, current);                                           (3)
12  } else {
13  current = 0; // a 0 ends the run
14  }
15  x >>>= 1; // unsigned shift so negatives are handled                      (5)
16  }
17  return best;
18  }
19
20  public static void main(String[] args) {
21  Scanner sc = new Scanner(System.in);
22  int x = sc.nextInt();
23  System.out.println(Integer.toBinaryString(x) + " -> longest run of 1s = " +
        longestRun(x));
24  }
25 }
```

Examines all 32 bits with shifts; no String needed, negative numbers work.

Sample input

```
56825
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	An int has exactly 32 bits, so 32 iterations cover every bit including leading zeros.
2	line 9	The lowest bit is 1: the current run grows.
3	line 11	Keep the best run seen so far.
4	line 6	A 0 ends the run.
5	line 15	Unsigned shift so the sign bit does not repeat for negatives.

FlipOneBit.java — Longest run after flipping one bit (optimised solution)

```
1  import java.util.Scanner;
2
3  public class FlipOneBit {
4   // Longest run of 1s you can get by flipping at most one 0 to 1 (32-bit int; negatives
        handled).
5   // Track the length of the current run and the run before the last 0.
6   static int longestAfterFlip(int x) {
7   if (x == -1) {                                                                    (1)
8   return 32; // already all ones: nothing to flip
9   }
10  int current = 0; // run of 1s ending at this bit
11  int previous = 0; // run of 1s just before the most recent 0
12  int best = 1; // flipping any 0 gives at least 1
13  for (int i = 0; i < 32; i++) {
14  if ((x & 1) == 1) {
15  current++;
16  } else {
17  previous = ((x & 2) == 0) ? 0 : current; // two zeros in a row: the flipped 0 cannot   (2)
        join two runs
18  current = 0;
19  }
20  best = Math.max(best, previous + current + 1);                                    (3)
21  x >>>= 1;
22  }
23  return Math.min(best, 32);                                                        (4)
24  }
25
26  public static void main(String[] args) {
27  Scanner sc = new Scanner(System.in);
28  int x = sc.nextInt();
29  System.out.println(Integer.toBinaryString(x) + " -> longest run after flipping one bit
        = " + longestAfterFlip(x));
30  }
31 }
```

Tracks `current` and `previous`; the candidate at every position is previous + current + 1.

Sample input

```
478
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	All 32 bits are already 1: there is nothing to flip, the answer is 32 (not 33).
2	line 17	On a 0: if the NEXT bit is also 0, flipping this 0 cannot connect to the earlier run (the other 0 is in the way), so previous restarts at 0; otherwise the run we just finished becomes `previous`.
3	line 20	Left run + the flipped bit + right run (the run we are in).
4	line 23	A 32-bit int cannot hold a longer run.

FlipCheck.java — Brute-force verification (demonstration)

```
public class FlipCheck {
 static int fast(int x) {
 if (x == -1) return 32;
 int current = 0, previous = 0, best = 1;
 for (int i = 0; i < 32; i++) {
 if ((x & 1) == 1) {
 current++;
 } else {
 previous = ((x & 2) == 0) ? 0 : current;
 current = 0;
 }
 best = Math.max(best, previous + current + 1);
 x >>>= 1;
 }
 return Math.min(best, 32);
 }

 // Try flipping each of the 32 bits and measure the longest run directly.
 static int brute(int x) {
 int best = 0;
 for (int bit = 0; bit < 32; bit++) {
 int y = x | (1 << bit);
 int run = 0, longest = 0;
 for (int i = 0; i < 32; i++) {
 if (((y >>> i) & 1) == 1) {
 run++;
 longest = Math.max(longest, run);
 } else run = 0;
 }
 best = Math.max(best, longest);
 }
 return best;
 }

 public static void main(String[] args) {
 java.util.Random rnd = new java.util.Random(7);
 int[] special = {0, 1, -1, 15, Integer.MIN_VALUE, Integer.MAX_VALUE, 0b111011110,
     0b1101110111111001};
 for (int x : special) {
 if (fast(x) != brute(x)) { System.out.println("MISMATCH " + x); return; }
 }
 for (int t = 0; t < 200000; t++) {
 int x = rnd.nextInt();
 if (fast(x) != brute(x)) { System.out.println("MISMATCH " + x); return; }
 }
 System.out.println("flip-one-bit matches brute force on 200000 random ints and the
     special cases");
 }
}
```

Tries flipping every one of the 32 bits and measures the run directly, comparing with the fast version.

Line-by-line walkthrough

#	Where	What happens
1	line 22	Force bit number `bit` to 1 (flipping a 0, or leaving a 1 unchanged).
2	line 42	200 000 random 32-bit integers, including negatives.

Dry run: watch it execute

478 = 111011110: flipping the 0 in the middle gives 3 + 1 + 4 = 8.

Algorithm: Longest run of 1s / flip one bit. **Input:** n = `478`.

Pseudocode

```
for each bit (low to high):
 if bit == 1: current++
 else: previous = current; current = 0
 best = max(best, previous + current + 1)
```

Step 1 of 11. 478 in binary is 111011110. We scan from the lowest bit (index 0) upwards, including one leading 0.

current = 0 · previous = 0 · best = 1

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 2 of 11. bit 0 is 0: previous = 0, current resets to 0.

current = 0 · previous = 0 · best = 1 · longest plain run = 0

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 3 of 11. bit 1 is 1: the current run grows to 1.

current = 1 · previous = 0 · best = 2 · longest plain run = 1

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 4 of 11. bit 2 is 1: the current run grows to 2.

current = 2 · previous = 0 · best = 3 · longest plain run = 2

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 5 of 11. bit 3 is 1: the current run grows to 3.

current = 3 · previous = 0 · best = 4 · longest plain run = 3

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 6 of 11. bit 4 is 1: the current run grows to 4.

current = 4 · previous = 0 · best = 5 · longest plain run = 4

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 7 of 11. bit 5 is 0: previous = 4, current resets to 0.

current = 0 · previous = 4 · best = 5 · longest plain run = 4

bits (index 0 = least significant)

0	1	1	1	1	0	1	1	1	0
0	1	2	3	4	5	6	7	8	9

Step 8 of 11. bit 6 is 1: the current run grows to 1.

current = 1 · previous = 4 · best = 6 · longest plain run = 4

bits (index 0 = least significant)

0	1	2	3	4	5	6	7	8	9
0	1	1	1	1	0	1	1	1	0

Step 9 of 11. bit 7 is 1: the current run grows to 2.

current = 2 · previous = 4 · best = 7 · longest plain run = 4

bits (index 0 = least significant)

0	1	2	3	4	5	6	7	8	9
0	1	1	1	1	0	1	1	1	0

Step 10 of 11. bit 8 is 1: the current run grows to 3.

current = 3 · previous = 4 · best = 8 · longest plain run = 4

bits (index 0 = least significant)

0	1	2	3	4	5	6	7	8	9
0	1	1	1	1	0	1	1	1	0

Step 11 of 11. bit 9 is 0: previous = 0, current resets to 0.

current = 0 · previous = 0 · best = 8 · longest plain run = 4

bits (index 0 = least significant)

0	1	2	3	4	5	6	7	8	9
0	1	1	1	1	0	1	1	1	0

> **RESULT**
> longest run of 1s = 4; after flipping one bit = 8

Complexity

Both programs look at each of the 32 bits once: **O(w)** where w = 32 is the word size (equivalently O(log n) for the number's value). Space O(1).

The String-based version on the slides is also O(log n) time but uses O(log n) memory and breaks for negative numbers, where `Integer.toBinaryString(-1)` has 32 ones and the +1 gives 33.

Common mistakes

Wrong decimal value in the slide example

Problem

```
0b1101110111111001 // "87769" and answer "5"
```

Why: 0b1101110111111001 is 56 825, and its longest run of 1s is 6, not 5.

Fix: Always verify examples by running them (java LongestOnes with 56825 prints 6).

Using toBinaryString on a negative number

Problem

```
Integer.toBinaryString(-1).length() // 32
```

Why: Works but then "flip one bit" returns 33 for -1; the result must be capped at the 32 bits.

Fix: if (x == -1) return 32;

Resetting current without saving previous

Problem

```
else { current = 0; } // flip variant
```

Why: You lose the run to the left of the zero; the join length cannot be computed.

Fix: previous = current; current = 0;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Longest run of 1s — *Beginner*

Read an int and print the length of the longest block of consecutive 1 bits in its 32-bit two's-complement representation.

Example input	Expected output
56825	6

Hint 1: Scan 32 bits with `x & 1` and `x >>>= 1`.

Exercise 2 · Flip one bit — *Intermediate*

Read an int. You may flip exactly one bit (0 → 1 or leave it). Print the longest run of 1s you can obtain among the 32 bits.

Example input	Expected output
478	8

Hint 1: Brute force is fine: try each of the 32 bits. **Hint 2:** Or track current and previous runs in one pass.

Exercise 3 · Flip up to k zeros — *Challenge*

Read an int x and k. Print the longest run of 1s in the 32-bit representation after flipping at most k zeros to ones.

- 0 ≤ k ≤ 32

Example input	Expected output
478 2	9

Hint 1: Sliding window over the 32 bits: keep the number of zeros inside the window ≤ k. **Hint 2:** Move the left end forward whenever the window has too many zeros.

Quick check

Answers are in the answer key at the end of this part.

Q1. What does this print?

```
int x = 0b0110_1110;
int cur = 0, best = 0;
for (int i = 0; i < 8; i++) {
 if ((x & 1) == 1) best = Math.max(best, ++cur);
 else cur = 0;
 x >>>= 1;
}
System.out.println(best);
```

Your answer: _____

Q2. In the flip variant, which value is the candidate on a 0?

- **A.** current
- **B.** previous
- **C.** previous + current + 1
- **D.** current × 2

Q3. The longest run of 1s after flipping one bit in -1 (all ones) is...

- **A.** 31
- **B.** 32
- **C.** 33
- **D.** 1

Q4. What is the time complexity of the bit scan?

- **A.** O(1)
- **B.** O(log n) (32 steps)
- **C.** O(n)
- **D.** O(n²)

Interview / exam questions

- Flip one bit to get the longest sequence of 1s — outline the O(w) solution.
- Why do you need `previous` in addition to `current`?
- What changes if you may flip k bits?

Watch & read

Video: Maximizing Contiguous Ones: Find the Longest Sequence in a Binary Array with a Single Flip | Medium — Tutorial Horizon · 12:00. *Longest run of 1s after one flip*

Video: L4. Max Consecutive Ones III | 2 Pointers and Sliding Window Playlist — take U forward · 29:58. *The sliding-window view*

REVISION CARD (30–60 SECONDS)

• Scan bits: current++ on 1, reset on 0, best = max.
• Flip variant: candidate = previous + current + 1 at each 0; previous = current (unless next bit is 0 too).
• All ones (−1) → 32.
• O(32) time, O(1) space.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide states that 0b1101110111111001 equals 87769 and that its longest run is 5. It equals 56825 and the longest run is 6. • The slide's Longafterflip code uses Integer.toBinaryString and char loops; its last line `maxLength = max(maxLength, currentLength + previousLength + 1)` yields 33 for -1. This lesson caps the answer at 32.

5.12 Josephus Problem

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · ★ on the minimum-viable path

People stand in a circle and every k-th one is removed. Who is left? A recursion that becomes a three-line loop.

YOU WILL BE ABLE TO

- Simulate the elimination with a list and print the order of elimination.
- Derive the recurrence J(n) = (J(n−1) + k) mod n and explain each term.
- Implement the recursive and iterative O(n) solutions and convert between 0-based and 1-based positions.
- Use the k = 2 shortcut 2·(n − 2^m) + 1.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Divisibility, GCD/LCM, Primes and Modular Arithmetic · Generics and Lists: ArrayList and LinkedList

The idea

N people stand in a circle, numbered 1..N. Starting at person 1, count k people; the k-th person leaves the circle. Counting continues from the next person. The last person left is the survivor. For N = 7, k = 3 the order of leaving is 3, 6, 2, 7, 5, 1 and person **4** survives.

```
N = 5, k = 2 1 2 3 4 5 start counting at 1
 kill 2 → 1 _ 3 4 5 next count starts at 3
 kill 4 → 1 _ 3 _ 5 next count starts at 5
 kill 1 → _ _ 3 _ 5 next count starts at 3
 kill 5 → _ _ 3 _ _ survivor: 3
```

WHY THIS WORKS

Suppose you know the answer for a circle of n−1 people, counted from the person right after the first victim. The only difference between that circle and the real circle of n people is that positions are shifted: the first victim stood at position k (0-based k−1), and the new count starts k places further round. Translating a position p of the small circle back to the big circle therefore adds k and wraps around n: **J(n) = (J(n−1) + k) mod n** (0-based). With 1 person the survivor is position 0, which seeds the loop.

Syntax

The `%` remainder operator wraps a position around a circle: `(index + k) % size` always lands inside 0..size−1. Lists (`ArrayList<Integer>`) support `remove(index)`, which shifts the later elements left.

A tiny example

```
int ans = 0; // 1 person: position 0
for (int people = 2; people <= n; people++)
 ans = (ans + k) % people;
return ans + 1; // 1-based
```

Guided example

Check the formula against the slide examples by running the recursive program: (5, 2) → 3, (7, 3) → 4, (6, 2) → 5, (14, 2) → 13.

JosephusSimulation.java — Josephus by simulation (simple solution)

```
1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.Scanner;
4
5  public class JosephusSimulation {
6   // Literally act out the circle: remove every k-th person until one is left.
7   static int survivor(int n, int k, List<Integer> order) {
8   List<Integer> circle = new ArrayList<>();
9   for (int person = 1; person <= n; person++) {
10  circle.add(person);
11  }
12  int index = 0; // where counting starts
13  while (circle.size() > 1) {
14  index = (index + k - 1) % circle.size(); // skip k-1 people, land on the k-th          (1)
15  order.add(circle.remove(index)); // that person is out; the next count starts at the  (2)
        same index
16  }
17  return circle.get(0);                                                                 (3)
18  }
19
20  public static void main(String[] args) {
21  Scanner sc = new Scanner(System.in);
22  int n = sc.nextInt();
23  int k = sc.nextInt();
24  List<Integer> order = new ArrayList<>();
25  int last = survivor(n, k, order);
26  System.out.println("eliminated in order: " + order);
27  System.out.println("survivor: " + last);
28  }
29 }
```

Keeps the circle as a list and removes the k-th person repeatedly. O(n²) because removal from the middle shifts elements.

Sample input

```
7 3
```

Line-by-line walkthrough

#	Where	What happens
1	line 14	Skip k−1 people (counting the current one as 1) and wrap with %.
2	line 15	Remove the victim. The person who was after them slides into `index`, which is exactly where the next count starts.
3	line 17	One person remains.

JosephusRecursive.java — Josephus recursively (optimised solution)

```
1  public class JosephusRecursive {
2   // survivor's position (1-based) among n people when every k-th is eliminated.
3   // Idea: after the first elimination there are n-1 people and counting restarts right
        after the
4   // eliminated person. Solve that smaller circle, then translate its answer back to the
        n-circle.
5   static int josephus(int n, int k) {
6   if (n == 1) {                                                                    (1)
7   return 1; // one person left: they survive
8   }
9   return (josephus(n - 1, k) + k - 1) % n + 1; // shift by k places and wrap around the   (3)
        circle of n
10  }
11
12  public static void main(String[] args) {
13  int[][] cases = {{5, 2}, {7, 3}, {6, 2}, {14, 2}};
14  for (int[] c : cases) {
15  System.out.println("josephus(" + c[0] + ", " + c[1] + ") = " + josephus(c[0], c[1]));
16  }
17  }
18 }
```

J(1) = 1; J(n) = (J(n−1) + k − 1) mod n + 1 in 1-based form.

Line-by-line walkthrough

#	Where	What happens
1	line 6	Base case: one person left.
2	line 9	Solve the smaller circle first.
3	line 9	Convert to 0-based (−1), add k, wrap in n people, convert back (+1). The slide folds the "−1 … +1" into this expression.

JosephusIterative.java — Josephus iteratively (optimised solution)

```
1  import java.util.Scanner;
2
3  public class JosephusIterative {
4   // The same recurrence without recursion: build the answer for 1, 2, 3, ... n people
        (0-based positions).
5   static int josephus(int n, int k) {
6   int ans = 0; // with 1 person the survivor is at position 0                      (1)
7   for (int people = 2; people <= n; people++) {
8   ans = (ans + k) % people;                                                       (2)
9   }
10  return ans + 1; // convert to 1-based                                           (3)
11  }
12
13  public static void main(String[] args) {
14  Scanner sc = new Scanner(System.in);
15  int n = sc.nextInt();
16  int k = sc.nextInt();
17  System.out.println(josephus(n, k));
18  }
19 }
```

Same recurrence, built bottom-up: O(n) time, O(1) space, no recursion depth problem.

Sample input

```
14 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	With one person the survivor is at (0-based) position 0.
2	line 8	Grow the circle by one person: shift by k and wrap.
3	line 10	Convert to 1-based numbering.

JosephusPowerOfTwo.java — Josephus for k = 2 (optimised solution)

```
1  import java.util.Scanner;
2
3  public class JosephusPowerOfTwo {
4   // Special case k = 2: write n = 2^m + r (largest power of two not above n). Answer =
        2r + 1.
5   static long josephusK2(long n) {
6   long highestPowerOfTwo = Long.highestOneBit(n);                    (1)
7   return 2 * (n - highestPowerOfTwo) + 1;                            (2)
8   }
9
10  public static void main(String[] args) {
11  Scanner sc = new Scanner(System.in);
12  long n = sc.nextLong();
13  System.out.println(josephusK2(n));
14  }
15 }
```

Write n = 2^m + r; the survivor is 2r + 1. O(1).

Sample input

```
14
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	The largest power of two not above n (e.g. 14 → 8).
2	line 7	After r = n − 2^m eliminations the circle has exactly 2^m people and the next person to count is 2r + 1 — and with k = 2 and a power of two people, the first person always survives.

Dry run: watch it execute

Watch the circle shrink as every 3rd person leaves.

Algorithm: Josephus problem. **Input:** n = `7`, k = `3`.

Pseudocode

```
circle = [1..n]; index = 0
while (circle.size() > 1) {
 index = (index + k - 1) % size
 remove circle[index]
}
survivor = circle[0]
```

Step 1 of 8. 7 people stand in a circle; counting starts at person 1.

index = 0 · size = 7

circle

1	2	3	4	5	6	7
0	1	2	3	4	5	6

Step 2 of 8. Count 3: person 3 leaves. Counting restarts at 4.

eliminated = 3 · index = 2 · size = 6

circle

1	2	4	5	6	7
0	1	2	3	4	5

eliminated so far

3
0

Step 3 of 8. Count 3: person 6 leaves. Counting restarts at 7.

eliminated = 6 · index = 4 · size = 5

circle

1	2	4	5	7
0	1	2	3	4

eliminated so far

3	6
0	1

Step 4 of 8. Count 3: person 2 leaves. Counting restarts at 4.

eliminated = 2 · index = 1 · size = 4

circle

1	4	5	7
0	1	2	3

eliminated so far

3	6	2
0	1	2

Step 5 of 8. Count 3: person 7 leaves. Counting restarts at 1.

eliminated = 7 · index = 0 · size = 3

circle

1	4	5
0	1	2

eliminated so far

3	6	2	7
0	1	2	3

Step 6 of 8. Count 3: person 5 leaves. Counting restarts at 1.

eliminated = 5 · index = 0 · size = 2

circle

1	4
0	1

eliminated so far

3	6	2	7	5
0	1	2	3	4

Step 7 of 8. Count 3: person 1 leaves. Counting restarts at 4.

eliminated = 1 · index = 0 · size = 1

circle

4
0

eliminated so far

3	6	2	7	5	1
0	1	2	3	4	5

Step 8 of 8. Only 4 is left: the survivor.

survivor = 4

circle

4
0

> **RESULT**
> survivor = 4 (order out: 3 6 2 7 5 1)

Complexity

Simulation: each of the n−1 removals costs O(n) (list shift) → O(n²). With a linked list or a Fenwick tree this improves but is still not simple.

Recurrence: n steps of O(1) → **O(n) time**. Recursive form uses O(n) stack space; the loop uses O(1).

k = 2: O(1) with a bit operation (O(log n) without highestOneBit).

> **NOTE**
> For huge n and small k there is an O(k log n) algorithm; for k = 2 it collapses to the bit trick above.

Common mistakes

Mixing 0-based and 1-based positions

Problem

```
return (josephus(n - 1, k) + k) % n; // returns 0..n-1 but the problem is 1-based
```

Why: The answer is off by one. Decide which indexing you use and convert once, at the end (or use the slide's +k−1 ... +1 form).

Fix: return (josephus(n - 1, k) + k - 1) % n + 1;

Skipping k instead of k − 1

Problem

```
index = (index + k) % circle.size(); // simulation
```

Why: The current person counts as the first; the k-th person is k−1 steps further. Using k kills the wrong person.

Fix: index = (index + k - 1) % circle.size();

Stack overflow on large n

Problem

```
josephus(1_000_000, 2) // recursive
```

Why: One stack frame per person: about 10^5–10^6 deep recursion throws StackOverflowError.

Fix: Use the iterative loop (same recurrence).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Elimination order — *Beginner*

Read N and k. Print the numbers in the order in which people are removed; the survivor is printed last.

Example input	Expected output
5 2	2 4 1 5 3

Hint 1: Keep an ArrayList 1..N and an index. **Hint 2:** index = (index + k − 1) % size, then remove(index).

Exercise 2 · Survivor for large N — *Intermediate*

Read N and k and print the survivor's position (1-based).

- $1 \le N \le 10^7$
- $1 \le k \le 10^9$

Example input	Expected output
10 3	4

Hint 1: Simulation is too slow for N = 10^7 — use the recurrence. **Hint 2:** Use long for (ans + k).

Exercise 3 · Josephus with k = 2 and N up to 10^{18} — *Challenge*

Every second person is removed (k = 2). Read N and print the survivor.

- $1 \le N \le 10^{18}$

Example input	Expected output
100	73

Hint 1: Write N = 2^m + r. What happens after r eliminations? **Hint 2:** Long.highestOneBit gives 2^m.

Quick check

Answers are in the answer key at the end of this part.

Q1. For N = 6 and k = 2, who survives?

- **A.** 1
- **B.** 3
- **C.** 5
- **D.** 6

Q2. What does this print?

```
int ans = 0;
for (int people = 2; people <= 5; people++)
 ans = (ans + 2) % people;
System.out.println(ans + 1);
```

Your answer: _____

Q3. What is the time complexity of the iterative Josephus recurrence?

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n^2)

Q4. Complete the recurrence for 0-based positions.

```
ans = (ans ______ k) % people;
```

Q5. For k = 2 and N = 20 (= 16 + 4) the survivor is...

- **A.** 9
- **B.** 17
- **C.** 8
- **D.** 4

Interview / exam questions

- Derive J(n) = (J(n−1) + k) mod n.
- How would you print the elimination order efficiently for N = 10^5?
- Is there a closed form for k = 2?
- What data structure speeds up the simulation?

Watch & read

Video: The Josephus Problem - Numberphile — Numberphile · 13:58. *The Josephus problem*

Video: Josephus Problem | GeeksforGeeks — GeeksforGeeks · 12:13. *The Josephus recurrence*

REVISION CARD (30–60 SECONDS)

- J(1) = 0; J(n) = (J(n−1) + k) mod n (0-based). Add 1 for 1-based.
- Simulation: index = (index + k − 1) % size; remove.
- k = 2: n = 2^m + r → 2r + 1.
- Iterative O(n) time, O(1) space; recursion depth n.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide's explanatory text says "recursively delete the Kth position until only one person is left", but its code does not delete anything: it uses the arithmetic recurrence. The slide diagram (steps 01–06) uses start = (start + k) % size with k = 1 in 0-based form, which is the same idea as the recurrence run forwards.

5.13 Block Swap Algorithm (Array Rotation)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · ★ on the minimum-viable path

Rotate an array left by d positions in O(n) time and O(1) extra space by swapping blocks.

YOU WILL BE ABLE TO

- Define left rotation by d and compute it by hand.
- Implement the O(n·d) one-by-one rotation and the O(n) temp-array version.
- Explain and trace the block swap algorithm that works in place.
- Compare it with the three-reversal trick and verify all variants by testing.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading

The idea

Rotating `{1,2,3,4,5,6,7}` left by 2 moves the first two elements to the end: `{3,4,5,6,7,1,2}`. Split the array into **A** = the first d elements and **B** = the rest. Rotation turns **AB** into **BA**.

```
d = 2: [ 1 2 | 3 4 5 6 7 ] A = [1 2] B = [3 4 5 6 7]
 A shorter than B → swap A with the LAST |A| elements of B:
 [ 6 7 | 3 4 5 | 1 2 ] → A is now in its final place at the end
 remaining: [6 7 | 3 4 5] , swap again:
 [ 3 4 | 6 7 | 5 ] ...
```

WHY THIS WORKS

Block swap is "Euclid's algorithm on blocks". If A is shorter than B, swapping A with the last |A| elements of B puts A in its final position (at the end) and leaves a smaller problem: rotate the front part `A_new B_front`. If A is longer, swapping B with the first |B| elements of A puts B at the front, and the remaining part `A_rest …` is a smaller rotation. Each step fixes some elements permanently and shrinks the problem like the subtraction form of Euclid's GCD, until both blocks are equal and one swap finishes.

Alternative: **reversal**. Reverse A, reverse B, then reverse everything: (A^r B^r)^r = B A. Three passes, 2 loops of swaps, trivial to code.

Syntax

```
int t = a[i]; a[i] = a[j]; a[j] = t; // swap two cells
System.arraycopy(a, from, b, to, len); // block copy (handles overlap)
int[] part = Arrays.copyOfRange(a, from, toExclusive); // new array
Collections.rotate(list, -d); // library rotation of a List by -d (left d)
static void reverse(int[] a, int lo, int hi) { // reverse a[lo..hi] in place
 while (lo < hi) { int t = a[lo]; a[lo++] = a[hi]; a[hi--] = t; }
}
```

A tiny example

```
int first = a[0];
for (int i = 0; i < n - 1; i++) a[i] = a[i + 1];
a[n - 1] = first; // rotate left by 1: O(n)
```

Guided example

Rotating by d with the tiny loop d times costs d·n moves. The block swap moves each element at most once or twice.

RotateOneByOne.java — Rotate one position at a time (simple solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class RotateOneByOne {
5   // Rotate left by one position, d times. Easy to understand, but d * n element moves.
6   static void leftRotate(int[] a, int d) {
7   int n = a.length;
8   d %= n;                                                    (1)
9   for (int step = 0; step < d; step++) {
10  int first = a[0];                                          (2)
11  for (int i = 0; i < n - 1; i++) {
12  a[i] = a[i + 1];                                           (3)
13  }
14  a[n - 1] = first;                                          (4)
15  }
16  }
17
18  public static void main(String[] args) {
19  Scanner sc = new Scanner(System.in);
20  int n = sc.nextInt();
21  int d = sc.nextInt();
22  int[] a = new int[n];
23  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
24  leftRotate(a, d);
25  System.out.println(Arrays.toString(a));
26  }
27 }
```

Left-rotates by 1, d times. O(n·d) time, O(1) space.

Sample input

```
8 2
7 9 8 0 5 1 6 4
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Rotating by n returns the array to the start, so only d mod n matters.
2	line 10	The element that falls off the front must be saved.
3	line 12	Everyone else moves one step left.
4	line 14	The saved element lands at the end.

BlockSwapRotate.java — Block swap rotation (optimised solution)

```java
import java.util.Arrays;
import java.util.Scanner;

public class BlockSwapRotate {
 // swap d elements starting at index fi with d elements starting at index si
 static void swap(int[] a, int fi, int si, int d) {
 for (int i = 0; i < d; i++) {
 int temp = a[fi + i];
 a[fi + i] = a[si + i];
 a[si + i] = temp;
 }
 }

 // Left-rotate a[0..n-1] by d using block swaps. A = a[0..d-1], B = a[d..n-1].
 static void leftRotate(int[] a, int d) {
 int n = a.length;
 if (n == 0) return;
 d %= n;                                                          (1)
 if (d == 0) return;
 int i = d; // size of block A (still to be placed)               (2)
 int j = n - d; // size of block B
 while (i != j) {
 if (i < j) { // A is shorter: swap A with the LAST i elements of B   (3)
 swap(a, d - i, d + j - i, i);                                    (4)
 j -= i; // those i elements of B are now final                   (5)
 } else { // A is longer: swap the LAST j elements of A with B
 swap(a, d - i, d, j);                                            (6)
 i -= j;
 }
 }
 swap(a, d - i, d, i); // equal sizes: one final block swap        (7)
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 leftRotate(a, d);
 System.out.println(Arrays.toString(a));
 }
}
```

A = a[0..d−1], B = a[d..n−1]. Repeatedly swap the shorter block with part of the longer one.

Sample input

```
10 5
6 8 7 9 0 5 1 3 2 4
```

Line-by-line walkthrough

#	Where	What happens
1	line 18	Normalise d (the slide handles d > n the same way).
2	line 20	i = size of A still to be placed; j = size of B. They shrink like the two numbers in Euclid's algorithm.
3	line 23	A is shorter: swap it with the LAST i elements of B. Those elements of B now hold A's old values and are in their final position.
4	line 24	Swap i elements starting at d−i (the end of A) with i elements starting at d+j−i (the tail of B).
5	line 25	The i swapped elements are final, so B effectively gets shorter.
6	line 27	A is longer: swap the last j elements of A with all of B; B is now in its final front position.
7	line 31	When i == j the two blocks are the same size and one swap finishes the job.

RotateReversal.java — Rotation by three reversals (optimised solution)

```
import java.util.Arrays;
import java.util.Scanner;

public class RotateReversal {
 static void reverse(int[] a, int lo, int hi) {
 while (lo < hi) {
 int t = a[lo];
 a[lo++] = a[hi];
 a[hi--] = t;
 }
 }

 // Reverse A, reverse B, then reverse the whole array: (A^r B^r)^r = B A
 static void leftRotate(int[] a, int d) {
 int n = a.length;
 d %= n;
 reverse(a, 0, d - 1);  (1)
 reverse(a, d, n - 1);  (2)
 reverse(a, 0, n - 1);  (3)
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 leftRotate(a, d);
 System.out.println(Arrays.toString(a));
 }
}
```

reverse(A), reverse(B), reverse(whole). Easy to remember, O(n) time, O(1) space.

Sample input

```
7 2
1 2 3 4 5 6 7
```

Line-by-line walkthrough

#	Where	What happens
1	line 17	Reverse the first block A in place.
2	line 18	Reverse the second block B in place.
3	line 19	Reversing everything turns (A^r B^r) into B A.

BlockSwapCheck.java — Exhaustive check (demonstration)

```java
import java.util.Arrays;

public class BlockSwapCheck {
 static void swap(int[] a, int fi, int si, int d) {
 for (int i = 0; i < d; i++) {
 int t = a[fi + i];
 a[fi + i] = a[si + i];
 a[si + i] = t;
 }
 }

 static void blockSwap(int[] a, int d) {
 int n = a.length;
 d %= n;
 if (d == 0) return;
 int i = d, j = n - d;
 while (i != j) {
 if (i < j) {
 swap(a, d - i, d + j - i, i);
 j -= i;
 } else {
 swap(a, d - i, d, j);
 i -= j;
 }
 }
 swap(a, d - i, d, i);
 }

 // Compares block swap with the obvious "rotate through a temp array" for every n <= 40 and every d.
 public static void main(String[] args) {
 int checked = 0;
 for (int n = 1; n <= 40; n++) {
 for (int d = 0; d <= 2 * n; d++) {
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = i + 1;
 int[] expected = new int[n];
 for (int i = 0; i < n; i++) expected[i] = a[(i + d) % n]; (1)
 blockSwap(a, d);
 if (!Arrays.equals(a, expected)) {
 System.out.println("MISMATCH n=" + n + " d=" + d + " got " + Arrays.toString(a));
 return;
 }
 checked++;
 }
 }
 System.out.println("block swap matches the reference for all " + checked + " (n, d) pairs");
 }
}
```

Compares the block swap output with `a[(i + d) % n]` for 1 680 combinations of n and d (including d > n and d = 0).

Line-by-line walkthrough

#	Where	What happens
1	line 37	The definition of a left rotation by d: element i of the result is element (i + d) mod n of the input.

Dry run: watch it execute

The two shrinking blocks i and j are swapped until they are equal.

Algorithm: Block swap rotation. **Input:** arr = `1,2,3,4,5,6,7`, d = `2`.

Pseudocode

```
i = d; j = n − d
while (i != j) {
 if (i < j) { swap(d−i, d+j−i, i); j −= i; }
 else { swap(d−i, d, j); i −= j; }
}
swap(d − i, d, i)
```

Step 1 of 5. Rotate left by 2. Block A = first 2 elements, block B = the other 5.

i = 2 · j = 5 · d = 2

array	1	2	3	4	5	6	7
	0	1	2	3	4	5	6

Step 2 of 5. A (size 2) is shorter: swap it with the last 2 elements of B; B shrinks to 3.

i = 2 · j = 3

array	6	7	3	4	5	1	2
	0	1	2	3	4	5	6

Step 3 of 5. A (size 2) is shorter: swap it with the last 2 elements of B; B shrinks to 1.

i = 2 · j = 1

array	4	5	3	6	7	1	2
	0	1	2	3	4	5	6

Step 4 of 5. A is longer: swap the last 1 elements of A with B; A shrinks to 1.

i = 1 · j = 1

array	3	5	4	6	7	1	2
	0	1	2	3	4	5	6

Step 5 of 5. Blocks are equal (size 1): one final swap.

i = 1 · j = 1

array	3	4	5	6	7	1	2
	0	1	2	3	4	5	6

> **RESULT**
> [3, 4, 5, 6, 7, 1, 2]

Complexity

One-by-one: d passes × n moves → O(n·d) time, O(1) space.

Temp array: copy the first d elements aside, shift the rest, copy back → O(n) time, O(d) space.

Block swap / reversal: every element is swapped O(1) times → **O(n) time, O(1) auxiliary space** (iterative).

> **CHECK THE CLAIM**
> The slide says the recursive version uses O(log n) space. Recursion depth of the subtraction-style block swap can reach about n (try d = 1), so O(log n) is not guaranteed; the iterative loop above avoids the issue entirely.

method	time	extra space
rotate by 1, d times	O(n·d)	O(1)
temp array	O(n)	O(d)
block swap (iterative)	O(n)	O(1)
three reversals	O(n)	O(1)

Common mistakes

Using d without taking d mod n

Problem

```
for (int i = 0; i < n; i++) result[i] = a[i + d];
```

Why: Throws ArrayIndexOutOfBoundsException for i + d ≥ n, and never wraps around.

Fix: result[i] = a[(i + d) % n];

Mixing up left and right rotation

Problem

```
// "rotate by 2" for {1,2,3,4,5} printed as {4,5,1,2,3}
```

Why: That is a RIGHT rotation. Left rotation by 2 gives {3,4,5,1,2}. Always state the direction.

Fix: Right rotation by d == left rotation by n − d.

Slide example 2 typo

Problem

```
// {6,8,7,9,0,5,1,3,2,4}, d = 5 → "{5,1,2,3,4,6,8,7,9,0}"
```

Why: The elements 3 and 2 are in the wrong order in the slide. The correct left rotation is {5,1,3,2,4,6,8,7,9,0} (verified by running the program).

Fix: Trace by hand: take indices 5..9 then 0..4.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Left rotation — *Beginner*

Read n, d and an array of n integers. Print the array after rotating it left by d positions.

- $1 \le n \le 10^5$
- $0 \le d \le 10^9$

Example input	Expected output
5 2 ⏎ 1 2 3 4 5	3 4 5 1 2

Hint 1: Only d mod n matters. **Hint 2:** Output element i is input element (i + d) mod n.

Exercise 2 · Right rotation in place — *Intermediate*

Read n, d and the array. Print the array rotated RIGHT by d, using O(1) extra memory (reversals).

Example input	Expected output
5 2 ⏎ 1 2 3 4 5	4 5 1 2 3

Hint 1: A right rotation by d is a left rotation by n − d. **Hint 2:** Reverse three times.

Exercise 3 · How many rotations? — *Challenge*

A sorted array of distinct integers was rotated left an unknown number of times. Read n and the array and print how many left rotations produced it (the index of the minimum). Use O(log n) time.

Example input	Expected output
6 ⏎ 15 18 2 3 6 12	2

Hint 1: The minimum is the only element smaller than its predecessor. **Hint 2:** Binary search: compare a[mid] with a[hi].

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
int[] a = {1, 2, 3, 4, 5};
int d = 2, n = a.length;
for (int i = 0; i < n; i++)
 System.out.print(a[(i + d) % n] + " ");
```

Your answer: _____

Q2. Which method rotates in O(n) time and O(1) extra space?

- **A.** Rotate by one, d times
- **B.** Copy to a temp array of size n
- **C.** Three reversals
- **D.** Sorting

Q3. Right rotation by 2 of a 7-element array equals left rotation by...

- **A.** 2
- **B.** 5
- **C.** 7
- **D.** 9

Q4. The time complexity of the one-by-one rotation by d is...

- **A.** O(n)
- **B.** O(n + d)
- **C.** O(n·d)
- **D.** O(log n)

Q5. After reversal rotation of {1,2,3,4,5,6} left by 2, what is the 1st element?

```
// reverse(0,1) -> 2 1 3 4 5 6
// reverse(2,5) -> 2 1 6 5 4 3
// reverse(0,5) -> ?
```

Your answer: _____

Interview / exam questions

- Rotate an array in place without extra space — name two ways.
- Why does the reversal method work? (A^r B^r)^r = ?
- What is the juggling algorithm and how does it use gcd(n, d)?
- How would you search a rotated sorted array?

Watch & read

Video: Block swap algorithm for Array rotation || easy explanation — Code with Med · 13:42. *Rotating an array by block swap*

Video: Rotate Array - Leetcode 189 - Python — NeetCode · 8:59. *Reversal algorithm for rotation (Python — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- Left rotate by d: result[i] = a[(i + d) % n]; use d mod n.
- AB → BA. Block swap shrinks like Euclid; three reversals is simplest.
- O(n) time, O(1) space.
- Right by d = left by n − d.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• Example 2 on the slide prints {5,1,2,3,4,6,8,7,9,0}; the correct result is {5,1,3,2,4,6,8,7,9,0}. • The slide says "O(log n)" auxiliary space for the recursive version. The depth can be O(n); the iterative version is O(1). • The slide's `leftRotate` and the helper `swap` appear in a confusing order across two slides; they belong to the same class.

5.14 Leaders in an Array

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 30 min · ★ on the minimum-viable path

An element is a leader if it is greater than everything to its right. Brute force vs a single right-to-left scan.

YOU WILL BE ABLE TO

- State the definition of a leader and find all leaders in a small array by hand.
- Write the O(n²) brute force with a `break`.
- Write the O(n) right-to-left scan and explain why it works.
- Adapt it to non-strict leaders and next-greater-element.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue

The idea

Imagine people standing in a line, all facing right. Someone is a **leader** if they are **taller than everybody standing to their right** — nobody ahead can look over them. In `{14, 20, 3, 6, 5}` the leaders are 20, 6 and 5 (the last person has nobody to their right, so is always a leader).

```
index : 0 1 2 3 4
value : 14 20 3 6 5
right max:20 6 6 5 –
leader? ✗ ✓ ✗ ✓ ✓ → 20, 6, 5
```

WHY THIS WORKS

Looking from the right, the only thing you need to know about "everything to my right" is its **maximum**. Walk from the last element to the first, keeping `maxFromRight`. If the current element is greater than that maximum it is a leader (and becomes the new maximum). One pass, one variable — the O(n²) comparisons collapse because the maximum summarises the whole suffix.

Syntax

```
int max = Integer.MIN_VALUE; // largest value seen so far from the right
List<Integer> leaders = new ArrayList<>();
for (int i = a.length - 1; i >= 0; i--) { // right-to-left loop
 if (a[i] > max) { leaders.add(a[i]); max = a[i]; }
}
Collections.reverse(leaders); // restore left-to-right order
```

A tiny example

```
int max = a[n - 1]; // the last element is always a leader
for (int i = n - 2; i >= 0; i--)
 if (a[i] > max) max = a[i]; // a[i] is a leader
```

Guided example

The lecture slides show both approaches and print the leaders of `{16, 17, 4, 3, 5, 2}`: **17, 5, 2** (left to right). The right-to-left scan naturally finds them in the order 2, 5, 17, so the lesson program stores them and prints in the original order.

LeadersBruteForce.java — Brute force (simple solution)

```
1  import java.util.Scanner;
2
3  public class LeadersBruteForce {
4   // An element is a leader if it is strictly greater than EVERY element to its right.
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   int[] a = new int[n];
9   for (int i = 0; i < n; i++) a[i] = sc.nextInt();
10
11  StringBuilder sb = new StringBuilder();
12  for (int i = 0; i < n; i++) {
13  boolean isLeader = true;
14  for (int j = i + 1; j < n; j++) { // compare with everything on the right   1
15  if (a[i] <= a[j]) {                                                        2
16  isLeader = false;
17  break; // one bigger-or-equal element is enough                           3
18  }
19  }
20  if (isLeader) {
21  sb.append(sb.length() == 0 ? "" : " ").append(a[i]);
22  }
23  }
24  System.out.println(sb);
25  }
26 }
```

For every element, compare it with every element on its right.

Sample input

```
6
16 17 4 3 5 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 14	Only elements to the RIGHT of i matter.
2	line 15	A single element that is greater than or equal disqualifies a[i] ("greater than all" means strictly greater).
3	line 17	No need to look further once a[i] has been disqualified.

LeadersRightScan.java — Right-to-left scan (optimised solution)

```
1  import java.util.Scanner;
2
3  public class LeadersRightScan {
4   // Scan from the right keeping the largest value seen so far. One pass, O(1) extra
        space for the scan.
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   int[] a = new int[n];
9   for (int i = 0; i < n; i++) a[i] = sc.nextInt();
10
11  int[] leaders = new int[n];
12  int count = 0;
13  int maxFromRight = Integer.MIN_VALUE;                                    (1)
14  for (int i = n - 1; i >= 0; i--) {
15  if (a[i] > maxFromRight) { // bigger than everything to its right        (2)
16  leaders[count++] = a[i];
17  maxFromRight = a[i];                                                     (3)
18  }
19  }
20  // leaders were found right-to-left; print them in the original left-to-right order
21  StringBuilder sb = new StringBuilder();
22  for (int i = count - 1; i >= 0; i--) {                                   (4)
23  sb.append(sb.length() == 0 ? "" : " ").append(leaders[i]);
24  }
25  System.out.println(sb);
26  }
27 }
```

One pass from the right with a running maximum. O(n) time.

Sample input

```
6
16 17 4 3 5 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	Nothing to the right of the last element, so any value beats this.
2	line 15	Strictly greater than every element to its right.
3	line 17	The new leader is also the new maximum of the suffix.
4	line 22	Reverse the stored leaders to print them in left-to-right order.

Dry run: watch it execute

Scan right to left; each new maximum is a leader.

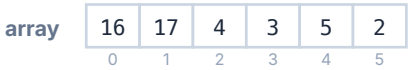
Algorithm: Leaders in an array. **Input:** arr = `16,17,4,3,5,2`.

Pseudocode

```
max = -infinity
for (i = n-1; i >= 0; i--)
 if (a[i] > max) {
 leader: a[i]
 max = a[i]
 }
```

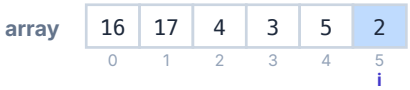
Step 1 of 7. Scan from the right. Nothing is to the right of the last element, so it is always a leader.

max = −∞ · leaders =



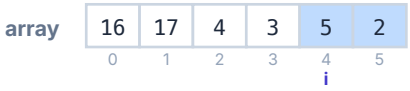
Step 2 of 7. 2 > max: leader! max becomes 2.

i = 5 · max = 2 · leaders = 2



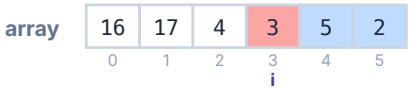
Step 3 of 7. 5 > max: leader! max becomes 5.

i = 4 · max = 5 · leaders = 5 2



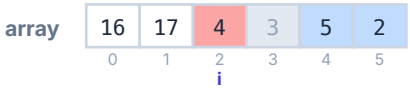
Step 4 of 7. 3 ≤ max (5): not a leader.

i = 3 · max = 5 · leaders = 5 2



Step 5 of 7. 4 ≤ max (5): not a leader.

i = 2 · max = 5 · leaders = 5 2



Step 6 of 7. 17 > max: leader! max becomes 17.

i = 1 · max = 17 · leaders = 17 5 2



Step 7 of 7. 16 ≤ max (17): not a leader.

i = 0 · max = 17 · leaders = 17 5 2



> **RESULT**
> leaders (left→right): 17 5 2

Complexity

Brute force: element i is compared with up to n−i−1 later elements. When every element is a leader (a strictly decreasing array) the inner loop never breaks, so the total is n(n−1)/2 comparisons → **O(n²)** worst case.

Right scan: one comparison per element → **O(n) time**, O(1) extra space besides the output.

n	brute force comparisons (worst)	right scan
1 000	≈ 500 000	1 000
100 000	≈ 5 × 10⁹	100 000

Common mistakes

Using >= and printing equal elements

Problem

```
if (a[i] >= maxFromRight) // for the standard (strict) definition
```

Why: With {7, 7} the first 7 is NOT greater than the 7 to its right. The strict definition excludes it.
Fix: if (a[i] > maxFromRight)

Forgetting the last element

Problem

```
for (int i = n - 2; i >= 0; i--) { ... } // never prints a[n-1]
```

Why: The last element has nothing to its right so it is always a leader and must be included.
Fix: start the scan at i = n − 1 with maxFromRight = MIN_VALUE

Initialising max to 0

Problem

```
int max = 0;
```

Why: For an all-negative array like {-5, -7} nothing is greater than 0, so no leader is found.
Fix: int max = Integer.MIN_VALUE;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Print the leaders — *Beginner*

Read n and an array. Print all leaders (strictly greater than every element to their right) from left to right.

Example input	Expected output
6 ⏎ 16 17 4 3 5 2	17 5 2

Hint 1: Scan from the right with a running maximum. **Hint 2:** Store, then print in reverse.

Exercise 2 · Non-strict leaders — *Intermediate*

Same as before, but an element counts as a leader when it is greater than OR EQUAL to every element on its right.

Example input	Expected output
5 ⏎ 7 10 4 10 6	10 10 6

Hint 1: Only the comparison changes.

Exercise 3 · Next greater element — *Challenge*

For each element print the first element to its right that is strictly greater, or -1 if there is none.

Example input	Expected output
4 ⏎ 4 5 2 25	5 25 25 −1

Hint 1: Scan from the right with a stack of candidates. **Hint 2:** Pop candidates that are not greater than the current element.

Quick check

Answers are in the answer key at the end of this part.

Q1. The leaders of {16, 17, 4, 3, 5, 2} are...

- **A.** 17 5 2
- **B.** 16 17 5 2
- **C.** 2 5 17
- **D.** 17 4 3 5 2

Q2. This leader finder gives wrong results for {-3, -5}. Which line is the bug?

```
1  int max = 0;
2  for (int i = n - 1; i >= 0; i--) {
3   if (a[i] > max) {
4   max = a[i];
5   print(a[i]);
6   }
7  }
```

Which line contains the bug? _____

Q3. The right-to-left scan runs in...

- **A.** O(1)
- **B.** O(n)
- **C.** O(n log n)
- **D.** O(n²)

Q4. Is the last element of an array always a leader?

- **A.** Yes
- **B.** No
- **C.** Only if positive
- **D.** Only if largest

Interview / exam questions

- Why is the last element always a leader?
- Can you find leaders in O(n) using O(1) extra space?
- How is the leader problem related to the "next greater element" problem?

Watch & read

Video: Leaders in an Array | Brute - Optimal | Strivers A2Z DSA Course — take U forward · 11:53. *Finding leaders by scanning from the right*

Video: Leaders in an Array | Right-to-Left Scan — Beyond the Function · 4:47. *The right-to-left scan*

REVISION CARD (30–60 SECONDS)

- Leader = strictly greater than all elements to its right; last element is always one.
- Scan from the right, keep max, a[i] > max → leader.
- O(n^2) brute force vs O(n) scan.
- Print in original order by reversing the stored list.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The Leaders deck is a screen-recorded video presentation (slides are images). Its right-to-left code prints leaders in reverse order (2 5 17) — the lesson prints them left-to-right to match the definition.

5.15 Majority Element (Boyer–Moore Voting)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · ★ on the minimum-viable path

Find the element that appears more than n/2 times: counting, sorting, and the clever O(1)-space voting algorithm.

YOU WILL BE ABLE TO

- State the majority-element definition (more than n/2 occurrences) precisely.
- Implement counting (two loops), sorting, and hash-map solutions and compare their costs.
- Explain Boyer–Moore voting: why cancelling different pairs leaves the majority, and why a verification pass is required.
- Extend the idea to elements appearing more than n/3 times.

Before this lesson: One-Dimensional Arrays · Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

The idea

An element is the **majority element** of an array of length n if it occurs **more than n/2 times**. In `{4,7,4,4,7,4,4,9,4,3}` (n = 10) the value 4 occurs 6 times > 5, so 4 is the majority. In `{1,7,8,2,6,8,1,3,2,8}` the best is 8 with 3 occurrences — there is **no** majority element.

Voting analogy: every element is a voter for itself. Pair off two voters who disagree and send both home: the winner (who has more than half the votes) can never be completely eliminated, because every pair removed contains at most one of its voters. Whoever is left must be the only possible winner — but if there was no real winner, a loser is left too, so we have to **count** to confirm.

```
4 7 4 4 7 4 4 9 4 3 candidate count
4 4 1
 7 (differs → cancel) 4 0
 4 (count==0 → new) 4 1
 4 4 2
 7 (cancel) 4 1
 4 4 2
 4 4 3
 9 (cancel) 4 2
 4 4 3
 3 (cancel) 4 2 → candidate 4, then verify: 4 appears 6 > 5 ✓
```

WHY THIS WORKS

Each time the counter decrements, one occurrence of the candidate is cancelled against one occurrence of a *different* value. If x appears more than n/2 times it has more occurrences than all other values combined, so after cancelling pairs of unequal elements at least one x remains. The candidate at the end is therefore x. If no value is a majority the algorithm still returns *some* value, so verify with one more pass.

Syntax

```
int candidate = 0, count = 0; // Boyer–Moore vote
for (int x : a) {
 if (count == 0) candidate = x;
 count += (x == candidate) ? 1 : -1;
}

Map<Integer, Integer> freq = new HashMap<>(); // counting with a map
for (int x : a) freq.merge(x, 1, Integer::sum); // or: freq.put(x, freq.getOrDefault(x, 0) + 1)
```

A tiny example

```
int candidate = 0, count = 0;
for (int x : a) {
 if (count == 0) { candidate = x; count = 1; }
 else if (x == candidate) count++;
 else count--;
}
```

Guided example

MajorityBruteForce.java — Two nested loops (simple solution)

```
1  import java.util.Scanner;
2
3  public class MajorityBruteForce {
4   // Count how many times each element occurs (two loops). O(n^2).
5   static Integer majority(int[] a) {
6   int n = a.length;
7   for (int i = 0; i < n; i++) {
8   int count = 0;
9   for (int j = 0; j < n; j++) {                                    (1)
10  if (a[j] == a[i]) {
11  count++;
12  }
13  }
14  if (count > n / 2) { // strictly more than half                  (2)
15  return a[i];
16  }
17  }
18  return null;
19  }
20
21  public static void main(String[] args) {
22  Scanner sc = new Scanner(System.in);
23  int n = sc.nextInt();
24  int[] a = new int[n];
25  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
26  Integer m = majority(a);
27  System.out.println(m == null ? "No majority element" : "Majority element: " + m);
28  }
29 }
```

Counts how often each element occurs. Simple, but O(n²).

Sample input

```
10
4 7 4 4 7 4 4 9 4 3
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	For each candidate a[i], scan the whole array and count equal elements.
2	line 14	"More than half": strictly greater than n/2 (integer division works: for n = 5, n/2 = 2, need ≥ 3).

MajoritySorting.java — Sort, then count runs (simple solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class MajoritySorting {
5   // After sorting, equal values sit together, so one pass counts every run. O(n log n).
6   static Integer majority(int[] original) {
7   int[] a = original.clone();                                    (1)
8   Arrays.sort(a);                                                (2)
9   int n = a.length;
10  int run = 1;
11  for (int i = 1; i <= n; i++) {
12  if (i < n && a[i] == a[i - 1]) {
13  run++;
14  } else {
15  if (run > n / 2) {                                             (3)
16  return a[i - 1];
17  }
18  run = 1;
19  }
20  }
21  return null;
22  }
23
24  public static void main(String[] args) {
25  Scanner sc = new Scanner(System.in);
26  int n = sc.nextInt();
27  int[] a = new int[n];
28  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
29  Integer m = majority(a);
30  System.out.println(m == null ? "No majority element" : "Majority element: " + m);
31  }
32 }
```

After sorting, equal values are adjacent, so one pass counts every run. O(n log n).

Sample input

```
10
1 7 8 2 6 8 1 3 2 8
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Sort a copy so the caller's array is not changed.
2	line 8	O(n log n) dominates the running time.
3	line 15	Check at the end of every run of equal values.

BoyerMoore.java — Boyer–Moore majority vote (optimised solution)

```
1  import java.util.Scanner;
2
3  public class BoyerMoore {
4   // Boyer–Moore voting: O(n) time, O(1) space.
5   static Integer majority(int[] a) {
6   // Step 1: find a CANDIDATE. Each different pair of elements "cancels out"; a true
        majority survives.
7   Integer candidate = null;
8   int count = 0;
9   for (int x : a) {
10  if (count == 0) {                                                    1
11  candidate = x;
12  count = 1;
13  } else if (x == candidate) {                                         2
14  count++;
15  } else {
16  count--;                                                             3
17  }
18  }
19  if (candidate == null) {
20  return null;
21  }
22  // Step 2: VERIFY. The survivor is only a candidate: it need not be a majority.   4
23  int occurrences = 0;
24  for (int x : a) {
25  if (x == candidate) {
26  occurrences++;
27  }
28  }
29  return occurrences > a.length / 2 ? candidate : null;                5
30  }
31
32  public static void main(String[] args) {
33  Scanner sc = new Scanner(System.in);
34  int n = sc.nextInt();
35  int[] a = new int[n];
36  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
37  Integer m = majority(a);
38  System.out.println(m == null ? "No majority element" : "Majority element: " + m);
39  }
40 }
```

Phase 1 finds the only possible candidate; phase 2 confirms it.

Sample input

```
5
2 1 2 4 7
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	No surviving voter: adopt the current element as the new candidate.
2	line 13	Same value: one more vote for the candidate.
3	line 16	Different value: the two cancel each other out.
4	line 22	The candidate is only a *possible* majority. Count its real occurrences.
5	line 29	Strict majority test; otherwise report no majority.

> **NOTE**
> The slide uses `{2,1,2,4,7}` to show why step 2 is needed: after step 1 the candidate is 7 with count 1, yet 7 appears only once in 5 elements. Try it: `BoyerMoore.in` contains exactly that array and prints "No majority element".

Dry run: watch it execute

Boyer–Moore: candidate and count after each element; the final pass verifies it.

Algorithm: Boyer–Moore majority vote. **Input:** arr = `4,7,4,4,7,4,4,9,4,3`.

Pseudocode

```
candidate = none; count = 0
for x in a:
 if count == 0: candidate = x; count = 1
 else if x == candidate: count++
 else count--
verify: occurrences(candidate) > n/2 ?
```

Step 1 of 12. Start with no candidate.

candidate = — · count = 0

array	4	7	4	4	7	4	4	9	4	3
	0	1	2	3	4	5	6	7	8	9

Step 2 of 12. count was 0: 4 becomes the candidate.

candidate = 4 · count = 1

array	4	7	4	4	7	4	4	9	4	3
	0	1	2	3	4	5	6	7	8	9

Step 3 of 12. 7 differs from 4: they cancel, count--.

candidate = 4 · count = 0

array	4	7	4	4	7	4	4	9	4	3
	0	1	2	3	4	5	6	7	8	9

Step 4 of 12. count was 0: 4 becomes the candidate.

candidate = 4 · count = 1

array	4	7	4	4	7	4	4	9	4	3
	0	1	2	3	4	5	6	7	8	9

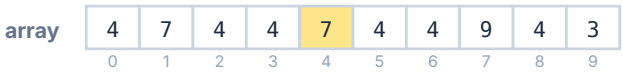
Step 5 of 12. 4 equals the candidate: count++.

candidate = 4 · count = 2



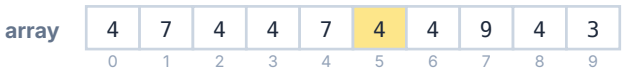
Step 6 of 12. 7 differs from 4: they cancel, count--.

candidate = 4 · count = 1



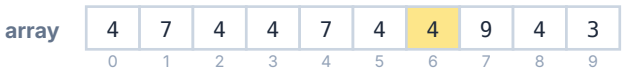
Step 7 of 12. 4 equals the candidate: count++.

candidate = 4 · count = 2



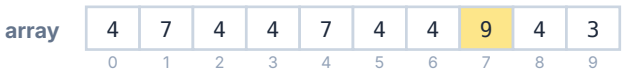
Step 8 of 12. 4 equals the candidate: count++.

candidate = 4 · count = 3



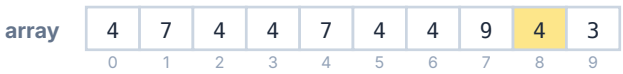
Step 9 of 12. 9 differs from 4: they cancel, count--.

candidate = 4 · count = 2



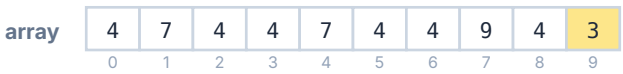
Step 10 of 12. 4 equals the candidate: count++.

candidate = 4 · count = 3



Step 11 of 12. 3 differs from 4: they cancel, count--.

candidate = 4 · count = 2



Step 12 of 12. Verify: 4 occurs 6 times; n/2 = 5. Majority confirmed.

candidate = 4 · occurrences = 6 · majority = true



> **RESULT**
> majority element = 4

Complexity

approach	time	space
two loops	O(n²)	O(1)
sorting	O(n log n)	O(1)–O(n)
HashMap of counts	O(n)	O(n)
Boyer–Moore	O(n)	O(1)

Boyer–Moore does exactly two passes over the array (2n comparisons) and uses two variables. This matches the slide's final statement: **Time O(n), Space O(1)**.

Common mistakes

Skipping the verification pass

Problem

```
return candidate; // after phase 1
```

Why: For {2, 1, 2, 4, 7} phase 1 ends with candidate 7, which is not a majority. Without phase 2 you report a wrong answer.

Fix: Count occurrences of the candidate and compare with n/2.

Using ≥ n/2 instead of > n/2

Problem

```
if (count >= n / 2)
```

Why: For {1, 2} the value 1 has exactly n/2 occurrences — not a majority.

Fix: if (count > n / 2)

Comparing Integer objects with ==

Problem

```
Integer candidate = ...;
if (x == candidate) ...
```

Why: When one side is an Integer object and the other an int, Java unboxes — fine; but comparing two Integer objects with == compares references (works only for −128..127).

Fix: Use int primitives, or candidate.equals(x).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Majority with a HashMap — *Beginner*

Read n and an array. Print the majority element or the text `No majority element`.

Example input	Expected output
5 ⏎ 2 2 1 1 1	1

Hint 1: Count frequencies with a HashMap<Integer,Integer>. **Hint 2:** Majority means more than n/2.

Exercise 2 · Boyer–Moore — *Intermediate*

Same task, but use O(1) extra memory: no sorting, no hash map.

- $1 \le n \le 10^6$

Example input	Expected output
7 ⏎ 3 3 4 2 4 4 2	No majority element

Hint 1: Two variables: candidate and count. **Hint 2:** Verify with a second pass.

Exercise 3 · Elements appearing more than n/3 times — *Challenge*

Print all values that occur more than $\lfloor n/3 \rfloor$ times, in increasing order, or `None`.

Example input	Expected output
8 ⏎ 1 1 1 3 3 2 2 2	1 2

Hint 1: At most two values can exceed n/3. **Hint 2:** Keep TWO candidates with two counters; decrement both when a third value arrives. **Hint 3:** Verify both candidates in a second pass.

Quick check

Answers are in the answer key at the end of this part.

Q1. In an array of 9 elements, how many occurrences make a value the majority?

- **A.** at least 4
- **B.** at least 5
- **C.** exactly 4
- **D.** at least 9

Q2. Run phase 1 of Boyer–Moore on {2, 1, 2, 4, 7}.

```
for (int x : a) {
 if (count == 0) { candidate = x; count = 1; }
 else if (x == candidate) count++;
 else count--;
}
```

- count after 2nd element (1): _____
- candidate after 3rd element (2): _____
- count after 4th element (4): _____
- final candidate: _____

Q3. Why is the second pass of Boyer–Moore needed?

- **A.** To sort the array
- **B.** The candidate may not be a majority
- **C.** To reset the counter
- **D.** To avoid overflow

Q4. Which approach uses O(1) extra space and O(n) time?

- **A.** HashMap counting
- **B.** Sorting
- **C.** Boyer–Moore
- **D.** Two loops

Q5. Sorting-based majority detection costs...

- **A.** O(n)
- **B.** O(n log n)
- **C.** $O(n^2)$
- **D.** O(log n)

Interview / exam questions

- Prove that Boyer–Moore returns the majority element if one exists.
- How would you find elements appearing more than n/k times?
- What if the array is too large to fit in memory?
- Why does the final verification matter in an interview answer?

Watch & read

Video: Majority Element I | Brute-Better-Optimal | Moore's Voting Algorithm | Intuition |Brute to Optimal — take U forward · 18:13. *Boyer–Moore majority vote*

Video: Majority Element - Leetcode 169 - Python — NeetCode · 14:39. *Majority element solutions (Python — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- Majority: count > n/2.
- Boyer–Moore: candidate/count; count == 0 → adopt; same → ++; different → −−.
- ALWAYS verify the candidate with a second pass.
- O(n) time, O(1) space; > n/3 needs two candidates.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide cites https://www.youtube.com/watch?v=fCGLjIsEsNA as the video for this topic. YouTube's own metadata shows that video is titled "Maximum average subarray of size k" (IDeserve) — it is not about majority elements. It is NOT used in this library.

ASSUMPTIONS MADE

- The slide deck is a set of images from an IDeserve lecture; the Java fragments in the screenshots are reconstructed into the complete program `BoyerMoore.java`.

5.16 Maximum Product Subarray

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · ★ on the minimum-viable path

Find the contiguous subarray with the largest product — negatives and zeros make this trickier than the sum version.

YOU WILL BE ABLE TO

- Define subarray and count them: n(n+1)/2.
- Solve with brute force in O(n²) and understand the cost.
- Explain why you must track BOTH the maximum and minimum product ending at each position.
- Handle zeros, all-negative arrays, and overflow, and verify against brute force.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Type Conversion, Casting and Overflow

The idea

A **subarray** is a block of consecutive elements. `{4, 0, −2, 6}` has 4·5/2 = 10 subarrays (1 + 2 + 3 + 4). Their products are 4, 0, 0, 0, 0, 0, −2, −12, 0 and 6 — so the maximum is **6** (the slide lists them all).

For sums, "best subarray ending here" needs only one number. For products it needs two, because multiplying by a **negative** number flips big positive into big negative and the other way round:

```
a = { −5, −4, −1, −2 }
 max ending here min ending here
 −5 −5 −5
 −4 20 (−5·−4) −4
 −1 4 (−4·−1) −20 (20·−1)
 −2 40 (−20·−2) −8 (4·−2) answer = 40
```

WHY THIS WORKS

A subarray ending at i is either just a[i] or a[i] times a subarray ending at i−1. If a[i] > 0 we want the biggest product ending at i−1 to make the biggest result, and the smallest to make the smallest result. If a[i] < 0 the roles are swapped: the **smallest** (most negative) previous product becomes the biggest. A zero resets both. That is why `max` and `min` are updated together and swapped when a[i] is negative.

Syntax

```
long best = a[0], curMax = a[0], curMin = a[0]; // products can overflow int: use long
for (int i = 1; i < a.length; i++) {
 long x = a[i];
 long hi = curMax * x, lo = curMin * x; // a negative x swaps the roles of max and min
 curMax = Math.max(x, Math.max(hi, lo));
 curMin = Math.min(x, Math.min(hi, lo));
 best = Math.max(best, curMax);
}
```

A tiny example

```
long mx = a[0], mn = a[0], best = a[0];
for (int i = 1; i < n; i++) {
 long x = a[i];
 if (x < 0) { long t = mx; mx = mn; mn = t; }
 mx = Math.max(x, mx * x);
 mn = Math.min(x, mn * x);
 best = Math.max(best, mx);
}
```

Guided example

MaxProductBrute.java — Brute force over all subarrays (simple solution)

```
1  import java.util.Scanner;
2
3  public class MaxProductBrute {
4   // Try every subarray (start i, end j): there are n(n+1)/2 of them. O(n^2).
5   static long maxProduct(int[] a) {
6   long best = a[0];
7   for (int i = 0; i < a.length; i++) {
8   long product = 1;                                                    (1)
9   for (int j = i; j < a.length; j++) {
10  product *= a[j]; // product of a[i..j], extended one element at a time  (2)
11  best = Math.max(best, product);                                      (3)
12  }
13  }
14  return best;
15  }
16
17  public static void main(String[] args) {
18  Scanner sc = new Scanner(System.in);
19  int n = sc.nextInt();
20  int[] a = new int[n];
21  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
22  System.out.println(maxProduct(a));
23  }
24 }
```

For each start i, extend the end j and keep a running product.

Sample input

```
4
4 0 -2 6
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Product of the empty prefix; reset for every new start index.
2	line 10	Extending the subarray by one element needs only one multiplication, not a recomputation.
3	line 11	Keep the largest product seen over all (i, j).

MaxProduct.java — Max/min product ending at each index (optimised solution)

```
1  import java.util.Scanner;
2
3  public class MaxProduct {
4   // Keep the best AND the worst product of a subarray ending at i. A negative number
        turns the
5   // worst (most negative) product into the best one, so both are needed. O(n) time, O(1)
        space.
6   static long maxProduct(int[] a) {
7   long maxEnding = a[0];                                                          (1)
8   long minEnding = a[0];                                                          (2)
9   long best = a[0];
10  for (int i = 1; i < a.length; i++) {
11  long x = a[i];
12  if (x < 0) { // multiplying by a negative swaps the roles                       (3)
13  long t = maxEnding;
14  maxEnding = minEnding;
15  minEnding = t;
16  }
17  maxEnding = Math.max(x, maxEnding * x); // extend the previous run, or start fresh at x  (4)
18  minEnding = Math.min(x, minEnding * x);
19  best = Math.max(best, maxEnding);                                               (5)
20  }
21  return best;
22  }
23
24  public static void main(String[] args) {
25  Scanner sc = new Scanner(System.in);
26  int n = sc.nextInt();
27  int[] a = new int[n];
28  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
29  System.out.println(maxProduct(a));
30  }
31 }
```

One pass, O(1) space. Swapping on negatives replaces the slide's three-way if/else.

Sample input

```
4
-5 -4 -1 -2
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Best product of a subarray ending at the current index.
2	line 8	Worst (most negative) product ending here — it may become the best after a negative factor.
3	line 12	A negative factor reverses the order of products: swap the roles.
4	line 17	Either start fresh at x (the earlier product hurts) or extend.
5	line 19	Track the overall best.

MaxProductCheck.java — Randomised check (demonstration)

```java
import java.util.Random;

public class MaxProductCheck {
 static long brute(int[] a) {
 long best = Long.MIN_VALUE;
 for (int i = 0; i < a.length; i++) {
 long p = 1;
 for (int j = i; j < a.length; j++) {
 p *= a[j];
 best = Math.max(best, p);
 }
 }
 return best;
 }

 static long fast(int[] a) {
 long mx = a[0], mn = a[0], best = a[0];
 for (int i = 1; i < a.length; i++) {
 long x = a[i];
 if (x < 0) {
 long t = mx;
 mx = mn;
 mn = t;
 }
 mx = Math.max(x, mx * x);
 mn = Math.min(x, mn * x);
 best = Math.max(best, mx);
 }
 return best;
 }

 // 20 000 random arrays with values in [-4, 4] (zeros and negatives included) against
     brute force.
 public static void main(String[] args) {
 Random rnd = new Random(2024);
 for (int t = 0; t < 20000; t++) {
 int n = 1 + rnd.nextInt(9);
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = rnd.nextInt(9) - 4;
 if (brute(a) != fast(a)) {
 System.out.println("MISMATCH on " + java.util.Arrays.toString(a));
 return;
 }
 }
 System.out.println("fast version matches brute force on 20000 random arrays");
 }
}
```

20 000 random arrays over {−4..4} including zeros; the fast and brute versions must agree.

Line-by-line walkthrough

#	Where	What happens
1	line 38	Values −4..4: lots of zeros and sign changes, which is where bugs hide.

Dry run: watch it execute

The slide's execution trace: max and min after each element.

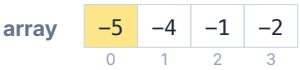
Algorithm: Maximum product subarray. **Input:** arr = `-5,-4,-1,-2`.

Pseudocode

```
mx = mn = best = a[0]
for i = 1..n-1:
 if a[i] < 0: swap(mx, mn)
 mx = max(a[i], mx * a[i])
 mn = min(a[i], mn * a[i])
 best = max(best, mx)
```

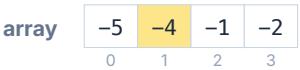
Step 1 of 4. Start: max = min = best = a[0] = -5.

max = -5 · min = -5 · best = -5



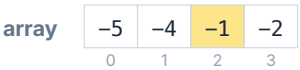
Step 2 of 4. a[1] = -4. Negative: swap max and min. max = 20, min = -4, best = 20.

max = 20 · min = -4 · best = 20



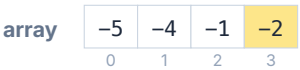
Step 3 of 4. a[2] = -1. Negative: swap max and min. max = 4, min = -20, best = 20.

max = 4 · min = -20 · best = 20



Step 4 of 4. a[3] = -2. Negative: swap max and min. max = 40, min = -8, best = 40.

max = 40 · min = -8 · best = 40



> **RESULT**
> maximum product = 40

Complexity

Brute force: n(n+1)/2 subarrays, O(1) each with the running product → **O(n²) time**, O(1) space.

Max/min: a single pass → **O(n) time, O(1) space**.

> **WATCH OUT**
> Products grow fast: 2^{63} is reached after only 63 elements equal to 2. Use `long` (or BigInteger) and state the constraints; the answer is guaranteed to fit in the problem statement.

Common mistakes

Tracking only the maximum

Problem

```
maxEnding = Math.max(a[i], maxEnding * a[i]);
```

Why: For {−2, 3, −4} the answer is 24, but a max-only version stores 3 then 3·(−4) = −12 → 3·... it misses (−2·3·−4). The minimum −6 is needed.

Fix: Keep maxEnding AND minEnding; swap when a[i] < 0.

Hard-coding the length (slide)

Problem

```
for (int i = 1; i < 4; i++) // array happens to have 4 elements
```

Why: Any other array length silently skips elements or throws.

Fix: for (int i = 1; i < a.length; i++)

Initialising best with 0

Problem

```
long best = 0;
```

Why: For {−3} the answer is −3 (a subarray must contain at least one element) but best stays 0.

Fix: long best = a[0];

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count subarrays with positive product — *Beginner*

Read n and an array. Print the number of subarrays whose product is strictly positive.

- 1 ≤ n ≤ 2000

Example input	Expected output
5 ⏎ 2 3 −2 4 0	4

Hint 1: A subarray is positive when it has no zero and an even number of negatives. **Hint 2:** For each start extend until a zero appears.

Exercise 2 · Maximum product subarray — *Intermediate*

Read n and an array of integers. Print the largest product of a contiguous non-empty subarray.

- 1 ≤ n ≤ 10^5
- −10 ≤ a[i] ≤ 10
- the answer fits in a 64-bit signed integer

Example input	Expected output
4 ⏎ 2 3 −2 4	6

Hint 1: Keep both max and min ending at i. **Hint 2:** Swap them when a[i] is negative.

Exercise 3 · Which subarray? — *Challenge*

Print three numbers: the maximum product, then the start and end index (0-based) of the first subarray that achieves it (earliest start, then earliest end).

- 1 ≤ n ≤ 1000

Example input	Expected output
4 ⏎ 2 3 −2 4	6 0 1

Hint 1: The brute force O(n²) is fine for n ≤ 1000. **Hint 2:** Only update on a STRICTLY larger product to keep the first one.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many subarrays does an array of 5 elements have?

- **A.** 10
- **B.** 15
- **C.** 20
- **D.** 25

Q2. What does the lesson's algorithm return for {−2, 3, −4}?

```
// max/min product ending at each index
```

Your answer: _____

Q3. Why do we keep the MINIMUM product ending at i?

- **A.** To find the minimum answer
- **B.** A negative number can turn it into the maximum
- **C.** To detect zeros
- **D.** For overflow protection

Q4. Time complexity of the max/min algorithm?

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n²)
- **D.** O(2ⁿ)

Q5. Max product of {0, 2}?

```
// a zero resets the running products
```

Your answer: _____

Interview / exam questions

- Why does the maximum-sum subarray (Kadane) not directly work for products?
- How do zeros and negatives affect the algorithm?
- What if the numbers are huge — how do you avoid overflow?
- Solve it by splitting at zeros and counting negatives.

Watch & read

Video: Maximum Product Subarray - Best Intuitive Approach Discussed — take U forward · 20:27. *Tracking max and min products*

Video: Maximum Product Subarray - Dynamic Programming - Leetcode 152 — NeetCode · 15:31. *Kadane-style solution*

REVISION CARD (30–60 SECONDS)

- Keep max & min product ending here; swap if a[i] < 0.
- max = max(a[i], max·a[i]); min = min(a[i], min·a[i]); best = max(best, max).
- Zero resets; initialise from a[0], not 0.
- O(n) time, O(1) space; use long.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide prints `{4, 0, −2, 6}` products with a stray "x1 = …" notation, but its conclusion (6) is correct. • The slide code loops `i < 4` regardless of array length and uses a three-way branch with a temp variable; the swap form in this lesson is equivalent and shorter.

5.17 Maximum Equilibrium Sum

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 35 min · ★ on the minimum-viable path

Find indices where the left sum equals the right sum, and report the largest such sum — in one pass with a running total.

YOU WILL BE ABLE TO

- Define an equilibrium index and find one by hand.
- Solve with brute force, then with a prefix/running sum in O(n).
- Compute the maximum equilibrium sum and distinguish the two common definitions.
- Handle negatives, a single element, and no equilibrium at all.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue

The idea

An **equilibrium index** is a position where the sum of everything to its left equals the sum of everything to its right (the element itself is on neither side). For `A = {−7, 1, 5, 2, −4, 3, 0}`: index 3 works because −7 + 1 + 5 = −1 and −4 + 3 + 0 = −1; index 6 works because the left side sums to 0 and the right side is empty (sum 0).

The **maximum equilibrium sum** is the largest of those equal side-sums. For `{1,2,3,4,5,4,3,2,1}` the only equilibrium is the middle 5, with both sides summing to **10**.

```
total = 25 left right = total − left − a[i]
i=0 a=1 left=0 right=24 0 24 no
i=1 a=2 left=1 right=22 1 22 no
i=2 a=3 left=3 right=19
i=3 a=4 left=6 right=15
i=4 a=5 left=10 right=10 ← equal! equilibrium, sum 10
```

WHY THIS WORKS

Instead of re-adding the right side for every i, note that `right = total - left - a[i]`. Keep `left` as a running sum and update it after testing each index. The total is computed once. This is the same idea as a prefix-sum array, but needs only one variable.

WATCH OUT

Two definitions are in circulation. The slides (and this lesson's main answer) report the side-sum *excluding* a[i]. Many online versions ask for the prefix sum *including* a[i] that equals the suffix sum including a[i]; the equilibrium indices are the same, but the reported sum is larger by a[i] (15 instead of 10 for the example). The program prints both.

Syntax

```
long[] prefix = new long[n + 1]; // prefix[i] = a[0] + … + a[i-1]
for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + a[i];
long left = prefix[i]; // sum of a[0 .. i-1]
long right = prefix[n] - prefix[i + 1]; // sum of a[i+1 .. n-1]
long rangeSum = prefix[r + 1] - prefix[l]; // any range a[l..r] in O(1)
```

A tiny example

```
long left = 0;
for (int i = 0; i < n; i++) {
 long right = total - left - a[i];
 if (left == right) { /* equilibrium at i */ }
 left += a[i];
}
```

Guided example

The slide animates the algorithm on `{1, 3, 5, 2, 2}`: total = 13; at index 2 (value 5) the left sum is 4 and the right sum 13 − 4 − 5 = 4 → equilibrium, position 3 in 1-based numbering (`pos = i + 1`).

EquilibriumBrute.java — Brute force (simple solution)

```
1  import java.util.Scanner;
2
3  public class EquilibriumBrute {
4   // For every index i, add up everything to its left and everything to its right. O(n^2).
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   long[] a = new long[n];
9   for (int i = 0; i < n; i++) a[i] = sc.nextLong();
10  boolean found = false;
11  for (int i = 0; i < n; i++) {
12  long left = 0, right = 0;
13  for (int j = 0; j < i; j++) left += a[j];                                    (1)
14  for (int j = i + 1; j < n; j++) right += a[j];                               (2)
15  if (left == right) {                                                         (3)
16  System.out.println("equilibrium index " + i + " (each side sums to " + left + ")");
17  found = true;
18  }
19  }
20  if (!found) System.out.println("no equilibrium index");
21  }
22 }
```

For each index add the left part and the right part separately. O(n²).

Sample input

```
7
-7 1 5 2 -4 3 0
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	Sum of everything strictly before i.
2	line 14	Sum of everything strictly after i.
3	line 15	Equilibrium: report the index and the side sum.

MaxEquilibriumSum.java — One pass (optimised solution)

```
1  import java.util.Scanner;
2
3  public class MaxEquilibriumSum {
4   // One pass with a running left sum. right = total - left - a[i]. O(n) time, O(1) space.
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   long[] a = new long[n];
9   long total = 0;
10  for (int i = 0; i < n; i++) {
11  a[i] = sc.nextLong();
12  total += a[i];                                                    (1)
13  }
14
15  long left = 0;
16  boolean found = false;
17  long bestExcluding = Long.MIN_VALUE; // the sum of ONE side (the slide's definition)
18  long bestIncluding = Long.MIN_VALUE; // prefix sum that also includes a[i] (a common
        textbook variant)
19  for (int i = 0; i < n; i++) {
20  long right = total - left - a[i];                                 (2)
21  if (left == right) {
22  found = true;
23  bestExcluding = Math.max(bestExcluding, left);                    (3)
24  bestIncluding = Math.max(bestIncluding, left + a[i]);             (4)
25  System.out.println("equilibrium index " + i + ": side sum = " + left + ", including
        a[i] = " + (left + a[i]));
26  }
27  left += a[i];                                                     (5)
28  }
29  if (!found) {
30  System.out.println("no equilibrium index");
31  } else {
32  System.out.println("max equilibrium sum (slide definition) = " + bestExcluding);
33  System.out.println("max prefix-including-element variant = " + bestIncluding);
34  }
35  }
36 }
```

Computes total once, then maintains left; right is derived. O(n) time, O(1) extra space.

Sample input

```
9
1 2 3 4 5 4 3 2 1
```

Line-by-line walkthrough

#	Where	What happens
1	line 12	First pass: the total of the whole array.
2	line 20	Right sum without looping: everything except the left part and a[i] itself.
3	line 23	The slide's definition: the side sum (left == right).
4	line 24	The alternative definition (prefix including a[i]).
5	line 27	Update AFTER the test, so that `left` means "sum before i" at the top of the loop.

Dry run: watch it execute

The slide's array: left sum and right sum at every index.

Algorithm: Equilibrium index. **Input:** arr = `1,3,5,2,2`.

Pseudocode

```
total = sum(a); left = 0
for i in 0..n-1:
 right = total - left - a[i]
 if left == right: equilibrium at i
 left += a[i]
```

Step 1 of 6. total = 13.

total = 13 · left = 0

	0	1	2	3	4
array	1	3	5	2	2

Step 2 of 6. left = 0, right = 12: not equal.

i = 0 · left = 0 · right = 12 · a[i] = 1

	0	1	2	3	4
array	1	3	5	2	2

Step 3 of 6. left = 1, right = 9: not equal.

i = 1 · left = 1 · right = 9 · a[i] = 3

	0	1	2	3	4
array	1	3	5	2	2

Step 4 of 6. left (4) == right (4): equilibrium index 2, side sum 4.

i = 2 · left = 4 · right = 4 · a[i] = 5

	0	1	2	3	4
array	1	3	5	2	2

Step 5 of 6. left = 9, right = 2: not equal.

i = 3 · left = 9 · right = 2 · a[i] = 2

	0	1	2	3	4
array	1	3	5	2	2

Step 6 of 6. left = 11, right = 0: not equal.

i = 4 · left = 11 · right = 0 · a[i] = 2

	0	1	2	3	4
array	1	3	5	2	2

> **RESULT**
> maximum equilibrium sum = 4

Complexity

Brute force: for each of n indices two sums of up to n elements → **O(n²)** time, O(1) space.

One pass: two linear scans → **O(n) time, O(1) space** (apart from the input).

Use `long` for the sums: 10^5 values of 10^9 overflow `int`.

Common mistakes

Updating left before comparing

Problem

```
left += a[i];
if (left == total - left) ...
```

Why: Then `left` already includes a[i] and the comparison tests the "including" variant with a different right side. Mixing the two definitions produces off-by-one-element results.

Fix: Compare first, then add a[i] to left.

Returning Integer.MIN_VALUE when nothing is found (slide)

Problem

```
return maxSum; // MIN_VALUE if no equilibrium
```

Why: The caller prints a huge negative number as if it were a real answer. Report "none" explicitly.

Fix: Track a `found` flag.

int overflow

Problem

```
int total = 0; for (...) total += a[i];
```

Why: Large inputs wrap around silently and create fake equilibria.

Fix: long total

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · First equilibrium index — *Beginner*

Read n and an array. Print the smallest equilibrium index (0-based) or -1.

Example input	Expected output
7 ⏎ −7 1 5 2 −4 3 0	3

Hint 1: right = total − left − a[i]. **Hint 2:** Stop at the first match.

Exercise 2 · Maximum equilibrium sum — *Intermediate*

Print the largest side-sum over all equilibrium indices (left sum excluding a[i]), or the word `none`.

Example input	Expected output
9 ⏎ 1 2 3 4 5 4 3 2 1	10

Hint 1: Collect a maximum over the equilibria. **Hint 2:** Use a boolean for "found" instead of a sentinel.

Exercise 3 · Balanced split points — *Challenge*

Count the split positions k (1 ≤ k ≤ n−1) such that the sum of a[0..k−1] equals the sum of a[k..n−1].

- 2 ≤ n ≤ 10^5

Example input	Expected output
6 ⏎ 1 −1 1 −1 1 −1	2

Hint 1: Both parts together sum to total, so the prefix must equal total/2. **Hint 2:** One pass with a running prefix.

Quick check

Answers are in the answer key at the end of this part.

Q1. For {1, 3, 5, 2, 2}, the equilibrium index (0-based) is...

- **A.** 1
- **B.** 2
- **C.** 3
- **D.** none

Q2. What is `right` at index 1 of {4, 1, 2, 1}?

```
long total = 8, left = 4;
long right = total - left - 1;
System.out.println(right);
```

Your answer: _____

Q3. How do you get the right sum in O(1)?

- **A.** Loop again
- **B.** total − left − a[i]
- **C.** left × 2
- **D.** sort first

Q4. Time complexity of the single-pass solution?

- **A.** O(1)
- **B.** O(n)
- **C.** O(n log n)
- **D.** $O(n^2)$

Interview / exam questions

- Find the equilibrium index in O(n) time and O(1) space.
- What if the array has negative numbers and zeros — can there be several equilibrium indices?
- How do you handle overflow?

Watch & read

Video: Equilibrium Index of Array | O(n) Prefix-Sum Solution | An easy detailed solution — dsa-made-easy · 12:38. *Prefix sums and equilibrium*

Video: Prefix Sum Array and Range Sum Queries — Profound Academy · 7:30. *The prefix-sum technique*

REVISION CARD (30–60 SECONDS)

- Equilibrium index: sum(left) == sum(right), a[i] excluded.
- right = total − left − a[i]; update left AFTER the test.
- Max equilibrium sum = max of left over equilibria (none → report none).
- O(n) time, O(1) space; use long.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The Java program on the last slide returns Integer.MIN_VALUE when there is no equilibrium index; the lesson uses an explicit "none". • The deck explains the algorithm with a sequence of animated screenshots using the 1-based position `pos = i + 1`; Java indices here are 0-based.

ASSUMPTIONS MADE

• "Maximum equilibrium sum" is taken as the maximum of the equal side-sum (the deck's code). The prefix-including-a[i] reading is shown as a labelled variant.

5.18 Maximum Sum of an Hourglass in a Matrix

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 30 min · ★ on the minimum-viable path

Slide a fixed 7-cell hourglass pattern over a 2-D array and keep the best sum.

YOU WILL BE ABLE TO

- Describe the hourglass shape and index its seven cells from a top-left corner.
- Count the hourglasses in an R×C matrix: (R−2)(C−2).
- Write two nested loops with correct bounds and a correct initial maximum.
- Generalise with 2-D prefix sums to any k×k window.

Before this lesson: Two-Dimensional Arrays (Matrices) · Loops: for, while, do-while, break and continue

The idea

An **hourglass** in a matrix is the 7-cell shape: a row of three cells, the single cell below its middle, and another row of three cells.

The hourglass is anchored by its top-left corner (i, j).

```
a b c (i, j) (i, j+1) (i, j+2)
d (i+1, j+1)
e f g (i+2, j) (i+2, j+1)(i+2, j+2)
```

For `4×4` matrix `1 2 3 4 / 5 6 7 8 / 9 1 2 3 / 4 5 6 7` there are (4−2)·(4−2) = 4 hourglasses with sums 24, 22, 34 and **41** — the slide's answer.

WHY THIS WORKS

The top-left corner can be at any row 0 … R−3 and any column 0 … C−3 (the hourglass needs 2 extra rows and columns). That gives (R−2) × (C−2) positions, and the problem becomes: evaluate a 7-term sum at each position and keep the largest.

Syntax

```
for (int r = 0; r + 2 < rows; r++) { // top-left corner of a 3 x 3 window
 for (int c = 0; c + 2 < cols; c++) {
 int sum = m[r][c] + m[r][c+1] + m[r][c+2]
 + m[r+1][c+1]
 + m[r+2][c] + m[r+2][c+1] + m[r+2][c+2];
 best = Math.max(best, sum);
 }
}
int best = Integer.MIN_VALUE; // NOT 0: all sums may be negative
```

A tiny example

```
int s = m[i][j] + m[i][j+1] + m[i][j+2]
 + m[i+1][j+1]
 + m[i+2][j] + m[i+2][j+1] + m[i+2][j+2];
```

Guided example

HourglassMax.java — Maximum hourglass sum (optimised solution)

```
1  import java.util.Scanner;
2
3  public class HourglassMax {
4   // An hourglass has its top-left corner at (i, j):
5   // a b c (i, j) (i, j+1) (i, j+2)
6   // d (i+1, j+1)
7   // e f g (i+2, j) (i+2, j+1) (i+2, j+2)
8   static int hourglassSum(int[][] m, int i, int j) {                    (1)
9   return m[i][j] + m[i][j + 1] + m[i][j + 2]
10  + m[i + 1][j + 1]
11  + m[i + 2][j] + m[i + 2][j + 1] + m[i + 2][j + 2];
12  }
13
14  public static void main(String[] args) {
15  Scanner sc = new Scanner(System.in);
16  int rows = sc.nextInt();
17  int cols = sc.nextInt();
18  int[][] m = new int[rows][cols];
19  for (int i = 0; i < rows; i++) {
20  for (int j = 0; j < cols; j++) {
21  m[i][j] = sc.nextInt();
22  }
23  }
24  if (rows < 3 || cols < 3) {                                           (2)
25  System.out.println("Not possible");
26  return;
27  }
28  int best = Integer.MIN_VALUE; // NOT 0: every hourglass sum may be negative   (3)
29  int bestRow = 0, bestCol = 0;
30  for (int i = 0; i + 2 < rows; i++) { // (rows - 2) possible top rows   (4)
31  for (int j = 0; j + 2 < cols; j++) { // (cols - 2) possible left columns   (5)
32  int s = hourglassSum(m, i, j);
33  if (s > best) {
34  best = s;
35  bestRow = i;
36  bestCol = j;
37  }
38  }
39  }
40  System.out.println("hourglasses: " + (rows - 2) * (cols - 2));
41  System.out.println("maximum sum: " + best + " (top-left corner at row " + bestRow + ",
        column " + bestCol + ")");
42  }
43 }
```

Two nested loops over the valid top-left corners; O(R·C) time.

Sample input

```
4 4
1 2 3 4
5 6 7 8
9 1 2 3
4 5 6 7
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Extracting the 7-term sum into a method makes the loops easy to read and the pattern easy to change.
2	line 24	No hourglass fits. Report it and stop (the slide calls System.exit(0) inside the method — returning is cleaner).
3	line 28	NOT 0: if every hourglass sum is negative, starting at 0 would report 0, which is not the sum of any hourglass.
4	line 30	Top row of the hourglass: it needs rows i, i+1, i+2.
5	line 31	Left column: it needs columns j, j+1, j+2.

Dry run: watch it execute

The four hourglasses of the slide's 4×4 example.

Algorithm: Hourglass sums. **Input:** rows = `4`, cols = `4`, data = `1 2 3 4 5 6 7 8 9 1 2 3 4 5 6 7`.

Pseudocode

```
best = -infinity
for i in 0..R-3:
 for j in 0..C-3:
 s = a[i][j..j+2] + a[i+1][j+1] + a[i+2][j..j+2]
 best = max(best, s)
```

Step 1 of 4. Hourglass with top-left corner (0, 0): 1 + 2 + 3 + 6 + 9 + 1 + 2 = 24. Best so far 24.

i = 0 · j = 0 · sum = 24 · best = 24

1	2	3	4
5	6	7	8
9	1	2	3
4	5	6	7

Step 2 of 4. Hourglass with top-left corner (0, 1): 2 + 3 + 4 + 7 + 1 + 2 + 3 = 22. Best so far 24.

i = 0 · j = 1 · sum = 22 · best = 24

1	2	3	4
5	6	7	8
9	1	2	3
4	5	6	7

Step 3 of 4. Hourglass with top-left corner (1, 0): 5 + 6 + 7 + 1 + 4 + 5 + 6 = 34. Best so far 34.

i = 1 · j = 0 · sum = 34 · best = 34

1	2	3	4
5	6	7	8
9	1	2	3
4	5	6	7

Step 4 of 4. Hourglass with top-left corner (1, 1): 6 + 7 + 8 + 2 + 5 + 6 + 7 = 41. Best so far 41.

i = 1 · j = 1 · sum = 41 · best = 41

1	2	3	4
5	6	7	8
9	1	2	3
4	5	6	7

> **RESULT**
> maximum hourglass sum = 41

Complexity

(R−2)(C−2) hourglasses × 7 additions = **O(R·C) time**, O(1) extra space. For a fixed shape the constant 7 disappears into the O.

For a general k×k window the naive cost is O(R·C·k²); with a 2-D prefix-sum table it drops to O(R·C) (challenge exercise).

Common mistakes

Initial maximum of 0 (slide)

Problem

```
int max_sum = 0;
```

Why: With all-negative hourglass sums the program prints 0, which is wrong.

Fix: int max = Integer.MIN_VALUE;

Loop bounds off by two

Problem

```
for (int i = 0; i < rows; i++) // then m[i + 2][...] → exception
```

Why: The pattern reaches two rows and two columns beyond the corner.

Fix: for (int i = 0; i + 2 < rows; i++)

Using the centre cell twice or the wrong middle row

Problem

```
m[i + 1][j] + m[i + 1][j + 1] + m[i + 1][j + 2]
```

Why: That is a full middle row (a 3×3 block), not an hourglass: only the CENTRE of the middle row belongs to it.

Fix: m[i + 1][j + 1]

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · One hourglass — *Beginner*

Read a 3×3 matrix (nine integers) and print its hourglass sum.

Example input	Expected output
1 1 1 ⏎ 0 1 0 ⏎ 1 1 1	7

Hint 1: Skip m[1][0] and m[1][2].

Exercise 2 · Maximum hourglass with negatives — *Intermediate*

Read R, C and the matrix (R, C ≥ 3). Print the maximum hourglass sum.

- 3 ≤ R, C ≤ 100
- −100 ≤ value ≤ 100

Example input	Expected output
3 3 ⏎ −1 −1 −1 ⏎ −1 −9 −1 ⏎ −1 −1 −1	−15

Hint 1: Start the maximum at Integer.MIN_VALUE.

Exercise 3 · Largest k×k block — *Challenge*

Read R, C, k and a matrix. Print the maximum sum of any k×k sub-matrix in O(R·C) time using 2-D prefix sums.

- 1 ≤ k ≤ min(R, C) ≤ 1000

Example input	Expected output
3 3 2 ⏎ 1 2 3 ⏎ 4 5 6 ⏎ 7 8 9	28

Hint 1: p[i][j] = sum of the rectangle (0,0)…(i−1,j−1). **Hint 2:** Block sum = p[i][j] − p[i−k][j] − p[i][j−k] + p[i−k][j−k].

Quick check

Answers are in the answer key at the end of this part.

Q1. How many hourglasses fit in a 5×6 matrix?

- **A.** 12
- **B.** 15
- **C.** 20
- **D.** 30

Q2. Which cells belong to the hourglass anchored at (i, j)?

- **A.** the full 3×3 block
- **B.** two full rows of 3 and the centre cell
- **C.** the main diagonals
- **D.** a plus sign

Q3. What is the hourglass sum of this 3×3? 1 2 3 / 4 5 6 / 7 8 9

```
// a b c / d / e f g
```

Your answer: _____

Q4. Why initialise the maximum with Integer.MIN_VALUE?

- **A.** It is faster
- **B.** All sums may be negative
- **C.** Arrays start at 0
- **D.** Required by Java

Interview / exam questions

- How many hourglasses are in an R×C matrix and why?
- How would you generalise to a k×k window in O(R·C)?
- What if the matrix is stored row by row in a 1-D array?

Watch & read

Video: 2D Array- DS - Hourglass |ARRAYS |HACKERRANK Solution| PYTHON | Interview |Programming for Beginners — Atharv Mudur · 6:53. *Scanning a 2D array for hourglasses (Python — the idea transfers to Java)*

Video: C_56 2D Arrays Program 1| Program to print Matrix and calculate Sum | C Programming — Jenny's Lectures CS IT · 17:33. *Window sums in a matrix (C — the idea transfers to Java)*

REVISION CARD (30–60 SECONDS)

- Hourglass at (i, j): m[i][j..j+2], m[i+1][j+1], m[i+2][j..j+2].
- Positions: i in 0..R−3, j in 0..C−3 → (R−2)(C−2).
- Initial max = Integer.MIN_VALUE.
- O(R·C) time, O(1) space.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide initialises `max_sum = 0` (wrong for negative data) and exits the JVM with `System.exit(0)` inside the helper. The lesson program avoids both. • The slide's worked sums (24, 22, 34, 41) are correct.

5.19 Toggle the Switch (100 Doors)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 40 min · ★ on the minimum-viable path

Monkeys flip doors by multiples. Why do exactly the perfect squares stay open?

YOU WILL BE ABLE TO

- Simulate the doors puzzle with nested loops and a boolean array.
- Explain why a door is toggled once per divisor of its number.
- Prove that only perfect squares have an odd number of divisors and write an O(1) solution.
- Compute integer square roots safely for 64-bit inputs.

Before this lesson: Loops: for, while, do-while, break and continue · One-Dimensional Arrays · Divisibility, GCD/LCM, Primes and Modular Arithmetic

The idea

There are 100 closed doors and 100 monkeys. Monkey 1 toggles every door; monkey 2 toggles doors 2, 4, 6, ...; monkey 3 toggles 3, 6, 9, ...; monkey k toggles every k-th door. How many doors are open at the end?

Door number **n** is touched by monkey k exactly when **k divides n**. So door 12 is toggled by monkeys 1, 2, 3, 4, 6, 12 — six times → closed again. Door 16 by monkeys 1, 2, 4, 8, 16 — five times → **open**.

WHY THIS WORKS

Divisors come in pairs (d, n/d). Door 12: (1,12), (2,6), (3,4) — an even count, so it ends closed. The count is odd only if one pair is "d with itself", i.e. d = n/d, i.e. $n = d^2$ is a perfect square. Hence the open doors are exactly 1, 4, 9, ..., 100: **10 doors**, and for n doors the answer is $\lfloor\sqrt{n}\rfloor$.

```
door: 1 2 3 4 5 6 7 8 9 10 ...
toggles: 1 2 2 3 2 4 2 4 3 4
final: O c c O c c c c O c (O = open: 1, 4, 9)
```

Syntax

```
boolean[] open = new boolean[n + 1]; // index 1..n, all closed
for (int pass = 1; pass <= n; pass++)
 for (int door = pass; door <= n; door += pass) // every pass-th door
 open[door] = !open[door]; // toggle
int count = (int) Math.sqrt(n); // the closed form: perfect squares <= n
```

A tiny example

```
for (int monkey = 1; monkey <= n; monkey++)
 for (int door = monkey; door <= n; door += monkey)
 open[door] = !open[door];
```

Guided example

DoorsSimulation.java — Doors by simulation (simple solution)

```
1  import java.util.Scanner;
2
3  public class DoorsSimulation {
4   // Monkey i visits doors i, 2i, 3i, ... and toggles each one.
5   public static void main(String[] args) {
6   Scanner sc = new Scanner(System.in);
7   int n = sc.nextInt();
8   boolean[] open = new boolean[n + 1]; // all doors start closed                (1)
9   for (int monkey = 1; monkey <= n; monkey++) {
10  for (int door = monkey; door <= n; door += monkey) {                          (2)
11  open[door] = !open[door];                                                     (3)
12  }
13  }
14  StringBuilder sb = new StringBuilder();
15  int count = 0;
16  for (int door = 1; door <= n; door++) {
17  if (open[door]) {
18  count++;
19  sb.append(door).append(' ');
20  }
21  }
22  System.out.println("open doors: " + count);
23  System.out.println(sb.toString().trim());
24  }
25 }
```

A boolean array and the two nested loops of the puzzle.

Sample input

```
100
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Index = door number (1-based), false = closed.
2	line 10	Monkey k visits k, 2k, 3k, ...: step by k.
3	line 11	Toggle: open ↔ closed.

DoorsSquares.java — Doors by mathematics (optimised solution)

```
1  import java.util.Scanner;
2
3  public class DoorsSquares {
4   // A door is toggled once for every divisor of its number. Divisors come in pairs (d,
        n/d), so the
5   // count is odd only when d == n/d, i.e. the number is a perfect square. Open doors =
        perfect squares.
6   static long isqrt(long n) {
7   long r = (long) Math.sqrt((double) n);                                          (1)
8   while (r * r > n) r--;                                                          (2)
9   while ((r + 1) * (r + 1) <= n) r++;                                             (3)
10  return r;
11  }
12
13  public static void main(String[] args) {
14  Scanner sc = new Scanner(System.in);
15  long n = sc.nextLong();
16  System.out.println("open doors: " + isqrt(n));
17  }
18 }
```

The number of perfect squares ≤ n is $\lfloor\sqrt{n}\rfloor$. Works for n up to 10^{18}.

Sample input

```
100
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Floating-point sqrt may be off by one for large n (double has 53 bits of precision).
2	line 8	Correct an overestimate...
3	line 9	...and an underestimate. Now r is exactly $\lfloor\sqrt{n}\rfloor$.

Dry run: watch it execute

After each monkey, which doors are open.

Algorithm: Toggle the doors. **Input:** n = `20`.

Pseudocode

```
for monkey = 1..n:
 for door = monkey; door <= n; door += monkey:
 open[door] = !open[door]
```

The full trace has 21 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 21. All doors start closed.

monkey = 0 · open = 0

doors (O = open, c = closed)

c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c	c
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Step 3 of 21. Monkey 2 toggles doors 2, 4, 6, … ≤ 20.

monkey = 2 · open = 10

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	0	c	0	c	0	c	0	c	0	c	0	c	0	c	0	c	0	c

Step 4 of 21. Monkey 3 toggles doors 3, 6, 9, … ≤ 20.

monkey = 3 · open = 10

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	c	0	0	0	c	c	c	0	0	0	c	c	c	0	0	0	c

Step 6 of 21. Monkey 5 toggles doors 5, 10, 15, … ≤ 20.

monkey = 5 · open = 13

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	0	0	0	c	0	0	c	0	c	0	0	0	0	0	c

Step 7 of 21. Monkey 6 toggles doors 6, 12, 18, … ≤ 20.

monkey = 6 · open = 12

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	0	0	c	0	0	0	0	c	0	0	0	c	0	c

Step 9 of 21. Monkey 8 toggles doors 8, 16, 24, … ≤ 20.

monkey = 8 · open = 10

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	c	c	c	0	0	0	0	0	0	c	0	c	0	c

Step 10 of 21. Monkey 9 toggles doors 9, 18, 27, … ≤ 20.

monkey = 9 · open = 12

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	c	c	0	0	0	0	0	0	0	c	0	0	0	c

Step 12 of 21. Monkey 11 toggles doors 11, 22, 33, … ≤ 20.

monkey = 11 · open = 11

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	c	c	0	c	c	0	0	0	0	c	0	0	0	0

Step 13 of 21. Monkey 12 toggles doors 12, 24, 36, … ≤ 20.

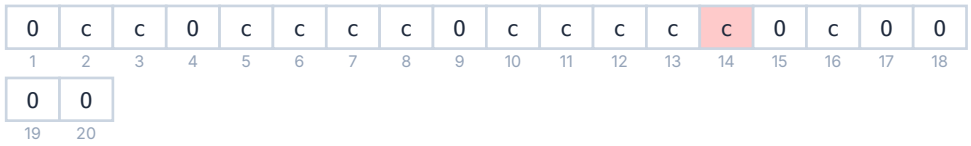
monkey = 12 · open = 10

doors (O = open, c = closed)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	c	c	0	c	c	c	c	0	c	c	c	0	0	0	c	0	0	0	0

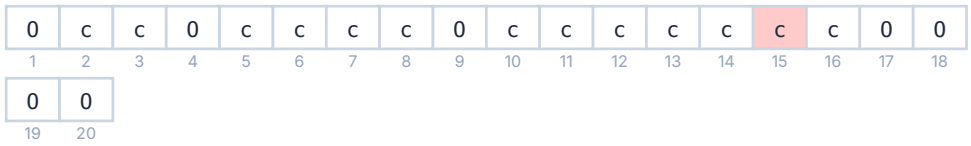
Step 15 of 21. Monkey 14 toggles doors 14, 28, 42, … ≤ 20.

monkey = 14 · open = 8



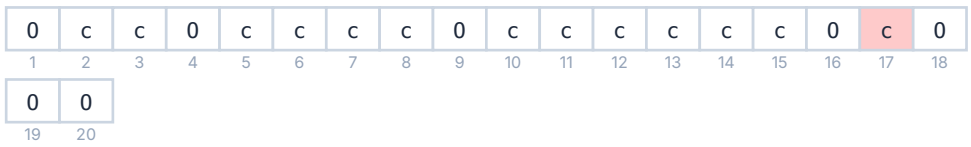
Step 16 of 21. Monkey 15 toggles doors 15, 30, 45, … ≤ 20.

monkey = 15 · open = 7



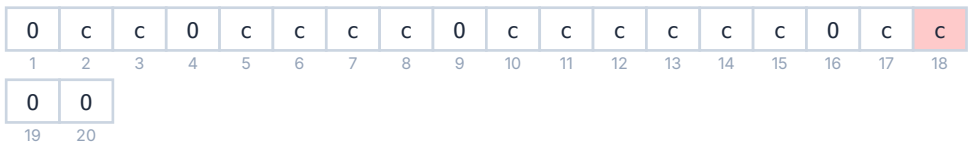
Step 18 of 21. Monkey 17 toggles doors 17, 34, 51, … ≤ 20.

monkey = 17 · open = 7



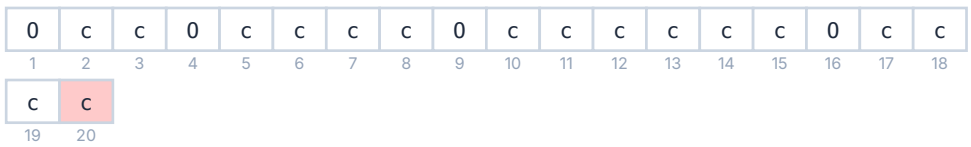
Step 19 of 21. Monkey 18 toggles doors 18, 36, 54, … ≤ 20.

monkey = 18 · open = 6



Step 21 of 21. Monkey 20 toggles doors 20, 40, 60, … ≤ 20.

monkey = 20 · open = 4



> **RESULT**
> 4 open doors: 1 4 9 16

Complexity

Simulation: monkey k does $\lfloor n/k \rfloor$ toggles; the total is $n \cdot (1 + 1/2 + \ldots + 1/n)$ = **O(n log n)** (harmonic series). Space O(n).

Mathematical: O(1) time and space (or O(log n) with a binary-search square root).

Common mistakes

Toggling door j instead of door j·i (the slide code)

Problem

```
for (i = 1; i <= n; i++)
 for (j = i; j * i <= n; j++)
 b[j] = !b[j]; // toggles door j, not door i*j
```

Why: The slide's loop never visits "every i-th door"; it toggles consecutive doors j with j·i ≤ n. For n = 100 it reports far more than 10 open doors.

Fix: for (j = i; j <= n; j += i) b[j] = !b[j];

Counting open doors with floating-point sqrt

Problem

```
int open = (int) Math.sqrt(n); // n up to 10^18
```

Why: For large n the double rounding can give 999999999 or 1000000001 instead of 10^9.

Fix: Adjust with the two while-loops after Math.sqrt.

Using int for n

Problem

```
int n = sc.nextInt(); // n = 10^18
```

Why: Does not fit in an int; Scanner throws InputMismatchException.

Fix: long n = sc.nextLong();

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count the open doors — *Beginner*

Read n (≤ 1000) and simulate the monkeys. Print the number of doors that are open at the end.

Example input	Expected output
10	3

Hint 1: boolean[] and two loops.

Exercise 2 · A trillion doors — *Intermediate*

Read n up to 10^{18}. Print how many doors are open.

Example input	Expected output
1000000000000000000	1000000000

Hint 1: Open doors are perfect squares. **Hint 2:** Use an exact integer square root.

Exercise 3 · Only the first m monkeys — *Challenge*

Only monkeys 1..m are released (m ≤ n). Read n and m and print how many doors are open. (n ≤ 10^6)

Example input	Expected output
10 3	4

Hint 1: A door is toggled once per divisor ≤ m. **Hint 2:** Simulation over m monkeys is O(n log m).

Quick check

Answers are in the answer key at the end of this part.

Q1. After all 100 monkeys, how many doors are open?

- **A.** 5
- **B.** 10
- **C.** 25
- **D.** 50

Q2. Door 36 is toggled how many times?

- **A.** 8
- **B.** 9
- **C.** 10
- **D.** 6

Q3. Why is the number of divisors odd only for perfect squares?

- **A.** Primes have 2 divisors
- **B.** Divisors pair up as (d, n/d) except when d = n/d
- **C.** Squares are even
- **D.** Because of overflow

Q4. The simulation's time complexity is...

- **A.** O(n)
- **B.** O(n log n)
- **C.** $O(n^2)$
- **D.** $O(\sqrt{n})$

Interview / exam questions

- Why do only perfect squares stay open?
- How would you compute $\lfloor\sqrt{n}\rfloor$ exactly for n up to 10^{18}?
- What changes if the monkeys toggle only doors up to some limit?

Watch & read

Video: 100 Doors in a Row | Perfect Square Numbers Magic | Train your Brain | Puzzle — PRiyanka Singh · 9:49. *Why only perfect squares remain open*

Video: 100 Doors Interview Puzzle || Microsoft Interview Puzzle — Simply Logical · 5:26. *The 100-doors puzzle*

REVISION CARD (30–60 SECONDS)

- Monkey k toggles multiples of k; door n flips once per divisor.
- Odd number of divisors ⇔ perfect square.
- Open doors = $\lfloor\sqrt{n}\rfloor$.
- Simulation O(n log n); formula O(1).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide's `Toggle` program toggles b[j] for j from i while j·i ≤ n. That is not "every i-th door"; running it for n = 100 reports many more than 10 open doors. The lesson uses the correct inner loop `door += monkey`. • The deck combines two problems (this one and "Alice's Apple Tree"); each has its own lesson.

5.20 Alice's Apple Tree (Minimum Garden Perimeter)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 45 min · complete path

Apples grow at every grid point; find the smallest square plot that contains enough — with a closed form and binary search.

YOU WILL BE ABLE TO

- Model the garden: tree (i, j) has |i| + |j| apples.
- Derive the apples in a ring and in a whole square: $12r^2$ and $2r(r+1)(2r+1)$.
- Find the smallest r with a loop, then with binary search on a monotone function.
- Use `long` everywhere and avoid overflow.

Before this lesson: Loops: for, while, do-while, break and continue · Toggle the Switch (100 Doors) · Searching: Linear and Binary Search

The idea

Alice stands at (0, 0) on an infinite grid. The tree at integer point (i, j) bears |i| + |j| apples. She wants a **square plot, centred at the origin and aligned with the axes**, that contains at least `neededApples` apples inside or on its border. What is the smallest possible **perimeter**?

A square with half-side r has corners (±r, ±r) and perimeter 8r. Going from half-side r−1 to r adds a "ring". Counting the apples in that ring gives **$12 \cdot r^2$** (the slide's `sum += 12*cnt*cnt`). Adding the rings 1..r:

$$\text{apples}(r) = \Sigma\ 12k^2\ (k = 1..r) = 2r(r+1)(2r+1) = 4r^3 + 6r^2 + 2r$$

That matches the slide's text: "The total apples is r·r·r·4 + r·r·6 + r·2." The answer is 8·r for the smallest r with apples(r) ≥ needed.

WHY THIS WORKS

apples(r) strictly increases with r, so the set of "good" values of r is an interval [r, ∞). *That monotone property is exactly what binary search** needs: test the middle, discard half. The loop version adds ring by ring and is just as correct but takes O(r) steps, and r can be 500 000 for needed = 10^{15}.

Syntax

```
long lo = 1, hi = 1_000_000; // search the answer (the radius) itself
while (lo < hi) {
 long mid = lo + (hi - lo) / 2;
 if (apples(mid) >= needed) hi = mid; else lo = mid + 1; // monotone yes/no test
}
return 8 * lo; // perimeter of the square
static long apples(long r) { return 2 * r * (r + 1) * (2 * r + 1); }
```

A tiny example

```
long apples = 0, r = 0;
while (apples < needed) {
 r++;
 apples += 12 * r * r; // the ring at distance r
}
return 8 * r;
```

Guided example

Check: needed = 13 → r = 2 (apples(1) = 12 < 13, apples(2) = 60) → perimeter 16. needed = 1 → r = 1 → 8. needed = 10^9 → r = 630 → perimeter **5040**.

AppleTreeLoop.java — Ring by ring (simple solution)

```
1  import java.util.Scanner;
2
3  public class AppleTreeLoop {
4   // Alice stands at the origin of an infinite grid; the tree at (i, j) has |i| + |j|
        apples.
5   // She buys a square plot [-r, r] x [-r, r] (perimeter 8r) and keeps every apple inside.
6   // The apples in the NEW ring when the plot grows from r-1 to r are exactly 12 * r * r.
7   public static void main(String[] args) {
8   Scanner sc = new Scanner(System.in);
9   long needed = sc.nextLong();                                                    (1)
10  long r = 0;
11  long apples = 0;
12  while (apples < needed) {
13  r++;
14  apples += 12 * r * r; // add the ring at distance r                             (2)
15  }
16  System.out.println("half-side r = " + r + ", total apples = " + apples + ", minimum   (3)
        perimeter = " + 8 * r);
17  }
18 }
```

Adds $12r^2$ for r = 1, 2, 3... until the total reaches the target. O(r).

Sample input

```
1000000000
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Targets up to 10^{15} do not fit in int.
2	line 14	All apples in the new ring: the slide uses exactly this term. `r` is long so r*r cannot overflow int.
3	line 16	A square of half-side r has perimeter 8r.

AppleTreeFormula.java — Binary search (optimised solution)

```
1  import java.util.Scanner;
2
3  public class AppleTreeFormula {
4   // Apples inside the square of half-side r: 2 r (r + 1) (2 r + 1) (= 4r^3 + 6r^2 + 2r)
5   static long applesInside(long r) {                                                    (1)
6   return 2 * r * (r + 1) * (2 * r + 1);
7   }
8
9   public static void main(String[] args) {
10  Scanner sc = new Scanner(System.in);
11  long needed = sc.nextLong();
12  long lo = 1;
13  long hi = 200_000; // 2 * 200000 * 200001 * 400001 > 3 * 10^16, enough for needed <=   (2)
        10^15
14  while (lo < hi) { // smallest r with applesInside(r) >= needed
15  long mid = (lo + hi) / 2;
16  if (applesInside(mid) >= needed) {                                                    (3)
17  hi = mid;
18  } else {
19  lo = mid + 1;                                                                         (4)
20  }
21  }
22  System.out.println(8 * lo);
23  }
24 }
```

Uses apples(r) = 2r(r+1)(2r+1). O(log R).

Sample input

```
1000000000000000
```

Line-by-line walkthrough

#	Where	What happens
1	line 5	The closed form of the ring sums.
2	line 13	An upper bound for r: apples(200000) ≈ 3.2 × 10^{16}, which exceeds the largest allowed target of 10^{15}, and the product still fits in a long.
3	line 16	mid is enough: the answer is mid or smaller.
4	line 19	mid is too small: the answer is larger.

Dry run: watch it execute

Rings add 12, 48, 108, ... apples.

Algorithm: Alice's apple tree. **Input:** needed = `13`.

Pseudocode

```
r = 0; apples = 0
while (apples < needed) {
 r++
 apples += 12 * r * r
}
return 8 * r
```

Step 1 of 3. Need at least 13 apples.

r = 0 · apples = 0 · perimeter = 0

Step 2 of 3. Ring 1 adds $12 \cdot 1^2 = 12$ apples. Total 12.

r = 1 · apples = 12 · perimeter = 8

Step 3 of 3. Ring 2 adds $12 \cdot 2^2 = 48$ apples. Total 60.

r = 2 · apples = 60 · perimeter = 16

> **RESULT**
> r = 2, perimeter = 16

Complexity

Loop: r iterations where r ≈ (needed/4)^(1/3) → **O(needed^(1/3))**. For needed = 10^{15} that is about 63 000... (r = 503 968 at 10^{15} — roughly half a million iterations, still instant).

Binary search: $\log_2(200\,000) \approx 18$ iterations → **O(log R)**.

> **WATCH OUT**
> Intermediate products: $2 \cdot r \cdot (r+1) \cdot (2r+1)$ for r = 200 000 is about 3.2×10^{16} << 9.2×10^{18}, so `long` is safe. With `int` it overflows already for r ≈ 800.

Common mistakes

Using int (the slide)

Problem

```
int apple = sc.nextInt();
int cnt = 0, sum = 0;
sum += 12 * cnt * cnt;
```

Why: The slide's code reads an int. $12 \cdot cnt^2$ overflows int for cnt > 13 000 and the target itself may be up to 10^{15}.

Fix: long needed; long r; long apples;

Returning r instead of the perimeter

Problem

```
System.out.println(r);
```

Why: The question asks for the perimeter 8r.

Fix: System.out.println(8 * r);

Binary search with an unsafe upper bound

Problem

```
long hi = 1000; // for needed up to 10^15
```

Why: apples(1000) ≈ $4 \cdot 10^9 < 10^{15}$ so the search cannot find the answer.

Fix: long hi = 200_000; or double hi until applesInside(hi) >= needed

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Apples in a square — *Beginner*

Read r. Print the number of apples inside the square with corners (±r, ±r).

Example input	Expected output
3	168

Hint 1: Use the closed form 2r(r+1)(2r+1). **Hint 2:** Check r = 1: 12.

Exercise 2 · Minimum perimeter (small) — *Intermediate*

Read neededApples (≤ 10^9) and print the minimum perimeter.

Example input	Expected output
13	16

Hint 1: Add the rings $12r^2$ until the total is ≥ needed.

Exercise 3 · Minimum perimeter (large) — *Challenge*

Read neededApples (≤ 10^{15}) and print the minimum perimeter, using binary search.

Example input	Expected output
1000000000000000	503968

Hint 1: apples(r) = 2r(r+1)(2r+1) is increasing. **Hint 2:** hi = 200000 is a valid upper bound.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many apples are in the ring at distance r = 2?

- **A.** 24
- **B.** 48
- **C.** 60
- **D.** 12

Q2. What does applesInside(2) return?

```
static long applesInside(long r) {
 return 2 * r * (r + 1) * (2 * r + 1);
}
```

Your answer: _____

Q3. Why can binary search be used here?

- **A.** The array is sorted
- **B.** apples(r) is monotonically increasing
- **C.** r is always even
- **D.** The answer is a prime

Q4. What is the perimeter of the square with half-side r?

- **A.** 4r
- **B.** 8r
- **C.** r^2

- **D.** 2r + 2

Interview / exam questions

- Derive the number of apples in the r-th ring.
- Why is binary search valid and what is its upper bound?
- How do you avoid overflow?

Watch & read

Video: Minimum Garden Perimeter To Collect Enough Apples leetcode weekly contest 252 Leetcode 1954. — CodeinDepth · 9:51. *Counting apples by layers; binary search on the answer*

Video: Binary Search | Monotonic Function | L-29 — Programming Pathshala · 11:57. *Binary search on the answer*

REVISION CARD (30–60 SECONDS)

- Ring r holds $12r^2$ apples; square of half-side r holds $2r(r+1)(2r+1)$.
- Perimeter = 8r.
- Smallest r with apples(r) ≥ needed → loop or binary search.
- Use long: 10^{15} needs r ≈ 503 968.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide's code uses int and a linear loop although the text on the previous slide says "then we apply binary search". Both appear here; the long type is required for the full constraint range (this is LeetCode 1954, "Minimum Garden Perimeter to Collect Enough Apples").

ASSUMPTIONS MADE

- The problem statement is only given in diagrams on the slides; the lesson uses the standard statement (square centred at the origin, tree at (i, j) with |i| + |j| apples).

5.21 Weighted Substring (Smallest String of a Given Weight)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 50 min · complete path

Letters have factorial-like weights; build the shortest, then alphabetically smallest, string with a given total weight.

YOU WILL BE ABLE TO

- Generate the letter weights A = 1, B = 3, C = 12, D = 60 … from the rule weight(n) = n·weight(n−1) + weight(n−1).
- Explain why the greedy "biggest letter first" gives the fewest letters.
- Sort the letters to get the lexicographically smallest string of that length.
- Use long (weights reach 10^{16}) and stop generating before overflow.

Before this lesson: Strings, StringBuilder and Characters · Searching: Linear and Binary Search · Sorting with Comparable and Comparator

The idea

The weight of a letter depends on the previous one: A = 1, B = 2·A + A = 3, C = 3·B + B = 12, D = 4·C + C = 60 … so each weight is **(position + 1) × previous weight** (B = 3·1, C = 4·3, D = 5·12). These are the numbers 2!/2, 3!/2, 4!/2, 5!/2 … = 1, 3, 12, 60, 360, 2520 …

The weight of a string is the sum of its letters' weights. Given a target weight W, find the **shortest** string with exactly that weight and, among shortest strings, the **lexicographically smallest**. For W = 25 the strings `ABBBBC`, `ACC`, `AAAAAAABBBBBB` all weigh 25; the shortest is `ACC`.

WHY THIS WORKS

Each weight divides the next one (1 | 3 | 12 | 60 | …), like coins in a currency where each coin is a multiple of the previous. For such "canonical" coin systems the **greedy** strategy — always take the largest coin that fits — uses the fewest coins. Which letters appear is therefore fixed; to make the string alphabetically smallest, simply put them in ascending order.

```
W = 20 largest weight ≤ 20 is C = 12 → C, remaining 8
 largest ≤ 8 is B = 3 → B, remaining 5
 B again → B, remaining 2
 A, A → remaining 0
letters C B B A A sorted → A A B B C
```

Syntax

```
char letter = (char) ('A' + index); // 0 -> 'A', 1 -> 'B', …
List<Character> letters = new ArrayList<>();
letters.add(letter);
Collections.sort(letters); // smallest letters first
StringBuilder sb = new StringBuilder();
for (char c : letters) sb.append(c);
String result = sb.toString();
long next = weight * (n + 1); // use long: weights grow factorially
```

A tiny example

```
long[] w = {1, 3, 12, 60, 360};
long remaining = 20;
for (int i = w.length - 1; i >= 0; i--)
 while (remaining >= w[i]) { /* take letter i */ remaining -= w[i]; }
```

Guided example

WeightedString.java — Smallest string of a given weight (optimised solution)

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class WeightedString {
 // weight(A) = 1, weight(B) = 2*A + A, weight(C) = 3*B + B, ... i.e. weight(letter n) = (n + 1) * weight(letter n - 1)
 static long[] buildWeights(long limit) {
 List<Long> w = new ArrayList<>();
 long cur = 1; // weight of 'A'                                                    (1)
 w.add(cur);
 for (int n = 2; n <= 26; n++) {
 long next = cur * (n + 1); // (n+1) * previous weight, from "n*prev + prev"      (2)
 if (next > limit || next / (n + 1) != cur) break; // stop before exceeding the limit / overflowing (3)
 w.add(next);
 cur = next;
 }
 long[] arr = new long[w.size()];
 for (int i = 0; i < arr.length; i++) arr[i] = w.get(i);
 return arr;
 }

 // Shortest string with the given weight, lexicographically smallest among those of that length.
 static String smallestString(long weight) {
 long[] w = buildWeights(weight);
 List<Character> letters = new ArrayList<>();
 long remaining = weight;
 for (int i = w.length - 1; i >= 0; i--) { // greedy: take the biggest letters first
 while (remaining >= w[i]) {                                                     (4)
 letters.add((char) ('A' + i));
 remaining -= w[i];
 }
 }
 Collections.sort(letters); // smallest letters first = lexicographically smallest (5)
 StringBuilder sb = new StringBuilder();
 for (char c : letters) sb.append(c);
 return sb.toString();
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long weight = sc.nextLong();
 System.out.println(smallestString(weight));
 }
}
```

Builds weights up to the limit, takes letters greedily from the largest, then sorts.

Sample input

```
20
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	weight(A) = 1.
2	line 13	weight(n) = (n+1) · weight(n−1): this is "n·prev + prev".
3	line 14	No letter heavier than the target is ever needed; stopping early also prevents overflow (26 letters would overflow long).
4	line 29	Greedy: use the current letter as many times as it fits.
5	line 34	Ascending order is the lexicographically smallest arrangement of this multiset.

> **NOTE**
> The slide uses a binary search (`findFloor`) to find the largest weight ≤ k each time. With at most 17 usable weights a simple backwards loop is just as fast; the binary search matters only if the alphabet were huge.

Dry run: watch it execute

Greedy from the heaviest letter: 100 = 60 + 3×12 + 3 + 1.

Algorithm: Smallest string of a given weight. **Input:** w = `100`.

Pseudocode

```
weights: A=1, B=3, C=12, D=60, E=360, …
remaining = weight
for letter from largest to smallest:
 while remaining >= weight(letter): take it
sort the letters (smallest first)
```

Step 1 of 6. Weights not exceeding 100: A=1, B=3, C=12, D=60.

weight = 100

letters	A	B	C	D
	0	1	2	3

weights	1	3	12	60
	0	1	2	3

Step 2 of 6. Letter D (weight 60) fits 1 time: take D. Remaining 40.

letter = D · taken = 1 · remaining = 40

letters	A	B	C	D
	0	1	2	3

letters taken so far	D
	0

Step 3 of 6. Letter C (weight 12) fits 3 times: take CCC. Remaining 4.

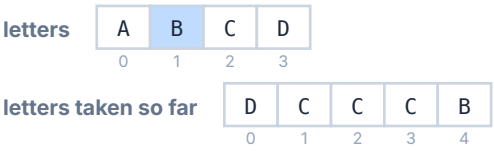
letter = C · taken = 3 · remaining = 4

letters	A	B	C	D
	0	1	2	3

letters taken so far	D	C	C	C
	0	1	2	3

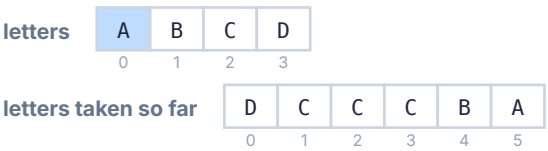
Step 4 of 6. Letter B (weight 3) fits 1 time: take B. Remaining 1.

letter = B · taken = 1 · remaining = 1



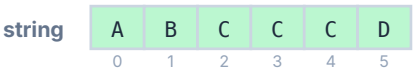
Step 5 of 6. Letter A (weight 1) fits 1 time: take A. Remaining 0.

letter = A · taken = 1 · remaining = 0



Step 6 of 6. Sort the letters so the string is lexicographically smallest: ABCCCD.

length = 6



> **RESULT**
> ABCCCD (6 letters)

Complexity

Weights grow factorially, so only about 17 letters are ever below 10^{16}. The greedy loop takes at most the sum over letters of (weight(next)/weight(this)) − 1 ≤ i + 1 copies each — at most ~160 letters. Time **O(L²)** or O(L log L) with the sort where L = answer length (tiny); space O(L).

Common mistakes

int weights (the slide)

Problem

```
static int[] values = new int[26];
```

Why: weight(M) = 14!/2 ≈ 4.4 × 10^{10} already overflows int and the constraint is up to 10^{16}.

Fix: long[] and a stopping rule

Computing all 26 weights

Problem

```
for (int i = 1; i < 26; i++) values[i] = (i + 1) * prev + prev;
```

Why: weight(Z) = 27!/2 ≈ 5 × 10^{27} overflows even long, giving negative nonsense that confuses the search.

Fix: Stop when the next weight exceeds the target.

Printing without sorting

Problem

```
print letters in the order they were chosen (largest first): "CBBAA"
```

Why: The task asks for the lexicographically smallest string, "AABBC".

Fix: sort ascending (the slide reverses its list, which gives the same effect)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Weight of a string — *Beginner*

Read a string of uppercase letters and print its total weight (A=1, B=3, C=12, ..., with weight(n) = (n+1)·weight(n−1)).

- length ≤ 10
- only letters A–J

Example input	Expected output
ACC	25

Hint 1: Build the weight table once. **Hint 2:** Sum table[c − 'A'].

Exercise 2 · Smallest string — *Intermediate*

Read a weight W ($1 \le W \le 10^{16}$) and print the shortest string of weight W, choosing the lexicographically smallest among the shortest.

Example input	Expected output
25	ACC
20	AABBC

Hint 1: Greedy from the heaviest letter that fits. **Hint 2:** Sort at the end.

Exercise 3 · Normalise a string — *Challenge*

Read a string of uppercase letters (up to 10^5 characters). Print the shortest, lexicographically smallest string with the same total weight — without ever computing the (possibly huge) total.

Example input	Expected output
AAAAB	ABB

Hint 1: (i + 2) copies of letter i weigh the same as one copy of letter i + 1 (A: 3 copies = B; B: 4 copies = C). **Hint 2:** Count letters, then carry from A upwards like long addition.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is the weight of D?

- **A.** 12
- **B.** 36
- **C.** 60
- **D.** 120

Q2. Shortest string of weight 25?

- **A.** ABBBBC
- **B.** ACC
- **C.** AAAAAAABBBBBB
- **D.** BBBBBBBBA

Q3. What string does the lesson's method produce for weight 7?

```
// weights A=1 B=3 C=12
```

Your answer: _____

Q4. Why does greedy work for these weights?

- **A.** They are prime
- **B.** Each weight divides the next
- **C.** They are powers of two
- **D.** Because letters are sorted

Interview / exam questions

- Why is greedy optimal when each coin divides the next?
- Give a coin system where greedy fails.
- How would you handle weights up to 10^{18} or more?

Watch & read

Video: Smallest String With A Given Numeric Value | Leetcode 1663 | Maths | Live coding session — Coding Decoded · 6:44. *Greedy construction of the smallest string*

Video: Introduction to Greedy Algorithms | GeeksforGeeks — GeeksforGeeks · 5:32. *Greedy reasoning*

REVISION CARD (30–60 SECONDS)

• weight(n) = (n+1)·weight(n−1): 1, 3, 12, 60, 360 ...
• Greedy largest-first = fewest letters; sort for lexicographic order.
• Use long; stop generating weights at the target.
• Answer length is tiny (≤ ~160).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide says the output "should be in the form of integer" but its sample output is the string AABBC for input 20 (a string, not an integer). • The slide stores weights in int[26] (overflows) and reads the weight with nextInt although the constraint is up to 10^{16}.

5.22 Move Hyphen to the Beginning

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 30 min · ★ on the minimum-viable path

Rearrange a string with `indexOf`, `substring` and StringBuilder — then do it for every hyphen in one pass.

YOU WILL BE ABLE TO

- Use `indexOf`, `substring`, `contains` and string concatenation correctly.
- Explain that Java Strings are immutable and what that means for repeated edits.
- Move all hyphens to the front, preserving the order of the other characters.
- Do it in one pass over a `char[]`, scanning from the right.

Before this lesson: Strings, StringBuilder and Characters · Methods: Parameters, Return Values, Scope and Overloading

The idea

The slide transforms `"face-prep"` into `"-faceprep"`: find the hyphen, take everything before it and everything after it, and put the hyphen first.

```
face-prep indexOf('-') = 4
face | - | prep
"-" + "face" + "prep" → -faceprep
```

The slide solves only the **first** hyphen. A more interesting problem moves **all** hyphens to the front without disturbing the other characters: `a-b-c-d` → `---abcd`. This is a *stable partition*: hyphens first, everything else in its original order.

WHY THIS WORKS

Scanning from the right, write every non-hyphen character into the rightmost free slot; whatever slots remain on the left must be hyphens (their count equals the number of hyphens). Because each character is read at or before the position where it is written, nothing is overwritten before being read — so one array and one pass suffice.

Syntax

method	returns	example (s = "face-prep")
s.indexOf('-')	index of first match, or −1	4
s.contains("-")	boolean	true
s.substring(a, b)	chars a … b−1	substring(0, 4) = "face"
s.substring(a)	chars a … end	substring(5) = "prep"
s.toCharArray()	a char[] copy	[f,a,c,e,-,p,r,e,p]

WATCH OUT

Strings are **immutable**: `s.substring(...)` returns a NEW string; `s` itself never changes. Repeated `+=` in a loop creates many temporary strings (O(n²)); use `StringBuilder`.

A tiny example

```
String s = "face-prep";
int i = s.indexOf('-');
String result = "-" + s.substring(0, i) + s.substring(i + 1); // -faceprep
```

Guided example

MoveFirstHyphen.java — Move the first hyphen (from the slides)

```
1  public class MoveFirstHyphen {
2   // The lecture version: only the FIRST hyphen moves to the front.
3   static String moveHyphenToBeginning(String s) {
4   int hyphenIndex = s.indexOf('-');                                    (1)
5   if (hyphenIndex < 0) {                                               (2)
6   return s; // no hyphen: nothing to do
7   }
8   return "-" + s.substring(0, hyphenIndex) + s.substring(hyphenIndex + 1);   (4)
9   }
10
11  public static void main(String[] args) {
12  System.out.println(moveHyphenToBeginning("face-prep"));
13  System.out.println(moveHyphenToBeginning("a-b-c"));
14  System.out.println(moveHyphenToBeginning("nohyphen"));
15  }
16 }
```

indexOf + substring. O(n) time (copying) and O(n) space for the new string.

Line-by-line walkthrough

#	Where	What happens
1	line 4	Position of the first hyphen, or −1 if there is none.
2	line 5	No hyphen: return the original (the slide uses `contains` then `indexOf`, i.e. searches twice).
3	line 8	Everything before the hyphen.
4	line 8	Everything after the hyphen (skip it with +1).

MoveAllHyphens.java — Move every hyphen (optimised solution)

```java
import java.util.Scanner;

public class MoveAllHyphens {
 // Move EVERY hyphen to the front; the other characters keep their original order.
 static String moveAll(String s) {
 StringBuilder hyphens = new StringBuilder();
 StringBuilder others = new StringBuilder();
 for (int i = 0; i < s.length(); i++) {
 char c = s.charAt(i);
 if (c == '-') {
 hyphens.append(c);
 } else {
 others.append(c);
 }
 }
 return hyphens.append(others).toString();
 }

 // The same result in ONE array, scanning from the right, no second builder.
 static String moveAllInPlace(String s) {
 char[] a = s.toCharArray();
 int write = a.length - 1; // next free slot from the right
 for (int read = a.length - 1; read >= 0; read--) {
 if (a[read] != '-') {
 a[write--] = a[read]; // push non-hyphens to the right end
 }
 }
 while (write >= 0) {
 a[write--] = '-'; // fill the rest with hyphens
 }
 return new String(a);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 System.out.println(moveAll(s));
 System.out.println(moveAllInPlace(s));
 }
}
```

Two versions of the same stable partition.

Sample input

```
a-b-c-d
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Two builders: hyphens and everything else. Appending to a StringBuilder is amortised O(1).
2	line 16	Hyphens first, then the others in their original order.
3	line 22	The in-place version: `write` is the next free slot from the right.
4	line 24	Copy non-hyphen characters towards the right end, preserving their order (read ≤ write always).
5	line 29	The remaining slots on the left are exactly the hyphens.

Dry run: watch it execute

Right-to-left copy, then fill the front with hyphens.

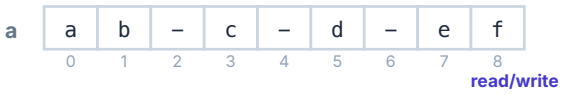
Algorithm: Move every hyphen to the front (in place). **Input:** s = `ab-c-d-ef`.

Pseudocode

```
write = n - 1
for read = n-1 down to 0:
 if a[read] != "-": a[write--] = a[read]
while write >= 0: a[write--] = "-"
```

Step 1 of 11. read and write both start at the last cell.

read = 8 · write = 8



Step 2 of 11. a[8] = "f" is not a hyphen: copy it to a[8].

read = 8 · write = 8



Step 3 of 11. a[7] = "e" is not a hyphen: copy it to a[7].

read = 7 · write = 7



Step 4 of 11. a[6] is a hyphen: skip it (only read moves left, so write falls behind).

read = 6 · write = 6



Step 5 of 11. a[5] = "d" is not a hyphen: copy it to a[6].

read = 5 · write = 6



Step 6 of 11. a[4] is a hyphen: skip it (only read moves left, so write falls behind).

read = 4 · write = 5



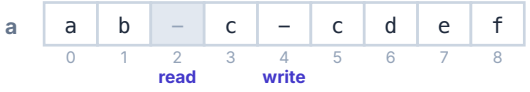
Step 7 of 11. a[3] = "c" is not a hyphen: copy it to a[5].

read = 3 · write = 5



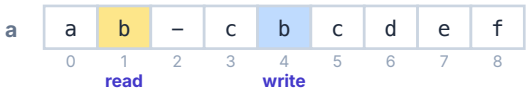
Step 8 of 11. a[2] is a hyphen: skip it (only read moves left, so write falls behind).

read = 2 · write = 4



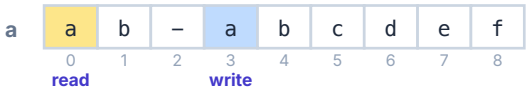
Step 9 of 11. a[1] = "b" is not a hyphen: copy it to a[4].

read = 1 · write = 4



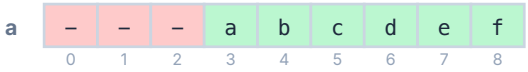
Step 10 of 11. a[0] = "a" is not a hyphen: copy it to a[3].

read = 0 · write = 3



Step 11 of 11. Fill the first 3 cells with hyphens.

hyphens = 3



> **RESULT**
> ---abcdef

Complexity

All versions are **O(n) time**. Memory: the substring/builder versions use O(n) extra characters; the `char[]` right-scan uses O(1) extra beyond the array itself.

The naive "remove each hyphen and insert it at index 0" costs O(n) per hyphen → O(n·h), up to O(n²).

Common mistakes

Expecting the string to change

Problem

```
s.substring(0, 3);
System.out.println(s); // unchanged
```

Why: Strings are immutable. The result of substring must be stored.

Fix: s = s.substring(0, 3);

Off-by-one on the second substring

Problem

```
s.substring(0, i) + s.substring(i)
```

Why: substring(i) still contains the hyphen: the output has the hyphen twice.

Fix: s.substring(i + 1)

Building the result with += in a loop

Problem

```
String out = "";
for (char c : s.toCharArray()) out += c;
```

Why: Each += copies the whole string so far: O(n²).

Fix: StringBuilder sb = new StringBuilder(); sb.append(c);

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · First hyphen to the front — *Beginner*

Read a line. If it contains a hyphen, move its FIRST hyphen to the beginning; otherwise print the line unchanged.

Example input	Expected output
face-prep	-faceprep

Hint 1: indexOf, then substring twice.

Exercise 2 · All hyphens to the front — *Intermediate*

Move every hyphen to the beginning of the line, keeping the other characters in order.

- length ≤ 10^6

Example input	Expected output
a-b-c	--abc

Hint 1: Two StringBuilders, or one char array scanned from the right.

Exercise 3 · Minimum adjacent swaps — *Challenge*

Print the minimum number of swaps of ADJACENT characters needed to bring all hyphens to the front.

- length ≤ 10^6 (answer may need long)

Example input	Expected output
a-b-c	3

Hint 1: Each hyphen must pass every non-hyphen character that stands before it. **Hint 2:** Sum, over hyphens, the number of non-hyphen characters seen so far.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is printed?

```
String s = "ab-cd";
int i = s.indexOf('-');
System.out.println("-" + s.substring(0, i) + s.substring(i + 1));
```

Your answer: _____

Q2. What does "face-prep".indexOf('x') return?

- **A.** 0
- **B.** -1
- **C.** null
- **D.** an exception

Q3. Why is building a long string with += in a loop slow?

- **A.** += is not allowed
- **B.** Strings are immutable so every += copies the string
- **C.** Because of garbage collection
- **D.** It is not slow

Q4. Time complexity of the one-pass right scan?

- **A.** O(1)
- **B.** O(n)
- **C.** O(n log n)
- **D.** O(n^2)

Interview / exam questions

- Why are Java Strings immutable?
- Move all zeros to the end of an array — how is it the same problem?
- How do you do a stable partition in place?

Watch & read

Video: Move special characters in String | String Assignment 1 | Programming Using Java — DeCode with Shreyas · 3:08. *Partitioning a string with two pointers*

Video: Lec-10: Two 2 Pointer Technique | Two 2 Sum Problem in Data Structure — Gate Smashers · 10:33. *The two-pointer technique*

REVISION CARD (30–60 SECONDS)

- indexOf, substring(a, b) = a..b−1, substring(a) = a..end.
- Strings are immutable; use StringBuilder for building.
- All hyphens: stable partition, right-to-left write pointer.
- O(n).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide moves only the first hyphen (by name it suggests "the hyphen"); the generalisation to all hyphens is added here and labelled as such.

5.23 Strobogrammatic Number

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 40 min · complete path

Numbers that look the same when rotated 180°: check one with two pointers and a map, generate them all with recursion.

YOU WILL BE ABLE TO

- State exactly which digits rotate to which: 0→0, 1→1, 8→8, 6→9, 9→6.
- Check a number with two pointers and a HashMap.
- Generate all n-digit strobogrammatic numbers by wrapping smaller ones.
- Count them with a formula and spot errors in informal definitions.

Before this lesson: Strings, StringBuilder and Characters · Sets and Maps: HashSet, TreeSet, HashMap, TreeMap · Recursion: Base Case, Recursive Case and the Call Stack

The idea

Turn a number **upside down** (rotate it 180°). Some digits look like a digit again: 0 → 0, 1 → 1, 8 → 8, 6 → 9, 9 → 6. A number is **strobogrammatic** if the rotated picture is the *same number*. Rotating reverses the order of digits as well as turning each one: `69` → rotate 9→6 and 6→9 and read from the other side → `69`. `818` → 818. `906` → 906. `25` → no (2 and 5 do not rotate into a digit in this definition).

```
number 6 9 rotate each digit 9 6
read backwards ← (6→9, 9→6) ↓ reverse → 6 9 ✓ same as the original
```

WHY THIS WORKS

Position i from the left must rotate into the digit at position i from the right. So walk two pointers from the two ends: for each pair (left, right) require `rotate(left) == right`. When the pointers meet in the middle (odd length) the digit must rotate into **itself** — only 0, 1 and 8 can sit in the middle, never 6 or 9. `while (left <= right)` with the same test handles that case automatically because left == right.

Syntax

```
static char rotate(char c) { // what a digit looks like upside down
 switch (c) {
 case '0': case '1': case '8': return c;
 case '6': return '9';
 case '9': return '6';
 default: return '?'; // not strobogrammatic
 }
}
int l = 0, r = s.length() - 1; // two pointers moving inwards
while (l <= r) { if (rotate(s.charAt(l)) != s.charAt(r)) return false; l++; r--; }
```

A tiny example

```
String from = "01689", to = "01986";
int i = from.indexOf(c); // c rotated = to.charAt(i)
```

Guided example

The slide solution builds a `HashMap<Character,Character>` and runs the two-pointer loop. The lesson adds the generator (LeetCode 247) so you can see all numbers of a given length.

StrobogrammaticCheck.java — Strobogrammatic check (optimised solution)

```
1  import java.util.HashMap;
2  import java.util.Map;
3  import java.util.Scanner;
4
5  public class StrobogrammaticCheck {
6   // Rotating a number by 180 degrees reverses the digit order AND rotates each digit:
7   // 0->0, 1->1, 8->8, 6->9, 9->6. Any other digit has no valid rotated form.
8   static boolean isStrobogrammatic(String num) {
9   Map<Character, Character> rotated = new HashMap<>();
10  rotated.put('0', '0');
11  rotated.put('1', '1');
12  rotated.put('8', '8');
13  rotated.put('6', '9');                                                    (1)
14  rotated.put('9', '6');
15
16  int left = 0;
17  int right = num.length() - 1;
18  while (left <= right) {                                                   (2)
19  Character r = rotated.get(num.charAt(left));
20  if (r == null || r != num.charAt(right)) { // left digit must rotate into the right   (3)
        digit
21  return false;
22  }
23  left++;
24  right--;
25  }
26  return true;
27  }
28
29  public static void main(String[] args) {
30  Scanner sc = new Scanner(System.in);
31  String n = sc.next();
32  System.out.println(n + " -> " + isStrobogrammatic(n));
33  }
34 }
```

A map of rotations and two pointers. O(n) time, O(1) space (the map has 5 entries).

Sample input

```
906
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	6 rotates into 9 and 9 into 6; 0, 1, 8 map to themselves. Any other digit is simply absent.
2	line 18	`<=` (not `<`) so that the middle digit of an odd-length number is also tested against itself.
3	line 20	Either the digit cannot rotate, or it rotates into something other than its mirrored partner.

StrobogrammaticGenerate.java — Generate all of length n (demonstration)

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class StrobogrammaticGenerate {
 // All strobogrammatic numbers with exactly n digits (no leading zero except the single
     digit "0").
 // Build from the middle outward: wrap every (n-2)-digit answer with a matching pair.
 static List<String> build(int n, int total) {
 if (n == 0) {                                                                    (1)
 return new ArrayList<>(List.of(""));
 }
 if (n == 1) {                                                                    (2)
 return new ArrayList<>(List.of("0", "1", "8"));
 }
 List<String> result = new ArrayList<>();
 for (String inner : build(n - 2, total)) {
 if (n != total) {                                                                (3)
 result.add("0" + inner + "0"); // zeros are fine on the inside, never on the outside
 }
 result.add("1" + inner + "1");
 result.add("6" + inner + "9");                                                   (4)
 result.add("8" + inner + "8");
 result.add("9" + inner + "6");
 }
 return result;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> all = build(n, n);
 System.out.println(all.size() + " numbers: " + all);
 }
}
```

build(n) wraps every build(n−2) result with 1…1, 6…9, 8…8, 9…6 (and 0…0 on the inside).

Sample input

```
3
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Base case for even lengths: the empty string.
2	line 12	Base case for odd lengths: the self-rotating digits 0, 1, 8.
3	line 17	A leading zero is not allowed: 0…0 is added only for the inner layers.
4	line 21	Wrapping with a rotating pair keeps the property.

Dry run: watch it execute

Compare rotate(left) with right while the pointers close in.

Algorithm: Strobogrammatic check. **Input:** s = `906`.

Pseudocode

```
left = 0; right = n - 1
while (left <= right) {
 if rotate(s[left]) != s[right]: return false
 left++; right--
}
return true
```

Step 1 of 3. Rotation map: 0→0, 1→1, 8→8, 6→9, 9→6.

left = 0 · right = 2

digits	9	0	6
	0	1	2

Step 2 of 3. rotate(9) = 6; s[2] = 6 → match.

left = 0 · right = 2 · rotated = 6

digits	9	0	6
	0 L	1	2 R

Step 3 of 3. rotate(0) = 0; s[1] = 0 → match.

left = 1 · right = 1 · rotated = 0

digits	9	0	6
	0	1 R	2

> **RESULT**
> 906 IS strobogrammatic

Complexity

Check: one pass over half the digits → O(n) time, O(1) space.

Generate: for length n there are 4·5^(n/2−1) numbers (n even) or 4·5^((n−1)/2−1)·3 (n odd, n ≥ 3); each has n characters, so the output size alone is Θ(n·5^(n/2)). Exponential — but optimal, since you must print them.

Counts of strobogrammatic numbers (no leading zero; the single digit 0 counts for n = 1).

n	1	2	3	4	5	6
count	3	4	12	20	60	100

Common mistakes

Only mapping, not mirroring

Problem

```
// treating "88" and "69" as the only valid ones because "digits look the same"
```

Why: The slide says 6 and 9 are "not strobogrammatic" digits; yet 69 is. The correct statement: each digit must rotate to a digit AND appear mirrored.

Fix: Use the pair map {6→9, 9→6, 0→0, 1→1, 8→8} with two pointers.

Using < instead of <= in the loop

Problem

```
while (left < right)
```

Why: "6" would be accepted: the middle digit is never tested. A single 6 is not strobogrammatic.

Fix: while (left <= right)

Claiming 2 and 5 work

Problem

```
// slide: "The digits 2 and 5 can also be considered strobogrammatic"
```

Why: On a seven-segment display 2 and 5 look alike when rotated, but in the standard definition (digits that turn into valid digits on a printed page) they do not. The standard set is {0, 1, 8, 6, 9}.

Fix: Stick to the stated definition and state it in the problem.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Is it strobogrammatic? — *Beginner*

Read a number as a string. Print YES if it looks the same rotated by 180°, else NO.

Example input	Expected output
69	YES
25	NO

Hint 1: Two pointers from both ends. **Hint 2:** Allowed rotations: 0-0, 1-1, 8-8, 6-9, 9-6.

Exercise 2 · How many of length n? — *Intermediate*

Read n (1 ≤ n ≤ 18). Print how many n-digit strobogrammatic numbers have no leading zero (for n = 1 the digit 0 counts).

Example input	Expected output
4	20

Hint 1: The outermost pair has 4 choices (11, 69, 88, 96). **Hint 2:** Each inner pair has 5 choices; an odd middle digit has 3.

Exercise 3 · Strobogrammatic in a range — *Challenge*

Read low and high (as strings of up to 15 digits, low ≤ high). Print how many strobogrammatic numbers lie in [low, high].

Example input	Expected output
50 100	3

Hint 1: Generate by length from len(low) to len(high). **Hint 2:** For equal lengths, string comparison equals numeric comparison.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which number is strobogrammatic?

- **A.** 25
- **B.** 962
- **C.** 69
- **D.** 1962

Q2. Which digits can be the MIDDLE digit of an odd-length strobogrammatic number?

- **A.** 0, 1, 8
- **B.** 6, 9
- **C.** 2, 5
- **D.** any digit

Q3. What does the check print for "88"?

```
// pairs: (8,8) → rotate(8) = 8 ✓
```

Your answer: _____

Q4. How many 2-digit strobogrammatic numbers (no leading zero) exist?

- **A.** 3
- **B.** 4
- **C.** 5
- **D.** 9

Interview / exam questions

- Check a strobogrammatic number in O(n).
- Generate all strobogrammatic numbers of length n (LeetCode 247).
- Why must the middle digit be 0, 1 or 8?

Watch & read

Video: STROBOGRAMMATIC NUMBER | LEETCODE # 246 | PYTHON SOLUTION — Cracking FAANG · 9:50.
Strobogrammatic number check (Python — the idea transfers to Java)

Video: Valid Palindrome Strings | Two Pointer Approach Explained | Python, C++ & Java — RisingBrain · 14:03.
Two-pointer symmetric checks (Python, C++ — the idea transfers to Java)

REVISION CARD (30–60 SECONDS)

- Rotations: 0↔0, 1↔1, 8↔8, 6↔9.
- Two pointers: rotate(s[l]) == s[r]; use l <= r.
- Middle digit ∈ {0, 1, 8}.
- Count for n digits: 4·5^(⌈n/2⌉−1)... (×3/÷5 for the odd middle).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide says digits 2 and 5 "can also be considered strobogrammatic" and that 6 and 9 "are not"; the standard definition (and the slide's own Java code) uses {0, 1, 8} as self-rotating and {6 ↔ 9} as a pair. • The slide explains 906 as "looks like 609". 906 is strobogrammatic because the rotated number equals 906 itself. • The slide program is split across two slides with numbered lines (22–30) that continue the class; the lesson program is the whole class.

5.24 Lexicographically First Palindromic Rearrangement

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 40 min · ★ on the minimum-viable path

Rearrange the letters of a string into the alphabetically smallest palindrome — using character counts, not permutations.

YOU WILL BE ABLE TO

- State when a multiset of letters can form a palindrome (at most one odd count).
- Build the smallest palindrome: half of each letter in alphabetical order, the odd letter in the middle, then the mirror.
- Explain why the brute-force permutation search is hopeless for long strings.
- Handle case and non-letter characters safely.

Before this lesson: Strings, StringBuilder and Characters · One-Dimensional Arrays · Sets and Maps: HashSet, TreeSet, HashMap, TreeMap

The idea

Given `malayalam` rearrange its letters so the result is a **palindrome** and, among all palindromes, the **alphabetically first**. Answer: `aalmymlaa`.

When is it possible? A palindrome mirrors itself, so every letter must appear an even number of times — except possibly one letter that sits alone in the middle. `malayalam` has a:4, l:2, m:2, y:1 → only `y` is odd ✓. `apple` has a:1, p:2, l:1, e:1 → three odd letters ✗ → "no palindromic string".

```
counts a:4 l:2 m:2 y:1
half a:2 l:1 m:1 front = a a l m (alphabetical)
middle y
back m l a a (front reversed)
result aalm + y + mlaa = aalmymlaa
```

WHY THIS WORKS

To make the string alphabetically small, the **left half** should be the smallest possible arrangement of the letters available for it: sort them. The right half is forced — it must mirror the left. Putting the single odd letter in the middle uses it up without affecting the order of the halves. Trying permutations one by one (the slide's "simple approach") finds the same answer but needs up to n! steps.

Syntax

```
int[] freq = new int[26];
for (char c : s.toCharArray()) freq[c - 'a']++; // count letters
StringBuilder half = new StringBuilder();
for (int i = 0; i < 26; i++)
 half.append(String.valueOf((char) ('a' + i)).repeat(freq[i] / 2)); // half of each letter, in
     order
String palindrome = half + middle + new StringBuilder(half).reverse();
```

A tiny example

```
int[] freq = new int[26];
for (char c : s.toCharArray()) freq[c - 'a']++;
```

Guided example

LexPalindromeBrute.java — Try permutations in order (simple solution)

```
import java.util.Arrays;
import java.util.Scanner;

public class LexPalindromeBrute {
 // Sorted string, then walk through successive permutations until one reads the same
     backwards.
 // Correct but factorial time: only usable for very short strings.
 static boolean nextPermutation(char[] a) {                                    (1)
 int i = a.length - 2;
 while (i >= 0 && a[i] >= a[i + 1]) i--;
 if (i < 0) return false;
 int j = a.length - 1;
 while (a[j] <= a[i]) j--;
 char t = a[i]; a[i] = a[j]; a[j] = t;
 for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {
 t = a[l]; a[l] = a[r]; a[r] = t;
 }
 return true;
 }

 static boolean isPalindrome(char[] a) {
 for (int l = 0, r = a.length - 1; l < r; l++, r--) {
 if (a[l] != a[r]) return false;
 }
 return true;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 char[] a = sc.next().toLowerCase().toCharArray();
 Arrays.sort(a);                                                               (2)
 do {
 if (isPalindrome(a)) {
 System.out.println(new String(a));
 return;
 }
 } while (nextPermutation(a));                                                 (3)
 System.out.println("no palindromic string");
 }
}
```

Sort the characters (the smallest permutation) and iterate next-permutation until one reads the same both ways. Correct, O(n·n!).

Sample input

```
Malayalam
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	The standard algorithm for the next lexicographic permutation (see the Sorted Unique Permutation lesson).
2	line 30	The very first permutation is the sorted one.
3	line 36	Stops after the last permutation → "no palindromic string".

LexPalindrome.java — Lexicographically first palindrome by counting (optimised solution)

```
1  import java.util.Scanner;
2  import java.util.TreeMap;
3
4  public class LexPalindrome {
5   // A string can be rearranged into a palindrome iff at most one character has an odd
        count
6   // (and exactly one when the length is odd). The smallest palindrome puts the smallest
        letters
7   // on the outside: front half in increasing order, middle = the odd character, back
        half mirrored.
8   static String firstPalindrome(String input) {
9   TreeMap<Character, Integer> freq = new TreeMap<>(); // TreeMap keeps characters sorted   1
10  for (char c : input.toLowerCase().toCharArray()) {
11  freq.merge(c, 1, Integer::sum);
12  }
13
14  String middle = "";
15  int oddCount = 0;
16  for (var e : freq.entrySet()) {
17  if (e.getValue() % 2 == 1) {                                                            2
18  oddCount++;
19  middle = String.valueOf(e.getKey());
20  }
21  }
22  if (oddCount > 1) {                                                                     3
23  return "no palindromic string";
24  }
25
26  StringBuilder front = new StringBuilder();
27  for (var e : freq.entrySet()) {
28  front.append(String.valueOf(e.getKey()).repeat(e.getValue() / 2)); // half of each count   4
29  }
30  String back = new StringBuilder(front).reverse().toString();                           5
31  return front + middle + back;
32  }
33
34  public static void main(String[] args) {
35  Scanner sc = new Scanner(System.in);
36  System.out.println(firstPalindrome(sc.next()));
37  }
38 }
```

O(n + σ log σ): count letters, test the odd-count rule, build half + middle + mirror.

Sample input

```
Malayalam
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	A TreeMap stores keys in sorted order — iteration is alphabetical for free.
2	line 17	Counting letters with an odd frequency.
3	line 22	More than one odd letter cannot be placed: no palindrome exists.
4	line 28	Each letter contributes half of its (even part of its) count to the front.
5	line 30	The right half is the mirror of the left.

Dry run: watch it execute

Counts → half → mirrored string.

Algorithm: Lexicographically first palindrome. **Input:** s = `malayalam`.

Pseudocode

```
count letters
odd = letters with odd count
if odd > 1: no palindromic string
front = (count/2 copies of each letter, a..z)
return front + oddLetter + reverse(front)
```

Step 1 of 4. Letter counts: a:4 l:2 m:2 y:1

a = 4 · l = 2 · m = 2 · y = 1

Step 2 of 4. Letters with an odd count: y.

odd letters = y

Step 3 of 4. Front half (sorted): "aalm".

front = aalm

Step 4 of 4. Result = front + "y" + mirror(front).

result = aalmymlaa

> **RESULT**
> aalmymlaa

Complexity

Permutation search: up to n!/(c₁!c₂!…) candidates × O(n) test → factorial time: n = 12 is already ~10⁸–10⁹ operations.

Counting: one pass to count (O(n)), σ distinct letters to build (O(σ log σ) with the TreeMap, O(σ) with a 26-array), → **O(n) time, O(σ) extra space** apart from the output.

Common mistakes

Crash on uppercase letters (slide)

Problem

```
freq[str.charAt(i) - 'a']++; // "Malayalam" → 'M' - 'a' = -20
```

Why: The slide's intro shows the input "Malayalam" but the code only works for lowercase: index −20 throws ArrayIndexOutOfBoundsException.

Fix: Normalise with toLowerCase() first, or use a Map / int[256].

A stray space in rear_str (slide)

Problem

```
String front_str = "", rear_str = " ";
```

Why: The back half starts with a space character, so the printed palindrome has a space in it ("aalmymlaa " ...). It is not a palindrome any more.

Fix: String rear = "";

Putting the odd letter at the front

Problem

```
return oddLetter + front + back;
```

Why: The odd letter has no partner, so it must stand alone in the exact middle; anywhere else the string is not symmetric.

Fix: return front + oddLetter + back;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Can it be a palindrome? — *Beginner*

Read a word (letters only, case-insensitive). Print YES if its letters can be rearranged into a palindrome, else NO.

Example input	Expected output
aabb	YES

Hint 1: Count letters; how many have an odd count?

Exercise 2 · Build the smallest palindrome — *Intermediate*

Read a word. Print the lexicographically first palindrome using ALL its letters (case-insensitive, print in lowercase), or `no palindromic string`.

Example input	Expected output
Malayalam	aalmymlaa
apple	no palindromic string

Hint 1: TreeMap<Character,Integer> keeps letters sorted. **Hint 2:** front = each letter repeated count/2 times.

Exercise 3 · How many palindromes? — *Challenge*

Read a word of up to 20 letters. Print the number of DISTINCT palindromes that can be formed from all its letters (0 if impossible).

Example input	Expected output
aabbcc	6

Hint 1: Only the left half can be chosen freely. **Hint 2:** Number of arrangements of a multiset: m! / (c_1! c_2! ...) where counts are halved.

Quick check

Answers are in the answer key at the end of this part.

Q1. Which word can be rearranged into a palindrome?

- **A.** apple
- **B.** level
- **C.** abcd
- **D.** aabbcd

Q2. Smallest palindrome from "bbaac"?

```
// counts a:2 b:2 c:1
```

Your answer: _____

Q3. Time complexity of the counting solution?

- **A.** O(n!)
- **B.** $O(n^2)$
- **C.** O(n)
- **D.** $O(2^n)$

Q4. In an even-length string, how many letters may have an odd count?

- **A.** 0
- **B.** 1
- **C.** 2
- **D.** any

Interview / exam questions

- How do you know if a palindrome permutation exists?
- Build the lexicographically smallest one in O(n).
- What about the lexicographically LARGEST?
- Count the number of distinct palindromic permutations.

Watch & read

Video: Lexicographically Smallest Palindrome | Leetcode 2697 | Approach + Code with explanation. — Pirai's Academy · 8:28. *Building the smallest palindrome from a multiset*

Video: LeetCode 266. Palindrome Permutation (Algorithm Explained) — Nick White · 5:53. *Frequency counting for palindromes*

REVISION CARD (30–60 SECONDS)

- Palindrome possible ⇔ ≤ 1 letter with odd count.
- Result = sorted half + odd letter + mirror.
- Normalise case; use TreeMap or int[26].
- O(n).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide program starts `rear_str = " "` (a space) and subtracts `'a'` from characters although its first example input is "Malayalam" with a capital M; the lesson program lowercases and fixes the initial value.
- The slide (slide 4) states the expected output for "Malayalam" as "aalmymlaa", which is correct for the lowercased word.

5.25 Manacher's Algorithm (Longest Palindromic Substring)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 65 min · complete path

Find the longest palindromic substring in linear time by reusing the symmetry of palindromes inside palindromes.

YOU WILL BE ABLE TO

- Define substring vs subsequence and find the longest palindromic substring by expanding around centres.
- Explain why inserting separators makes all palindromes odd-length.
- Describe the mirror idea and trace the centre/right-edge bookkeeping.
- Argue why the total work is O(n) and verify the code with a brute-force test.

Before this lesson: Strings, StringBuilder and Characters · Lexicographically First Palindromic Rearrangement · Analysing Loops: Counting Iterations

The idea

A **palindromic substring** is a block of consecutive characters that reads the same both ways. The longest in `forgeeksskeegfor` is `geeksskeeg` (length 10). The straightforward method is to treat every position as the **centre** of a palindrome and grow outwards while the two ends match — O(n²) in the worst case (try `aaaa…a`).

Manacher's observation: **a palindrome inside a bigger palindrome is mirrored**. If we already know a big palindrome centred at C that reaches to R, then for a position i inside it, the palindrome around i is at least as large as the palindrome around its mirror position 2C − i (up to the border R). So we never need to start from radius 0 again.

```
C−radius ... mirror C i ... R
[ . . . . m . . . C . . i . . . ]
what is known on the left of C also holds on the right of C
→ radius[i] = min(radius[mirror], R − i), then try to extend only beyond R
```

WHY THIS WORKS

To treat even and odd palindromes uniformly, insert a separator between all characters: `abba` → `#a#b#b#a#`. Every palindrome of the original is now an odd-length palindrome centred on a real character or a `#`. The radius in the transformed string equals the length of the palindrome in the original. Because R only ever moves right, and every successful extension moves R right by one, the `while` loops run at most 2n times in total — linear.

Syntax

```
StringBuilder t = new StringBuilder("#"); // "abba" -> "#a#b#b#a#"
for (char c : s.toCharArray()) t.append(c).append('#');
int[] p = new int[t.length()]; // p[i] = palindrome radius around i
int centre = 0, right = 0;
for (int i = 0; i < t.length(); i++) {
 if (i < right) p[i] = Math.min(right - i, p[2 * centre - i]); // reuse the mirror
 while (i - p[i] - 1 >= 0 && i + p[i] + 1 < t.length()
 && t.charAt(i - p[i] - 1) == t.charAt(i + p[i] + 1)) p[i]++; // expand
 if (i + p[i] > right) { centre = i; right = i + p[i]; }
}
```

A tiny example

```
// expand around centre i
int l = i, r = i;
while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
// palindrome is s.substring(l + 1, r)
```

Guided example

ExpandAroundCenter.java — Expand around centres (simple solution)

```
1  import java.util.Scanner;
2
3  public class ExpandAroundCenter {
4   // Every palindrome has a centre (a character, or a gap between two characters). From
        each of the
5   // 2n-1 centres grow outwards while the ends match. O(n^2) worst case, O(1) space.
6   static String longestPalindrome(String s) {
7   if (s.isEmpty()) {
8   return "";
9   }
10  int bestStart = 0;
11  int bestLen = 1;
12  for (int centre = 0; centre < s.length(); centre++) {
13  for (int type = 0; type < 2; type++) { // type 0: odd length, type 1: even length      (1)
14  int l = centre;
15  int r = centre + type;                                                               (2)
16  while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
17  l--;
18  r++;
19  }
20  int len = r - l - 1;                                                                 (3)
21  if (len > bestLen) {
22  bestLen = len;
23  bestStart = l + 1;
24  }
25  }
26  }
27  return s.substring(bestStart, bestStart + bestLen);
28  }
29
30  public static void main(String[] args) {
31  Scanner sc = new Scanner(System.in);
32  String s = sc.next();
33  String p = longestPalindrome(s);
34  System.out.println("longest palindromic substring: " + p + " (length " + p.length() + ")
        ");
35  }
36 }
```

For each centre (a character or a gap) grow while the characters match. O(n²) time, O(1) space.

Sample input

```
babad
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	type 0 = odd palindrome centred on a character; type 1 = even palindrome centred between two characters.
2	line 15	For even palindromes the right pointer starts one step to the right.
3	line 20	After the loop l and r are one step past the palindrome on both sides.

Manacher.java — Manacher's algorithm (optimised solution)

```
import java.util.Scanner;

public class Manacher {
 // Manacher's algorithm: longest palindromic substring in O(n).
 // 1. Insert '#' between characters so every palindrome has odd length: "abba" ->
     "#a#b#b#a#".
 // 2. radius[i] = how far the palindrome centred at i extends (in the transformed
     string).
 // 3. Reuse the mirror image of i inside the biggest palindrome seen so far instead of
     starting from 0.
 static int[] radii(String s) {
 StringBuilder sb = new StringBuilder("#");
 for (char c : s.toCharArray()) {
 sb.append(c).append('#');                                                    (1)
 }
 String t = sb.toString();
 int n = t.length();
 int[] radius = new int[n];
 int center = 0; // centre of the palindrome that reaches furthest right      (2)
 int right = 0; // its right edge (exclusive)
 for (int i = 0; i < n; i++) {
 if (i < right) {                                                             (3)
 int mirror = 2 * center - i;
 radius[i] = Math.min(right - i, radius[mirror]); // free information from symmetry (4)
 }
 while (i - radius[i] - 1 >= 0 && i + radius[i] + 1 < n                       (5)
 && t.charAt(i - radius[i] - 1) == t.charAt(i + radius[i] + 1)) {
 radius[i]++; // extend only as far as symmetry could not tell us
 }
 if (i + radius[i] > right) {                                                 (6)
 center = i;
 right = i + radius[i];
 }
 }
 return radius;
 }

 static String longestPalindrome(String s) {
 if (s.isEmpty()) {
 return "";
 }
 int[] r = radii(s);
 int bestCentre = 0;
 for (int i = 1; i < r.length; i++) {
 if (r[i] > r[bestCentre]) {
 bestCentre = i;
 }
 }
 int start = (bestCentre - r[bestCentre]) / 2; // map back to an index in the original (7)
     string
 return s.substring(start, start + r[bestCentre]);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 String p = longestPalindrome(s);
```

(code continued)

```
    System.out.println("longest palindromic substring: " + p + " (length " + p.length() + ")
        ");
  }
}
```

Radius array over the "#"-padded string with centre/right-edge reuse. O(n) time and space.

Sample input

```
forgeeksskeegfor
```

Line-by-line walkthrough

#	Where	What happens
1	line 11	Pad: "#a#b#b#a#". Every palindrome is odd-length now.
2	line 16	Centre of the palindrome whose right edge `right` is the furthest seen so far.
3	line 19	i lies inside the known palindrome, so its mirror gives free information.
4	line 21	The mirrored radius, limited by the border (beyond `right` nothing is known).
5	line 23	Extend only while characters match — each success pushes the border right.
6	line 27	New furthest palindrome: update centre and right edge.
7	line 46	Map back: a radius r around transformed index c covers original indices (c − r)/2 … (c − r)/2 + r − 1.

ManacherCheck.java — Brute-force verification (demonstration)

```
import java.util.Random;

public class ManacherCheck {
 static int[] radii(String s) {
 StringBuilder sb = new StringBuilder("#");
 for (char c : s.toCharArray()) sb.append(c).append('#');
 String t = sb.toString();
 int n = t.length();
 int[] radius = new int[n];
 int center = 0, right = 0;
 for (int i = 0; i < n; i++) {
 if (i < right) radius[i] = Math.min(right - i, radius[2 * center - i]);
 while (i - radius[i] - 1 >= 0 && i + radius[i] + 1 < n && t.charAt(i - radius[i] - 1)
     == t.charAt(i + radius[i] + 1)) radius[i]++;
 if (i + radius[i] > right) { center = i; right = i + radius[i]; }
 }
 return radius;
 }

 static int longestFast(String s) {
 int best = 0;
 for (int r : radii(s)) best = Math.max(best, r);
 return best;
 }

 static int longestBrute(String s) {
 int best = 0;
 for (int i = 0; i < s.length(); i++) {
 for (int j = i; j < s.length(); j++) {
 boolean pal = true;
 for (int l = i, r = j; l < r; l++, r--) if (s.charAt(l) != s.charAt(r)) { pal = false;
     break; }
 if (pal) best = Math.max(best, j - i + 1);
 }
 }
 return best;
 }

 // 20000 random strings over a tiny alphabet (many palindromes) compared with a
     triple-loop brute force.
 public static void main(String[] args) {
 Random rnd = new Random(11);
 for (int t = 0; t < 20000; t++) {
 int n = 1 + rnd.nextInt(14);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append((char) ('a' + rnd.nextInt(2)));
 String s = sb.toString();
 if (longestFast(s) != longestBrute(s)) {
 System.out.println("MISMATCH on " + s);
 return;
 }
 }
 System.out.println("Manacher matches brute force on 20000 random strings");
 }
}
```

The O(n) result must equal the longest palindrome found by checking every substring.

Line-by-line walkthrough

#	Where	What happens
1	line 43	A two-letter alphabet creates many palindromes and repeated structure — exactly where bugs in the mirror logic show up.

Dry run: watch it execute

Radii in the transformed string; the mirror trick saves re-expansion.

Algorithm: Manacher's algorithm. **Input:** s = `abaxabaxabb`.

Pseudocode

```
t = "#a#b#..#"
for each i:
 if i < right: radius[i] = min(right − i, radius[2*center − i])
 expand while t[i−r−1] == t[i+r+1]
 if i + radius[i] > right: center = i; right = i + radius[i]
```

The full trace has 24 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 24. Padded string t = "#a#b#a#x#a#b#a#x#a#b#b#" (length 23).

center = 0 · right = 0

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

0	22
	0

Step 3 of 24. i = 1 is outside any known palindrome: start at 0. Extended to 1. New center 1, right edge 2.

i = 1 · radius[i] = 1 · center = 1 · right = 2

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

0	22
	0

Step 5 of 24. i = 3 is outside any known palindrome: start at 0. Extended to 3. New center 3, right edge 6.

i = 3 · radius[i] = 3 · center = 3 · right = 6

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

0	22
	0

Step 6 of 24. i = 4 is inside the palindrome (center 3, right 6); mirror = 2 → start with radius 0. No extension possible.

i = 4 · radius[i] = 0 · center = 3 · right = 6



Step 8 of 24. i = 6 is outside any known palindrome: start at 0. No extension possible.

i = 6 · radius[i] = 0 · center = 3 · right = 6



Step 10 of 24. i = 8 is inside the palindrome (center 7, right 14); mirror = 6 → start with radius 0. No extension possible.

i = 8 · radius[i] = 0 · center = 7 · right = 14



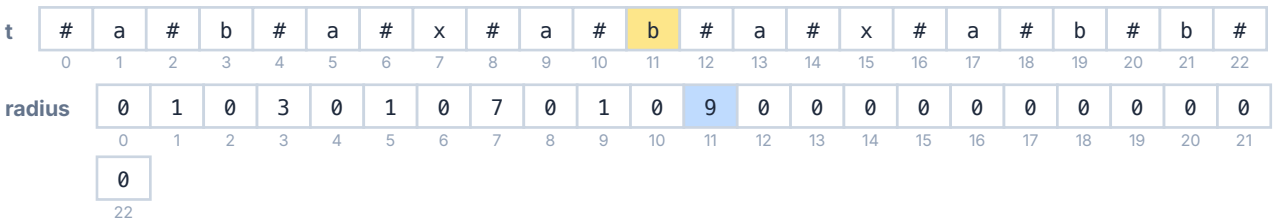
Step 12 of 24. i = 10 is inside the palindrome (center 7, right 14); mirror = 4 → start with radius 0. No extension possible.

i = 10 · radius[i] = 0 · center = 7 · right = 14



Step 13 of 24. i = 11 is inside the palindrome (center 7, right 14); mirror = 3 → start with radius 3. Extended to 9. New center 11, right edge 20.

i = 11 · radius[i] = 9 · center = 11 · right = 20



Step 15 of 24. i = 13 is inside the palindrome (center 11, right 20); mirror = 9 → start with radius 1. No extension possible.

i = 13 · radius[i] = 1 · center = 11 · right = 20



Step 17 of 24. i = 15 is inside the palindrome (center 11, right 20); mirror = 7 → start with radius 5. No extension possible.

i = 15 · radius[i] = 5 · center = 11 · right = 20



Step 19 of 24. i = 17 is inside the palindrome (center 11, right 20); mirror = 5 → start with radius 1. No extension possible.

i = 17 · radius[i] = 1 · center = 11 · right = 20



Step 20 of 24. i = 18 is inside the palindrome (center 11, right 20); mirror = 4 → start with radius 0. No extension possible.

i = 18 · radius[i] = 0 · center = 11 · right = 20



Step 22 of 24. i = 20 is outside any known palindrome: start at 0. Extended to 2. New center 20, right edge 22.

i = 20 · radius[i] = 2 · center = 20 · right = 22



Step 24 of 24. i = 22 is outside any known palindrome: start at 0. No extension possible.

i = 22 · radius[i] = 0 · center = 20 · right = 22

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
t	#	a	#	b	#	a	#	x	#	a	#	b	#	a	#	x	#	a	#	b	#	b	#
radius	0	1	0	3	0	1	0	7	0	1	0	9	0	1	0	5	0	1	0	1	2	1	0

> **RESULT**
> longest palindromic substring: "baxabaxab" (length 9)

Complexity

Centre expansion: each of the 2n−1 centres may expand up to n/2 steps → **O(n²)** worst case (`aaaa…`), O(1) space.

Manacher: `right` never decreases and increases at most 2n+1 times in total; each outer iteration does O(1) work apart from successful extensions → **O(n) time, O(n) space** for the radius array and padded string.

Common mistakes

Forgetting even-length palindromes

Problem

```
// expand only from characters: misses "bb" in "cbbd"
```

Why: Palindromes of even length have a centre between two characters.

Fix: Try both (i, i) and (i, i + 1) — or insert separators.

Reading beyond the border

Problem

```
radius[i] = radius[mirror]; // without min(right − i, …)
```

Why: The mirror's palindrome may extend past the known palindrome; nothing is known beyond `right`, so the copied radius can be too large.

Fix: radius[i] = Math.min(right - i, radius[mirror]);

Wrong mapping back to the original string

Problem

```
s.substring(centre − radius, centre + radius)
```

Why: In the padded string indices are doubled. A radius r around transformed index c corresponds to original start (c − r) / 2 and length r.

Fix: int start = (c - r) / 2; s.substring(start, start + r)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Is it a palindrome? — *Beginner*

Read a word. Print YES if it is a palindrome, else NO.

Example input	Expected output
racecar	YES

Hint 1: Two pointers moving inwards.

Exercise 2 · Longest palindromic substring (expand) — *Intermediate*

Read a word (length ≤ 2000). Print its longest palindromic substring; if several have the same length, print the one that starts first.

Example input	Expected output
cbbd	bb

Hint 1: Two kinds of centres. **Hint 2:** Update only when strictly longer to keep the earliest.

Exercise 3 · Count palindromic substrings — *Challenge*

Read a word of up to 10^5 letters. Print how many substrings (counted by position) are palindromes, in O(n).

Example input	Expected output
abba	6

Hint 1: In the padded string a radius r around a centre contributes $\lceil r/2 \rceil$ palindromes. **Hint 2:** Use long.

Quick check

Answers are in the answer key at the end of this part.

Q1. Why does Manacher's algorithm insert "#" between characters?

- **A.** To speed up comparison
- **B.** To make every palindrome odd-length
- **C.** To sort the string
- **D.** To avoid overflow

Q2. The time complexity of Manacher's algorithm is...

- **A.** $O(n^2)$
- **B.** O(n log n)
- **C.** O(n)
- **D.** $O(n^3)$

Q3. The padded form of "abba" is...

```
// insert # at the start, end, and between characters
```

Your answer: _____

Q4. The worst case input for centre expansion is...

- **A.** abcdef
- **B.** aaaaaa
- **C.** abababa

- **D.** a

Interview / exam questions

- Explain Manacher's algorithm in two minutes.
- Why is it linear although it has nested loops?
- Substring vs subsequence?
- Compare with a suffix-array / hashing approach.

Watch & read

Video: Longest Palindromic Substring Manacher's Algorithm — Tushar Roy - Coding Made Simple · 16:46. *Manacher in linear time*

Video: 5. Longest Palindromic Substring | Part-1: Expand around centers #leetcode — CodeWonka : Get started with programming · 7:55. *Expand-around-center for comparison*

REVISION CARD (30–60 SECONDS)

- Pad with # → all palindromes odd. radius[i] = palindrome radius at i.
- Keep (center, right). If i < right: radius[i] = min(right − i, radius[2·center − i]).
- Extend while chars match; update center/right.
- O(n) time & space; original start = (c − r)/2, length r.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide listing of the Java program stops in the middle of the main loop (slide 5 ends after the first while loop and slide 6 is "Thank you"), so the code is incomplete. The lesson provides a complete, tested implementation. • The slide text (steps 1–12) describes the version with sentinels "@" and "$" and array `p`; the Java snippet uses a different (GeeksforGeeks-style) layout with array `L`. The lesson implements the classical radius-array formulation that both describe.

ASSUMPTIONS MADE

• The unfinished Java fragment on the slide is replaced by the standard algorithm; the explanation follows the 12 steps on slides 3–4.

5.26 Natural Sort Order

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 40 min · complete path

Sort "file2" before "file10" by splitting names into text and number chunks and writing a Comparator.

YOU WILL BE ABLE TO

- Explain why plain string comparison puts "file10" before "file2".
- Split a name into alternating digit and non-digit chunks.
- Write a `Comparator<String>` that compares number chunks numerically.
- Handle ties (leading zeros, case) deterministically.

Before this lesson: Sorting with Comparable and Comparator · Strings, StringBuilder and Characters · Generics and Lists: ArrayList and LinkedList

The idea

Java's default String order compares characters one at a time using their codes. `"file10"` vs `"file2"`: the first difference is `1` vs `2`; `1` < `2`, so `file10` sorts before `file2`. Humans expect the opposite, because 10 is bigger than 2.

```
plain sort : file1.txt file10.txt file2.txt file20.txt file3.txt
natural sort : file1.txt file2.txt file3.txt file10.txt file20.txt
```

WHY THIS WORKS

Natural order cuts each name into chunks where digits and non-digits alternate: `file10.txt` → `file`, `10`, `.txt`. Two names are compared chunk by chunk: if both chunks are numbers, compare them as numbers (`2 < 10`); otherwise compare them as text. The first difference decides. A name that is a prefix of the other (fewer chunks) comes first.

Syntax

A `Comparator<T>` is an object with `int compare(T a, T b)`: negative if a goes first, 0 if equal, positive if b goes first. In Java 8+ you can write it as a lambda: `(a, b) -> …`. `list.sort(comparator)` sorts in place and is **stable**.

```
Comparator<String> byLength = (a, b) -> Integer.compare(a.length(), b.length());
names.sort(byLength);
```

A tiny example

```
boolean isD = Character.isDigit('7'); // true
String num = "file10".substring(4); // "10"
int c = new java.math.BigInteger("0010").compareTo(new java.math.BigInteger("10")); // 0
```

Guided example

NaturalSort.java — Natural-order comparator (optimised solution)

```
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

public class NaturalSort {
 // Split a name into alternating chunks of digits and non-digits: "file10.txt" ->
     "file", "10", ".txt".
 static List<String> chunks(String s) {                                          (1)
 List<String> out = new ArrayList<>();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 boolean digit = Character.isDigit(s.charAt(i));
 while (j < s.length() && Character.isDigit(s.charAt(j)) == digit) {
 j++;
 }
 out.add(s.substring(i, j));
 i = j;
 }
 return out;
 }

 // Compare chunk by chunk. Two number chunks are compared as numbers; anything else as
     text.
 static final Comparator<String> NATURAL = (a, b) -> {
 List<String> x = chunks(a);
 List<String> y = chunks(b);
 for (int i = 0; i < Math.min(x.size(), y.size()); i++) {
 String p = x.get(i);
 String q = y.get(i);
 int cmp;
 if (Character.isDigit(p.charAt(0)) && Character.isDigit(q.charAt(0))) {
 cmp = new BigInteger(p).compareTo(new BigInteger(q)); // numeric value, any length  (2)
 } else {
 cmp = p.compareToIgnoreCase(q);                                                 (3)
 }
 if (cmp != 0) {
 return cmp;
 }
 }
 return Integer.compare(x.size(), y.size()); // shorter list of chunks first       (4)
 };

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 List<String> names = new ArrayList<>();
 while (sc.hasNext()) {
 names.add(sc.next());
 }
 List<String> lexicographic = new ArrayList<>(names);
 Collections.sort(lexicographic);
 names.sort(NATURAL);                                                            (5)
```

(code continued)

```
    System.out.println("plain sort : " + lexicographic);
    System.out.println("natural sort: " + names);
    }
  }
```

Splits both names into chunks and compares them pairwise; O(L) per comparison for names of length L.

Sample input

```
file1.txt file10.txt file2.txt file20.txt file3.txt
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	Splits "file10.txt" into ["file", "10", ".txt"] by walking while the character class (digit / non-digit) stays the same.
2	line 34	Numeric comparison of arbitrarily long digit strings (leading zeros ignored).
3	line 36	Text chunks compared case-insensitively so that "File2" and "file2" are neighbours.
4	line 42	All shared chunks are equal: the name with fewer chunks goes first.
5	line 53	List.sort is a stable merge sort (TimSort): equal names keep their input order.

> **NOTE**
> The slide code compares `List<String>` objects returned by `extractNumericParts` with `Comparator.comparing(...)`. A `List` is not `Comparable`, so that line **does not compile**, and even if it did it would compare "10" before "2" as text. The lesson compares chunk by chunk and uses `BigInteger` for number chunks of any length.

Dry run: watch it execute

Numbers inside names are compared by value.

Algorithm: Natural-order comparison. **Input:** a = `file10.txt`, b = `file9.txt`.

Pseudocode

```
chunks("file10.txt") = file | 10 | .txt
for i in 0..min(len)−1:
 both numbers → compare as numbers
 otherwise → compare text (ignore case)
 first difference decides
shorter chunk list first
```

Step 1 of 4. Split each name into alternating chunks of non-digits and digits.

chunks of "file10.txt"

file	10	.txt
0	1	2

chunks of "file9.txt"

file	9	.txt
0	1	2

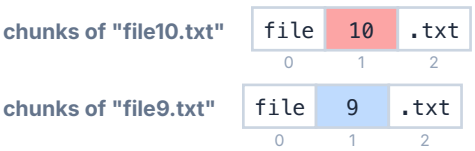
Step 2 of 4. Chunk 0: text "file" vs "file" compared ignoring case → equal.

chunk = 0 · comparison = 0



Step 3 of 4. Chunk 1: numbers 10 vs 9 compared by VALUE (10 vs 9) → greater.

chunk = 1 · comparison = 1



Step 4 of 4. For comparison, plain compareTo gives "file10.txt" first because it compares character by character ('1' < '9').

plain compareTo = -1 · natural = 1

> **RESULT**
> "file9.txt" comes before "file10.txt" (natural order)

Complexity

Each comparison costs O(L) (splitting and comparing). Sorting n names: O(n log n) comparisons → **O(n · L · log n)** time, O(n·L) space for the chunk lists. Pre-computing each name's chunk list once (a "Schwartzian transform") avoids re-splitting at every comparison.

Common mistakes

Comparing numeric chunks as text

Problem

```
"10".compareTo("2") // negative: "10" < "2"
```

Why: Strings are ordered by character codes, so a longer number can come first.

Fix: Parse as number (BigInteger for unlimited length) before comparing.

Using Integer.parseInt on arbitrary chunks

Problem

```
Integer.parseInt("99999999999")
```

Why: Throws NumberFormatException for numbers above $2^{31}-1$.

Fix: new BigInteger(chunk) or compare by (length after stripping zeros, then text)

A comparator that is not a total order

Problem

```
return 0; // for everything that is not obviously different
```

Why: If equal-looking names (file01 vs file1) compare equal while others are ordered inconsistently, sort may throw "Comparison method violates its general contract".

Fix: Add tie-breakers: length, then compareTo (see exercise 3).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Sort file names naturally — *Beginner*

Read n and then n file names. Print them in natural order, one per line (case-insensitive text, numbers compared numerically).

Example input	Expected output
`4 ⏎ img12.png ⏎ img10.png ⏎ img2.png ⏎ IMG1.png`	`IMG1.png ⏎ img2.png ⏎ img10.png ⏎ img12.png`

Hint 1: Chunk, compare chunk by chunk. **Hint 2:** List.sort(comparator).

Exercise 2 · Sort version numbers — *Intermediate*

Read n and n version strings like 1.2.10. Print them in increasing version order on one line (a missing component counts as 0).

Example input	Expected output
`4 ⏎ 1.10 ⏎ 1.2.10 ⏎ 1.2.9 ⏎ 1.9`	`1.2.9 1.2.10 1.9 1.10`

Hint 1: split("\\.") and compare component by component as numbers.

Exercise 3 · Natural order with deterministic ties — *Challenge*

Sort names naturally, ignoring case in text chunks. If two names are numerically equal chunk by chunk (e.g. file01 and file1), the SHORTER name comes first, then plain compareTo. Print on one line separated by spaces.

Example input	Expected output
`4 ⏎ file01 ⏎ file1 ⏎ File2 ⏎ file10`	`file1 file01 File2 file10`

Hint 1: Compare chunk by chunk, then the number of chunks, then total length, then compareTo.

Quick check

Answers are in the answer key at the end of this part.

Q1. Plain String ordering puts which first?

- **A.** file2
- **B.** file10
- **C.** they are equal
- **D.** depends on locale

Q2. How should two digit chunks "07" and "7" compare in natural order?

- **A.** "07" < "7"
- **B.** equal numerically (then tie-break)
- **C.** "07" > "7"
- **D.** error

Q3. What does a Comparator return when a should come before b?

- **A.** a positive number
- **B.** zero
- **C.** a negative number
- **D.** true

Q4. List.sort in Java is...

- **A.** stable
- **B.** unstable
- **C.** randomised
- **D.** not allowed for Strings

Interview / exam questions

- How would you implement natural sort for file names like "a10b2" vs "a10b10"?
- What makes a comparator valid?
- Why use BigInteger for the number chunks?

Watch & read

Video: Java Comparator Sorting in Java | Practical Tutorial with Eclipse Examples | Hands on | Red9SysTech — Red9SysTech · 3:52. *Comparing strings with embedded numbers*

Video: Sort Strings in Descending Order in Java | Custom Comparator Example | Java Programming — The Corporate Villager · 5:30. *Writing a custom comparator*

REVISION CARD (30–60 SECONDS)

- Split into digit / non-digit chunks.
- Number chunks → numeric compare (BigInteger); text → compareToIgnoreCase.
- First difference decides; fewer chunks first.
- Add tie-breakers so the order is total.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide's `Comparator.comparing(NaturalSort::extractNumericParts)` does not compile (List is not Comparable) and compares digits as text. The lesson replaces it with a chunk comparator. • The slide text describes sorting by the numeric part "in ascending order, otherwise regular string comparison" — which ignores that prefix text must be compared first.

5.27 Maneuvering (Counting Grid Paths)

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 45 min · ★ on the minimum-viable path

Count the right/down paths across a grid — recursion, then dynamic programming, then a combinatorics formula.

YOU WILL BE ABLE TO

- Set up the recurrence paths(m, n) = paths(m−1, n) + paths(m, n−1).
- Show why plain recursion is exponential (it recomputes the same cells).
- Fill a table bottom-up (dynamic programming) in O(m·n).
- Obtain the closed form C(m+n−2, m−1) and handle obstacles.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Two-Dimensional Arrays (Matrices) · Combinations (nCr)

The idea

You stand in the top-left cell of an m × n grid and want to reach the bottom-right cell moving only **right** or **down**. How many different paths exist? For a 2 × 2 grid there are 2; for 3 × 3 there are 6.

Think about the **last** move. You entered the destination from the cell above or from the cell to the left. So `paths(m, n) = paths(m−1, n) + paths(m, n−1)`. A single row or column has exactly one path.

```
3×3 grid: paths into each cell
1 1 1
1 2 3
1 3 6 → 6 paths to the bottom-right
```

WHY THIS WORKS

The recursive code asks for the same sub-grids again and again: paths(2, 2) is needed by both paths(2, 3) and paths(3, 2). The number of calls roughly doubles with every step → exponential. **Dynamic programming** just stores each answer in a table the first time it is computed. The closed form comes from counting arrangements: every path is a sequence of (m−1) downs and (n−1) rights, so choose which of the m+n−2 moves are downs: C(m+n−2, m−1).

Syntax

```
long[][] dp = new long[m][n]; // dp[i][j] = number of paths to cell (i, j)
for (int i = 0; i < m; i++) dp[i][0] = 1;
for (int j = 0; j < n; j++) dp[0][j] = 1;
for (int i = 1; i < m; i++)
 for (int j = 1; j < n; j++) dp[i][j] = dp[i - 1][j] + dp[i][j - 1];

static long paths(int r, int c) { // plain recursion (exponential): same recurrence
 if (r == 0 || c == 0) return 1;
 return paths(r - 1, c) + paths(r, c - 1);
}
```

A tiny example

```
static int paths(int m, int n) {
 if (m == 1 || n == 1) return 1;
 return paths(m - 1, n) + paths(m, n - 1);
}
```

Guided example

The lecture trace for `numberOfPaths(3, 3)` is exactly the call tree; the recursive program below also counts the calls (11 for a 3 × 3 grid) so you can see the growth.

> **NOTE**
> The commented line `// + numberOfPaths(m-1, n-1)` on the slide adds the diagonal move; the numbers that result are the **Delannoy numbers**.

PathsRecursive.java — Recursive path counting (from the slides)

```
1  import java.util.Scanner;
2
3  public class PathsRecursive {
4   static long calls = 0;                                                    (1)
5
6   // Number of paths from the top-left to the bottom-right of an m x n grid using only
        right/down moves.
7   static long numberOfPaths(int m, int n) {
8   calls++;
9   if (m == 1 || n == 1) {                                                   (2)
10  return 1; // a single row or column: only one way
11  }
12  return numberOfPaths(m - 1, n) + numberOfPaths(m, n - 1); // arrive from above or from   (3)
        the left
13  }
14
15  public static void main(String[] args) {
16  Scanner sc = new Scanner(System.in);
17  int m = sc.nextInt();
18  int n = sc.nextInt();
19  System.out.println("paths = " + numberOfPaths(m, n));
20  System.out.println("recursive calls = " + calls);
21  }
22 }
```

Direct translation of the recurrence. Exponential time.

Sample input

```
3 3
```

Line-by-line walkthrough

#	Where	What happens
1	line 4	A counter to expose the amount of repeated work.
2	line 9	Base case: one row or one column — a single straight path.
3	line 12	Arrive from above, or from the left. Add the two counts.

PathsDP.java — Bottom-up table (optimised solution)

```
1  import java.util.Scanner;
2
3  public class PathsDP {
4   // dp[i][j] = number of ways to reach cell (i, j). Each cell is computed once: O(m * n).
5   static long numberOfPaths(int m, int n) {
6   long[][] dp = new long[m][n];
7   for (int i = 0; i < m; i++) {
8   for (int j = 0; j < n; j++) {
9   if (i == 0 || j == 0) {                                    (1)
10  dp[i][j] = 1; // first row / first column: one way
11  } else {
12  dp[i][j] = dp[i - 1][j] + dp[i][j - 1];                    (2)
13  }
14  }
15  }
16  return dp[m - 1][n - 1];                                   (3)
17  }
18
19  public static void main(String[] args) {
20  Scanner sc = new Scanner(System.in);
21  int m = sc.nextInt();
22  int n = sc.nextInt();
23  System.out.println(numberOfPaths(m, n));
24  }
25 }
```

dp[i][j] = dp[i−1][j] + dp[i][j−1]. O(m·n) time and space (O(n) with a single row).

Sample input

```
18 18
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	First row and first column: exactly one way to reach each.
2	line 12	The same recurrence, but both terms are already computed.
3	line 16	The bottom-right cell is the answer.

PathsFormula.java — Closed form (optimised solution)

```
1  import java.util.Scanner;
2
3  public class PathsFormula {
4   // Every path has (m−1) downs and (n−1) rights in some order: choose where the downs go.
5   // paths = C(m + n − 2, m − 1)
6   static long paths(int m, int n) {
7   int total = m + n − 2;                                                    (1)
8   int k = Math.min(m − 1, n − 1);                                           (2)
9   long result = 1;
10  for (int i = 1; i <= k; i++) {
11  result = result * (total − k + i) / i; // exact at every step             (3)
12  }
13  return result;
14  }
15
16  public static void main(String[] args) {
17  Scanner sc = new Scanner(System.in);
18  int m = sc.nextInt();
19  int n = sc.nextInt();
20  System.out.println(paths(m, n));
21  }
22 }
```

C(m+n−2, m−1) with the multiply-then-divide loop from the Combination lesson. O(min(m, n)) time, O(1) space.

Sample input

```
18 18
```

Line-by-line walkthrough

#	Where	What happens
1	line 7	Total number of moves in any path.
2	line 8	C(total, k) = C(total, total − k): use the smaller side.
3	line 11	Exact at every step (see the Combination lesson).

Dry run: watch it execute

Each cell holds the number of paths that reach it.

Algorithm: Grid paths DP. **Input:** m = `4`, n = `5`.

Pseudocode

```
for i in 0..m−1:
 for j in 0..n−1:
 if i == 0 or j == 0: dp[i][j] = 1
 else dp[i][j] = dp[i−1][j] + dp[i][j−1]
return dp[m−1][n−1]
```

The full trace has 20 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 20. Cell (0, 0) is on the first row/column: only one way.

i = 0 · j = 0 · dp[i][j] = 1

1	·	·	·	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 2 of 20. Cell (0, 1) is on the first row/column: only one way.

i = 0 · j = 1 · dp[i][j] = 1

1	1	·	·	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 4 of 20. Cell (0, 3) is on the first row/column: only one way.

i = 0 · j = 3 · dp[i][j] = 1

1	1	1	1	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 5 of 20. Cell (0, 4) is on the first row/column: only one way.

i = 0 · j = 4 · dp[i][j] = 1

1	1	1	1	1
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 7 of 20. Cell (1, 1) = 1 (from above) + 1 (from the left) = 2.

i = 1 · j = 1 · dp[i][j] = 2

1	1	1	1	1
1	2	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 8 of 20. Cell (1, 2) = 1 (from above) + 2 (from the left) = 3.

i = 1 · j = 2 · dp[i][j] = 3

1	1	1	1	1
1	2	3	·	·
·	·	·	·	·
·	·	·	·	·

Step 10 of 20. Cell (1, 4) = 1 (from above) + 4 (from the left) = 5.

i = 1 · j = 4 · dp[i][j] = 5

1	1	1	1	1
1	2	3	4	5
·	·	·	·	·
·	·	·	·	·

Step 11 of 20. Cell (2, 0) is on the first row/column: only one way.

i = 2 · j = 0 · dp[i][j] = 1

1	1	1	1	1
1	2	3	4	5
1	·	·	·	·
·	·	·	·	·

Step 13 of 20. Cell (2, 2) = 3 (from above) + 3 (from the left) = 6.

i = 2 · j = 2 · dp[i][j] = 6

1	1	1	1	1
1	2	3	4	5
1	3	6	·	·
·	·	·	·	·

Step 14 of 20. Cell (2, 3) = 4 (from above) + 6 (from the left) = 10.

i = 2 · j = 3 · dp[i][j] = 10

1	1	1	1	1
1	2	3	4	5
1	3	6	10	·
·	·	·	·	·

Step 16 of 20. Cell (3, 0) is on the first row/column: only one way.

i = 3 · j = 0 · dp[i][j] = 1

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	·	·	·	·

Step 17 of 20. Cell (3, 1) = 3 (from above) + 1 (from the left) = 4.

i = 3 · j = 1 · dp[i][j] = 4

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	4	·	·	·

Step 19 of 20. Cell (3, 3) = 10 (from above) + 10 (from the left) = 20.

i = 3 · j = 3 · dp[i][j] = 20

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	4	10	20	·

Step 20 of 20. Cell (3, 4) = 15 (from above) + 20 (from the left) = 35.

i = 3 · j = 4 · dp[i][j] = 35

1	1	1	1	1
1	2	3	4	5
1	3	6	10	15
1	4	10	20	35

> **RESULT**
> 35 paths

Complexity

Recursion: T(m, n) = T(m−1, n) + T(m, n−1) + 1 → about C(m+n−2, m−1) leaf calls — **exponential** (for 18 × 18 it is over 2 billion calls).

DP: every cell once → **O(m·n)** time, O(m·n) space (reducible to O(n)).

Formula: O(min(m, n)).

grid	paths	recursive calls
3 × 3	6	11
10 × 10	48 620	97 239
18 × 18	2 333 606 220	≈ 4.7 billion

Common mistakes

int overflow of the count

Problem

```
int paths = ...; // 18 x 18 grid
```

Why: 2 333 606 220 > 2 147 483 647 — the result wraps to a negative number.
Fix: long (or BigInteger / modulus when the problem asks for it)

Confusing grid size with number of moves

Problem

```
C(m + n, m)
```

Why: An m × n grid of CELLS needs m−1 downs and n−1 rights: C(m+n−2, m−1).
Fix: total moves = m + n − 2

Blocked cells treated as 1

Problem

```
dp[i][j] = (i == 0 || j == 0) ? 1 : ... // with obstacles
```

Why: In the first row, a blocked cell cuts off every cell to its right; setting them all to 1 overcounts.
Fix: dp[i][j] = blocked ? 0 : (from above + from left), with dp[0][0] = 1

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Paths in a rectangle — *Beginner*

Read m and n and print the number of right/down paths from the top-left to the bottom-right of an m × n grid.

- 1 ≤ m, n ≤ 18

Example input	Expected output
3 7	28

Hint 1: dp table with first row and column set to 1.

Exercise 2 · Paths with obstacles — *Intermediate*

Read R, C and a grid of 0 (free) and 1 (blocked). Print the number of right/down paths from (0,0) to (R−1,C−1) that avoid blocked cells (0 if the start or end is blocked).

- 1 ≤ R, C ≤ 100
- the answer fits in a long

Example input	Expected output
3 3 ⏎ 0 0 0 ⏎ 0 1 0 ⏎ 0 0 0	2

Hint 1: A blocked cell contributes 0. **Hint 2:** dp[0][0] = 1 only if the start is free.

Exercise 3 · Huge grid modulo a prime — *Challenge*

Read m and n (≤ 100 000). Print the number of paths modulo 1 000 000 007.

Example input	Expected output
3 7	28

Hint 1: A DP table of 10^{10} cells is impossible — use C(m+n−2, m−1). **Hint 2:** Precompute factorials mod p and use a modular inverse (Fermat: a^(p−2)).

Quick check

Answers are in the answer key at the end of this part.

Q1. Number of right/down paths in a 4 × 4 grid of cells?

- **A.** 12
- **B.** 20
- **C.** 24
- **D.** 70

Q2. How many paths does this return?

```
int[][] dp = new int[3][3];
for (int i = 0; i < 3; i++)
 for (int j = 0; j < 3; j++)
 dp[i][j] = (i == 0 || j == 0) ? 1 : dp[i-1][j] + dp[i][j-1];
System.out.println(dp[2][2]);
```

Your answer: _____

Q3. Why is the recursive solution slow?

- **A.** Java recursion is slow
- **B.** It recomputes the same sub-grids many times
- **C.** The base case is wrong
- **D.** Arrays are slow

Q4. Time complexity of the DP table solution?

- **A.** O(m + n)
- **B.** O(m·n)
- **C.** O(2^(m+n))
- **D.** O(log n)

Interview / exam questions

- Explain overlapping sub-problems using this example.
- Reduce the DP space to O(n).
- Count paths with obstacles; with diagonal moves.
- Why is the answer a binomial coefficient?

Watch & read

Video: Unique Paths - Dynamic Programming - Leetcode 62 — NeetCode · 10:48. *Counting grid paths*

Video: Ways to Reach Origin | GFG Problem of the Day | Grid Paths DP & nCr Explained — Gopo workspace · 10:43. *The combinatorial formula*

REVISION CARD (30–60 SECONDS)

- paths(m, n) = paths(m−1, n) + paths(m, n−1); first row/col = 1.
- Recursion is exponential; DP is O(m·n).
- Closed form: C(m+n−2, m−1).
- Obstacle ⇒ 0; use long / modulus.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide says the problem is "Maneuvering a Cave"; the deck contains only the grid-path recurrence (a classic problem). Its trace of numberOfPaths(3, 3) is correct and ends at 6. • The slide mentions "already calculated" for numberOfPaths(2, 2) — the plain recursive code does NOT cache; the second call is recomputed (that is the inefficiency this lesson addresses).

5.28 Sorted Distinct Permutations

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 50 min · complete path

Print every distinct permutation of a string with duplicates, in alphabetical order.

YOU WILL BE ABLE TO

- Count distinct permutations of a multiset: n! / (c_1!·c_2!·...).
- Generate them in sorted order with the next-permutation algorithm.
- Generate them in sorted order with count-based backtracking.
- Recognise that the slide's swap-based generator does not produce sorted output.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Strings, StringBuilder and Characters · Combinations (nCr)

The idea

The string `AAB` has 3! = 6 arrangements, but only 3 are different: `AAB`, `ABA`, `BAA`. In general the number of **distinct** permutations is `n! / (c₁! · c₂! · …)` where c_i is how often each letter occurs (the slide: "geeks" has 5!/2! = 60).

We want them **sorted** (dictionary order). Two reliable ways:

1. **Next permutation.** Start from the sorted string and repeatedly turn the current arrangement into the next larger one: find the rightmost position that can still grow, swap in the smallest larger letter to its right, and reverse the tail.
2. **Counting backtracking.** Keep how many copies of each letter are left. At every position try the letters in alphabetical order (each distinct letter once, if copies remain) and recurse. Duplicates are never generated and the order is automatically sorted.

```
ABC → ACB → BAC → BCA → CAB → CBA (next-permutation order)
```

WHY THIS WORKS

Next-permutation works because the suffix to the right of the pivot is in descending order — the largest arrangement of those letters. To get the next larger permutation you must raise the pivot by the smallest possible amount and then make the suffix as small as possible (ascending, i.e. reversed).

Syntax

```
char[] c = s.toCharArray();
Arrays.sort(c); // start from the smallest arrangement
String again = new String(c); // char[] back to String
boolean[] used = new boolean[c.length]; // backtracking: which letters are taken
static void swap(char[] c, int i, int j) { char t = c[i]; c[i] = c[j]; c[j] = t; }
Set<Character> seen = new HashSet<>(); // skip repeated letters at one position
if (!seen.add(c[i])) continue; // add() returns false when already present
```

A tiny example

```
int i = a.length - 2;
while (i >= 0 && a[i] >= a[i + 1]) i--; // pivot: first place from the right that is smaller
    than its successor
```

Guided example

NextPermutation.java — Next lexicographic permutation (optimised solution)

```
1   import java.util.Arrays;
2   import java.util.Scanner;
3
4   public class NextPermutation {
5    // Rearrange a into the next lexicographically larger permutation; return false if a is
         the largest.
6    static boolean next(char[] a) {
7    int i = a.length - 2;
8    while (i >= 0 && a[i] >= a[i + 1]) {                                                    (1)
9    i--; // 1. find the rightmost position that can still grow
10   }
11   if (i < 0) {
12   return false;
13   }
14   int j = a.length - 1;
15   while (a[j] <= a[i]) {                                                                  (2)
16   j--; // 2. smallest value to its right that is bigger
17   }
18   char t = a[i];
19   a[i] = a[j];                                                                            (3)
20   a[j] = t; // 3. swap them
21   for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {                                (4)
22   t = a[l];
23   a[l] = a[r];
24   a[r] = t; // 4. the tail was decreasing: reverse it to make it smallest
25   }
26   return true;
27   }
28
29   public static void main(String[] args) {
30   Scanner sc = new Scanner(System.in);
31   char[] a = sc.next().toCharArray();
32   Arrays.sort(a); // start from the smallest arrangement                                  (5)
33   StringBuilder sb = new StringBuilder();
34   do {
35   sb.append(new String(a)).append(' ');
36   } while (next(a));
37   System.out.println(sb.toString().trim());
38   }
39  }
```

The classic 4-step algorithm; O(n) per permutation, O(1) space.

Sample input

```
AAB
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	1. Find the pivot: the rightmost i with a[i] < a[i+1]. Everything right of it is non-increasing.
2	line 15	2. Find the smallest letter in the suffix that is larger than the pivot (the suffix is descending, so scan from the right).
3	line 19	3. Swap pivot and that letter.
4	line 21	4. The suffix is still descending: reverse it to ascending to make it the smallest tail.
5	line 32	Starting from the sorted string guarantees we enumerate ALL arrangements, in order.

PermutationsBacktrack.java — Counting backtracking (optimised solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class PermutationsBacktrack {
5   static StringBuilder out = new StringBuilder();
6
7   // counts[c] = how many copies of letter c are still unused. Trying letters in
        alphabetical order
8   // produces the permutations in sorted order, and each DISTINCT letter is tried once
        per position.
9   static void build(int[] counts, char[] current, int pos) {
10  if (pos == current.length) {
11  out.append(new String(current)).append(' ');
12  return;
13  }
14  for (int c = 0; c < counts.length; c++) {
15  if (counts[c] == 0) {                                                    1
16  continue;
17  }
18  counts[c]--; // choose                                                   2
19  current[pos] = (char) ('A' + c);
20  build(counts, current, pos + 1); // explore                              3
21  counts[c]++; // un-choose                                                4
22  }
23  }
24
25  public static void main(String[] args) {
26  Scanner sc = new Scanner(System.in);
27  String s = sc.next();
28  int[] counts = new int[26];
29  for (char ch : s.toCharArray()) {
30  counts[ch - 'A']++;
31  }
32  build(counts, new char[s.length()], 0);
33  System.out.println(out.toString().trim());
34  }
35 }
```

Recursion over a counts array: sorted order and no duplicates by construction.

Sample input

```
BAC
```

Line-by-line walkthrough

#	Where	What happens
1	line 15	Skip letters with no copies left.
2	line 18	Choose: use one copy of letter c.
3	line 20	Explore the remaining positions.
4	line 21	Un-choose: put the letter back so another branch can use it.

SwapBasedPermutations.java — Lecture program (swap-based) (from the slides)

```java
import java.util.Arrays;
import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class SwapBasedPermutations {
 // The lecture's approach (sort the input, then generate by swapping, skipping repeated letters).
 // It produces every DISTINCT permutation exactly once, but NOT in sorted order.
 static StringBuilder out = new StringBuilder();

 static void permute(char[] chars, int index) {
 if (index == chars.length - 1) {
 out.append(String.valueOf(chars)).append(' ');
 return;
 }
 Set<Character> used = new HashSet<>();                    (1)
 for (int i = index; i < chars.length; i++) {
 if (used.contains(chars[i])) {
 continue;
 }
 used.add(chars[i]);
 swap(chars, index, i);                                    (2)
 permute(chars, index + 1);
 swap(chars, index, i);                                    (3)
 }
 }

 static void swap(char[] a, int i, int j) {
 char t = a[i];
 a[i] = a[j];
 a[j] = t;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 char[] chars = sc.next().toCharArray();
 Arrays.sort(chars);
 permute(chars, 0);
 System.out.println(out.toString().trim());
 }
}
```

HashSet per level avoids duplicate letters; swapping makes the order non-lexicographic.

Sample input

```
BAC
```

Line-by-line walkthrough

#	Where	What happens
1	line 16	Letters already tried at this position — prevents duplicate permutations.
2	line 22	Bring chars[i] to the current position.
3	line 24	Swap back to restore the array — but the restored order is not the same as the sorted order for the next branch, which is why output is not sorted.

THE SLIDE CODE DOES NOT SORT ITS OUTPUT

Run `SwapBasedPermutations` with input `BAC`: it prints `ABC ACB BAC BCA CBA CAB` — the last two are out of order (CAB must precede CBA). The swap-and-restore technique generates every permutation exactly once, but not in dictionary order, even if the input was sorted first.

Dry run: watch it execute

Successive next-permutation steps.

Algorithm: Next permutation. **Input:** s = `AAB`.

Pseudocode

```
sort(a)
repeat:
 print a
 i = rightmost index with a[i] < a[i+1]
 j = rightmost index with a[j] > a[i]
 swap(a[i], a[j]); reverse(a[i+1..])
```

Step 1 of 3. Permutation #1: AAB. Pivot index 1 (A).

= 1 · permutation = AAB · pivot = 1

current	A	A	B
	0	1	2

Step 2 of 3. Permutation #2: ABA. Pivot index 0 (A).

= 2 · permutation = ABA · pivot = 0

current	A	B	A
	0	1	2

Step 3 of 3. Permutation #3: BAA — the last one (no pivot).

= 3 · permutation = BAA · pivot = —

current	B	A	A
	0	1	2

RESULT

3 distinct permutations

Complexity

There are up to n! permutations, each taking O(n) to print, so any algorithm is Ω**(n · n!/(∏c_i!))** — factorial in the worst case. Next-permutation uses O(1) extra space; backtracking uses O(n) for the recursion stack and the counts.

For n = 10 distinct letters: 3 628 800 permutations — fine. For n = 13 it is 6.2 × 10⁹ — too many to print.

Common mistakes

Assuming swap-based recursion is sorted

Problem

```
Arrays.sort(chars); permute(chars, 0); // slide
```

Why: After swap/restore operations the remaining letters are no longer in sorted order. For "ABC" it prints CBA before CAB.

Fix: Use next-permutation or count-based backtracking, or sort the final list.

Duplicates from repeated letters

Problem

```
for (int i = index; i < n; i++) { swap; recurse; swap; } // no used-set
```

Why: "AAB" produces 6 outputs with repeats.

Fix: Skip a letter if it was already tried at this position (set/counts).

Not resetting the counts

Problem

```
counts[c]--; build(...); // missing counts[c]++
```

Why: Later branches see too few letters and never complete.

Fix: Always undo the choice after the recursive call.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · How many distinct permutations? — *Beginner*

Read a word (≤ 20 letters). Print the number of distinct permutations.

Example input	Expected output
AAB	3

Hint 1: n! divided by the factorial of every letter count. **Hint 2:** long is enough for n ≤ 20.

Exercise 2 · Print them sorted — *Intermediate*

Read a word (≤ 8 letters). Print every distinct permutation, one per line, in alphabetical order.

Example input	Expected output
ABA	AAB ⏎ ABA ⏎ BAA

Hint 1: Counts array over the 256 characters. **Hint 2:** Try letters in increasing order.

Exercise 3 · The k-th permutation — *Challenge*

Read a word (≤ 15 letters) and k. Print the k-th distinct permutation in sorted order (1-based) WITHOUT generating the others, or `NONE` if k is too large.

Example input	Expected output
DBCA 10	BCDA

Hint 1: At each position, for each letter, count how many permutations start with it: arrangements of the remaining multiset. **Hint 2:** If k ≤ that count, take the letter; otherwise subtract and continue.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many distinct permutations does "AABB" have?

- **A.** 4
- **B.** 6
- **C.** 12
- **D.** 24

Q2. What follows "ACB" in lexicographic order of "ABC"?

```
// permutations of A, B, C
```

Your answer: _____

Q3. Which statement about the slide's swap-based generator is true?

- **A.** It prints duplicates
- **B.** It prints every distinct permutation once, but not in sorted order
- **C.** It prints sorted output
- **D.** It never terminates

Q4. Time to print all permutations of 8 distinct letters is...

- **A.** O(8)
- **B.** O(8^2)
- **C.** O(8·8!)
- **D.** O(2^8)

Interview / exam questions

- Generate all distinct permutations with duplicates.
- Explain next-permutation in 4 steps.
- Find the k-th permutation without enumerating.
- Why is the lecture's "sort first" insufficient?

Watch & read

Video: Permutations of an Array/String | Recursion & Backtracking — Apna College · 22:55. *Distinct permutations with duplicates*

Video: Next Permutation - Intuition in Detail | Brute to Optimal — take U forward · 28:15. *The next-permutation algorithm*

REVISION CARD (30–60 SECONDS)

- Distinct permutations = n! / Π(c_i!).
- Next permutation: pivot, swap with next larger on the right, reverse suffix.
- Count-based backtracking: letters in alphabetical order, choose/explore/un-choose.
- Swap-based generation is NOT lexicographic.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide claims its program prints distinct permutations "in sorted order". Running it on "BAC" prints ABC ACB BAC BCA CBA CAB, which is not sorted. The slide's own "Steps" (sort, count permutations, then find the next greater string repeatedly) describe the NEXT-PERMUTATION method, which IS sorted — this is what `NextPermutation.java` implements. • The slide's example output for "DBCA" lists the 24 sorted permutations (correct).

5.29 N-Queens (Backtracking)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 60 min · ★ on the minimum-viable path

Place N queens on an N × N board so that none attack each other — the textbook backtracking problem.

YOU WILL BE ABLE TO

- Explain what a queen attacks and why at most one queen fits in each row, column and diagonal.
- Describe backtracking as "choose, explore, un-choose" and trace it on the 4-queens board.
- Write isSafe and the recursive row-by-row solver, then speed up the safety test with three boolean arrays.
- Count solutions for N up to 10 and state the complexity honestly.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Two-Dimensional Arrays (Matrices) · Algorithm Design Paradigms

The idea

A chess **queen** attacks along its row, its column and both diagonals. N-Queens asks for N queens on an N × N board with **no two attacking each other**. For N = 4 there are exactly 2 solutions; for N = 8 there are 92.

Because two queens in the same row attack each other, a solution has **exactly one queen per row**. So we place queens row by row: for row 0 try column 0; if that is safe go to row 1 and try its columns; if some row has **no** safe column, we made a bad choice earlier — **backtrack**: undo the previous queen and try its next column.

```
N = 4 (row by row)
row 0: Q . . . row 1: cols 0,1 attacked → Q at col 2: . . Q .
row 2: every column attacked → dead end → BACKTRACK to row 1 → try col 3 ...
...eventually: . Q . .
 . . . Q
 Q . . .
 . . Q .
```

WHY THIS WORKS

Backtracking explores a tree of partial placements but **prunes** every branch as soon as it breaks a rule, instead of building all N^N full boards and checking them. The key to correctness is **un-choosing**: after the recursive call returns without success, the queen must be removed so the board is exactly as before and the next column can be tried. The key to speed is the early rejection.

Syntax

```java
int[] col = new int[n]; // col[row] = column of the queen in that row
boolean[] cols = new boolean[n]; // column used?
boolean[] diag1 = new boolean[2 * n - 1]; // "/" diagonals: row + col
boolean[] diag2 = new boolean[2 * n - 1]; // "" diagonals: row - col + n - 1
if (cols[c] || diag1[r + c] || diag2[r - c + n - 1]) continue; // attacked: prune
cols[c] = diag1[r + c] = diag2[r - c + n - 1] = true; // place
solve(r + 1);
cols[c] = diag1[r + c] = diag2[r - c + n - 1] = false; // remove (backtrack)
```

A tiny example

Two queens on the same diagonal satisfy |Δrow| = |Δcol|.

```
// safe if no earlier row has a queen in the same column or on a diagonal
for (int r = 0; r < row; r++) {
 if (queenCol[r] == col || Math.abs(queenCol[r] - col) == row - r) return false;
}
return true;
```

Guided example

NQueensFirst.java — First solution by backtracking (simple solution)

```
1  import java.util.Scanner;
2
3  public class NQueensFirst {
4   static int n;
5   static int[][] board;
6
7   // Can a queen stand at (row, col)? Check the column and both diagonals ABOVE the row
8   // (rows below are still empty because we fill the board one row at a time).
9   static boolean isSafe(int row, int col) {                                    (1)
10  for (int r = 0; r < row; r++) {
11  if (board[r][col] == 1) {
12  return false; // same column
13  }
14  }
15  for (int r = row - 1, c = col - 1; r >= 0 && c >= 0; r--, c--) {             (2)
16  if (board[r][c] == 1) {
17  return false; // upper-left diagonal
18  }
19  }
20  for (int r = row - 1, c = col + 1; r >= 0 && c < n; r--, c++) {              (3)
21  if (board[r][c] == 1) {
22  return false; // upper-right diagonal
23  }
24  }
25  return true;
26  }
27
28  // Place one queen per row. Returns true as soon as every row has a queen.
29  static boolean solve(int row) {
30  if (row == n) {                                                              (4)
31  return true; // all N queens placed
32  }
33  for (int col = 0; col < n; col++) {
34  if (isSafe(row, col)) {
35  board[row][col] = 1; // choose                                               (5)
36  if (solve(row + 1)) { // explore the next row                               (6)
37  return true;
38  }
39  board[row][col] = 0; // un-choose: BACKTRACK and try the next column         (7)
40  }
41  }
42  return false; // no column works in this row -> previous row must move
43  }
44
45  public static void main(String[] args) {
```

(code continued)

```
    Scanner sc = new Scanner(System.in);
    n = sc.nextInt();
    board = new int[n][n];
    if (!solve(0)) {
    System.out.println("Solution does not exist");
    return;
    }
    for (int[] row : board) {
    StringBuilder sb = new StringBuilder();
    for (int cell : row) {
    sb.append(sb.length() == 0 ? "" : " ").append(cell);
    }
    System.out.println(sb);
    }
    }
  }
```

Row-by-row search with a board scan for the safety test. Stops at the first solution.

Sample input

```
4
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Only rows ABOVE `row` can already hold queens, so only those need checking.
2	line 15	Walk up-left along the diagonal.
3	line 20	Walk up-right along the other diagonal.
4	line 30	Base case: every row has a queen — success.
5	line 35	CHOOSE: place a queen.
6	line 36	EXPLORE: try to place the rest; propagate success upwards.
7	line 39	UN-CHOOSE (backtrack): remove the queen and try the next column.

NQueensCount.java — Counting all solutions (optimised solution)

```java
import java.util.Scanner;

public class NQueensCount {
 static int n;
 static boolean[] column; // column c already has a queen
 static boolean[] diagDown; // diagonal r - c + (n - 1)
 static boolean[] diagUp; // anti-diagonal r + c
 static int solutions;

 // O(1) safety test: three boolean arrays instead of scanning the board.
 static void place(int row) {
 if (row == n) {
 solutions++;
 return;
 }
 for (int col = 0; col < n; col++) {
 int d1 = row - col + n - 1;
 int d2 = row + col;
 if (column[col] || diagDown[d1] || diagUp[d2]) {
 continue;
 }
 column[col] = diagDown[d1] = diagUp[d2] = true; // choose
 place(row + 1); // explore
 column[col] = diagDown[d1] = diagUp[d2] = false; // un-choose
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 column = new boolean[n];
 diagDown = new boolean[2 * n - 1];
 diagUp = new boolean[2 * n - 1];
 place(0);
 System.out.println("N = " + n + ": " + solutions + " solutions");
 }
}
```

Three boolean arrays make every safety test O(1). 92 solutions for N = 8.

Sample input

```
8
```

Line-by-line walkthrough

#	Where	What happens
1	line 6	Cells on the same "\" diagonal share row − col; shifted by n−1 to be a valid index.
2	line 18	Cells on the same "/" diagonal share row + col.
3	line 22	Mark the three lines as occupied.
4	line 24	Free them again on the way back.
5	line 13	Do not stop at the first solution — keep exploring to count all of them.

The slide fills the board column by column and prints `0 0 1 0 / 1 0 0 0 / 0 0 0 1 / 0 1 0 0`. Filling row by row finds the mirror-image solution `0 1 0 0 / 0 0 0 1 / 1 0 0 0 / 0 0 1 0`; both are valid

because the 4-queens solutions are mirror images of each other.

Dry run: watch it execute

Placing, rejecting and backtracking row by row for N = 4.

Algorithm: N-Queens backtracking. **Input:** n = `4`.

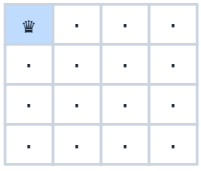
Pseudocode

```
solve(row):
 if row == n: success
 for col in 0..n-1:
 if safe(row, col):
 place; if solve(row+1): return true
 remove (backtrack)
 return false
```

The full trace has 30 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

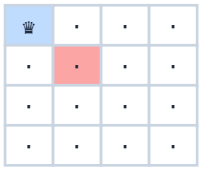
Step 1 of 30. Row 0: column 0 is safe → place a queen.

row = 0 · col = 0 · queens = 1



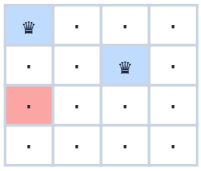
Step 3 of 30. Row 1: column 1 is attacked — skip.

row = 1 · col = 1 · queens = 1



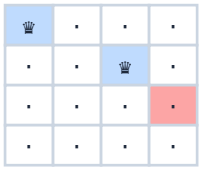
Step 5 of 30. Row 2: column 0 is attacked — skip.

row = 2 · col = 0 · queens = 2



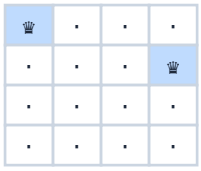
Step 8 of 30. Row 2: column 3 is attacked — skip.

row = 2 · col = 3 · queens = 2



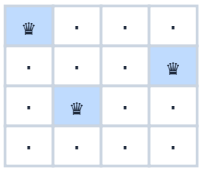
Step 10 of 30. Row 1: column 3 is safe → place a queen.

row = 1 · col = 3 · queens = 2



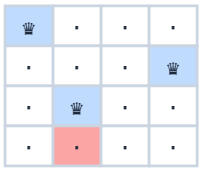
Step 12 of 30. Row 2: column 1 is safe → place a queen.

row = 2 · col = 1 · queens = 3



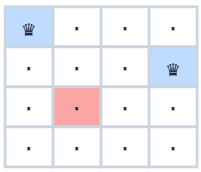
Step 14 of 30. Row 3: column 1 is attacked — skip.

row = 3 · col = 1 · queens = 3



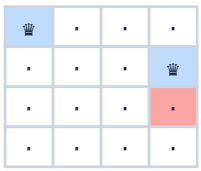
Step 17 of 30. Row 3 had no safe column → remove the queen from row 2, column 1 (backtrack).

row = 2 · col = 1 · queens = 2



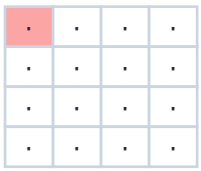
Step 19 of 30. Row 2: column 3 is attacked — skip.

row = 2 · col = 3 · queens = 2



Step 21 of 30. Row 1 had no safe column → remove the queen from row 0, column 0 (backtrack).

row = 0 · col = 0 · queens = 0



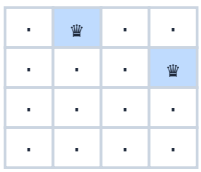
Step 23 of 30. Row 1: column 0 is attacked — skip.

row = 1 · col = 0 · queens = 1



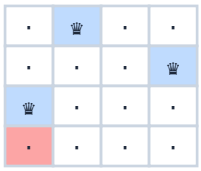
Step 26 of 30. Row 1: column 3 is safe → place a queen.

row = 1 · col = 3 · queens = 2



Step 28 of 30. Row 3: column 0 is attacked — skip.

row = 3 · col = 0 · queens = 3



Step 30 of 30. Row 3: column 2 is safe → place a queen.

row = 3 · col = 2 · queens = 4



> **RESULT**
> solution (column per row): 2 4 1 3

Complexity

There are N^N ways to put one queen in each row; pruning shrinks that dramatically but the worst case is still exponential. The number of nodes visited grows roughly like N! (fewer in practice): N = 8 takes about 15 000 recursive calls; N = 12 about 10 million; N = 15 several billion.

Time: **O(N!)** upper bound (honest worst case). Space: O(N) for the recursion and the three arrays (the slide's N×N board is O(N^2)).

N	1	2	3	4	5	6	7	8	9	10
solutions	1	0	0	2	10	4	40	92	352	724

Common mistakes

Forgetting to un-place the queen

Problem

```
board[row][col] = 1;
solve(row + 1); // no reset
```

Why: Dead-end branches leave queens on the board, which then block valid placements, so correct solutions are missed.

Fix: if (!solve(row + 1)) board[row][col] = 0;

Checking the wrong diagonal index

Problem

```
diagDown[row − col] // negative index for col > row
```

Why: row − col ranges from −(n−1) to n−1: negative indices crash.

Fix: diagDown[row - col + n - 1]

Stopping at the first solution when counting

Problem

```
if (place(row + 1)) return true; // in a counting program
```

Why: The function returns as soon as one solution is found; the count is 1.

Fix: Count in the base case and return nothing (void).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Validate a placement — *Beginner*

Read n and then n integers: the column (1-based) of the queen in each row. Print VALID if no two queens attack each other, else INVALID.

Example input	Expected output
4 ⏎ 2 4 1 3	VALID

Hint 1: Same column, or |Δrow| == |Δcol|, means attack.

Exercise 2 · Count solutions — *Intermediate*

Read N (1 ≤ N ≤ 10) and print the number of distinct ways to place N non-attacking queens.

Example input	Expected output
6	4

Hint 1: Row by row; three boolean arrays for column and diagonals.

Exercise 3 · Lexicographically first solution — *Challenge*

Read N (1 ≤ N ≤ 12). Print the solution that is smallest when read as the column of the queen in row 1, row 2, ... (columns 1-based, separated by spaces), or NONE.

Example input	Expected output
8	1 5 8 6 3 7 2 4

Hint 1: Trying columns in increasing order and stopping at the first success gives the lexicographically first.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many solutions does the 4-queens problem have?

- **A.** 1
- **B.** 2
- **C.** 4
- **D.** 24

Q2. Two queens at (1, 2) and (3, 4) attack each other because...

- **A.** same row
- **B.** same column
- **C.** same diagonal
- **D.** they do not

Q3. Complete the backtracking step.

```
board[row][col] = 1;
if (solve(row + 1)) return true;
board[row][col] = ______;
```

Q4. What does "backtracking" mean here?

- **A.** Sorting the board
- **B.** Undoing the last choice and trying the next option
- **C.** Starting the whole search again
- **D.** Reading the board backwards

Q5. Which strategy fits "find all arrangements that satisfy constraints"?

- **A.** Greedy
- **B.** Binary search
- **C.** Backtracking
- **D.** Sorting

Interview / exam questions

- Why can you place exactly one queen per row?
- How do you check diagonals in O(1)?
- What is the complexity? Can you do better with bitmasks?
- How many solutions are there for N = 8 and how many are distinct up to symmetry (12)?

Watch & read

Video: 6.1 N Queens Problem using Backtracking — Abdul Bari · 13:41. *N-Queens by backtracking*

Video: Visualization of N Queens (5×5 board) — G Bhat · 2:39. *N-Queens implementation*

REVISION CARD (30–60 SECONDS)

- One queen per row; try columns left→right.
- choose → explore → un-choose.
- Conflict: same column, or row − col equal, or row + col equal.
- Counts: 1,0,0,2,10,4,40,92,352,724 for N = 1…10. O(N!) worst case.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide fills the board column by column (`col`), so its printed solution is the transpose/mirror of the one a row-by-row search prints. Both are valid. • The slide says "queen can move any number of steps in any direction … can't change direction" and then lists only rows and columns; diagonals are an essential part of the attack rule.

5.30 Maze Solving (Rat in a Maze)

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · ★ on the minimum-viable path

Find a path from the entry to the exit of a grid maze by depth-first backtracking, then the shortest path with BFS.

YOU WILL BE ABLE TO

- Model a maze as a 2-D array and neighbours as direction offsets.
- Write a recursive depth-first search that marks the solution path and backtracks from dead ends.
- Explain why a `visited` array is required when moves may go back.
- Use a queue (BFS) to find the shortest route.

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Two-Dimensional Arrays (Matrices) · N-Queens (Backtracking)

The idea

A maze is a grid of **walls** and **open cells** with one entry and one exit. The slide encodes it as: `0` road, `1` wall, `2` entry, `3` exit, `4` a cell on the path that the algorithm found. A rat enters at the entry and tries to reach the exit.

Depth-first search with backtracking is how you would explore a maze with chalk: walk forward, mark where you have been, and if you reach a dead end, walk back and try the next turn.

```
2 0 0 1 0 2 4 4 1 0 (4 = the path found)
1 1 0 1 0 → 1 1 4 1 0
0 0 0 0 0 4 4 4 0 0
0 1 1 1 1 4 1 1 1 1
0 0 0 0 3 4 4 4 4 3
```

WHY THIS WORKS

A cell is on a path to the exit if it is the exit, or if at least one of its neighbours leads to the exit. The recursion asks exactly that: `dfs(cell)` returns true if the exit is reachable from here without revisiting cells. The `visited` array turns an endless walk in circles into a finite search; marking the successful route only on the way **back** (when `dfs` returns true) means dead-end explorations leave no trace in the final path.

Depth-first search finds *a* path, not necessarily the shortest one. **Breadth-first search** (BFS) explores the maze in rings of increasing distance using a queue, so the first time it reaches the exit it has found a shortest route.

Syntax

Direction arrays replace four copies of nearly identical code.

```
int[] dr = {1, 0, -1, 0}; // down, right, up, left
int[] dc = {0, 1, 0, -1};
for (int d = 0; d < 4; d++) {
 int nr = r + dr[d], nc = c + dc[d];
}
```

A tiny example

```
static boolean isSafe(int[][] maze, int x, int y) {
 return x >= 0 && x < N && y >= 0 && y < N && maze[x][y] == 1;
}
```

Guided example

RatMazeDownRight.java — Rat in a maze (down/right only) (from the slides)

```
1  public class RatMazeDownRight {
2   static final int N = 4;
3
4   static boolean isSafe(int[][] maze, int x, int y) {
5   return x >= 0 && x < N && y >= 0 && y < N && maze[x][y] == 1; // inside the grid and an (1)
        open cell
6   }
7
8   // The lecture's version: the rat may only move DOWN or RIGHT.
9   static boolean solve(int[][] maze, int x, int y, int[][] sol) {
10  if (x == N - 1 && y == N - 1) { (2)
11  sol[x][y] = 1; (3)
12  return true; // reached the exit
13  }
14  if (isSafe(maze, x, y)) {
15  sol[x][y] = 1; // mark this cell as part of the path
16  if (solve(maze, x + 1, y, sol)) return true; // try down (4)
17  if (solve(maze, x, y + 1, sol)) return true; // try right (5)
18  sol[x][y] = 0; // dead end: un-mark (backtrack) (6)
19  }
20  return false;
21  }
22
23  public static void main(String[] args) {
24  int[][] maze = {
25  {1, 0, 0, 0},
26  {1, 1, 0, 1},
27  {0, 1, 0, 0},
28  {1, 1, 1, 1}
29  };
30  int[][] sol = new int[N][N];
31  if (!solve(maze, 0, 0, sol)) {
32  System.out.println("Solution doesn't exist");
33  return;
34  }
35  for (int[] row : sol) {
36  StringBuilder sb = new StringBuilder();
37  for (int v : row) sb.append(' ').append(v).append(' ');
38  System.out.println(sb);
39  }
40  }
41 }
```

The lecture's recursive backtracker. `sol` marks the path; dead ends are unmarked.

Line-by-line walkthrough

#	Where	What happens
1	line 5	Safe = inside the grid AND an open cell. Bounds are checked first so the array access cannot fail.
2	line 10	Reached the bottom-right exit: mark it and succeed.
3	line 11	Tentatively mark this cell as part of the path.
4	line 16	Try moving down first...
5	line 17	...then right.
6	line 18	Both failed: this cell is a dead end. Un-mark it and return false so the caller tries its next option.

MazeSolver.java — General maze solver (4 directions) (optimised solution)

```java
import java.util.Scanner;

public class MazeSolver {
 // Encoding from the lecture slide:
 // 0 road, 1 wall, 2 entry, 3 exit, 4 cell on the path found
 static int rows, cols;
 static int[][] g;
 static boolean[][] visited;
 static final int[] DR = {1, 0, -1, 0}; // down, right, up, left (1)
 static final int[] DC = {0, 1, 0, -1};

 static boolean dfs(int r, int c) {
 if (r < 0 || r >= rows || c < 0 || c >= cols || g[r][c] == 1 || visited[r][c]) {
 return false; // outside, wall, or already explored
 }
 visited[r][c] = true; // remember: never walk in circles (2)
 if (g[r][c] == 3) { (3)
 return true; // exit found
 }
 for (int d = 0; d < 4; d++) {
 if (dfs(r + DR[d], c + DC[d])) { (4)
 if (g[r][c] == 0) g[r][c] = 4; // this cell lies on the successful route (5)
 return true;
 }
 }
 return false; // all four directions failed: dead end
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 rows = sc.nextInt();
 cols = sc.nextInt();
 g = new int[rows][cols];
 visited = new boolean[rows][cols];
 int sr = 0, sc0 = 0;
 for (int i = 0; i < rows; i++) {
 for (int j = 0; j < cols; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) {
 sr = i;
 sc0 = j;
 }
 }
 }
 if (!dfs(sr, sc0)) {
 System.out.println("No path from entry to exit");
 return;
 }
 for (int[] row : g) {
 StringBuilder sb = new StringBuilder();
 for (int v : row) sb.append(sb.length() == 0 ? "" : " ").append(v);
 System.out.println(sb);
 }
 }
}
```

Depth-first search with a visited array; reads the maze from standard input; marks the route with 4.

Sample input

```
5 5
2 0 0 1 0
1 1 0 1 0
0 0 0 0 0
0 1 1 1 1
0 0 0 0 3
```

Line-by-line walkthrough

#	Where	What happens
1	line 9	Four neighbours as offset arrays.
2	line 16	Never enter a cell twice — otherwise moving up/left would loop forever.
3	line 17	Exit cell: the base case.
4	line 21	Explore each neighbour; the first one that succeeds finishes the search.
5	line 22	Mark the route only on the way back, and keep the entry (2) and exit (3) labels untouched.

> **NOTE**
> The slide shows the 4×4 maze with 1 = path in the code but describes the model 0 = road, 1 = wall in the text. The two programs above keep each convention explicit; mixing them is the most common bug when adapting this code.

Dry run: watch it execute

The rat explores down first, then right, marking and un-marking cells.

Algorithm: Rat in a maze (down/right). **Input:** grid = `1000,1101,0100,1111`.

Pseudocode

```
solve(x, y):
 if (x, y) is the exit: mark; return true
 if safe(x, y):
 mark sol[x][y]
 if solve(x+1, y): return true (down)
 if solve(x, y+1): return true (right)
 unmark sol[x][y] (dead end)
 return false
```

Step 1 of 7. Mark (0, 0) as part of the path.

x = 0 · y = 0

→	▓	▓	▓
1	1	▓	1
▓	1	▓	▓
1	1	1	1

Step 2 of 7. Mark (1, 0) as part of the path.

x = 1 · y = 0

→	▓	▓	▓
→	1	▓	1
▓	1	▓	▓
1	1	1	1

Step 3 of 7. Mark (1, 1) as part of the path.

x = 1 · y = 1

→	▓	▓	▓
→	→	▓	1
▓	1	▓	▓
1	1	1	1

Step 4 of 7. Mark (2, 1) as part of the path.

x = 2 · y = 1

→	▓	▓	▓
→	→	▓	1
▓	→	▓	▓
1	1	1	1

Step 5 of 7. Mark (3, 1) as part of the path.

x = 3 · y = 1

→	▓	▓	▓
→	→	▓	1
▓	→	▓	▓
1	→	1	1

Step 6 of 7. Mark (3, 2) as part of the path.

x = 3 · y = 2

→	▓	▓	▓
→	→	▓	1
▓	→	▓	▓
1	→	→	1

Step 7 of 7. (3, 3) is the exit — success!

x = 3 · y = 3

→	▓	▓	▓
→	→	▓	1
▓	→	▓	▓
1	→	→	→

> **RESULT**
> path found

Complexity

DFS with a visited array: each cell is entered at most once and examines 4 neighbours → **O(R·C) time**, O(R·C) space for `visited` plus the recursion stack (up to R·C deep).

Without a visited array (down/right only) each cell is on a monotone path, so the search is still finite but can be exponential: O(2^(R+C)) in the worst case.

BFS: O(R·C) time and space and it returns the shortest path length.

> **WATCH OUT**
> Recursion depth can reach R·C. For big mazes (1000 × 1000) use an explicit stack/queue to avoid StackOverflowError.

Common mistakes

No visited array when moving in 4 directions

Problem

```
solve(x + 1, y) || solve(x, y + 1) || solve(x − 1, y) || solve(x, y − 1)
```

Why: The rat steps right, then left, then right again ... infinite recursion → StackOverflowError.

Fix: Mark cells as visited before exploring neighbours.

Checking maze[x][y] before the bounds

Problem

```
maze[x][y] == 1 && x >= 0 && x < N
```

Why: ArrayIndexOutOfBounds for x = −1 or x = N: the operands of && are evaluated left to right.

Fix: Test the bounds first.

Marking the path on the way in

Problem

```
g[r][c] = 4; // before exploring neighbours
```

Why: Dead-end cells stay marked as "path". Mark when dfs returns true, or un-mark on failure.

Fix: mark after the successful recursive call (or reset on failure)

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · How many cells can the rat reach? — *Beginner*

Read R, C and an R × C maze (0 road, 1 wall, 2 entry, 3 exit). Print the number of non-wall cells reachable from the entry by moving up, down, left or right (the entry itself counts).

Example input	Expected output
3 3 ⏎ 2 0 1 ⏎ 1 0 1 ⏎ 1 0 3	5

Hint 1: Flood fill with recursion or a stack. **Hint 2:** Mark cells visited.

Exercise 2 · Is there a way out? — *Intermediate*

Same input. Print YES if the exit can be reached from the entry, else NO.

Example input	Expected output
`3 3 ⏎ 2 1 3 ⏎ 0 1 0 ⏎ 0 0 0`	YES

Hint 1: DFS returning boolean.

Exercise 3 · Shortest path — *Challenge*

Same input. Print the minimum number of moves from the entry to the exit, or -1.

- 1 ≤ R, C ≤ 500

Example input	Expected output
`3 3 ⏎ 2 0 0 ⏎ 1 1 0 ⏎ 3 0 0`	6

Hint 1: Breadth-first search with a queue and a distance array.

Quick check

Answers are in the answer key at the end of this part.

Q1. Why does a 4-direction maze solver need a visited array?

- **A.** To save memory
- **B.** To avoid walking in circles forever
- **C.** To count steps
- **D.** To find the shortest path

Q2. Which algorithm guarantees the SHORTEST path in an unweighted maze?

- **A.** DFS
- **B.** BFS
- **C.** Binary search
- **D.** Quick sort

Q3. What does `sol[x][y] = 0;` after both recursive calls do?

- **A.** Mark success
- **B.** Un-mark a dead-end cell (backtrack)
- **C.** Reset the maze
- **D.** Nothing

Q4. DFS on an R × C maze with a visited array runs in...

- **A.** O(R + C)
- **B.** O(R·C)
- **C.** O(2^(R·C))
- **D.** O($R^2 \cdot C^2$)

Interview / exam questions

- DFS vs BFS for maze solving — when would you choose each?
- How do you print the actual shortest path with BFS?
- How would you avoid recursion depth problems?

Watch & read

Video: L19. Rat in A Maze | Backtracking — take U forward · 25:10. *Rat in a maze*

Video: Rat in a Maze Problem | Backtracking — Apna College · 32:45. *Backtracking on a grid*

REVISION CARD (30–60 SECONDS)

- Maze = grid; neighbours via dr/dc arrays; bounds before wall test.
- DFS: visited, recurse, mark route when success returns true, backtrack on failure.
- BFS (queue) = shortest path.
- O(R·C).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide models the maze as 0 road / 1 wall / 2 entry / 3 exit / 4 path, but its Java program uses 1 for an open cell and only moves down/right. The lesson provides one program for each convention and states which is which. • The slide's printed solution `1 0 0 0 / 1 1 0 0 / 0 1 0 0 / 0 1 1 1` matches `RatMazeDownRight`. • The 5×5 matrix at the end of the slide has no stated solution; it needs up/left moves, which is why a visited-array solver is provided.

5.31 Hamiltonian Cycle

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · complete path

Does a graph contain a cycle that visits every vertex exactly once? Backtracking over an adjacency matrix.

YOU WILL BE ABLE TO

- Define a graph, adjacency matrix and Hamiltonian cycle.
- Write the backtracking search with a path array and an `isSafe` test.
- Explain why the start vertex can be fixed and why the problem is exponential.
- Run it on both slide graphs and predict the outcome.

Before this lesson: Trees, Graphs and Heaps · N-Queens (Backtracking) · Two-Dimensional Arrays (Matrices)

The idea

A **graph** is a set of vertices connected by edges. A **Hamiltonian cycle** is a closed walk that visits **every vertex exactly once** and returns to the start. (Compare an Eulerian cycle, which uses every *edge* once — a much easier problem.)

```
(0)--(1)--(2) cycle: 0 → 1 → 2 → 4 → 3 → 0
| / | \ |
| / | \ |
(3)---+--(4) edges 0–1 1–2 2–4 4–3 3–0 are all present
```

The graph is stored as an **adjacency matrix**: `graph[i][j] == 1` if there is an edge between i and j. The search builds a `path` array: place vertex 0 first (any cycle can be rotated to start there), then at each position try every unused vertex that is adjacent to the previous one. When all V vertices are placed, the last must also be adjacent to the first.

WHY THIS WORKS

There is no known fast algorithm for this problem — it is NP-complete. Backtracking works because most partial paths fail quickly (no edge, or a vertex reused); the `path[pos] = -1` reset is the un-choose step that lets the loop try the next candidate. The cost is bounded by (V−1)! complete orderings.

Syntax

```
int[][] graph = {{0,1,0,1,0}, {1,0,1,1,1}, {0,1,0,0,1}, {1,1,0,0,1}, {0,1,1,1,0}}; // adjacency
    matrix
int[] path = new int[n]; Arrays.fill(path, -1); // path[pos] = vertex placed there
path[0] = 0; // fix the start: a cycle can start anywhere
boolean ok = graph[path[pos - 1]][v] == 1; // an edge from the previous vertex?
boolean fresh = true; // v not already on the path
for (int i = 0; i < pos; i++) if (path[i] == v) fresh = false;
// the cycle closes only if the LAST vertex is adjacent to path[0]
```

A tiny example

```
// is there an edge from the previous vertex, and is the candidate unused?
if (graph[path[pos - 1]][v] == 0) return false;
for (int i = 0; i < pos; i++) if (path[i] == v) return false;
return true;
```

Guided example

The two slide graphs (5 vertices) are `graph1` — which has the cycle 0 1 2 4 3 0 — and `graph2`, where vertices 3 and 4 both have only one neighbour besides vertex 1, which makes a cycle impossible.

HamiltonianCycle.java — Hamiltonian cycle by backtracking (optimised solution)

```
import java.util.Arrays;
import java.util.Scanner;

public class HamiltonianCycle {
 static int v;
 static int[][] graph;
 static int[] path;

 // Can vertex `candidate` be placed at position `pos` of the path?
 static boolean isSafe(int candidate, int pos) {                    (1)
 if (graph[path[pos - 1]][candidate] == 0) {                        (2)
 return false; // there is no edge from the previous vertex
 }
 for (int i = 0; i < pos; i++) {
 if (path[i] == candidate) {                                        (3)
 return false; // each vertex may be used only once
 }
 }
 return true;
 }

 static boolean extend(int pos) {
 if (pos == v) {                                                    (4)
 return graph[path[pos - 1]][path[0]] == 1; // all placed: the last must connect back to
     the start
 }
 for (int candidate = 1; candidate < v; candidate++) {
 if (isSafe(candidate, pos)) {
 path[pos] = candidate; // choose                                   (5)
 if (extend(pos + 1)) {
 return true;
 }
 path[pos] = -1; // un-choose                                       (6)
 }
 }
 return false;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 v = sc.nextInt();
 graph = new int[v][v];
 for (int i = 0; i < v; i++) {
 for (int j = 0; j < v; j++) {
```

(code continued)

```
    graph[i][j] = sc.nextInt();
    }
    }
    path = new int[v];
    Arrays.fill(path, -1);
    path[0] = 0; // a cycle can start anywhere: fix vertex 0
    if (!extend(1)) {
    System.out.println("Solution does not exist");
    return;
    }
    StringBuilder sb = new StringBuilder();
    for (int x : path) sb.append(x).append(' ');
    sb.append(path[0]);
    System.out.println("Solution exists: " + sb);
    }
   }
```

7

Complete version of the (fragmented) slide program, reading its graph from standard input.

Sample input

```
5
0 1 0 1 0
1 0 1 1 1
0 1 0 0 1
1 1 0 0 1
0 1 1 1 0
```

Line-by-line walkthrough

#	Where	What happens
1	line 10	Can `candidate` be the vertex at position `pos` of the path?
2	line 11	There must be an edge from the previously placed vertex.
3	line 15	And the vertex must not already be on the path.
4	line 23	All vertices placed: the cycle must close — the last vertex needs an edge back to vertex 0.
5	line 28	CHOOSE.
6	line 32	UN-CHOOSE: free the slot for the next candidate.
7	line 49	Fix the start. A cycle has no natural start, so every cycle appears as one that begins at vertex 0.

Dry run: watch it execute

Extending and abandoning partial paths.

Algorithm: Hamiltonian cycle. **Input:** matrix = `01010,10111,01001,11001,01110`.

Pseudocode

```
path[0] = 0
extend(pos):
 if pos == V: return edge(path[V-1], path[0])
 for v in 1..V-1:
 if edge(path[pos-1], v) and v not in path:
 path[pos] = v; if extend(pos+1): return true
 path[pos] = -1
 return false
```

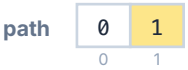
Step 1 of 6. Fix the start at vertex 0.

pos = 1



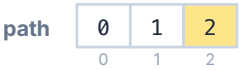
Step 2 of 6. Position 1: vertex 1 is adjacent to 0 and unused → place it.

pos = 1 · v = 1



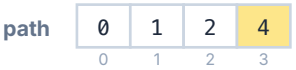
Step 3 of 6. Position 2: vertex 2 is adjacent to 1 and unused → place it.

pos = 2 · v = 2



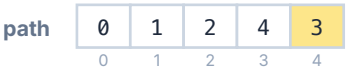
Step 4 of 6. Position 3: vertex 4 is adjacent to 2 and unused → place it.

pos = 3 · v = 4



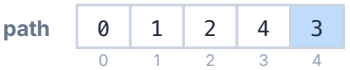
Step 5 of 6. Position 4: vertex 3 is adjacent to 4 and unused → place it.

pos = 4 · v = 3



Step 6 of 6. All 5 vertices placed and 3 → 0 is an edge: cycle closed!

pos = 5



> **RESULT**
> Hamiltonian cycle: 0 → 1 → 2 → 4 → 3 → 0

Complexity

Worst case the search examines every permutation of the remaining V−1 vertices: **O(V!)** time. Space: O(V) for `path` and the recursion stack (O(V²) for the matrix).

Smarter algorithms exist (Held–Karp dynamic programming: O(2^V · V²)), but nothing polynomial is known.

Common mistakes

Skipping the closing edge

Problem

```
if (pos == v) return true;
```

Why: A path through all vertices is a Hamiltonian PATH. A cycle also needs an edge from the last vertex back to the start.

Fix: return graph[path[pos - 1]][path[0]] == 1;

Not resetting path[pos]

Problem

```
path[pos] = v; if (extend(pos + 1)) return true; // no reset
```

Why: A stale value remains in the array; isSafe will think that vertex is already used.

Fix: path[pos] = -1;

Expecting the slide output for the second graph

Problem

```
// slide shows "0 1 2 4 3 0" after graph2
```

Why: That is the answer for graph1. graph2 has no Hamiltonian cycle: the program prints "Solution does not exist".

Fix: Run both and compare.

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Verify a cycle — *Beginner*

Read V, the V × V adjacency matrix, and then V+1 vertex numbers. Print YES if they form a Hamiltonian cycle (start = end, every vertex once, consecutive vertices adjacent).

Example input	Expected output
4 ⏎ 0 1 0 1 ⏎ 1 0 1 0 ⏎ 0 1 0 1 ⏎ 1 0 1 0 ⏎ 0 1 2 3 0	YES

Hint 1: Check first = last, no repeats among the first V, and an edge between consecutive vertices.

Exercise 2 · Find a cycle — *Intermediate*

Read V and the matrix. Print `YES` followed by the first Hamiltonian cycle found starting at 0 (trying vertices in increasing order) — e.g. `YES 0 1 2 4 3 0` — or `NO`.

- 3 ≤ V ≤ 10

Example input	Expected output
5 ⏎ 0 1 0 1 0 ⏎ 1 0 1 1 1 ⏎ 0 1 0 0 1 ⏎ 1 1 0 0 0 ⏎ 0 1 1 0 0	NO

Hint 1: Backtracking with path[] and the closing check.

Exercise 3 · Count Hamiltonian cycles — *Challenge*

Read V (≤ 10) and the matrix. Count the Hamiltonian cycles that start and end at vertex 0, counting a cycle and its reverse as two different cycles.

Example input	Expected output
4 ⏎ 0 1 1 1 ⏎ 1 0 1 1 ⏎ 1 1 0 1 ⏎ 1 1 1 0	6

Hint 1: Same search but do not stop at the first success. **Hint 2:** K_4 has 3! = 6 orderings of the other vertices.

Quick check

Answers are in the answer key at the end of this part.

Q1. A Hamiltonian cycle visits...

- **A.** every edge once
- **B.** every vertex exactly once and returns to the start
- **C.** every vertex at least once
- **D.** the shortest route

Q2. Why can the search always start at vertex 0?

- **A.** Vertex 0 has most edges
- **B.** A cycle can be rotated to start anywhere
- **C.** Because arrays start at 0
- **D.** It cannot

Q3. Time complexity of the backtracking search?

- **A.** O(V)
- **B.** O(V²)
- **C.** O(V!)
- **D.** O(log V)

Q4. A vertex with only one neighbour makes a Hamiltonian cycle...

- **A.** easier
- **B.** impossible
- **C.** unchanged
- **D.** a path

Interview / exam questions

- Hamiltonian vs Eulerian cycle?
- Is Hamiltonian cycle NP-complete? What does that imply?
- Bitmask DP for Hamiltonian cycle: complexity?
- How does this relate to the travelling salesman problem?

Watch & read

Video: 6.4 Hamiltonian Cycle - Backtracking — Abdul Bari · 18:35. *Hamiltonian cycle by backtracking*

Video: Euler Graph in Graph Theory | Euler Path & Euler Circuit with examples — Gate Smashers · 6:45. *Hamiltonian vs Eulerian paths*

REVISION CARD (30–60 SECONDS)

• Hamiltonian cycle: every vertex once, back to start.
• path[0] = 0; place vertices with edge from previous & unused; close with edge to path[0].
• choose / explore / reset to −1.
• O(V!) — NP-complete.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide listing of the Java class is scrambled over two slides (the `hamCycle` method header is missing and part of `main` is duplicated), so it does not compile as printed. The lesson program is the complete, working version. • The slide's final output line "0 1 2 4 3 0" is printed after `graph2`, but that graph has NO Hamiltonian cycle; the output belongs to `graph1`.

5.32 Knight's Tour & Warnsdorff's Rule

Archive Algorithm Mastery · Advanced · Archive algorithm knowledge · about 55 min · complete path

Move a knight so that it visits every square once — plain backtracking explodes, a one-line heuristic solves 8×8 instantly.

YOU WILL BE ABLE TO

- List the eight knight moves and check them against the board edges.
- Describe Warnsdorff's rule: always move to the square with the fewest onward moves.
- Implement the heuristic with a `degree` function and a deterministic tie-break.
- Compare it with exhaustive backtracking and verify a tour.

Before this lesson: N-Queens (Backtracking) · Maze Solving (Rat in a Maze) · Two-Dimensional Arrays (Matrices)

The idea

A **knight** moves in an "L": two squares in one direction and one square sideways — up to eight possible moves. A **knight's tour** visits every square of the board exactly once. It is a Hamiltonian path on the "knight graph".

Exhaustive backtracking (try a move, undo if stuck) works for 5 × 5 but explodes for 8 × 8. **Warnsdorff's rule** (1823) is a heuristic: from the current square move to the unvisited square that has the **fewest onward moves**. The idea: squares with few exits are the most likely to be stranded later, so visit them first.

```
knight at (x, y) can jump to
(x±1, y±2) and (x±2, y±1)
degree(square) = number of unvisited squares the knight could jump to from it
```

WHY THIS WORKS

Squares with few remaining exits (corners start with only 2) are dangerous: if you leave them for last you may never be able to enter them. Always visiting the *most constrained* square first is a classic "fail-first" principle from constraint solving. It is only a **heuristic** — it usually works for standard boards, but there is no proof it never fails; that is why the slide's program restarts with a random tie-break until it succeeds.

Syntax

```
final int[] DX = { 1, 2, 2, 1, -1, -2, -2, -1}; // the eight knight moves
final int[] DY = { 2, 1, -1, -2, -2, -1, 1, 2};
int nx = x + DX[k], ny = y + DY[k];
boolean inside = nx >= 0 && ny >= 0 && nx < N && ny < N;
boolean free = inside && board[nx][ny] == -1; // -1 = not visited yet
// Warnsdorff: among free moves choose the one with the fewest onward free moves
```

A tiny example

```
int[] dx = {1, 1, 2, 2, -1, -1, -2, -2};
int[] dy = {2, -2, 1, -1, 2, -2, 1, -1};
```

Guided example

WarnsdorffTour.java — Warnsdorff's heuristic (optimised solution)

```
1  import java.util.Scanner;
2
3  public class WarnsdorffTour {
4   static final int[] DX = {1, 1, 2, 2, -1, -1, -2, -2}; // the eight knight moves          1
5   static final int[] DY = {2, -2, 1, -1, 2, -2, 1, -1};
6
7   static int n;
8   static int[][] board; // board[y][x] = move number (1..n*n), 0 = not visited yet
9
10  static boolean free(int x, int y) {
11  return x >= 0 && y >= 0 && x < n && y < n && board[y][x] == 0;
12  }
13
14  // degree = how many unvisited squares a knight standing on (x, y) could still jump to
15  static int degree(int x, int y) {                                                        2
16  int count = 0;
17  for (int m = 0; m < 8; m++) {
18  if (free(x + DX[m], y + DY[m])) {
19  count++;
20  }
21  }
22  return count;
23  }
24
25  // Warnsdorff's rule: always jump to the neighbour with the FEWEST onward moves.
26  // Ties are broken by the first move in the DX/DY order (so the result is deterministic)
        .
27  static boolean tour(int startX, int startY) {
28  int x = startX;
29  int y = startY;
30  board[y][x] = 1;
31  for (int step = 2; step <= n * n; step++) {
32  int bestMove = -1;
33  int bestDegree = 9;
34  for (int m = 0; m < 8; m++) {
35  int nx = x + DX[m];
36  int ny = y + DY[m];
37  if (free(nx, ny)) {
38  int d = degree(nx, ny);
39  if (d < bestDegree) {                                                                    3
40  bestDegree = d;
41  bestMove = m;
42  }
43  }
44  }
45  if (bestMove == -1) {                                                                    5
46  return false; // stuck: the heuristic failed from this start
47  }
48  x += DX[bestMove];
```

(code continued)

```
    y += DY[bestMove];
    board[y][x] = step;                                                    4
    }
    return true;
    }

    public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    n = sc.nextInt();
    int sx = sc.nextInt();
    int sy = sc.nextInt();
    board = new int[n][n];
    if (!tour(sx, sy)) {
    System.out.println("Warnsdorff's rule got stuck");
    return;
    }
    for (int y = 0; y < n; y++) {
    StringBuilder sb = new StringBuilder();
    for (int x = 0; x < n; x++) {
    sb.append(String.format("%3d", board[y][x]));
    }
    System.out.println(sb);
    }
    }
   }
```

Greedy choice by minimum degree. O(N²·8·8) ≈ O(N²) steps.

Sample input

```
8 0 0
```

Line-by-line walkthrough

#	Where	What happens
1	line 4	The eight knight moves; the order also decides which move wins a tie.
2	line 15	Counts unvisited squares reachable from (x, y) — the number of "exits".
3	line 39	Strictly smaller degree wins, so ties go to the first move in the list (deterministic).
4	line 50	Number the square with its move number.
5	line 45	No unvisited neighbour: the heuristic got stuck before covering the board.

KnightBacktracking.java — Backtracking knight's tour (simple solution)

```
import java.util.Scanner;

public class KnightBacktracking {
 static final int[] DX = {2, 1, -1, -2, -2, -1, 1, 2};
 static final int[] DY = {1, 2, 2, 1, -1, -2, -2, -1};
 static int n;
 static int[][] board;
 static long calls = 0;

 // Plain backtracking: try every move in a fixed order, undo when stuck. Exponential —
     only fine for tiny boards.
 static boolean solve(int x, int y, int step) {
 calls++;
 board[y][x] = step;                                                    (1)
 if (step == n * n) {                                                   (2)
 return true;
 }
 for (int m = 0; m < 8; m++) {
 int nx = x + DX[m], ny = y + DY[m];
 if (nx >= 0 && ny >= 0 && nx < n && ny < n && board[ny][nx] == 0) {
 if (solve(nx, ny, step + 1)) {
 return true;
 }
 }
 }
 board[y][x] = 0; // backtrack                                          (3)
 return false;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 board = new int[n][n];
 boolean ok = solve(0, 0, 1);
 System.out.println(ok ? "tour found after " + calls + " calls" : "no tour");
 if (ok) {
 for (int y = 0; y < n; y++) {
 StringBuilder sb = new StringBuilder();
 for (int x = 0; x < n; x++) sb.append(String.format("%3d", board[y][x]));
 System.out.println(sb);
 }
 }
 }
}
```

Tries moves in a fixed order and undoes them when stuck; counts the recursive calls.

Sample input

```
5
```

Line-by-line walkthrough

#	Where	What happens
1	line 13	Choose: put the move number on the square.
2	line 14	Every square visited: success.
3	line 25	Un-choose and return false.

> **NOTE**
> The slide's program searches for a **closed** tour (the last square is a knight move from the first) with a random starting move index, repeating `while (!findClosedTour())`. That randomness is what makes it eventually succeed; the lesson program is deterministic and finds an open tour.

Dry run: watch it execute

Warnsdorff on a 5×5 board: each step shows the chosen square.

Algorithm: Warnsdorff's knight's tour. **Input:** n = `5`, x = `0`, y = `0`.

Pseudocode

```
board[start] = 1
for step = 2..n*n:
 among unvisited knight moves choose
 the one with the smallest degree
 (first move wins ties)
 move there; board = step
```

The full trace has 25 steps; 14 evenly spaced ones are shown. Run the rest in the website's Dry-run lab.

Step 1 of 25. Start on (0, 0).

x = 0 · y = 0 · step = 1

1	·	·	·	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·
·	·	·	·	·

Step 3 of 25. Move 3: go to (0, 4), the square with the fewest onward moves (1).

x = 0 · y = 4 · step = 3 · degree chosen = 1

1	·	·	·	·
·	·	·	·	·
·	2	·	·	·
·	·	·	·	·
3	·	·	·	·

Step 5 of 25. Move 5: go to (4, 4), the square with the fewest onward moves (1).

x = 4 · y = 4 · step = 5 · degree chosen = 1

1	·	·	·	·
·	·	·	·	·
·	2	·	·	·
·	·	4	·	·
3	·	·	·	5

Step 7 of 25. Move 7: go to (4, 0), the square with the fewest onward moves (1).

x = 4 · y = 0 · step = 7 · degree chosen = 1

1	·	·	·	7
·	·	·	·	·
·	2	·	6	·
·	·	4	·	·
3	·	·	·	5

Step 8 of 25. Move 8: go to (2, 1), the square with the fewest onward moves (4).

x = 2 · y = 1 · step = 8 · degree chosen = 4

1	·	·	·	7
·	·	8	·	·
·	2	·	6	·
·	·	4	·	·
3	·	·	·	5

Step 10 of 25. Move 10: go to (4, 1), the square with the fewest onward moves (2).

x = 4 · y = 1 · step = 10 · degree chosen = 2

1	·	·	·	7
·	·	8	·	10
·	2	·	6	·
·	·	4	9	·
3	·	·	·	5

Step 12 of 25. Move 12: go to (0, 1), the square with the fewest onward moves (2).

x = 0 · y = 1 · step = 12 · degree chosen = 2

1	·	11	·	7
12	·	8	·	10
·	2	·	6	·
·	·	4	9	·
3	·	·	·	5

Step 14 of 25. Move 14: go to (3, 4), the square with the fewest onward moves (2).

x = 3 · y = 4 · step = 14 · degree chosen = 2

1	·	11	·	7
12	·	8	·	10
·	2	·	6	·
·	13	4	9	·
3	·	·	14	5

Step 16 of 25. Move 16: go to (3, 0), the square with the fewest onward moves (2).

x = 3 · y = 0 · step = 16 · degree chosen = 2

1	·	11	16	7
12	·	8	·	10
·	2	·	6	15
·	13	4	9	·
3	·	·	14	5

Step 18 of 25. Move 18: go to (0, 3), the square with the fewest onward moves (2).

x = 0 · y = 3 · step = 18 · degree chosen = 2

1	·	11	16	7
12	17	8	·	10
·	2	·	6	15
18	13	4	9	·
3	·	·	14	5

Step 19 of 25. Move 19: go to (2, 4), the square with the fewest onward moves (1).

x = 2 · y = 4 · step = 19 · degree chosen = 1

1	·	11	16	7
12	17	8	·	10
·	2	·	6	15
18	13	4	9	·
3	·	19	14	5

Step 21 of 25. Move 21: go to (3, 1), the square with the fewest onward moves (1).

x = 3 · y = 1 · step = 21 · degree chosen = 1

1	·	11	16	7
12	17	8	21	10
·	2	·	6	15
18	13	4	9	20
3	·	19	14	5

Step 23 of 25. Move 23: go to (2, 2), the square with the fewest onward moves (1).

x = 2 · y = 2 · step = 23 · degree chosen = 1

1	22	11	16	7
12	17	8	21	10
·	2	23	6	15
18	13	4	9	20
3	·	19	14	5

Step 25 of 25. Move 25: go to (0, 2), the square with the fewest onward moves (0).

x = 0 · y = 2 · step = 25 · degree chosen = 0

1	22	11	16	7
12	17	8	21	10
25	2	23	6	15
18	13	4	9	20
3	24	19	14	5

> **RESULT**
> tour completed: all 25 squares visited

Complexity

Warnsdorff: N^2 steps; each step looks at ≤ 8 neighbours and computes a degree (≤ 8 more) → **$O(N^2)$** with a constant of about 64. Space $O(N^2)$ for the board.

Backtracking: worst case $O(8^{(N^2)})$; the 5 × 5 board from corner (0,0) already needs thousands of calls, 8 × 8 is infeasible.

Common mistakes

Counting the current square in the degree

Problem

```
degree includes squares already visited
```

Why: The rule counts only UNVISITED onward squares, as the slide states: "we do not count moves that revisit any square already visited".

Fix: use isEmpty(x, y) in the degree loop

Using the heuristic as a proof

Problem

```
// "Warnsdorff always finds a tour"
```

Why: It is a heuristic: it can get stuck on some boards/starts (there are known failures for large boards). Detect the failure and retry with another tie-break.

Fix: check the return value and handle failure

x and y swapped when printing

Problem

```
a[x * N + y] in one place and a[y * N + x] in another
```

Why: Mixed row/column indexing produces a board that looks random and fails verification.

Fix: Use one convention: board[y][x] (row = y).

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Verify a tour — *Beginner*

Read n and an n × n board of move numbers 1..n^2. Print YES if every number appears once and consecutive numbers are a knight's move apart, else NO.

Example input	Expected output
5 ⏎ 1 6 15 10 21 ⏎ 14 9 20 5 16 ⏎ 19 2 7 22 11 ⏎ 8 13 24 17 4 ⏎ 25 18 3 12 23	YES

Hint 1: Store the position of each number. **Hint 2:** A knight move has |dx|, |dy| = (1, 2) or (2, 1).

Exercise 2 · Degree of a square — *Intermediate*

Read n, x, y (0-based) and print how many squares an unrestricted knight on (x, y) can move to on an n × n board.

Example input	Expected output
8 0 0	2

Hint 1: Test all eight moves against the board limits.

Exercise 3 · Reachable squares — *Challenge*

Read n and k. A knight starts at (0, 0) on an n × n board. Print how many squares it can occupy after EXACTLY k moves (it may revisit squares).

Example input	Expected output
8 2	10

Hint 1: Keep a boolean grid of "possible positions" and advance it k times.

Quick check

Answers are in the answer key at the end of this part.

Q1. How many moves can a knight in the middle of a large board make?

- **A.** 4
- **B.** 6
- **C.** 8
- **D.** 12

Q2. Warnsdorff's rule chooses the next square with...

- **A.** the most onward moves
- **B.** the fewest onward moves
- **C.** the smallest row index
- **D.** a random choice

Q3. Is Warnsdorff's rule guaranteed to find a tour?

- **A.** Yes, always
- **B.** No, it is a heuristic
- **C.** Only on 8×8
- **D.** Only from corners

Q4. The heuristic's running time on an N × N board is...

- **A.** O(N)
- **B.** $O(N^2)$
- **C.** O(N!)
- **D.** O(2^N)

Interview / exam questions

- Why is a knight's tour a Hamiltonian path problem?
- What is the "fail-first" principle?
- How would you detect and recover when the heuristic gets stuck?
- Open vs closed tour?

Watch & read

Video: CSES Knight's Tour | Backtracking + Warnsdorff's Rule Explained | Full Intuition + Code in C++ — Yash Poonia · 16:32. *Knight's tour and Warnsdorff's rule (C++, C — the idea transfers to Java)*

Video: Knight's Tour - Numberphile — Numberphile · 4:51. *Knight's tour*

REVISION CARD (30–60 SECONDS)

- Knight: 8 moves (±1,±2),(±2,±1).
- Warnsdorff: go to the unvisited neighbour with the fewest unvisited neighbours.
- Heuristic — can fail; tie-break order matters.
- Backtracking exponential; heuristic O(N²).

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• This deck is named `downloadCourseMaterialFacultyPdf.pptx` and has no topic in its file name; its content is Warnsdorff's algorithm for the knight's tour. • The slide's class is not complete Java (e.g. `int y` without a semicolon, and the method `findClosedTour` is declared without a `return` on all paths in the screenshot). The lesson program is a clean, tested rewrite. • The slide uses `ThreadLocalRandom` to pick the first move to examine and loops until a CLOSED tour is found; its sample output is a closed tour starting at square (3, 2).

ASSUMPTIONS MADE

• The deterministic tie-break (first move in the DX/DY list) is an educational simplification of the slide's randomised version.

5.33 Selection Sort

Archive Algorithm Mastery · Beginner · Archive algorithm knowledge · about 35 min · ★ on the minimum-viable path

Repeatedly select the smallest remaining element and swap it into place — simple, always O(n²), at most n−1 swaps.

YOU WILL BE ABLE TO

- Trace selection sort on an array and show the sorted/unsorted split after every pass.
- Write the nested loops with a `minIndex` variable and a single swap per pass.
- State its time (O(n²) always), space (O(1)), stability (not stable) and swap count.
- Explain when its few swaps matter.

Before this lesson: One-Dimensional Arrays · Loops: for, while, do-while, break and continue · Methods: Parameters, Return Values, Scope and Overloading

The idea

Imagine sorting a hand of cards. Look through the whole hand, pick the **smallest** card and put it at the left end. Then look through the remaining cards, pick the smallest and place it second. Repeat. After each pass the left part of the array is sorted **and final**.

```
64 25 12 22 11 pass 1: minimum is 11 → swap with 64
11 25 12 22 64 pass 2: minimum of [25 12 22 64] is 12 → swap with 25
11 12 25 22 64 pass 3: minimum of [25 22 64] is 22 → swap with 25
11 12 22 25 64 pass 4: minimum of [25 64] is 25 → already in place
11 12 22 25 64 sorted
```

WHY THIS WORKS

After pass i the first i elements are the i smallest values in order — they can never need to move again. Each pass shrinks the unsorted part by one, so n−1 passes sort the array. The swap puts the minimum in its final position in a single move, which is why selection sort performs at most **n−1 swaps** — the fewest of the simple sorts.

Syntax

```
for (int i = 0; i < n - 1; i++) { // i = first index of the unsorted part
 int min = i;
 for (int j = i + 1; j < n; j++) // scan the rest
 if (a[j] < a[min]) min = j;
 if (min != i) { int t = a[i]; a[i] = a[min]; a[min] = t; } // at most n - 1 swaps overall
}
```

A tiny example

```
int minIndex = i;
for (int j = i + 1; j < n; j++)
 if (a[j] < a[minIndex]) minIndex = j;
int t = a[minIndex]; a[minIndex] = a[i]; a[i] = t;
```

Guided example

SelectionSort.java — Selection sort (simple solution)

```
1   import java.util.Arrays;
2   import java.util.Scanner;
3
4   public class SelectionSort {
5    // Repeatedly find the smallest value in the unsorted part and swap it to the front of
         that part.
6    static void sort(int[] a) {
7    int n = a.length;
8    for (int i = 0; i < n - 1; i++) {                                                    (1)
9    int minIndex = i; // position of the smallest value seen so far                      (2)
10   for (int j = i + 1; j < n; j++) {
11   if (a[j] < a[minIndex]) {                                                            (3)
12   minIndex = j;
13   }
14   }
15   int temp = a[minIndex]; // one swap per pass                                         (4)
16   a[minIndex] = a[i];
17   a[i] = temp;
18   System.out.println("after pass " + (i + 1) + ": " + Arrays.toString(a));
19   }
20   }
21
22   public static void main(String[] args) {
23   Scanner sc = new Scanner(System.in);
24   int n = sc.nextInt();
25   int[] a = new int[n];
26   for (int i = 0; i < n; i++) a[i] = sc.nextInt();
27   sort(a);
28   System.out.println("sorted: " + Arrays.toString(a));
29   }
30  }
```

In-place, O(n²) comparisons, at most n−1 swaps.

Sample input

```
5
64 25 12 22 11
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	Pass i fixes position i. The last element is automatically in place after n−1 passes.
2	line 9	Assume the first unsorted element is the smallest…
3	line 11	…and look for a smaller one in the rest. Strict `<` keeps the first of equal minima.
4	line 15	Exactly one swap per pass puts the minimum at index i.

> **NOTE**
> On the slide the intermediate arrays are drawn as "[11 | 64 25 12 22]" — i.e. the minimum moved to the front and the other elements shifted right. Real selection sort **swaps**, so the rest of the array is `[25 12 22 64]` (64 goes to where 11 was). Both end sorted, but the program prints what the code really does.

Dry run: watch it execute

The sorted prefix grows by one element per pass.

Algorithm: Selection sort. **Input:** arr = `64,25,12,22,11`.

Pseudocode

```
for i in 0..n-2:
 minIndex = i
 for j in i+1..n-1:
 if a[j] < a[minIndex]: minIndex = j
 swap(a[i], a[minIndex])
```

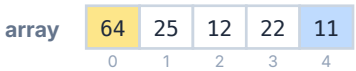
Step 1 of 9. Unsorted array.

i = —



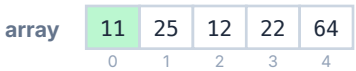
Step 2 of 9. Pass 1: the minimum of a[0..4] is 11 at index 4.

i = 0 · minIndex = 4 · comparisons = 4



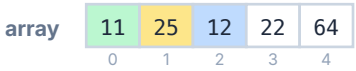
Step 3 of 9. Swap a[0] and a[4]. The first 1 elements are final.

i = 0 · swaps = 1



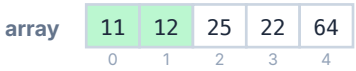
Step 4 of 9. Pass 2: the minimum of a[1..4] is 12 at index 2.

i = 1 · minIndex = 2 · comparisons = 7



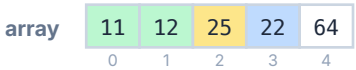
Step 5 of 9. Swap a[1] and a[2]. The first 2 elements are final.

i = 1 · swaps = 2



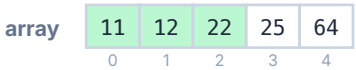
Step 6 of 9. Pass 3: the minimum of a[2..4] is 22 at index 3.

i = 2 · minIndex = 3 · comparisons = 9



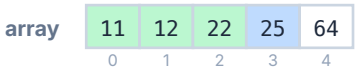
Step 7 of 9. Swap a[2] and a[3]. The first 3 elements are final.

i = 2 · swaps = 3



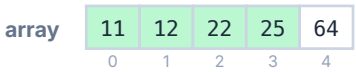
Step 8 of 9. Pass 4: the minimum of a[3..4] is 25 at index 3.

i = 3 · minIndex = 3 · comparisons = 10



Step 9 of 9. Already in place. The first 4 elements are final.

i = 3 · swaps = 3



> **RESULT**
> [11, 12, 22, 25, 64] — 10 comparisons, 3 swaps

Complexity

Pass i makes n−1−i comparisons whatever the data looks like → (n−1) + (n−2) + … + 1 = n(n−1)/2 comparisons → **O(n²) in best, average and worst case**. Swaps ≤ n−1. Space O(1) (in place).

n	comparisons	swaps (max)
10	45	9
1 000	499 500	999
100 000	≈ 5 × 10⁹	99 999

Not stable: swapping a far-away minimum over equal elements can reorder them (`[3a, 3b, 1]` → `[1, 3b, 3a]`). Useful when writes are expensive (flash memory) because swaps are minimal.

Common mistakes

Swapping inside the inner loop

Problem

```
if (a[j] < a[i]) swap(a, i, j); // inside the j-loop
```

Why: This "exchange sort" does up to O(n²) swaps and is a different (worse) algorithm, though it also sorts.
Fix: Remember minIndex and swap once after the loop.

Off-by-one in the outer loop

Problem

```
for (int i = 0; i < n; i++) { // inner loop j = i + 1 → fine
```

Why: It works, but the last pass does nothing. Using i ≤ n−1 elsewhere with a[j+1] causes an out-of-bounds read.
Fix: for (int i = 0; i < n - 1; i++)

Using <= to compare

Problem

```
if (a[j] <= a[minIndex]) minIndex = j;
```

Why: Picks the LAST of several equal minima, increasing the number of reorderings of equal keys.
Fix: if (a[j] < a[minIndex])

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Sort and count swaps — *Beginner*

Read n and an array. Sort it with selection sort and print the sorted array on one line, then the number of swaps that actually moved something (skip a swap when minIndex = i).

Example input	Expected output
5 ⏎ 64 25 12 22 11	11 12 22 25 64 ⏎ 3

Hint 1: Count a swap only when minIndex != i.

Exercise 2 · Largest first — *Intermediate*

Sort an array in DESCENDING order using the same idea (select the maximum). Print on one line.

Example input	Expected output
5 ⏎ 3 1 4 1 5	5 4 3 1 1

Hint 1: Only the comparison changes.

Exercise 3 · K smallest, cheaply — *Challenge*

Read n, k and an array. Print the k smallest values in increasing order using only k passes of selection (O(n·k)).

- 1 ≤ k ≤ n ≤ 5000

Example input	Expected output
6 3 ⏎ 7 10 4 3 20 15	3 4 7

Hint 1: You do not have to finish the sort: stop after k passes.

Quick check

Answers are in the answer key at the end of this part.

Q1. What is the array after the 2nd pass?

```
int[] a = {29, 10, 14, 37, 13};
// selection sort
```

Your answer: _____

Q2. How many comparisons does selection sort make on 6 elements, whatever their order?

- **A.** 5
- **B.** 10
- **C.** 15
- **D.** 36

Q3. Which property is TRUE of selection sort?

- **A.** Stable
- **B.** At most n−1 swaps
- **C.** O(n) best case
- **D.** Uses extra O(n) memory

Q4. Best-case time complexity of selection sort?

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^2)
- **D.** O(1)

Interview / exam questions

- Why is selection sort not stable? Give an example.
- When would you prefer selection sort?
- How many swaps does it make in the worst case?
- Compare with insertion sort.

Watch & read

Video: 7.5 Selection Sort in Data Structure | Selection Sort Algorithm with C Program — Jenny's Lectures CS IT · 17:49. *How selection sort works*

Video: L-3.8: Selection Sort | Time Complexity(Best, Avg & Worst) Analysis | Stable or Not | Inplace or Not — Gate Smashers · 11:37. *Selection sort in Java*

REVISION CARD (30–60 SECONDS)

- Find minimum of the unsorted part, swap to the front, repeat.
- O(n^2) always; ≤ n−1 swaps; in place; not stable.
- After pass i: first i+1 elements final.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

• The slide shows each iteration as "sorted [11], unsorted [64, 25, 12, 22]", which looks like the minimum was inserted at the front and the rest shifted. The slide's code swaps instead, giving [25, 12, 22, 64] after pass 1. The program output matches the code. • The slide describes Quick sort and Selection sort in the same deck; they are split into two lessons here.

5.34 Quick Sort

Archive Algorithm Mastery · Intermediate · Archive algorithm knowledge · about 55 min · ★ on the minimum-viable path

Pick a pivot, partition the array around it, and recursively sort the two parts: divide and conquer in place.

YOU WILL BE ABLE TO

- Explain divide and conquer and why one partition places the pivot in its final position.
- Implement Lomuto partitioning and the recursive driver.
- Derive best/average O(n log n) and worst-case O(n²) and say what triggers the worst case.
- Use quickselect to find the k-th smallest element in expected O(n).

Before this lesson: Recursion: Base Case, Recursive Case and the Call Stack · Selection Sort · Algorithm Design Paradigms

The idea

Quick sort is divide and conquer. Choose one element as the **pivot**. Rearrange the array so that everything ≤ pivot is on its left and everything > pivot is on its right — this is called **partitioning**. The pivot is now in its final sorted position! Then sort the left part and the right part the same way.

```
[8 3 1 7 0 10 2] pivot = last element = 2
 partition → [1 0 | 2 | 7 3 10 8]
 ≤2 pivot >2
 sort [1 0] → [0 1] sort [7 3 10 8] → [3 7 8 10]
 result [0 1 2 3 7 8 10]
```

WHY THIS WORKS

Partitioning does O(n) work and fixes one element permanently. If the pivot splits the array roughly in half, the recursion depth is log n and every level does O(n) total work: O(n log n). If the pivot is always the smallest or largest element (e.g. a sorted array with a last-element pivot) one part is empty each time, the depth is n and the total work is O(n²). Randomising the pivot or choosing the median of three makes the bad case very unlikely.

Lomuto partition in words

Keep an index `i` marking the end of the "≤ pivot" region. Scan j from left to right; whenever `a[j] ≤ pivot` grow the region (`i++`) and swap `a[j]` into it. Finally swap the pivot into position `i + 1`.

Syntax

```
static void quickSort(int[] a, int lo, int hi) {
 if (lo >= hi) return; // 0 or 1 element
 int p = partition(a, lo, hi); // pivot lands on its final index p
 quickSort(a, lo, p - 1);
 quickSort(a, p + 1, hi);
}
static int partition(int[] a, int lo, int hi) { // Lomuto: pivot = last element
 int pivot = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++)
 if (a[j] <= pivot) { i++; int t = a[i]; a[i] = a[j]; a[j] = t; }
 int t = a[i + 1]; a[i + 1] = a[hi]; a[hi] = t;
 return i + 1;
}
// randomised pivot avoids the sorted-input worst case:
int r = java.util.concurrent.ThreadLocalRandom.current().nextInt(lo, hi + 1); // then swap a[r]
    and a[hi]
```

A tiny example

```
int pivot = a[high];
int i = low - 1;
for (int j = low; j < high; j++)
 if (a[j] <= pivot) { i++; swap(a, i, j); }
swap(a, i + 1, high);
```

Guided example

QuickSort.java — Quick sort with Lomuto partition (optimised solution)

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class QuickSort {
5   // Lomuto partition: the pivot is the LAST element. Afterwards everything <= pivot is
        on its left,
6   // everything > pivot on its right, and the pivot stands at its final sorted position.
7   static int partition(int[] a, int low, int high) {
8   int pivot = a[high];                                                              (1)
9   int i = low - 1; // end of the "<= pivot" region                                  (2)
10  for (int j = low; j < high; j++) {
11  if (a[j] <= pivot) {                                                              (3)
12  i++;
13  int t = a[i];
14  a[i] = a[j];
15  a[j] = t;
16  }
17  }
18  int t = a[i + 1]; // drop the pivot between the two regions                       (4)
19  a[i + 1] = a[high];
20  a[high] = t;
21  return i + 1;
22  }
23
24  static void quickSort(int[] a, int low, int high) {
25  if (low < high) { // 0 or 1 element: already sorted (base case)                  (5)
26  int p = partition(a, low, high);
27  System.out.println("pivot " + a[p] + " placed at index " + p + ": " + Arrays.toString(a)
        );
28  quickSort(a, low, p - 1);                                                         (6)
29  quickSort(a, p + 1, high);                                                        (7)
30  }
31  }
32
33  public static void main(String[] args) {
34  Scanner sc = new Scanner(System.in);
35  int n = sc.nextInt();
36  int[] a = new int[n];
37  for (int i = 0; i < n; i++) a[i] = sc.nextInt();
38  quickSort(a, 0, n - 1);
39  System.out.println("sorted: " + Arrays.toString(a));
40  }
41 }
```

Expected O(n log n), worst O(n²), O(log n) average recursion depth.

Sample input

```
7
8 3 1 7 0 10 2
```

Line-by-line walkthrough

#	Where	What happens
1	line 8	The last element is the pivot (simple, but bad for already-sorted input).
2	line 9	`i` is the last index of the "≤ pivot" region; it starts empty.
3	line 11	This element belongs in the left region: extend it and swap it in.
4	line 18	Put the pivot between the two regions; it is now final.
5	line 25	Base case: 0 or 1 elements are already sorted.
6	line 28	Sort everything left of the pivot…
7	line 29	…and everything right of it. The pivot itself needs no more work.

SortCheck.java — Randomised verification (demonstration)

```
import java.util.Arrays;
import java.util.Random;

public class SortCheck {
 static void selection(int[] a) {
 for (int i = 0; i < a.length - 1; i++) {
 int m = i;
 for (int j = i + 1; j < a.length; j++) if (a[j] < a[m]) m = j;
 int t = a[m]; a[m] = a[i]; a[i] = t;
 }
 }

 static int partition(int[] a, int lo, int hi) {
 int p = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++) if (a[j] <= p) { i++; int t = a[i]; a[i] = a[j]; a[j] = t;
     }
 int t = a[i + 1]; a[i + 1] = a[hi]; a[hi] = t;
 return i + 1;
 }

 static void quick(int[] a, int lo, int hi) {
 if (lo < hi) {
 int p = partition(a, lo, hi);
 quick(a, lo, p - 1);
 quick(a, p + 1, hi);
 }
 }

 // 5000 random arrays (with many duplicates) sorted by both algorithms and compared
     with Arrays.sort.
 public static void main(String[] args) {
 Random rnd = new Random(5);
 for (int t = 0; t < 5000; t++) {
 int n = rnd.nextInt(30);
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = rnd.nextInt(10) - 5;
 int[] expected = a.clone();
 Arrays.sort(expected);
 int[] s = a.clone();
 selection(s);
 int[] q = a.clone();
 quick(q, 0, n - 1);
 if (!Arrays.equals(s, expected) || !Arrays.equals(q, expected)) {
 System.out.println("MISMATCH on " + Arrays.toString(a));
 return;
 }
 }
 System.out.println("selection sort and quick sort agree with Arrays.sort on 5000 random
     arrays");
 }
}
```

Sorts 5 000 random arrays with many duplicates and negative numbers and compares with Arrays.sort.

Line-by-line walkthrough

#	Where	What happens
1	line 34	Small value range = many duplicates, which is where partition bugs (`<` vs `<=`) show up.

> **NOTE**
> For the slide's array `[8, 3, 1, 7, 0, 10, 2]` the slide says one partition gives `[1, 0, 2, 7, 8, 10, 3]`. The slide's own Lomuto code produces `[1, 0, 2, 7, 3, 10, 8]` — also a valid partition (everything left of 2 is ≤ 2, everything right is > 2) with the pivot at index 2.

Dry run: watch it execute

Pivot placements and the shrinking sub-arrays.

Algorithm: Quick sort (Lomuto). **Input:** arr = `8,3,1,7,0,10,2`.

Pseudocode

```
quickSort(low, high):
 if low < high:
 pivot = a[high]; i = low - 1
 for j in low..high-1:
 if a[j] <= pivot: i++; swap(a[i], a[j])
 swap(a[i+1], a[high]) → p = i+1
 quickSort(low, p-1); quickSort(p+1, high)
```

Step 1 of 5. Unsorted array.

low = 0 · high = 6

	0	1	2	3	4	5	6
array	8	3	1	7	0	10	2

Step 2 of 5. Pivot 2 (last of a[0..6]): 2 element(s) ≤ pivot go left. The pivot lands at index 2 — its final position.

low = 0 · high = 6 · pivot = 2 · p = 2

	0	1	2	3	4	5	6
array	1	0	2	7	3	10	8

Step 3 of 5. Pivot 0 (last of a[0..1]): 0 element(s) ≤ pivot go left. The pivot lands at index 0 — its final position.

low = 0 · high = 1 · pivot = 0 · p = 0

	0	1	2	3	4	5	6
array	0	1	2	7	3	10	8

Step 4 of 5. Pivot 8 (last of a[3..6]): 2 element(s) ≤ pivot go left. The pivot lands at index 5 — its final position.

low = 3 · high = 6 · pivot = 8 · p = 5

	0	1	2	3	4	5	6
array	0	1	2	7	3	8	10

Step 5 of 5. Pivot 3 (last of a[3..4]): 0 element(s) ≤ pivot go left. The pivot lands at index 3 — its final position.

low = 3 · high = 4 · pivot = 3 · p = 3

	0	1	2	3	4	5	6
array	0	1	2	3	7	8	10

> **RESULT**
> [0, 1, 2, 3, 7, 8, 10]

Complexity

Let T(n) be the time to sort n elements. A partition costs n. A balanced split gives T(n) = 2T(n/2) + n = **O(n log n)**; an unbalanced split T(n) = T(n−1) + n = **O(n²)**.

case	time	when
best	O(n log n)	pivot near the median every time
average	O(n log n)	random data / random pivot
worst	O(n²)	sorted/reverse-sorted input with first/last pivot, or all equal elements with ≤ partitioning

Space: **O(log n)** average for the recursion stack (O(n) worst case). In place, not stable. Recurse into the smaller part first (and loop on the larger) to guarantee O(log n) stack.

Common mistakes

Quick sort on sorted data with last-element pivot

Problem

```
quickSort(sortedArray) // n = 100 000
```

Why: Every partition peels one element: depth 100 000 → StackOverflowError and O(n²) time.

Fix: Random pivot or median-of-three: swap a random element into a[high] first.

Infinite recursion from the pivot index

Problem

```
quickSort(a, low, p); // should be p - 1
```

Why: The pivot is already in place; including it can leave the sub-problem the same size forever.

Fix: quickSort(a, low, p - 1); quickSort(a, p + 1, high);

Wrong swap temp

Problem

```
a[i] = a[j]; a[j] = a[i];
```

Why: The first assignment destroys the old a[i]; both end up equal.

Fix: int t = a[i]; a[i] = a[j]; a[j] = t;

Practice

Try each exercise yourself first. Reference solutions are in the Solutions chapter at the end of this part (so you are not tempted to peek).

Exercise 1 · Count the partitions — *Beginner*

Quick sort the array with the lesson's Lomuto code. Read n and the array; print the sorted array. (Implement quick sort yourself; the output is simply the sorted array.)

Example input	Expected output
7 ⏎ 8 3 1 7 0 10 2	0 1 2 3 7 8 10

Hint 1: partition returns the pivot's final index. **Hint 2:** Recurse on both sides.

Exercise 2 · Descending quick sort — *Intermediate*

Sort the array in descending order using quick sort.

Example input	Expected output
5 ⏎ 3 1 4 1 5	5 4 3 1 1

Hint 1: Flip the comparison in the partition: move elements ≥ pivot to the left region.

Exercise 3 · k-th smallest with quickselect — *Challenge*

Read n, k and an array. Print the k-th smallest element (1-based) in expected O(n) time without fully sorting.

Example input	Expected output
6 3 ⏎ 7 10 4 3 20 15	7

Hint 1: After a partition the pivot is at index p. If p == k−1 you are done; otherwise only recurse into ONE side.

Quick check

Answers are in the answer key at the end of this part.

Q1. After one partition, the pivot is...

- **A.** at the middle always
- **B.** in its final sorted position
- **C.** at index 0
- **D.** removed

Q2. Worst-case time of quick sort?

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n^2)
- **D.** O(2^n)

Q3. What does partition return for this call?

```
int[] a = {3, 8, 2, 5, 1, 4, 7, 6};
int p = partition(a, 0, 7); // pivot = a[7] = 6
System.out.println(p);
```

Your answer: _____

Q4. Which strategy is used by quick sort?

- **A.** Greedy
- **B.** Divide and conquer
- **C.** Dynamic programming
- **D.** Brute force

Q5. Is quick sort stable?

- **A.** Yes
- **B.** No

Interview / exam questions

- Why is quick sort usually faster than merge sort in practice?
- How do you avoid the O(n²) worst case?
- What is quickselect and its complexity?
- Hoare vs Lomuto partitioning?

Watch & read

Video: 2.8.1 QuickSort Algorithm — Abdul Bari · 13:43. *Partitioning and recursion*

Video: Quicksort Sort Algorithm in Java - Full Tutorial With Source — Coding with John · 24:58. *Quick sort implementation and pivot choice*

REVISION CARD (30–60 SECONDS)

- Pivot → partition (≤ left, > right) → recurse both sides.
- Lomuto: i = low−1; scan j; if a[j] ≤ pivot {i++; swap}; swap pivot to i+1.
- Average O(n log n), worst O(n²), space O(log n), in place, unstable.
- Quickselect: recurse on one side only.

WHERE THE SLIDES WERE WRONG OR UNCLEAR

- The slide's worked example gives the partition result `[1, 0, 2, 7, 8, 10, 3]`; the code produces `[1, 0, 2, 7, 3, 10, 8]`. Then the slide partitions `[7, 8, 10, 3]` with pivot 3 as `[7, 8, 3, 10]`, which is also not what Lomuto does (`[3, 8, 10, 7]`). The final sorted result on the slide is correct.

Solutions to the exercises

5.1 Euclid's Algorithm (GCD)

Exercise 1 · Reduce a fraction

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static int gcd(int a, int b) {
 while (b != 0) {
 int r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int numerator = sc.nextInt();
 int denominator = sc.nextInt();
 int g = gcd(numerator, denominator);
 System.out.println((numerator / g) + "/" + (denominator / g));
 }
}
```

Why it works: Reduce by the gcd: dividing numerator and denominator by their greatest common divisor leaves no common factor, so the fraction is in lowest terms.

Verified against 3 test cases: `12 18⏎` → `2/3`; `7 5⏎` → `7/5`; `100 25⏎` → `4/1`

Exercise 2 · LCM of a list

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static long gcd(long a, long b) {
 while (b != 0) {
 long r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long lcm = 1;
 for (int i = 0; i < n; i++) {
 long x = sc.nextLong();
 // divide first, then multiply: keeps the intermediate value small
 lcm = lcm / gcd(lcm, x) * x;
 }
 System.out.println(lcm);
 }
}
```

Why it works: The LCM of a list is built one number at a time with lcm(x, y) = x / gcd(x, y) · y. Dividing first keeps intermediate values as small as possible.

Verified against 3 test cases: `3⏎4 6 8⏎` → `24`; `2⏎7 13⏎` → `91`; `4⏎1 2 3 4⏎` → `12`

Exercise 3 · Modular inverse

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // returns {g, x, y} with a*x + b*y = g = gcd(a, b)
 static long[] extendedGcd(long a, long b) {
 if (b == 0) {
 return new long[] {a, 1, 0};
 }
 long[] next = extendedGcd(b, a % b);
 long g = next[0];
 long x = next[2];
 long y = next[1] - (a / b) * next[2];
 return new long[] {g, x, y};
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long a = sc.nextLong();
 long m = sc.nextLong();
 long[] r = extendedGcd(a, m);
 if (r[0] != 1) {
 System.out.println(-1); // a and m share a factor: no inverse exists
 } else {
 long inverse = ((r[1] % m) + m) % m; // shift a possibly-negative x into [0, m-1]
 System.out.println(inverse);
 }
 }
}
```

Why it works: Extended Euclid carries two extra numbers back up the recursion so that a·x + b·y = gcd(a, b) holds at every level. When the gcd is 1, a·x ≡ 1 (mod m), so x is the inverse.

Verified against 3 test cases: `3 11⏎` → `4`; `10 17⏎` → `12`; `6 9⏎` → `-1`

5.2 Euler's Phi (Totient) Function

Exercise 1 · List the coprime numbers

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 static int gcd(int a, int b) {
 while (b != 0) {
 int r = a % b;
 a = b;
 b = r;
 }
 return a;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int k = 1; k <= n; k++) {
 if (gcd(k, n) == 1) {
 sb.append(k).append(' ');
 }
 }
 System.out.println(sb.toString().trim());
 }
}
```

Why it works: For each k test gcd(k, n) == 1 and collect the passes.

Verified against 3 test cases: `24⏎` → `1 5 7 11 13 17 19 23`; `12⏎` → `1 5 7 11`; `7⏎` → `1 2 3 4 5 6`

Exercise 2 · Sum of totients

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] phi = new int[n + 1];
 for (int i = 0; i <= n; i++) {
 phi[i] = i;
 }
 for (int p = 2; p <= n; p++) {
 if (phi[p] == p) { // p is prime
 for (int m = p; m <= n; m += p) {
 phi[m] -= phi[m] / p;
 }
 }
 }
 long sum = 0;
 for (int i = 1; i <= n; i++) {
 sum += phi[i];
 }
 System.out.println(sum);
 }
}
```

Why it works: The sieve-style table computes all φ values in O(n log log n); the sum is a plain loop.

Verified against 3 test cases: `10⏎` → `32`; `1⏎` → `1`; `100⏎` → `3044`

Exercise 3 · Coprime pairs

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Number of ordered pairs (a, b) with 1 <= a, b <= n and gcd(a, b) = 1.
 // For each larger value b = 2..n there are phi(b) smaller partners a; double for order; add (1, 1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] phi = new int[n + 1];
 for (int i = 0; i <= n; i++) {
 phi[i] = i;
 }
 for (int p = 2; p <= n; p++) {
 if (phi[p] == p) {
 for (int m = p; m <= n; m += p) {
 phi[m] -= phi[m] / p;
 }
 }
 }
 long sum = 0;
 for (int i = 1; i <= n; i++) {
 sum += phi[i];
 }
 System.out.println(2 * sum - 1);
 }
}
```

Why it works: For each b ≥ 2 there are φ(b) values a < b that are coprime to it. Counting both orders gives 2·Σφ(b) for b = 2..n, plus the pair (1,1): 2·(Σ_{i=1..n} φ(i)) − 1.

Verified against 3 test cases: `3⏎` → `7`; `5⏎` → `19`; `1⏎` → `1`

5.3 Chinese Remainder Theorem

Exercise 1 · Two congruences

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int a = sc.nextInt();
 int r1 = sc.nextInt();
 int b = sc.nextInt();
 int r2 = sc.nextInt();
 for (int x = 0; x < a * b; x++) {
 if (x % a == r1 && x % b == r2) {
 System.out.println(x);
 return;
 }
 }
 }
}
```

Why it works: CRT guarantees exactly one solution in [0, a·b). A simple loop finds it.

Verified against 3 test cases: `3 2 5 3⏎` → `8`; `4 3 9 5⏎` → `23`; `7 0 11 0⏎` → `0`

Exercise 2 · CRT with the formula

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static long[] extGcd(long a, long b) {
 if (b == 0) {
 return new long[] {a, 1, 0};
 }
 long[] r = extGcd(b, a % b);
 return new long[] {r[0], r[2], r[1] - (a / b) * r[2]};
 }

 static long inverse(long a, long m) {
 long x = extGcd(((a % m) + m) % m, m)[1];
 return ((x % m) + m) % m;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] mod = new long[n];
 long[] rem = new long[n];
 long M = 1;
 for (int i = 0; i < n; i++) {
 mod[i] = sc.nextLong();
 rem[i] = sc.nextLong();
 M *= mod[i];
 }
 long x = 0;
 for (int i = 0; i < n; i++) {
 long Mi = M / mod[i];
 long term = rem[i] % M * Mi % M * inverse(Mi, mod[i]) % M;
 x = (x + term) % M;
 }
 System.out.println(x);
 }
}
```

Why it works: Each term r_i·M_i·inv(M_i) is ≡ r_i for modulus m_i and ≡ 0 for all others. Summing and reducing mod M gives the unique answer.

Verified against 3 test cases: `3⏎3 2⏎5 3⏎7 2⏎` → `23`; `2⏎5 4⏎11 7⏎` → `29`; `1⏎13 6⏎` → `6`

Exercise 3 · Non-coprime moduli

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Moduli need NOT be coprime. Walk through x = r1, r1 + m1, r1 + 2*m1, ... : only these satisfy
 // the first congruence. The remainders mod m2 repeat after m2 steps, so m2 tries are enough.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long r1 = sc.nextLong();
 long m1 = sc.nextLong();
 long r2 = sc.nextLong();
 long m2 = sc.nextLong();
 long x = r1 % m1;
 for (long k = 0; k < m2; k++) {
 if (x % m2 == r2 % m2) {
 System.out.println(x);
 return;
 }
 x += m1;
 }
 System.out.println(-1);
 }
}
```

Why it works: Walk through the candidates r1 + k·m1. Their residues mod m2 form a cycle of length ≤ m2, so after m2 candidates every possibility has been checked.

Verified against 3 test cases: `2 6 4 10⏎` → `14`; `2 6 1 4⏎` → `-1`; `0 5 0 5⏎` → `0`

5.4 Combinations (nCr)

Exercise 1 · nCr as a number

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 long result = 1;
 for (int i = 1; i <= r; i++) {
 result = result * (n - r + i) / i; // always exact: result is C(n-r+i, i)
 }
 System.out.println(result);
 }
}
```

Why it works: The loop keeps the exact value C(n−r+i, i) after step i, so each division is exact.

Verified against 3 test cases: `5 2⏎` → `10`; `30 15⏎` → `155117520`; `7 0⏎` → `1`

Exercise 2 · nCr modulo 1 000 000 007

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 final long MOD = 1_000_000_007L;
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 long[][] c = new long[n + 1][r + 1];
 for (int i = 0; i <= n; i++) {
 c[i][0] = 1;
 for (int j = 1; j <= Math.min(i, r); j++) {
 c[i][j] = (c[i - 1][j - 1] + (j <= i - 1 ? c[i - 1][j] : 0)) % MOD;
 }
 }
 System.out.println(c[n][r]);
 }
}
```

Why it works: Pascal's recurrence uses only additions, so applying `% MOD` at each step keeps every value small.

Verified against 3 test cases: `10 3⏎` → `120`; `1000 500⏎` → `159835829`; `5 5⏎` → `1`

Exercise 3 · List all combinations

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static StringBuilder out = new StringBuilder();

 // choose(start, chosen): extend the current selection with numbers >= start
 static void choose(int n, int r, int start, int[] picked, int count) {
 if (count == r) {
 for (int i = 0; i < r; i++) {
 out.append(i == 0 ? "" : " ").append(picked[i]);
 }
 out.append('\n');
 return;
 }
 for (int v = start; v <= n; v++) {
 picked[count] = v;
 choose(n, r, v + 1, picked, count + 1);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int r = sc.nextInt();
 choose(n, r, 1, new int[r], 0);
 System.out.print(out);
 }
}
```

Why it works: Recursive loop from `start` upwards: choose a value, recurse with start = value + 1. Because values only increase, each subset appears once and in lexicographic order.

Verified against 3 test cases: `4 2⏎` → `1 2⏎1 3⏎1 4⏎2 3⏎2 4⏎3 4`; `3 3⏎` → `1 2 3`; `3 1⏎` → `1⏎2⏎3`

5.5 Karatsuba's Multiplication

Exercise 1 · School-book multiplication

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // School-book multiplication of two digit strings: O(n * m) digit multiplications.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String x = sc.next();
 String y = sc.next();
 int[] result = new int[x.length() + y.length()];
 for (int i = x.length() - 1; i >= 0; i--) {
 for (int j = y.length() - 1; j >= 0; j--) {
 int product = (x.charAt(i) - '0') * (y.charAt(j) - '0');
 int pos = i + j + 1; // units position of this partial product
 int sum = product + result[pos];
 result[pos] = sum % 10;
 result[pos - 1] += sum / 10; // carry into the next position to the left
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int digit : result) {
 if (!(sb.length() == 0 && digit == 0)) {
 sb.append(digit);
 }
 }
 System.out.println(sb.length() == 0 ? "0" : sb);
 }
}
```

Why it works: The classic method: n·m single-digit products, each added into the right position with carry.

Verified against 3 test cases: `12345 6789⏎` → `83810205`; `0 98765⏎` → `0`; `99999999999 99999999999⏎` → `9999999999800000000001`

Exercise 2 · Count the base multiplications

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static int baseCases = 0;

 static long pow10(int k) {
 long p = 1;
 for (int i = 0; i < k; i++) p *= 10;
 return p;
 }

 static long multiply(long x, long y) {
 if (x < 10 || y < 10) {
 baseCases++;
 return x * y;
 }
 int n = Math.max(Long.toString(x).length(), Long.toString(y).length());
 int half = n / 2;
 long p = pow10(half);
 long a = x / p, b = x % p, c = y / p, d = y % p;
 long ac = multiply(a, c);
 long bd = multiply(b, d);
 long abcd = multiply(a + b, c + d);
 return ac * p * p + (abcd - ac - bd) * p + bd;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 multiply(sc.nextLong(), sc.nextLong());
 System.out.println(baseCases);
 }
}
```

Why it works: A static counter in the base case counts leaf calls of the recursion tree.

Verified against 3 test cases: `1234 5678⏎` → `11`; `5 7⏎` → `1`; `12 34⏎` → `3`

Exercise 3 · Big Karatsuba

Ex3.java

```java
import java.math.BigInteger;
import java.util.Scanner;

public class Ex3 {
 static BigInteger multiply(BigInteger x, BigInteger y) {
 int n = Math.max(x.bitLength(), y.bitLength());
 if (n <= 30) { // both factors < 2^30, so the product < 2^60 fits in a long
 return BigInteger.valueOf(x.longValue() * y.longValue());
 }
 int half = n / 2;
 BigInteger a = x.shiftRight(half);
 BigInteger b = x.subtract(a.shiftLeft(half));
 BigInteger c = y.shiftRight(half);
 BigInteger d = y.subtract(c.shiftLeft(half));
 BigInteger ac = multiply(a, c);
 BigInteger bd = multiply(b, d);
 BigInteger mid = multiply(a.add(b), c.add(d)).subtract(ac).subtract(bd);
 return ac.shiftLeft(2 * half).add(mid.shiftLeft(half)).add(bd);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 BigInteger x = new BigInteger(sc.next());
 BigInteger y = new BigInteger(sc.next());
 System.out.println(multiply(x, y));
 }
}
```

Why it works: Same three-product identity as the long version, but with BigInteger and bit shifts; the base case is a plain 30-bit multiplication.

Verified against 3 test cases: `123456789 987654321⏎` → `121932631112635269`; `10000000000000000000 10000000000000000000⏎` → `100000000000000000000000000000000000000000`; `0 5⏎` → `0`

5.6 Booth's Algorithm (signed binary multiplication)

Exercise 1 · n-bit two's complement

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Print x in n-bit two's complement (works for negative x too).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int n = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int bit = n - 1; bit >= 0; bit--) {
 sb.append((x >> bit) & 1); // >> on a negative int keeps the sign, so high bits come out as 1s
 }
 System.out.println(sb);
 }
}
```

Why it works: Java stores int in two's complement, so shifting and masking one bit at a time reads off the pattern directly.

Verified against 3 test cases: `-5 8⏎` → `11111011`; `7 4⏎` → `0111`; `-1 4⏎` → `1111`

Exercise 2 · Booth's operations

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // For an n-bit multiplier given as a bit string, list Booth's operation for each cycle
     (rightmost bit first).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String q = sc.next();
 int previous = 0; // Q-1 starts as 0
 StringBuilder sb = new StringBuilder();
 for (int i = q.length() - 1; i >= 0; i--) {
 int current = q.charAt(i) - '0';
 String op;
 if (current == 1 && previous == 0) {
 op = "-M";
 } else if (current == 0 && previous == 1) {
 op = "+M";
 } else {
 op = "0";
 }
 sb.append(sb.length() == 0 ? "" : " ").append(op);
 previous = current;
 }
 System.out.println(sb);
 }
}
```

Why it works: The current bit with the previously processed (lower) bit forms the pair Q_0Q_{-1}; the table of conditions gives the action.

Verified against 3 test cases: `0110⏎` → `0 -M 0 +M`; `0011⏎` → `-M 0 +M 0`; `1111⏎` → `-M 0 0 0`

Exercise 3 · Booth multiplication

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int q = sc.nextInt();
 int width = 16; // inputs fit in 16-bit signed numbers
 if (m == -(1 << (width - 1))) {
 width++;
 }
 long mask = (1L << width) - 1;
 long M = m & mask;
 long negM = (-m) & mask;
 long ac = 0;
 long qr = q & mask;
 int qm1 = 0;
 for (int i = 0; i < width; i++) {
 int q0 = (int) (qr & 1);
 if (q0 == 1 && qm1 == 0) {
 ac = (ac + negM) & mask;
 } else if (q0 == 0 && qm1 == 1) {
 ac = (ac + M) & mask;
 }
 qm1 = (int) (qr & 1);
 qr = ((qr >> 1) | ((ac & 1) << (width - 1))) & mask;
 ac = (ac >> 1) | (ac & (1L << (width - 1)));
 }
 long raw = (ac << width) | qr;
 long sign = 1L << (2 * width - 1);
 long product = (raw & sign) != 0 ? raw - (1L << (2 * width)) : raw; // read the 2n-bit pattern
     as signed
 System.out.println(product);
 }
}
```

Why it works: Direct simulation of the hardware loop; the final 2n-bit pattern is converted to a signed number.

Verified against 3 test cases: `-3 4⏎` → `-12`; `7 3⏎` → `21`; `-8 -8⏎` → `64`

5.7 Simple Sieve of Eratosthenes

Exercise 1 · Count primes

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 boolean[] composite = new boolean[n + 1];
 int count = 0;
 for (int i = 2; i <= n; i++) {
 if (!composite[i]) {
 count++;
 for (long m = (long) i * i; m <= n; m += i) {
 composite[(int) m] = true;
 }
 }
 }
 System.out.println(count);
 }
}
```

Why it works: Standard sieve; every i not crossed out when reached is prime and is counted, then its multiples are crossed out.

Verified against 3 test cases: `100⏎` → `25`; `1⏎` → `0`; `1000000⏎` → `78498`

Exercise 2 · The k-th prime

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int k = sc.nextInt();
 int limit = 2_000_000; // the 100 000th prime is 1 299 709, so this is enough
 boolean[] composite = new boolean[limit + 1];
 int seen = 0;
 for (int i = 2; i <= limit; i++) {
 if (!composite[i]) {
 seen++;
 if (seen == k) {
 System.out.println(i);
 return;
 }
 for (long m = (long) i * i; m <= limit; m += i) {
 composite[(int) m] = true;
 }
 }
 }
}
```

Why it works: A single sieve to 2 000 000 contains the 100 000th prime (1 299 709); count survivors until the k-th.

Verified against 3 test cases: `1000⏎` → `7919`; `1⏎` → `2`; `100000⏎` → `1299709`

Exercise 3 · Prime factorisation queries

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 int limit = 1_000_000;
 int[] spf = new int[limit + 1]; // spf[x] = smallest prime factor of x
 for (int i = 2; i <= limit; i++) {
 if (spf[i] == 0) {
 for (int m = i; m <= limit; m += i) {
 if (spf[m] == 0) {
 spf[m] = i;
 }
 }
 }
 }
 Scanner sc = new Scanner(System.in);
 int q = sc.nextInt();
 StringBuilder out = new StringBuilder();
 for (int t = 0; t < q; t++) {
 int x = sc.nextInt();
 int original = x;
 StringBuilder line = new StringBuilder();
 while (x > 1) {
 int p = spf[x];
 int e = 0;
 while (x % p == 0) {
 x /= p;
 e++;
 }
 if (line.length() > 0) {
 line.append(" * ");
 }
 line.append(p);
 if (e > 1) {
 line.append('^').append(e);
 }
 }
 out.append(original).append(" = ").append(original == 1 ? "1" : line).append('\n');
 }
 System.out.print(out);
 }
}
```

Why it works: A "smallest prime factor" sieve stores for each number its smallest prime divisor, so every factorisation takes O(log x) steps.

Verified against 2 test cases: `3⏎12⏎97⏎360⏎` → `12 = 2^2 * 3⏎97 = 97⏎360 = 2^3 * 3^2 * 5`; `2⏎2⏎1000000⏎` → `2 = 2⏎1000000 = 2^6 * 5^6`

5.8 Segmented & Incremental Sieve

Exercise 1 · Primes in a small range

Ex4.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex4 {
 // Count primes in [L, R] where R can be as large as 10^12 but R - L <= 10^6.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;

 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 int count = 0;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 count++;
 }
 }
 System.out.println(count);
 }
}
```

Why it works: This reference solution already uses the segmented method, which also covers small ranges.

Verified against 3 test cases: `10 30⏎` → `6`; `1 1⏎` → `0`; `100 130⏎` → `6`

Exercise 2 · Primes in a huge range

Ex4.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex4 {
 // Count primes in [L, R] where R can be as large as 10^12 but R - L <= 10^6.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;

 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 int count = 0;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 count++;
 }
 }
 System.out.println(count);
 }
}
```

Why it works: Segmented sieve: base primes up to √R (≤ 10⁶) and a window array of at most 10⁶ + 1 cells.

Verified against 3 test cases: `1000000000000 1000000000100⏎` → `4`; `2 2⏎` → `1`; `999999999900 1000000000000⏎` → `4`

Exercise 3 · Largest prime gap in a window

Ex5.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex5 {
 // Largest gap between two consecutive primes inside [L, R]; print -1 if fewer than two primes.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long low = sc.nextLong();
 long high = sc.nextLong();
 int root = (int) Math.sqrt((double) high);
 while ((long) root * root > high) root--;
 while ((long) (root + 1) * (root + 1) <= high) root++;
 boolean[] small = new boolean[root + 1];
 List<Integer> base = new ArrayList<>();
 for (int i = 2; i <= root; i++) {
 if (!small[i]) {
 base.add(i);
 for (long m = (long) i * i; m <= root; m += i) {
 small[(int) m] = true;
 }
 }
 }
 boolean[] composite = new boolean[(int) (high - low + 1)];
 for (int p : base) {
 long start = Math.max((long) p * p, (low + p - 1) / p * p);
 for (long m = start; m <= high; m += p) {
 composite[(int) (m - low)] = true;
 }
 }
 long prev = -1;
 long best = -1;
 for (long n = Math.max(low, 2); n <= high; n++) {
 if (!composite[(int) (n - low)]) {
 if (prev != -1) {
 best = Math.max(best, n - prev);
 }
 prev = n;
 }
 }
 System.out.println(best);
 }
}
```

Why it works: After the segmented sieve, one pass over the window tracks the last prime seen and the maximum gap.

Verified against 3 test cases: `1 100⏎` → `8`; `90 96⏎` → `-1`; `2 3⏎` → `1`

5.9 Binary Palindrome

Exercise 1 · Is it a binary palindrome?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 System.out.println(rev == x ? "YES" : "NO");
 }
}
```

Why it works: Reverse the bits into a second variable with shifts and compare. 0 never enters the loop, reversed stays 0 and equals n.

Verified against 4 test cases: `5⏎` → `YES`; `10⏎` → `NO`; `0⏎` → `YES`

Exercise 2 · Count binary palindromes up to n

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static boolean pal(int x) {
 int rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 return rev == x;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int count = 0;
 for (int i = 1; i <= n; i++) if (pal(i)) count++;
 System.out.println(count);
 }
}
```

Why it works: Test every i in 1..n with the O(log i) check. Total O(n log n).

Verified against 3 test cases: `10⏎` → `5`; `1⏎` → `1`; `100⏎` → `19`

Exercise 3 · Next binary palindrome

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static boolean pal(long x) {
 long rev = 0, rest = x;
 while (rest > 0) {
 rev = (rev << 1) | (rest & 1);
 rest >>= 1;
 }
 return rev == x;
 }

 // Smallest binary palindrome strictly greater than n.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long x = n + 1;
 while (!pal(x)) x++;
 System.out.println(x);
 }
}
```

Why it works: Simple forward search is fast enough because the next palindrome is always within about 2^(len/2) steps (for a 30-bit number, at most ~33 000 steps).

Verified against 3 test cases: `7⏎` → `9`; `0⏎` → `1`; `51⏎` → `63`

5.10 Swap Two Nibbles in a Byte

Exercise 1 · Swap the nibbles

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt() & 0xFF;
 System.out.println(((x & 0xF0) >>> 4) | ((x & 0x0F) << 4));
 }
}
```

Why it works: Mask each nibble, shift into the other's place, OR together.

Verified against 4 test cases: `171⏎` → `186`; `0⏎` → `0`; `255⏎` → `255`

Exercise 2 · Swap adjacent bits

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Swap every pair of adjacent bits (bit0<->bit1, bit2<->bit3, ...) of a 32-bit value; print as unsigned.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long input = sc.nextLong();
 int x = (int) input;
 int even = x & 0x55555555; // bits 0, 2, 4, ...
 int odd = x & 0xAAAAAAAA; // bits 1, 3, 5, ...
 int swapped = (even << 1) | (odd >>> 1);
 System.out.println(swapped & 0xFFFFFFFFL);
 }
}
```

Why it works: The two masks split the word into non-overlapping bit sets; each set moves by one position into the other set's slots.

Verified against 3 test cases: `23⏎` → `43`; `0⏎` → `0`; `4294967295⏎` → `4294967295`

Exercise 3 · Reverse 32 bits

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Reverse the 32 bits of an unsigned 32-bit integer.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long input = sc.nextLong();
 int x = (int) input;
 int result = 0;
 for (int i = 0; i < 32; i++) {
 result = (result << 1) | (x & 1);
 x >>>= 1;
 }
 System.out.println(result & 0xFFFFFFFFL);
 }
}
```

Why it works: Same peel-and-push technique as the binary-palindrome lesson, but for exactly 32 iterations so leading zeros are included.

Verified against 3 test cases: `43261596⏎` → `964176192`; `1⏎` → `2147483648`; `0⏎` → `0`

5.11 Longest Sequence of 1s (and after flipping a bit)

Exercise 1 · Longest run of 1s

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int cur = 0, best = 0;
 for (int i = 0; i < 32; i++) {
 if ((x & 1) == 1) best = Math.max(best, ++cur);
 else cur = 0;
 x >>>= 1;
 }
 System.out.println(best);
 }
}
```

Why it works: Count while bits are 1, reset on 0, remember the maximum.

Verified against 4 test cases: `56825⏎` → `6`; `0⏎` → `0`; `-1⏎` → `32`

Exercise 2 · Flip one bit

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int best = 0;
 for (int bit = 0; bit < 32; bit++) {
 int y = x | (1 << bit);
 int run = 0;
 for (int i = 0; i < 32; i++) {
 if (((y >>> i) & 1) == 1) best = Math.max(best, ++run);
 else run = 0;
 }
 }
 System.out.println(best);
 }
}
```

Why it works: Set each bit in turn (OR with 1 << bit) and measure the longest run; the maximum over 32 trials is the answer. The one-pass method from the lesson gives the same result.

Verified against 4 test cases: `478⏎` → `8`; `0⏎` → `1`; `-1⏎` → `32`

Exercise 3 · Flip up to k zeros

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Longest run of 1s after flipping at most k zeros: sliding window over the 32 bits containing
     <= k zeros.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int x = sc.nextInt();
 int k = sc.nextInt();
 int left = 0, zeros = 0, best = 0;
 for (int right = 0; right < 32; right++) {
 if (((x >>> right) & 1) == 0) zeros++;
 while (zeros > k) {
 if (((x >>> left) & 1) == 0) zeros--;
 left++;
 }
 best = Math.max(best, right - left + 1);
 }
 System.out.println(best);
 }
}
```

Why it works: Classic two-pointer window: the right end moves one step per bit; the left end only moves forward, so total work is O(32).

Verified against 3 test cases: `478 2⏎` → `9`; `0 5⏎` → `5`; `-1 0⏎` → `32`

5.12 Josephus Problem

Exercise 1 · Elimination order

Ex1.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 List<Integer> circle = new ArrayList<>();
 for (int i = 1; i <= n; i++) circle.add(i);
 StringBuilder sb = new StringBuilder();
 int idx = 0;
 while (!circle.isEmpty()) {
 idx = (idx + k - 1) % circle.size();
 sb.append(sb.length() == 0 ? "" : " ").append(circle.remove(idx));
 }
 System.out.println(sb);
 }
}
```

Why it works: Direct simulation; the last remaining person is removed at the end of the loop so the output ends with the survivor.

Verified against 3 test cases: `5 2⏎` → `2 4 1 5 3`; `7 3⏎` → `3 6 2 7 5 1 4`; `1 5⏎` → `1`

Exercise 2 · Survivor for large N

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long k = sc.nextLong();
 long ans = 0;
 for (long people = 2; people <= n; people++) {
 ans = (ans + k) % people;
 }
 System.out.println(ans + 1);
 }
}
```

Why it works: The loop ans = (ans + k) % people for people = 2..N takes O(N) time and O(1) space.

Verified against 3 test cases: `10 3⏎` → `4`; `14 2⏎` → `13`; `1 7⏎` → `1`

Exercise 3 · Josephus with k = 2 and N up to 10^{18}

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 System.out.println(2 * (n - Long.highestOneBit(n)) + 1);
 }
}
```

Why it works: After r eliminations 2^m people remain and the counting position is person 2r+1; with k = 2 and a power-of-two circle that person survives.

Verified against 4 test cases: `100⏎` → `73`; `1⏎` → `1`; `64⏎` → `1`

5.13 Block Swap Algorithm (Array Rotation)

Exercise 1 · Left rotation

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt() % n;
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) {
 sb.append(i == 0 ? "" : " ").append(a[(i + d) % n]);
 }
 System.out.println(sb);
 }
}
```

Why it works: The index formula (i + d) % n gives the result directly, without moving anything in place.

Verified against 3 test cases: `5 2⏎1 2 3 4 5⏎` → `3 4 5 1 2`; `4 0⏎9 8 7 6⏎` → `9 8 7 6`; `3 7⏎1 2 3⏎` → `2 3 1`

Exercise 2 · Right rotation in place

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static void reverse(int[] a, int lo, int hi) {
 while (lo < hi) {
 int t = a[lo];
 a[lo++] = a[hi];
 a[hi--] = t;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int d = sc.nextInt() % n;
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 // right rotation by d == left rotation by n - d
 int left = (n - d) % n;
 reverse(a, 0, left - 1);
 reverse(a, left, n - 1);
 reverse(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Convert to a left rotation by (n − d) mod n and apply the three-reversal trick.

Verified against 3 test cases: `5 2⏎1 2 3 4 5⏎` → `4 5 1 2 3`; `6 6⏎1 2 3 4 5 6⏎` → `1 2 3 4 5 6`; `2 1⏎1 2⏎` → `2 1`

Exercise 3 · How many rotations?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // The smallest element sits right after the "break" where a[i] > a[i+1].
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int lo = 0, hi = n - 1;
 while (lo < hi) {
 int mid = (lo + hi) / 2;
 if (a[mid] > a[hi]) {
 lo = mid + 1; // the minimum is to the right of mid
 } else {
 hi = mid; // the minimum is at mid or to its left
 }
 }
 System.out.println(lo); // index of the minimum == number of left rotations
 }
}
```

Why it works: If a[mid] > a[hi] the minimum is right of mid; otherwise it is at mid or left of it. The loop narrows to the minimum's index in O(log n).

Verified against 3 test cases: `6⏎15 18 2 3 6 12⏎` → `2`; `5⏎1 2 3 4 5⏎` → `0`; `4⏎2 3 4 1⏎` → `3`

5.14 Leaders in an Array

Exercise 1 · Print the leaders

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] out = new int[n];
 int c = 0;
 int max = Integer.MIN_VALUE;
 for (int i = n - 1; i >= 0; i--) {
 if (a[i] > max) {
 out[c++] = a[i];
 max = a[i];
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = c - 1; i >= 0; i--) sb.append(sb.length() == 0 ? "" : " ").append(out[i]);
 System.out.println(sb);
 }
}
```

Why it works: The running maximum of the suffix decides each element in O(1).

Verified against 4 test cases: `6⏎16 17 4 3 5 2⏎` → `17 5 2`; `5⏎14 20 3 6 5⏎` → `20 6 5`; `3⏎5 4 3⏎` → `5 4 3`

Exercise 2 · Non-strict leaders

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Non-strict leaders: an element counts if it is >= every element to its right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] out = new int[n];
 int c = 0;
 int max = Integer.MIN_VALUE;
 for (int i = n - 1; i >= 0; i--) {
 if (a[i] >= max) {
 out[c++] = a[i];
 max = a[i];
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = c - 1; i >= 0; i--) sb.append(sb.length() == 0 ? "" : " ").append(out[i]);
 System.out.println(sb);
 }
}
```

Why it works: Replace `>` by `>=` in the running-maximum test.

Verified against 3 test cases: `5⏎7 10 4 10 6⏎` → `10 10 6`; `3⏎2 2 2⏎` → `2 2 2`; `1⏎9⏎` → `9`

Exercise 3 · Next greater element

Ex3.java

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class Ex3 {
 // Next greater element to the right (or -1), in O(n) with a stack of candidates scanned from the right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int[] ans = new int[n];
 Deque<Integer> stack = new ArrayDeque<>(); // values, decreasing from top... top is nearest candidate
 for (int i = n - 1; i >= 0; i--) {
 while (!stack.isEmpty() && stack.peek() <= a[i]) {
 stack.pop(); // can never be the next-greater for anything to the left
 }
 ans[i] = stack.isEmpty() ? -1 : stack.peek();
 stack.push(a[i]);
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(ans[i]);
 System.out.println(sb);
 }
}
```

Why it works: The stack holds the elements that could still be "next greater" for something on the left, in decreasing order. Each element is pushed and popped at most once: O(n).

Verified against 3 test cases: `4⏎4 5 2 25⏎` → `5 25 25 -1`; `4⏎13 7 6 12⏎` → `-1 12 12 -1`; `3⏎3 2 1⏎` → `-1 -1 -1`

5.15 Majority Element (Boyer–Moore Voting)

Exercise 1 · Majority with a HashMap

Ex1.java

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 Map<Integer, Integer> freq = new HashMap<>();
 Integer winner = null;
 for (int i = 0; i < n; i++) {
 int x = sc.nextInt();
 int c = freq.merge(x, 1, Integer::sum);
 if (c > n / 2) winner = x;
 }
 System.out.println(winner == null ? "No majority element" : winner);
 }
}
```

Why it works: merge(x, 1, Integer::sum) increments the counter; any value whose count passes n/2 is the answer.

Verified against 3 test cases: `5⏎2 2 1 1 1⏎` → `1`; `4⏎1 2 1 2⏎` → `No majority element`; `1⏎7⏎` → `7`

Exercise 2 · Boyer–Moore

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int candidate = 0, count = 0;
 for (int x : a) {
 if (count == 0) {
 candidate = x;
 count = 1;
 } else if (x == candidate) count++;
 else count--;
 }
 int occ = 0;
 for (int x : a) if (x == candidate) occ++;
 System.out.println(occ > n / 2 ? String.valueOf(candidate) : "No majority element");
 }
}
```

Why it works: Phase 1 keeps a candidate and a vote counter; phase 2 counts the real occurrences.

Verified against 3 test cases: `5⏎2 2 1 1 1⏎` → `1`; `7⏎3 3 4 2 4 4 2⏎` → `No majority element`; `9⏎2 2 2 6 2 2 8 2 1⏎` → `2`

Exercise 3 · Elements appearing more than n/3 times

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Elements appearing more than n/3 times: at most TWO of them exist, so keep two candidates.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int c1 = 0, c2 = 1, n1 = 0, n2 = 0;
 for (int x : a) {
 if (x == c1) n1++;
 else if (x == c2) n2++;
 else if (n1 == 0) { c1 = x; n1 = 1; }
 else if (n2 == 0) { c2 = x; n2 = 1; }
 else { n1--; n2--; }
 }
 int o1 = 0, o2 = 0;
 for (int x : a) {
 if (x == c1) o1++;
 else if (x == c2) o2++;
 }
 StringBuilder sb = new StringBuilder();
 if (o1 > n / 3) sb.append(c1);
 if (o2 > n / 3) sb.append(sb.length() == 0 ? "" : " ").append(c2);
 if (sb.length() > 0 && o1 > n / 3 && o2 > n / 3 && c1 > c2) {
 sb = new StringBuilder().append(c2).append(' ').append(c1); // print in increasing order
 }
 System.out.println(sb.length() == 0 ? "None" : sb);
 }
}
```

Why it works: Generalised Boyer–Moore: removing three different values at a time cannot eliminate a value that has more than n/3 occurrences.

Verified against 3 test cases: `8⏎1 1 1 3 3 2 2 2⏎` → `1 2`; `3⏎3 2 3⏎` → `3`; `4⏎1 2 3 4⏎` → `None`

5.16 Maximum Product Subarray

Exercise 1 · Count subarrays with positive product

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Count subarrays whose product is strictly positive.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int count = 0;
 for (int i = 0; i < n; i++) {
 int negatives = 0;
 for (int j = i; j < n; j++) {
 if (a[j] == 0) break; // a zero makes every longer subarray's product 0
 if (a[j] < 0) negatives++;
 if (negatives % 2 == 0) count++;
 }
 }
 System.out.println(count);
 }
}
```

Why it works: Track the parity of negatives while extending; stop at a zero.

Verified against 3 test cases: `5⏎2 3 -2 4 0⏎` → `4`; `3⏎-1 -2 -3⏎` → `2`; `3⏎0 0 0⏎` → `0`

Exercise 2 · Maximum product subarray

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long mx = 0, mn = 0, best = 0;
 for (int i = 0; i < n; i++) {
 long x = sc.nextInt();
 if (i == 0) {
 mx = mn = best = x;
 continue;
 }
 if (x < 0) {
 long t = mx;
 mx = mn;
 mn = t;
 }
 mx = Math.max(x, mx * x);
 mn = Math.min(x, mn * x);
 best = Math.max(best, mx);
 }
 System.out.println(best);
 }
}
```

Why it works: The O(n) max/min recurrence from the lesson.

Verified against 4 test cases: `4⏎2 3 -2 4⏎` → `6`; `3⏎-2 0 -1⏎` → `0`; `4⏎-5 -4 -1 -2⏎` → `40`

Exercise 3 · Which subarray?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Print "product start end" for the maximum-product subarray (0-based, earliest start, then
     earliest end).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextLong();
 long best = Long.MIN_VALUE;
 int bs = 0, be = 0;
 for (int i = 0; i < n; i++) {
 long p = 1;
 for (int j = i; j < n; j++) {
 p *= a[j];
 if (p > best) {
 best = p;
 bs = i;
 be = j;
 }
 }
 }
 System.out.println(best + " " + bs + " " + be);
 }
}
```

Why it works: Enumerate (i, j) in order and update the record only when the product is strictly greater.

Verified against 3 test cases: `4⏎2 3 -2 4⏎` → `6 0 1`; `1⏎5⏎` → `5 0 0`; `3⏎-1 -1 -1⏎` → `1 0 1`

5.17 Maximum Equilibrium Sum

Exercise 1 · First equilibrium index

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long left = 0;
 for (int i = 0; i < n; i++) {
 if (left == total - left - a[i]) {
 System.out.println(i);
 return;
 }
 left += a[i];
 }
 System.out.println(-1);
 }
}
```

Why it works: Running left sum and a precomputed total give the right sum in O(1).

Verified against 3 test cases: `7⏎-7 1 5 2 -4 3 0⏎` → `3`; `3⏎1 2 3⏎` → `-1`; `1⏎5⏎` → `0`

Exercise 2 · Maximum equilibrium sum

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Print the maximum equilibrium sum (one side's sum), or "none".
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long left = 0;
 boolean found = false;
 long best = Long.MIN_VALUE;
 for (int i = 0; i < n; i++) {
 if (left == total - left - a[i]) {
 found = true;
 best = Math.max(best, left);
 }
 left += a[i];
 }
 System.out.println(found ? String.valueOf(best) : "none");
 }
}
```

Why it works: Same scan; update the maximum whenever left == right.

Verified against 3 test cases: `9⏎1 2 3 4 5 4 3 2 1⏎` → `10`; `7⏎-7 1 5 2 -4 3 0⏎` → `0`; `2⏎1 2⏎` → `none`

Exercise 3 · Balanced split points

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Count split points k (1 <= k <= n-1) where sum(a[0..k-1]) == sum(a[k..n-1]).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long[] a = new long[n];
 long total = 0;
 for (int i = 0; i < n; i++) {
 a[i] = sc.nextLong();
 total += a[i];
 }
 long prefix = 0;
 int count = 0;
 for (int k = 1; k < n; k++) {
 prefix += a[k - 1];
 if (prefix == total - prefix) count++;
 }
 System.out.println(count);
 }
}
```

Why it works: prefix == total − prefix at each split point; count them.

Verified against 3 test cases: `6⏎1 -1 1 -1 1 -1⏎` → `2`; `4⏎2 2 2 2⏎` → `1`; `3⏎1 2 3⏎` → `1`

5.18 Maximum Sum of an Hourglass in a Matrix

Exercise 1 · One hourglass

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Read a 3x3 matrix and print the single hourglass sum.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int[][] m = new int[3][3];
 for (int i = 0; i < 3; i++) for (int j = 0; j < 3; j++) m[i][j] = sc.nextInt();
 System.out.println(m[0][0] + m[0][1] + m[0][2] + m[1][1] + m[2][0] + m[2][1] + m[2][2]);
 }
}
```

Why it works: Add the two full outer rows and the centre cell.

Verified against 3 test cases: `1 1 1⏎0 1 0⏎1 1 1⏎` → `7`; `0 0 0⏎5 5 5⏎0 0 0⏎` → `5`; `-1 -1 -1⏎-1 -1 -1⏎-1 -1 -1⏎` → `-7`

Exercise 2 · Maximum hourglass with negatives

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] m = new int[r][c];
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) m[i][j] = sc.nextInt();
 int best = Integer.MIN_VALUE;
 for (int i = 0; i + 2 < r; i++) {
 for (int j = 0; j + 2 < c; j++) {
 int s = m[i][j] + m[i][j + 1] + m[i][j + 2] + m[i + 1][j + 1] + m[i + 2][j] + m[i + 2][j + 1] +
 m[i + 2][j + 2];
 best = Math.max(best, s);
 }
 }
 System.out.println(best);
 }
}
```

Why it works: All (R−2)(C−2) hourglass sums are compared; the initial value must be below any possible sum.

Verified against 3 test cases: `3 3⏎-1 -1 -1⏎-1 -9 -1⏎-1 -1 -1⏎` → `-15`; `4 4⏎1 2 3 4⏎5 6 7 8⏎9 1 2 3⏎4 5 6 7⏎` → `41`; `3 4⏎1 1 1 1⏎1 9 1 1⏎1 1 1 1⏎` → `15`

Exercise 3 · Largest k×k block

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Largest sum of any k x k block, using a 2-D prefix-sum table so each block costs O(1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int k = sc.nextInt();
 long[][] p = new long[r + 1][c + 1]; // p[i][j] = sum of the rectangle rows 0..i-1, cols 0..j-1
 for (int i = 1; i <= r; i++) {
 for (int j = 1; j <= c; j++) {
 p[i][j] = sc.nextLong() + p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1];
 }
 }
 long best = Long.MIN_VALUE;
 for (int i = k; i <= r; i++) {
 for (int j = k; j <= c; j++) {
 long s = p[i][j] - p[i - k][j] - p[i][j - k] + p[i - k][j - k];
 best = Math.max(best, s);
 }
 }
 System.out.println(best);
 }
}
```

Why it works: The prefix table lets any rectangle sum be evaluated in four lookups, independent of its size.

Verified against 3 test cases: `3 3 2⏎1 2 3⏎4 5 6⏎7 8 9⏎` → `28`; `2 2 2⏎-1 -2⏎-3 -4⏎` → `-10`; `2 3 1⏎5 -1 3⏎0 2 -4⏎` → `5`

5.19 Toggle the Switch (100 Doors)

Exercise 1 · Count the open doors

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 boolean[] open = new boolean[n + 1];
 for (int m = 1; m <= n; m++) for (int d = m; d <= n; d += m) open[d] = !open[d];
 int c = 0;
 for (int d = 1; d <= n; d++) if (open[d]) c++;
 System.out.println(c);
 }
}
```

Why it works: Simulate every monkey and count the true cells.

Verified against 3 test cases: `10⏎` → `3`; `100⏎` → `10`; `1⏎` → `1`

Exercise 2 · A trillion doors

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long n = sc.nextLong();
 long r = (long) Math.sqrt((double) n);
 while (r * r > n) r--;
 while ((r + 1) * (r + 1) <= n) r++;
 System.out.println(r);
 }
}
```

Why it works: $\lfloor\sqrt{n}\rfloor$ with a floating-point estimate corrected by two while-loops.

Verified against 3 test cases: `1000000000000000000⏎` → `1000000000`; `99⏎` → `9`; `999999999999999999⏎` → `999999999`

Exercise 3 · Only the first m monkeys

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Only the first m monkeys are released. Count doors that are open afterwards.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int m = sc.nextInt();
 boolean[] open = new boolean[n + 1];
 for (int monkey = 1; monkey <= Math.min(m, n); monkey++) {
 for (int d = monkey; d <= n; d += monkey) open[d] = !open[d];
 }
 int c = 0;
 for (int d = 1; d <= n; d++) if (open[d]) c++;
 System.out.println(c);
 }
}
```

Why it works: The shortcut no longer applies (a door's toggle count is the number of divisors ≤ m), so simulate only the m monkeys.

Verified against 3 test cases: `10 3⏎` → `4`; `10 10⏎` → `3`; `6 1⏎` → `6`

5.20 Alice's Apple Tree (Minimum Garden Perimeter)

Exercise 1 · Apples in a square

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long r = sc.nextLong();
 System.out.println(2 * r * (r + 1) * (2 * r + 1));
 }
}
```

Why it works: Direct evaluation of the closed form in long arithmetic.

Verified against 3 test cases: `3⏎` → `168`; `1⏎` → `12`; `2⏎` → `60`

Exercise 2 · Minimum perimeter (small)

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long need = sc.nextLong();
 long r = 0, sum = 0;
 while (sum < need) {
 r++;
 sum += 12 * r * r;
 }
 System.out.println(8 * r);
 }
}
```

Why it works: Ring-by-ring accumulation in long.

Verified against 3 test cases: `13⏎` → `16`; `1⏎` → `8`; `1000000000⏎` → `5040`

Exercise 3 · Minimum perimeter (large)

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static long inside(long r) {
 return 2 * r * (r + 1) * (2 * r + 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long need = sc.nextLong();
 long lo = 1, hi = 200_000;
 while (lo < hi) {
 long mid = (lo + hi) / 2;
 if (inside(mid) >= need) hi = mid;
 else lo = mid + 1;
 }
 System.out.println(8 * lo);
 }
}
```

Why it works: Binary search for the smallest r with apples(r) ≥ needed; answer 8r.

Verified against 3 test cases: `1000000000000000⏎` → `503968`; `13⏎` → `16`; `1⏎` → `8`

5.21 Weighted Substring (Smallest String of a Given Weight)

Exercise 1 · Weight of a string

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Total weight of a string made of A..Z, with weight(A)=1 and weight(next) = (index+1) *
     weight(previous).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long[] w = new long[26];
 w[0] = 1;
 for (int i = 1; i < 26; i++) w[i] = w[i - 1] * (i + 2);
 long sum = 0;
 for (char c : s.toCharArray()) sum += w[c - 'A'];
 System.out.println(sum);
 }
}
```

Why it works: Table lookup per character; sum in long.

Verified against 3 test cases: `ACC⏎` → `25`; `AABBC⏎` → `20`; `D⏎` → `60`

Exercise 2 · Smallest string

Ex2.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 long weight = sc.nextLong();
 List<Long> w = new ArrayList<>();
 long cur = 1;
 w.add(cur);
 for (int n = 2; n <= 26; n++) {
 if (cur > weight / (n + 1)) break;
 cur *= (n + 1);
 w.add(cur);
 }
 List<Character> out = new ArrayList<>();
 for (int i = w.size() - 1; i >= 0; i--) {
 while (weight >= w.get(i)) {
 out.add((char) ('A' + i));
 weight -= w.get(i);
 }
 }
 Collections.sort(out);
 StringBuilder sb = new StringBuilder();
 for (char c : out) sb.append(c);
 System.out.println(sb);
 }
}
```

Why it works: Each weight divides the next, so greedy gives the minimum count; ascending order gives the smallest string.

Verified against 4 test cases: `25⏎` → `ACC`; `20⏎` → `AABBC`; `1⏎` → `A`

Exercise 3 · Normalise a string

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Normalise: print the shortest, lexicographically smallest string with the same weight as s.
 // Carry like a number system: (letter i+1 has weight (i+2) times letter i), so (i+2) copies of
     letter i
 // become one copy of letter i+1.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long[] count = new long[27];
 for (char c : s.toCharArray()) count[c - 'A']++;
 for (int i = 0; i < 25; i++) {
 long base = i + 3; // weight(letter i+1) / weight(letter i): 3 for A->B, 4 for B->C, ...
 count[i + 1] += count[i] / base;
 count[i] %= base;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < 26; i++) {
 for (long k = 0; k < count[i]; k++) sb.append((char) ('A' + i));
 }
 System.out.println(sb);
 }
}
```

Why it works: A mixed-radix number system: carry count[i] / (i + 2) into the next letter and keep the remainder.

Verified against 4 test cases: `AAAAB⏎` → `ABB`; `AAA⏎` → `B`; `BBBB⏎` → `C`

5.22 Move Hyphen to the Beginning

Exercise 1 · First hyphen to the front

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 int i = s.indexOf('-');
 System.out.println(i < 0 ? s : "-" + s.substring(0, i) + s.substring(i + 1));
 }
}
```

Why it works: Rebuild the string as "-" + before + after.

Verified against 3 test cases: `face-prep⏎` → `-faceprep`; `abc⏎` → `abc`; `a-b-c⏎` → `-ab-c`

Exercise 2 · All hyphens to the front

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 StringBuilder h = new StringBuilder();
 StringBuilder o = new StringBuilder();
 for (char c : s.toCharArray()) (c == '-' ? h : o).append(c);
 System.out.println(h.append(o));
 }
}
```

Why it works: A stable partition into hyphens and non-hyphens, concatenated.

Verified against 3 test cases: `a-b-c⏎` → `--abc`; `---⏎` → `---`; `abc⏎` → `abc`

Exercise 3 · Minimum adjacent swaps

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Minimum number of ADJACENT swaps to bring all hyphens to the front:
 // each hyphen must jump over every non-hyphen character that stands before it.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.nextLine();
 long nonHyphensSeen = 0;
 long swaps = 0;
 for (char c : s.toCharArray()) {
 if (c == '-') swaps += nonHyphensSeen;
 else nonHyphensSeen++;
 }
 System.out.println(swaps);
 }
}
```

Why it works: Hyphens never need to cross each other (that would be a wasted swap), so the cost is the number of (non-hyphen, hyphen) inversions.

Verified against 3 test cases: `a-b-c⏎` → `3`; `--ab⏎` → `0`; `ab--⏎` → `4`

5.23 Strobogrammatic Number

Exercise 1 · Is it strobogrammatic?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 String from = "01689";
 String to = "01986";
 int l = 0, r = s.length() - 1;
 boolean ok = true;
 while (l <= r) {
 int i = from.indexOf(s.charAt(l));
 if (i < 0 || to.charAt(i) != s.charAt(r)) {
 ok = false;
 break;
 }
 l++;
 r--;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: The digit at the left pointer must rotate into the digit at the right pointer; the middle digit must be 0, 1 or 8.

Verified against 5 test cases: `69⏎` → `YES`; `818⏎` → `YES`; `962⏎` → `NO`

Exercise 2 · How many of length n?

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Count of n-digit strobogrammatic numbers (n >= 1, no leading zero; the single digit 0 counts
     for n = 1).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 long count;
 if (n == 1) {
 count = 3; // 0, 1, 8
 } else {
 count = 4; // outer pair: 11, 69, 88, 96
 for (int i = 1; i < n / 2; i++) count *= 5; // each inner pair: 00, 11, 69, 88, 96
 if (n % 2 == 1) count *= 3; // middle digit: 0, 1, 8
 }
 System.out.println(count);
 }
}
```

Why it works: Multiplication principle: 4 · 5^(pairs−1) · (3 if n is odd).

Verified against 4 test cases: `4⏎` → `20`; `1⏎` → `3`; `3⏎` → `12`

Exercise 3 · Strobogrammatic in a range

Ex3.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 static List<String> build(int n, int total) {
 if (n == 0) return new ArrayList<>(List.of(""));
 if (n == 1) return new ArrayList<>(List.of("0", "1", "8"));
 List<String> res = new ArrayList<>();
 for (String inner : build(n - 2, total)) {
 if (n != total) res.add("0" + inner + "0");
 res.add("1" + inner + "1");
 res.add("6" + inner + "9");
 res.add("8" + inner + "8");
 res.add("9" + inner + "6");
 }
 return res;
 }

 // Count strobogrammatic numbers in [low, high] given as strings of up to 15 digits.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String low = sc.next();
 String high = sc.next();
 int count = 0;
 for (int len = low.length(); len <= high.length(); len++) {
 for (String s : build(len, len)) {
 if (len == low.length() && s.compareTo(low) < 0) continue; // same length: string order =
     numeric order
 if (len == high.length() && s.compareTo(high) > 0) continue;
 count++;
 }
 }
 System.out.println(count);
 }
}
```

Why it works: Generate all candidates for each length and filter the ones outside the bounds.

Verified against 3 test cases: `50 100⏎` → `3`; `0 0⏎` → `1`; `1 1000⏎` → `18`

5.24 Lexicographically First Palindromic Rearrangement

Exercise 1 · Can it be a palindrome?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next().toLowerCase();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 int odd = 0;
 for (int f : freq) if (f % 2 == 1) odd++;
 System.out.println(odd <= 1 ? "YES" : "NO");
 }
}
```

Why it works: At most one letter may have an odd frequency.

Verified against 3 test cases: `aabb⏎` → `YES`; `apple⏎` → `NO`; `Malayalam⏎` → `YES`

Exercise 2 · Build the smallest palindrome

Ex2.java

```java
import java.util.Scanner;
import java.util.TreeMap;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 TreeMap<Character, Integer> f = new TreeMap<>();
 for (char c : sc.next().toLowerCase().toCharArray()) f.merge(c, 1, Integer::sum);
 String mid = "";
 int odd = 0;
 for (var e : f.entrySet()) {
 if (e.getValue() % 2 == 1) {
 odd++;
 mid = String.valueOf(e.getKey());
 }
 }
 if (odd > 1) {
 System.out.println("no palindromic string");
 return;
 }
 StringBuilder front = new StringBuilder();
 for (var e : f.entrySet()) front.append(String.valueOf(e.getKey()).repeat(e.getValue() / 2));
 System.out.println(front + mid + new StringBuilder(front).reverse());
 }
}
```

Why it works: Front half sorted, optional odd letter in the middle, mirrored back half.

Verified against 4 test cases: `Malayalam⏎` → `aalmymlaa`; `aabbcc⏎` → `abccba`; `apple⏎` → `no palindromic string`

Exercise 3 · How many palindromes?

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Number of DISTINCT palindromic permutations: arrange the first half freely.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next().toLowerCase();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 int odd = 0;
 for (int f : freq) if (f % 2 == 1) odd++;
 if (odd > 1) {
 System.out.println(0);
 return;
 }
 long[] fact = new long[21];
 fact[0] = 1;
 for (int i = 1; i <= 20; i++) fact[i] = fact[i - 1] * i;
 int half = s.length() / 2;
 long ways = fact[half];
 for (int f : freq) ways /= fact[f / 2];
 System.out.println(ways);
 }
}
```

Why it works: The palindrome is determined by its left half; count the distinct arrangements of the half-multiset.

Verified against 4 test cases: `aabbcc⏎` → `6`; `aabb⏎` → `2`; `abc⏎` → `0`

5.25 Manacher's Algorithm (Longest Palindromic Substring)

Exercise 1 · Is it a palindrome?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 boolean ok = true;
 for (int l = 0, r = s.length() - 1; l < r; l++, r--) {
 if (s.charAt(l) != s.charAt(r)) {
 ok = false;
 break;
 }
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: Compare the characters at both ends and move inwards; any mismatch decides NO.

Verified against 3 test cases: `racecar⏎` → `YES`; `abca⏎` → `NO`; `a⏎` → `YES`

Exercise 2 · Longest palindromic substring (expand)

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int bestStart = 0, bestLen = 1;
 for (int c = 0; c < s.length(); c++) {
 for (int t = 0; t < 2; t++) {
 int l = c, r = c + t;
 while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
 l--;
 r++;
 }
 if (r - l - 1 > bestLen) {
 bestLen = r - l - 1;
 bestStart = l + 1;
 }
 }
 }
 System.out.println(s.substring(bestStart, bestStart + bestLen));
 }
}
```

Why it works: Centre expansion with odd and even centres; O(n²).

Verified against 4 test cases: `cbbd⏎` → `bb`; `babad⏎` → `bab`; `a⏎` → `a`

Exercise 3 · Count palindromic substrings

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Count ALL palindromic substrings (by position) in O(n) using Manacher's radii.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 StringBuilder sb = new StringBuilder("#");
 for (char c : s.toCharArray()) sb.append(c).append('#');
 String t = sb.toString();
 int n = t.length();
 int[] rad = new int[n];
 int center = 0, right = 0;
 long total = 0;
 for (int i = 0; i < n; i++) {
 if (i < right) rad[i] = Math.min(right - i, rad[2 * center - i]);
 while (i - rad[i] - 1 >= 0 && i + rad[i] + 1 < n && t.charAt(i - rad[i] - 1) == t.charAt(i +
     rad[i] + 1)) rad[i]++;
 if (i + rad[i] > right) {
 center = i;
 right = i + rad[i];
 }
 total += (rad[i] + 1) / 2; // a radius of r in the transformed string gives ceil(r/2) palindromes
 }
 System.out.println(total);
 }
}
```

Why it works: Manacher's radii give, for every centre, the number of palindromes centred there; summing them is O(n).

Verified against 3 test cases: `abba⏎` → `6`; `aaa⏎` → `6`; `abc⏎` → `3`

5.26 Natural Sort Order

Exercise 1 · Sort file names naturally

Ex1.java

```java
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex1 {
 static List<String> chunks(String s) {
 List<String> out = new ArrayList<>();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 boolean d = Character.isDigit(s.charAt(i));
 while (j < s.length() && Character.isDigit(s.charAt(j)) == d) j++;
 out.add(s.substring(i, j));
 i = j;
 }
 return out;
 }

 static int cmp(String a, String b) {
 List<String> x = chunks(a), y = chunks(b);
 for (int i = 0; i < Math.min(x.size(), y.size()); i++) {
 String p = x.get(i), q = y.get(i);
 int c = (Character.isDigit(p.charAt(0)) && Character.isDigit(q.charAt(0))) ? new BigInteger(p)
 .compareTo(new BigInteger(q)) : p.compareToIgnoreCase(q);
 if (c != 0) return c;
 }
 return Integer.compare(x.size(), y.size());
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> names = new ArrayList<>();
 for (int i = 0; i < n; i++) names.add(sc.next());
 names.sort(Ex1::cmp);
 for (String s : names) System.out.println(s);
 }
}
```

Why it works: The comparator from the lesson, applied with List.sort.

Verified against 3 test cases: `4⏎img12.png⏎img10.png⏎img2.png⏎IMG1.png⏎` → `IMG1.png⏎img2.png⏎img10.png⏎img12.png`; `2⏎b2⏎a10⏎` → `a10⏎b2`; `3⏎x9⏎x10⏎x1⏎` → `x1⏎x9⏎x10`

Exercise 2 · Sort version numbers

Ex2.java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex2 {
 // Sort version numbers such as 1.2.10 and 1.10 numerically, field by field.
 static int cmp(String a, String b) {
 String[] x = a.split("\\.");
 String[] y = b.split("\\.");
 for (int i = 0; i < Math.max(x.length, y.length); i++) {
 long p = i < x.length ? Long.parseLong(x[i]) : 0;
 long q = i < y.length ? Long.parseLong(y[i]) : 0;
 if (p != q) return Long.compare(p, q);
 }
 return 0;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> v = new ArrayList<>();
 for (int i = 0; i < n; i++) v.add(sc.next());
 v.sort(Ex2::cmp);
 System.out.println(String.join(" ", v));
 }
}
```

Why it works: Numeric comparison per dotted component; pad the shorter version with zeros.

Verified against 3 test cases: `4⏎1.10⏎1.2.10⏎1.2.9⏎1.9⏎` → `1.2.9 1.2.10 1.9 1.10`; `2⏎2.0⏎1.9.9⏎` → `1.9.9 2.0`; `2⏎1.0⏎1.0.0⏎` → `1.0 1.0.0`

Exercise 3 · Natural order with deterministic ties

Ex3.java

```java
import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ex3 {
 static List<String> chunks(String s) {
 List<String> out = new ArrayList<>();
 int i = 0;
 while (i < s.length()) {
 int j = i;
 boolean d = Character.isDigit(s.charAt(i));
 while (j < s.length() && Character.isDigit(s.charAt(j)) == d) j++;
 out.add(s.substring(i, j));
 i = j;
 }
 return out;
 }

 // Natural order, ignoring case. When two names are equal numerically (file01 vs file1),
 // the one with FEWER leading zeros (the shorter text) comes first, then plain compareTo.
 static int cmp(String a, String b) {
 List<String> x = chunks(a), y = chunks(b);
 for (int i = 0; i < Math.min(x.size(), y.size()); i++) {
 String p = x.get(i), q = y.get(i);
 int c = (Character.isDigit(p.charAt(0)) && Character.isDigit(q.charAt(0))) ? new BigInteger(p)
 .compareTo(new BigInteger(q)) : p.compareToIgnoreCase(q);
 if (c != 0) return c;
 }
 if (x.size() != y.size()) return Integer.compare(x.size(), y.size());
 if (a.length() != b.length()) return Integer.compare(a.length(), b.length());
 return a.compareTo(b);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 List<String> names = new ArrayList<>();
 for (int i = 0; i < n; i++) names.add(sc.next());
 names.sort(Ex3::cmp);
 System.out.println(String.join(" ", names));
 }
}
```

Why it works: Adding tie-breakers makes the comparator a strict total order, so the output is deterministic.

Verified against 3 test cases: `4⏎file01⏎file1⏎File2⏎file10⏎` → `file1 file01 File2 file10`; `3⏎a1⏎A1⏎a01⏎` → `A1 a1 a01`; `1⏎zzz⏎` → `zzz`

5.27 Maneuvering (Counting Grid Paths)

Exercise 1 · Paths in a rectangle

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int n = sc.nextInt();
 long[][] dp = new long[m][n];
 for (int i = 0; i < m; i++)
 for (int j = 0; j < n; j++)
 dp[i][j] = (i == 0 || j == 0) ? 1 : dp[i - 1][j] + dp[i][j - 1];
 System.out.println(dp[m - 1][n - 1]);
 }
}
```

Why it works: Bottom-up DP with long values.

Verified against 3 test cases: `3 7⏎` → `28`; `1 5⏎` → `1`; `18 18⏎` → `2333606220`

Exercise 2 · Paths with obstacles

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // Grid of 0 (free) and 1 (blocked). Count right/down paths from top-left to bottom-right.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] g = new int[r][c];
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) g[i][j] = sc.nextInt();
 long[][] dp = new long[r][c];
 for (int i = 0; i < r; i++) {
 for (int j = 0; j < c; j++) {
 if (g[i][j] == 1) {
 dp[i][j] = 0;
 } else if (i == 0 && j == 0) {
 dp[i][j] = 1;
 } else {
 dp[i][j] = (i > 0 ? dp[i - 1][j] : 0) + (j > 0 ? dp[i][j - 1] : 0);
 }
 }
 }
 System.out.println(dp[r - 1][c - 1]);
 }
}
```

Why it works: Same recurrence; blocked cells are forced to 0 so no path goes through them.

Verified against 3 test cases: `3 3⏎0 0 0⏎0 1 0⏎0 0 0⏎` → `2`; `2 2⏎0 1⏎1 0⏎` → `0`; `1 1⏎0⏎` → `1`

Exercise 3 · Huge grid modulo a prime

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static final long MOD = 1_000_000_007L;

 static long power(long b, long e) {
 long r = 1;
 b %= MOD;
 while (e > 0) {
 if ((e & 1) == 1) r = r * b % MOD;
 b = b * b % MOD;
 e >>= 1;
 }
 return r;
 }

 // C(m+n-2, m-1) modulo 1e9+7 for m, n up to 100000: factorials and a modular inverse (Fermat).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int m = sc.nextInt();
 int n = sc.nextInt();
 int total = m + n - 2;
 long[] fact = new long[total + 1];
 fact[0] = 1;
 for (int i = 1; i <= total; i++) fact[i] = fact[i - 1] * i % MOD;
 long denom = fact[m - 1] * fact[n - 1] % MOD;
 System.out.println(fact[total] * power(denom, MOD - 2) % MOD);
 }
}
```

Why it works: nCr mod p = n! · inverse(r!) · inverse((n−r)!) mod p, with the inverse by fast exponentiation.

Verified against 3 test cases: `3 7⏎` → `28`; `1 1⏎` → `1`; `100000 100000⏎` → `691090292`

5.28 Sorted Distinct Permutations

Exercise 1 · How many distinct permutations?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Number of distinct permutations = n! / (c1! * c2! * ...)
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int[] freq = new int[256];
 for (char c : s.toCharArray()) freq[c]++;
 long[] f = new long[21];
 f[0] = 1;
 for (int i = 1; i <= 20; i++) f[i] = f[i - 1] * i;
 long ways = f[s.length()];
 for (int x : freq) ways /= f[x];
 System.out.println(ways);
 }
}
```

Why it works: Multinomial coefficient from letter counts.

Verified against 4 test cases: `AAB⏎` → `3`; `ABCD⏎` → `24`; `AAAA⏎` → `1`

Exercise 2 · Print them sorted

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static StringBuilder out = new StringBuilder();

 static void build(int[] counts, char[] cur, int pos) {
 if (pos == cur.length) {
 out.append(new String(cur)).append('\n');
 return;
 }
 for (int c = 0; c < counts.length; c++) {
 if (counts[c] == 0) continue;
 counts[c]--;
 cur[pos] = (char) c;
 build(counts, cur, pos + 1);
 counts[c]++;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 int[] counts = new int[256];
 for (char c : s.toCharArray()) counts[c]++;
 build(counts, new char[s.length()], 0);
 System.out.print(out);
 }
}
```

Why it works: Count-based backtracking enumerates in sorted order without duplicates.

Verified against 3 test cases: `ABA⏎` → `AAB⏎ABA⏎BAA`; `BAC⏎` → `ABC⏎ACB⏎BAC⏎BCA⏎CAB⏎CBA`; `X⏎` → `X`

Exercise 3 · The k-th permutation

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static long[] fact = new long[21];

 // number of distinct arrangements of the remaining letters
 static long arrangements(int[] counts, int remaining) {
 long r = fact[remaining];
 for (int c : counts) r /= fact[c];
 return r;
 }

 // k-th (1-based) distinct permutation in sorted order, without generating the others.
 public static void main(String[] args) {
 fact[0] = 1;
 for (int i = 1; i <= 20; i++) fact[i] = fact[i - 1] * i;
 Scanner sc = new Scanner(System.in);
 String s = sc.next();
 long k = sc.nextLong();
 int[] counts = new int[256];
 for (char c : s.toCharArray()) counts[c]++;
 int n = s.length();
 if (k > arrangements(counts, n)) {
 System.out.println("NONE");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int pos = 0; pos < n; pos++) {
 for (int c = 0; c < 256; c++) {
 if (counts[c] == 0) continue;
 counts[c]--;
 long block = arrangements(counts, n - pos - 1); // permutations that start with this letter here
 if (k <= block) {
 sb.append((char) c);
 break;
 }
 k -= block;
 counts[c]++;
 }
 }
 System.out.println(sb);
 }
}
```

Why it works: Ranking via multinomial counts: O(n · σ) instead of enumerating up to n! permutations.

Verified against 3 test cases: `DBCA 10⏎` → `BCDA`; `AAB 3⏎` → `BAA`; `AB 3⏎` → `NONE`

5.29 N-Queens (Backtracking)

Exercise 1 · Validate a placement

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Read n and, for each row, the column of its queen. Print VALID if no two queens attack each other.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] col = new int[n];
 for (int i = 0; i < n; i++) col[i] = sc.nextInt();
 boolean ok = true;
 for (int i = 0; i < n && ok; i++) {
 for (int j = i + 1; j < n; j++) {
 if (col[i] == col[j] || Math.abs(col[i] - col[j]) == j - i) {
 ok = false;
 break;
 }
 }
 }
 System.out.println(ok ? "VALID" : "INVALID");
 }
}
```

Why it works: Check every pair of rows: equal columns or equal row and column distance mean attack.

Verified against 3 test cases: `4⏎2 4 1 3⏎` → `VALID`; `4⏎1 2 3 4⏎` → `INVALID`; `1⏎1⏎` → `VALID`

Exercise 2 · Count solutions

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static int n, solutions;
 static boolean[] c, d1, d2;

 static void place(int r) {
 if (r == n) {
 solutions++;
 return;
 }
 for (int col = 0; col < n; col++) {
 int a = r - col + n - 1, b = r + col;
 if (c[col] || d1[a] || d2[b]) continue;
 c[col] = d1[a] = d2[b] = true;
 place(r + 1);
 c[col] = d1[a] = d2[b] = false;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 c = new boolean[n];
 d1 = new boolean[2 * n];
 d2 = new boolean[2 * n];
 place(0);
 System.out.println(solutions);
 }
}
```

Why it works: Exhaustive backtracking with O(1) conflict checks.

Verified against 4 test cases: `6⏎` → `4`; `8⏎` → `92`; `3⏎` → `0`

Exercise 3 · Lexicographically first solution

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static int n;
 static int[] pos;
 static boolean[] c, d1, d2;

 static boolean place(int r) {
 if (r == n) return true;
 for (int col = 0; col < n; col++) {
 int a = r - col + n - 1, b = r + col;
 if (c[col] || d1[a] || d2[b]) continue;
 c[col] = d1[a] = d2[b] = true;
 pos[r] = col;
 if (place(r + 1)) return true;
 c[col] = d1[a] = d2[b] = false;
 }
 return false;
 }

 // The lexicographically first solution as "column of the queen in each row" (1-based), or NONE.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 n = sc.nextInt();
 pos = new int[n];
 c = new boolean[n];
 d1 = new boolean[2 * n];
 d2 = new boolean[2 * n];
 if (!place(0)) {
 System.out.println("NONE");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(pos[i] + 1);
 System.out.println(sb);
 }
}
```

Why it works: Depth-first search trying columns left to right returns the lexicographically smallest solution first.

Verified against 3 test cases: `8⏎` → `1 5 8 6 3 7 2 4`; `4⏎` → `2 4 1 3`; `3⏎` → `NONE`

5.30 Maze Solving (Rat in a Maze)

Exercise 1 · How many cells can the rat reach?

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Count the cells reachable from the entry (2) moving up/down/left/right through non-walls
     (flood fill).
 static int r, c;
 static int[][] g;
 static boolean[][] seen;

 static int fill(int i, int j) {
 if (i < 0 || j < 0 || i >= r || j >= c || g[i][j] == 1 || seen[i][j]) return 0;
 seen[i][j] = true;
 return 1 + fill(i + 1, j) + fill(i - 1, j) + fill(i, j + 1) + fill(i, j - 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 r = sc.nextInt();
 c = sc.nextInt();
 g = new int[r][c];
 seen = new boolean[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 System.out.println(fill(si, sj));
 }
}
```

Why it works: A flood fill counts every cell entered by the DFS exactly once.

Verified against 3 test cases: `3 3⏎2 0 1⏎1 0 1⏎1 0 3⏎` → `5`; `1 3⏎2 1 3⏎` → `1`; `2 2⏎2 0⏎0 3⏎` → `4`

Exercise 2 · Is there a way out?

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 static int r, c;
 static int[][] g;
 static boolean[][] seen;

 static boolean reach(int i, int j) {
 if (i < 0 || j < 0 || i >= r || j >= c || g[i][j] == 1 || seen[i][j]) return false;
 if (g[i][j] == 3) return true;
 seen[i][j] = true;
 return reach(i + 1, j) || reach(i - 1, j) || reach(i, j + 1) || reach(i, j - 1);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 r = sc.nextInt();
 c = sc.nextInt();
 g = new int[r][c];
 seen = new boolean[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 System.out.println(reach(si, sj) ? "YES" : "NO");
 }
}
```

Why it works: Depth-first search with a visited array; true when the exit cell is reached.

Verified against 3 test cases: `3 3⏎2 1 3⏎0 1 0⏎0 0 0⏎` → YES; `1 3⏎2 1 3⏎` → NO; `1 2⏎2 3⏎` → YES

Exercise 3 · Shortest path

Ex3.java

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class Ex3 {
 // Length (number of moves) of the SHORTEST route from entry (2) to exit (3), or -1.
     Breadth-first search.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int r = sc.nextInt();
 int c = sc.nextInt();
 int[][] g = new int[r][c];
 int si = 0, sj = 0;
 for (int i = 0; i < r; i++) for (int j = 0; j < c; j++) {
 g[i][j] = sc.nextInt();
 if (g[i][j] == 2) { si = i; sj = j; }
 }
 int[][] dist = new int[r][c];
 for (int[] row : dist) java.util.Arrays.fill(row, -1);
 Deque<int[]> q = new ArrayDeque<>();
 q.add(new int[] {si, sj});
 dist[si][sj] = 0;
 int[] dr = {1, -1, 0, 0};
 int[] dc = {0, 0, 1, -1};
 while (!q.isEmpty()) {
 int[] cur = q.poll();
 if (g[cur[0]][cur[1]] == 3) {
 System.out.println(dist[cur[0]][cur[1]]);
 return;
 }
 for (int d = 0; d < 4; d++) {
 int ni = cur[0] + dr[d], nj = cur[1] + dc[d];
 if (ni < 0 || nj < 0 || ni >= r || nj >= c || g[ni][nj] == 1 || dist[ni][nj] != -1) continue;
 dist[ni][nj] = dist[cur[0]][cur[1]] + 1;
 q.add(new int[] {ni, nj});
 }
 }
 System.out.println(-1);
 }
}
```

Why it works: BFS visits cells in order of distance; the first time the exit is dequeued its distance is minimal.

Verified against 3 test cases: `3 3⏎2 0 0⏎1 1 0⏎3 0 0⏎` → `6`; `1 3⏎2 1 3⏎` → `-1`; `1 2⏎2 3⏎` → `1`

5.31 Hamiltonian Cycle

Exercise 1 · Verify a cycle

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Input: v, adjacency matrix, then a vertex sequence of length v+1. Is it a Hamiltonian cycle?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int v = sc.nextInt();
 int[][] g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 int[] p = new int[v + 1];
 for (int i = 0; i <= v; i++) p[i] = sc.nextInt();
 boolean ok = p[0] == p[v];
 boolean[] seen = new boolean[v];
 for (int i = 0; i < v && ok; i++) {
 if (seen[p[i]]) ok = false;
 seen[p[i]] = true;
 if (g[p[i]][p[i + 1]] == 0) ok = false;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: Linear scan with a seen array.

Verified against 3 test cases: `4⏎0 1 0 1⏎1 0 1 0⏎0 1 0 1⏎1 0 1 0⏎0 1 2 3 0⏎` → `YES`; `4⏎0 1 0 1⏎1 0 1 0⏎0 1 0 1⏎1 0 1 0⏎0 2 1 3 0⏎` → `NO`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎0 1 2 0⏎` → `YES`

Exercise 2 · Find a cycle

Ex2.java

```java
import java.util.Arrays;
import java.util.Scanner;

public class Ex2 {
 static int v;
 static int[][] g;
 static int[] path;

 static boolean extend(int pos) {
 if (pos == v) return g[path[v - 1]][path[0]] == 1;
 for (int c = 1; c < v; c++) {
 boolean ok = g[path[pos - 1]][c] == 1;
 for (int i = 0; i < pos && ok; i++) if (path[i] == c) ok = false;
 if (!ok) continue;
 path[pos] = c;
 if (extend(pos + 1)) return true;
 path[pos] = -1;
 }
 return false;
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 v = sc.nextInt();
 g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 path = new int[v];
 Arrays.fill(path, -1);
 path[0] = 0;
 if (!extend(1)) {
 System.out.println("NO");
 return;
 }
 StringBuilder sb = new StringBuilder();
 for (int x : path) sb.append(x).append(' ');
 sb.append(0);
 System.out.println("YES " + sb);
 }
}
```

Why it works: Depth-first backtracking, candidates in increasing order, close the cycle at the end.

Verified against 3 test cases: `5⏎0 1 0 1 0⏎1 0 1 1 1⏎0 1 0 0 1⏎1 1 0 0 0⏎0 1 1 0 0⏎` → `NO`; `5⏎0 1 0 1 0⏎1 0 1 1 1⏎0 1 0 0 1⏎1 1 0 0 1⏎0 1 1 1 0⏎` → `YES 0 1 2 4 3 0`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎` → `YES 0 1 2 0`

Exercise 3 · Count Hamiltonian cycles

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 static int v, count;
 static int[][] g;
 static boolean[] used;

 static void dfs(int cur, int placed) {
 if (placed == v) {
 if (g[cur][0] == 1) count++;
 return;
 }
 for (int nxt = 1; nxt < v; nxt++) {
 if (!used[nxt] && g[cur][nxt] == 1) {
 used[nxt] = true;
 dfs(nxt, placed + 1);
 used[nxt] = false;
 }
 }
 }

 // Count Hamiltonian cycles that start and end at vertex 0, counting a cycle and its reverse separately.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 v = sc.nextInt();
 g = new int[v][v];
 for (int i = 0; i < v; i++) for (int j = 0; j < v; j++) g[i][j] = sc.nextInt();
 used = new boolean[v];
 used[0] = true;
 dfs(0, 1);
 System.out.println(count);
 }
}
```

Why it works: Count every complete path that closes back to vertex 0.

Verified against 3 test cases: `4⏎0 1 1 1⏎1 0 1 1⏎1 1 0 1⏎1 1 1 0⏎` → `6`; `3⏎0 1 1⏎1 0 1⏎1 1 0⏎` → `2`; `4⏎0 1 0 0⏎1 0 1 0⏎0 1 0 1⏎0 0 1 0⏎` → `0`

5.32 Knight's Tour & Warnsdorff's Rule

Exercise 1 · Verify a tour

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Input: n, then an n x n board of move numbers. Is it a valid knight's tour (1..n*n, each step
     a knight move)?
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] px = new int[n * n + 1];
 int[] py = new int[n * n + 1];
 boolean ok = true;
 for (int y = 0; y < n; y++) {
 for (int x = 0; x < n; x++) {
 int v = sc.nextInt();
 if (v < 1 || v > n * n || px[v] != 0) { // out of range or a repeated number
 ok = false;
 } else {
 px[v] = x + 1;
 py[v] = y + 1;
 }
 }
 }
 for (int v = 1; v < n * n && ok; v++) {
 if (px[v] == 0 || px[v + 1] == 0) {
 ok = false;
 break;
 }
 int dx = Math.abs(px[v] - px[v + 1]);
 int dy = Math.abs(py[v] - py[v + 1]);
 if (!((dx == 1 && dy == 2) || (dx == 2 && dy == 1))) ok = false;
 }
 System.out.println(ok ? "YES" : "NO");
 }
}
```

Why it works: Positions per number, then check each consecutive pair.

Verified against 3 test cases: `5⏎1 6 15 10 21⏎14 9 20 5 16⏎19 2 7 22 11⏎8 13 24 17 4⏎25 18 3 12 23⏎` → YES; `3⏎1 2 3⏎4 5 6⏎7 8 9⏎` → NO; `1⏎1⏎` → YES

Exercise 2 · Degree of a square

Ex2.java

```java
import java.util.Scanner;

public class Ex2 {
 // How many squares can a knight on (x, y) reach on an empty n x n board? (the "degree"
     Warnsdorff uses)
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int x = sc.nextInt();
 int y = sc.nextInt();
 int[] dx = {1, 1, 2, 2, -1, -1, -2, -2};
 int[] dy = {2, -2, 1, -1, 2, -2, 1, -1};
 int c = 0;
 for (int m = 0; m < 8; m++) {
 int nx = x + dx[m], ny = y + dy[m];
 if (nx >= 0 && ny >= 0 && nx < n && ny < n) c++;
 }
 System.out.println(c);
 }
}
```

Why it works: Count the legal moves; corners have 2, the centre 8.

Verified against 4 test cases: `8 0 0⏎` → `2`; `8 3 3⏎` → `8`; `5 2 2⏎` → `8`

Exercise 3 · Reachable squares

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Count the squares of an n x n board that a knight can reach from (0,0) in exactly k moves
 // (squares may be visited again). Plain breadth-first layers.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] dx = {1, 1, 2, 2, -1, -1, -2, -2};
 int[] dy = {2, -2, 1, -1, 2, -2, 1, -1};
 boolean[][] cur = new boolean[n][n];
 cur[0][0] = true;
 for (int step = 0; step < k; step++) {
 boolean[][] next = new boolean[n][n];
 for (int y = 0; y < n; y++) {
 for (int x = 0; x < n; x++) {
 if (!cur[y][x]) continue;
 for (int m = 0; m < 8; m++) {
 int nx = x + dx[m], ny = y + dy[m];
 if (nx >= 0 && ny >= 0 && nx < n && ny < n) next[ny][nx] = true;
 }
 }
 }
 cur = next;
 }
 int c = 0;
 for (boolean[] row : cur) for (boolean b : row) if (b) c++;
 System.out.println(c);
 }
}
```

Why it works: Layer-by-layer breadth-first expansion: the set of squares reachable in exactly i moves.

Verified against 3 test cases: `8 2⏎` → `10`; `8 1⏎` → `2`; `8 0⏎` → `1`

5.33 Selection Sort

Exercise 1 · Sort and count swaps

Ex1.java

```java
import java.util.Scanner;

public class Ex1 {
 // Selection sort; print how many swaps that actually move something (minIndex != i) were
     performed.
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 int swaps = 0;
 for (int i = 0; i < n - 1; i++) {
 int m = i;
 for (int j = i + 1; j < n; j++) if (a[j] < a[m]) m = j;
 if (m != i) {
 int t = a[m];
 a[m] = a[i];
 a[i] = t;
 swaps++;
 }
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 System.out.println(swaps);
 }
}
```

Why it works: Standard selection sort with a counter around the swap.

Verified against 3 test cases: `5⏎64 25 12 22 11⏎` → `11 12 22 25 64⏎3`; `3⏎1 2 3⏎` → `1 2 3⏎0`; `4⏎4 3 2 1⏎` → `1 2 3 4⏎2`

Exercise 2 · Largest first

SelEx2.java

```java
import java.util.Scanner;

public class SelEx2 {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 0; i < n - 1; i++) {
 int maxIndex = i;
 for (int j = i + 1; j < n; j++) {
 if (a[j] > a[maxIndex]) maxIndex = j;
 }
 int t = a[maxIndex];
 a[maxIndex] = a[i];
 a[i] = t;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Select the largest remaining value in each pass.

Verified against 3 test cases: `5⏎3 1 4 1 5⏎` → `5 4 3 1 1`; `1⏎7⏎` → `7`; `3⏎-1 -5 -3⏎` → `-1 -3 -5`

Exercise 3 · K smallest, cheaply

SelEx3.java

```java
import java.util.Scanner;

public class SelEx3 {
 // The k smallest values in increasing order using only k selection passes: O(n * k).
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 for (int i = 0; i < k; i++) {
 int m = i;
 for (int j = i + 1; j < n; j++) {
 if (a[j] < a[m]) m = j;
 }
 int t = a[m];
 a[m] = a[i];
 a[i] = t;
 }
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < k; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Reuse selection: after k passes the first k positions hold the k smallest in order.

Verified against 3 test cases: `6 3⏎7 10 4 3 20 15⏎` → `3 4 7`; `3 3⏎3 2 1⏎` → `1 2 3`; `4 1⏎9 8 7 6⏎` → `6`

5.34 Quick Sort

Exercise 1 · Count the partitions

QuickSortEx1.java

```java
import java.util.Scanner;

public class QuickSortEx1 {
 static int partition(int[] a, int low, int high) {
 int pivot = a[high];
 int i = low - 1;
 for (int j = low; j < high; j++) {
 if (a[j] <= pivot) {
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[high];
 a[high] = t;
 return i + 1;
 }

 static void quickSort(int[] a, int low, int high) {
 if (low < high) {
 int p = partition(a, low, high);
 quickSort(a, low, p - 1);
 quickSort(a, p + 1, high);
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 quickSort(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: The lesson's algorithm with the output formatted as one line.

Verified against 3 test cases: `7⏎8 3 1 7 0 10 2⏎` → `0 1 2 3 7 8 10`; `1⏎5⏎` → `5`; `5⏎2 2 2 2 2⏎` → `2 2 2 2 2`

Exercise 2 · Descending quick sort

QuickSortEx2.java

```java
import java.util.Scanner;

public class QuickSortEx2 {
 static void quick(int[] a, int lo, int hi) {
 if (lo >= hi) return;
 int pivot = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++) {
 if (a[j] >= pivot) { // descending order: larger values go to the left part
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[hi];
 a[hi] = t;
 quick(a, lo, i);
 quick(a, i + 2, hi);
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 quick(a, 0, n - 1);
 StringBuilder sb = new StringBuilder();
 for (int i = 0; i < n; i++) sb.append(i == 0 ? "" : " ").append(a[i]);
 System.out.println(sb);
 }
}
```

Why it works: Partition by "≥ pivot goes left" and recurse as before.

Verified against 3 test cases: `5⏎3 1 4 1 5⏎` → `5 4 3 1 1`; `2⏎1 2⏎` → `2 1`; `1⏎0⏎` → `0`

Exercise 3 · k-th smallest with quickselect

Ex3.java

```java
import java.util.Scanner;

public class Ex3 {
 // Quickselect: k-th smallest (1-based) without sorting everything. Average O(n).
 static int select(int[] a, int k) {
 int lo = 0, hi = a.length - 1;
 while (true) {
 int pivot = a[hi], i = lo - 1;
 for (int j = lo; j < hi; j++) {
 if (a[j] <= pivot) {
 i++;
 int t = a[i];
 a[i] = a[j];
 a[j] = t;
 }
 }
 int t = a[i + 1];
 a[i + 1] = a[hi];
 a[hi] = t;
 int p = i + 1;
 if (p == k - 1) return a[p];
 if (p > k - 1) hi = p - 1;
 else lo = p + 1;
 }
 }

 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int k = sc.nextInt();
 int[] a = new int[n];
 for (int i = 0; i < n; i++) a[i] = sc.nextInt();
 System.out.println(select(a, k));
 }
}
```

Why it works: Quickselect discards the side that cannot contain the k-th element, so the work forms a geometric series ≈ 2n.

Verified against 3 test cases: `6 3⏎7 10 4 3 20 15⏎` → `7`; `3 1⏎9 8 7⏎` → `7`; `4 4⏎1 2 3 4⏎` → `4`

Quiz answer key

5.1 Euclid's Algorithm (GCD)

	Answer	Why
Q1	B. 6	48 = 2·18 + 12, 18 = 1·12 + 6, 12 = 2·6 + 0 → gcd = 6.
Q2	12	84 mod 36 = 12 → (36, 12); 36 mod 12 = 0 → (12, 0). a = 12.
Q3	line 4	Line 4 runs after a was overwritten, so `a % b` is `b % b` = 0. Save the remainder first.
Q4	0 | a | b	gcd(a, 0) = a, otherwise gcd(a, b) = gcd(b, a mod b).
Q5	B. O(log min(a, b))	The remainder is more than halved every two steps, so only about $2 \cdot \log_2(\min)$ steps are needed.
Q6	a after iteration 1 = 12; b after iteration 1 = 2; b after iteration 2 = 0; final a = 2	(50, 12) → (12, 2) → (2, 0). The answer is the last a.

5.2 Euler's Phi (Totient) Function

	Answer	Why
Q1	C. 16	17 is prime: every k from 1 to 16 is coprime to it. φ(p) = p − 1.
Q2	A. 16	$60 = 2^2 \cdot 3 \cdot 5$ → 60 × 1/2 × 2/3 × 4/5 = 16. (The lecture slide says 32 — that is a mistake.)
Q3	12	$36 = 2^2 \cdot 3^2$ → 36 − 18 = 18 (p = 2); then 18 − 6 = 12 (p = 3).
Q4	-=	result − result/p = result · (1 − 1/p).
Q5	B. A prime factor larger than √n can remain	The loop only searches p ≤ √n. A single bigger prime factor can survive as the remaining value of n.
Q6	φ(1) → 1; φ(9) → 6; φ(10) → 4; φ(13) → 12	φ(9) = 9 − 3 = 6; φ(10) = 4 (1,3,7,9); φ(13) = 12 (prime).

5.3 Chinese Remainder Theorem

	Answer	Why
Q1	A. 23	233 mod 105 = 23 and 23 checks out for all three.
Q2	C. Pairwise coprime	Pairwise coprime moduli make each M_i invertible modulo m_i.
Q3	17	Numbers ≡ 1 (mod 4): 1, 5, 9, 13, 17 → 17 % 5 = 2.
Q4	B. M / m_i	M_i = M / m_i is the product of all the other moduli.
Q5	C. no solution	x ≡ 1 (mod 6) forces x odd; x ≡ 2 (mod 4) forces x even.

5.4 Combinations (nCr)

	Answer	Why
Q1	B. 15	6!/(2!·4!) = 720/(2·24) = 15.
Q2	C. 30	6 · 5 = 30. Dividing by 2! gives the 15 combinations.
Q3	15	i=1: 1·5/1 = 5; i=2: 5·6/2 = 15.
Q4	B. fact(15) overflows int	$15! \approx 1.3 \times 10^{12} > 2.1 \times 10^9$.
Q5	A. C(n, n−r)	Choosing r to include is the same as choosing n−r to leave out.

5.5 Karatsuba's Multiplication

	Answer	Why
Q1	B. 3	ac, bd and (a+b)(c+d).
Q2	A. (a+b)(c+d) − ac − bd	Expanding (a+b)(c+d) = ac + ad + bc + bd.
Q3	C. O(n^1.585)	T(n) = 3T(n/2) + O(n) gives n^($\log_2$ 3).
Q4	2840	46 × 134 = 6164; 6164 − 672 − 2652 = 2840 = ad + bc.
Q5	B. Karatsuba has overhead that dominates for small n	Recursion and extra additions cost more than 1-2 machine multiplications.

5.6 Booth's Algorithm (signed binary multiplication)

	Answer	Why
Q1	C. 10	"10" marks the start of a run of 1s.
Q2	1001	1000 + 0001 = 1001 = −7.
Q3	B. 1110	The sign bit (1) is copied into the vacated left position.
Q4	A. 00010101	0001 0101 = 21 = 7 × 3.
Q5	B. 11111111	A single long run of 1s needs just one subtract (and the result is just a shift).

5.7 Simple Sieve of Eratosthenes

	Answer	Why
Q1	B. Because every composite ≤ n has a prime factor ≤ √n	A composite m = a·b with a ≤ b has a ≤ √m ≤ √n.
Q2	9 12 15 18 21 24 27 30	Multiples of 3 starting from 9.
Q3	B. O(n log log n)	Σ n/p over primes ≈ n log log n.
Q4	line 4	Starting at m = p crosses out p itself, so every prime is removed. Start at 2·p (or p·p).
Q5	B. 15	2 3 5 7 11 13 17 19 23 29 31 37 41 43 47.

5.8 Segmented & Incremental Sieve

	Answer	Why
Q1	C. √R	Every composite ≤ R has a prime factor ≤ √R.
Q2	105	(100 + 6) / 7 = 15; 15 · 7 = 105.
Q3	B. To avoid marking p itself and values below p·p	Smaller multiples were handled by smaller primes; p itself must stay unmarked.
Q4	B. Produces primes one after another, remembering upcoming composites	It stores for each upcoming composite the primes that will cross it out.

5.9 Binary Palindrome

	Answer	Why
Q1	B. 9 (1001)	1001 reads the same both ways.
Q2	3	6 = 110 → reversed 011 = 3.
Q3	B. multiplies by 2	A left shift moves every bit one place to the left: multiplication by 2.

	Answer	Why
Q4	&	`rest & 1` isolates the lowest bit.
Q5	B. O(log n)	One iteration per bit; the number of bits is $\log_2 n$.

5.10 Swap Two Nibbles in a Byte

	Answer	Why
Q1	B. 4	A nibble is half a byte: 4 bits.
Q2	195	0x3C → 0xC3 = 195.
Q3	B. 0xF0	0xF0 = 1111 0000.
Q4	B. To remove sign extension and get 0–255	Bytes are signed; & 0xFF keeps the low 8 bits as an unsigned value.

5.11 Longest Sequence of 1s (and after flipping a bit)

	Answer	Why
Q1	3	01101110: runs of 1s are 3 (bits 1–3) and 2 (bits 5–6).
Q2	C. previous + current + 1	Left run + flipped bit + right run.
Q3	B. 32	All 32 bits are already 1; a 32-bit int cannot hold more.
Q4	B. O(log n) (32 steps)	One step per bit; the number of bits is logarithmic in the value.

5.12 Josephus Problem

	Answer	Why
Q1	C. 5	Order: 2, 4, 6, 3, 1 → survivor 5.
Q2	3	ans: 0 → (0+2)%2=0 → (0+2)%3=2 → (2+2)%4=0 → (0+2)%5=2; +1 = 3.
Q3	C. O(n)	One loop iteration per person.
Q4	+	Shift by k places and wrap around the circle of `people`.
Q5	A. 9	2·4 + 1 = 9.

5.13 Block Swap Algorithm (Array Rotation)

	Answer	Why
Q1	3 4 5 1 2	Element i of the output is element (i + 2) mod 5 of the input.
Q2	C. Three reversals	Reverse(A), reverse(B), reverse(all) uses only swaps.
Q3	B. 5	n − d = 5.
Q4	C. O(n·d)	d passes of n element moves.
Q5	3	Final array: 3 4 5 6 1 2.

5.14 Leaders in an Array

	Answer	Why
Q1	A. 17 5 2	17 > all to its right; 5 > 2; 2 is last. 16 is smaller than 17.
Q2	line 1	Initialising max to 0 hides negative leaders; use Integer.MIN_VALUE.
Q3	B. O(n)	One pass.

	Answer	Why
Q4	A. Yes	Nothing to its right can be larger.

5.15 Majority Element (Boyer–Moore Voting)

	Answer	Why
Q1	B. at least 5	More than 9/2 = 4.5 → at least 5.
Q2	count after 2nd element (1) = 0; candidate after 3rd element (2) = 2; count after 4th element (4) = 0; final candidate = 7	(2,1) → (2,0) → (2,1) → (2,0) → (7,1). 7 is only a candidate; verification fails.
Q3	B. The candidate may not be a majority	If no majority exists, phase 1 still returns some value.
Q4	C. Boyer–Moore	Voting needs only a candidate and a counter.
Q5	B. O(n log n)	The sort dominates.

5.16 Maximum Product Subarray

	Answer	Why
Q1	B. 15	n(n+1)/2 = 15.
Q2	24	(−2)·3·(−4) = 24.
Q3	B. A negative number can turn it into the maximum	Most negative × negative = largest positive.
Q4	A. O(n)	One pass with O(1) work per element.
Q5	2	The subarray {2} has product 2.

5.17 Maximum Equilibrium Sum

	Answer	Why
Q1	B. 2	Left 1+3 = 4, right 2+2 = 4 at index 2.
Q2	3	8 − 4 − 1 = 3 (the elements 2 and 1).
Q3	B. total − left − a[i]	Everything not on the left and not a[i] is on the right.
Q4	B. O(n)	Two linear passes.

5.18 Maximum Sum of an Hourglass in a Matrix

	Answer	Why
Q1	A. 12	(5−2)(6−2) = 12.
Q2	B. two full rows of 3 and the centre cell	Top row, centre of the middle row, bottom row.
Q3	35	1+2+3+5+7+8+9 = 35 (the slide's own example).
Q4	B. All sums may be negative	Otherwise the result 0 would not correspond to any hourglass.

5.19 Toggle the Switch (100 Doors)

	Answer	Why
Q1	B. 10	The perfect squares 1, 4, 9, ..., 100.
Q2	B. 9	Divisors of 36: 1, 2, 3, 4, 6, 9, 12, 18, 36 — nine (odd), so it is open.
Q3	B. Divisors pair up as (d, n/d) except when d = n/d	The unpaired divisor √n makes the count odd.
Q4	B. O(n log n)	Σ n/k = n H(n).

5.20 Alice's Apple Tree (Minimum Garden Perimeter)

	Answer	Why
Q1	B. 48	$12 \cdot 2^2 = 48$ (and 12 + 48 = 60 inside the r = 2 square).
Q2	60	2·2·3·5 = 60.
Q3	B. apples(r) is monotonically increasing	More rings always mean more apples.
Q4	B. 8r	Side length 2r, four sides: 8r.

5.21 Weighted Substring (Smallest String of a Given Weight)

	Answer	Why
Q1	C. 60	D = 4·C + C = 5·12 = 60.
Q2	B. ACC	12 + 12 + 1 = 25 with just 3 letters.
Q3	ABB	Greedy takes B (3), B (3), then A (1): 3 + 3 + 1 = 7. Sorted: ABB.
Q4	B. Each weight divides the next	Divisibility makes the coin system canonical.

5.22 Move Hyphen to the Beginning

	Answer	Why
Q1	-abcd	Before: "ab", after: "cd".
Q2	B. -1	indexOf returns −1 when there is no match.
Q3	B. Strings are immutable so every += copies the string	Each step creates a new String of increasing length.
Q4	B. O(n)	Every character is read once and written once.

5.23 Strobogrammatic Number

	Answer	Why
Q1	C. 69	69 rotated by 180° is 69. 25 and 1962 contain 2 (no valid rotation); 962 contains 2 as well.
Q2	A. 0, 1, 8	The middle digit must rotate into itself.
Q3	true	8 rotates to 8 and 88 reads the same.
Q4	B. 4	11, 69, 88, 96.

5.24 Lexicographically First Palindromic Rearrangement

	Answer	Why
Q1	B. level	level: l:2, e:2, v:1 — one odd count.

	Answer	Why
Q2	abcba	front "ab", middle "c", back "ba".
Q3	C. O(n)	One pass to count plus a pass over the alphabet.
Q4	A. 0	The counts add up to an even number, so the number of odd counts is even; for a palindrome it must be ≤ 1, hence 0.

5.25 Manacher's Algorithm (Longest Palindromic Substring)

	Answer	Why
Q1	B. To make every palindrome odd-length	Even palindromes get a "#" centre.
Q2	C. O(n)	The right edge only moves forward.
Q3	#a#b#b#a#	n characters → 2n+1 characters.
Q4	B. aaaaaa	Every centre expands to the borders.

5.26 Natural Sort Order

	Answer	Why
Q1	B. file10	Compare "1" with "2": 1 < 2, so file10 comes first.
Q2	B. equal numerically (then tie-break)	Numerically they are both 7; add a tie-breaker for a deterministic order.
Q3	C. a negative number	Negative, zero, positive: less, equal, greater.
Q4	A. stable	It uses TimSort, a stable sort.

5.27 Maneuvering (Counting Grid Paths)

	Answer	Why
Q1	B. 20	C(6, 3) = 20.
Q2	6	Rows: 1 1 1 / 1 2 3 / 1 3 6.
Q3	B. It recomputes the same sub-grids many times	Overlapping sub-problems → exponential calls.
Q4	B. O(m·n)	Each cell is filled once.

5.28 Sorted Distinct Permutations

	Answer	Why
Q1	B. 6	4!/(2!·2!) = 6.
Q2	BAC	Pivot is A (A < C); smallest larger letter to its right is B; swap → BCA, reverse the tail "CA" → "AC" → BAC.
Q3	B. It prints every distinct permutation once, but not in sorted order	The HashSet prevents duplicates; swapping disturbs the order.
Q4	C. O(8·8!)	8! permutations × O(8) to build/print each.

5.29 N-Queens (Backtracking)

	Answer	Why
Q1	B. 2	Two mirror-image solutions.
Q2	C. same diagonal	|Δrow| = |Δcol| = 2.

	Answer	Why
Q3	0	Remove the queen again (un-choose).
Q4	B. Undoing the last choice and trying the next option	Return to the previous decision and change it.
Q5	C. Backtracking	Systematic search with pruning.

5.30 Maze Solving (Rat in a Maze)

	Answer	Why
Q1	B. To avoid walking in circles forever	Without it the search can bounce between two cells indefinitely.
Q2	B. BFS	BFS explores in rings of increasing distance.
Q3	B. Un-mark a dead-end cell (backtrack)	The cell is not on any path to the exit.
Q4	B. O(R·C)	Each cell is expanded once.

5.31 Hamiltonian Cycle

	Answer	Why
Q1	B. every vertex exactly once and returns to the start	Edges once = Eulerian; vertices once = Hamiltonian.
Q2	B. A cycle can be rotated to start anywhere	Any cycle through all vertices contains vertex 0.
Q3	C. O(V!)	Up to (V−1)! orderings.
Q4	B. impossible	A cycle needs two edges at every vertex.

5.32 Knight's Tour & Warnsdorff's Rule

	Answer	Why
Q1	C. 8	Eight L-shaped jumps.
Q2	B. the fewest onward moves	Visit the most constrained square first.
Q3	B. No, it is a heuristic	It can get stuck; check the result.
Q4	B. O(N^2)	N^2 moves with constant work each.

5.33 Selection Sort

	Answer	Why
Q1	10 13 14 37 29	Pass 1: swap 10 with 29 → 10 29 14 37 13. Pass 2: min of rest is 13 → swap with 29 → 10 13 14 37 29.
Q2	C. 15	5 + 4 + 3 + 2 + 1 = 15.
Q3	B. At most n−1 swaps	One swap per pass at most.
Q4	C. O(n^2)	It always scans the entire unsorted part.

5.34 Quick Sort

	Answer	Why
Q1	B. in its final sorted position	Everything left is ≤ and everything right is >.
Q2	C. O(n^2)	Highly unbalanced partitions.

	Answer	Why
Q3	5	Five elements (3, 2, 5, 1, 4) are ≤ 6 so the pivot goes to index 5.
Q4	B. Divide and conquer	Partition, then solve the two halves independently.
Q5	B. No	Partitioning swaps distant elements, so equal keys may be reordered.